Diskrete Strukturen

© F. Brandt, J. Esparza, S. Jegelka und M. Luttenberger[1] (26w)

Department of Computer Science
TUM CIT

20. Oktober 2026

[1] Erstautor

No c[illegible], only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

1. Einführung
2. Grundlagen
3. Graphentheorie
4. Kombinatorik
5. Logik
6. Grundlagen der Gruppen- und Zahlentheorie

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

1 Einführung

2 Grundlagen

3 Graphentheorie

4 Kombinatorik

5 Logik

6 Grundlagen der Gruppen- und Zahlentheorie

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Was sind „Diskrete Strukturen“?

- Diskrete Mathematik ist ein Sammelbegriff für die Bereiche der Mathematik, die sich speziell mit diskreten Mengen, Größen, Systemen bzw. Strukturen beschäftigen.
  - „diskret“ von „discernere“: unterscheidbar
  - „diskret“ bedeutet hier grob, dass alle betrachteten Objekt eindeutig durch natürliche Zahlen (IDs) beschriftet werden können (sie sind abzählbar)
- Der Begriff „diskret“ steht im Gegensatz zu „kontinuierlich“.
  - „kontinuierlich“ von continere: verbunden
  - In der klassischen Physik ist die Standardannahme, dass Raum und Zeit (und Bewegungen) kontinuierlich sind, d.h. „keine Löcher haben“ und keine „Sprünge“ machen.
    (Zumindest, wenn man nicht zu genau hinschaut/misst.)
  - Im Gegensatz zu $\mathbb{N}, \mathbb{Z}, \mathbb{Q}$ ist $\mathbb{R}$ nicht abzählbar (kurz: „überabzählbar“).
- Ziel dieser Vorlesung ist es, die mathematischen Grundlagen (insbesondere Grundbegriffe) der Informatik und „formales Denken“ zu vermitteln.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Überblick

- In dieser Vorlesung behandeln wir insbesondere die folgenden Bereiche:
  - Grundlagen: informelle Logik, Mengen, Relationen, Funktionen, Kardinalität

    Grundbegriffe für die weiteren Themen, aber auch z.B. für Pseudocode und Datentypen
  - Graphentheorie: Eigenschaften von Graphen bestehend aus Knoten und Kanten,

    z.B. Grundlage für verzeigerte Datentypen, Routenplanung, soziale Netzwerke
  - Kombinatorik: Effizientes Abzählen von implizit gegebenen Mengen,

    z.B. Grundlage für diskrete Wahrscheinlichkeitstheorie und Laufzeitabschätzung
  - Logik: Syntax und Semantik aussagenlogischer Formeln

    z.B. Grundlagen für Beweise, Programmiersprachen, Prozessoren, Spezifikationen, Verifikation
  - Algebra: Eigenschaften von Gruppen

    z.B. Grundlage für Kryptographie, lineare Algebra, fehlerkorrigierende Codes, Hashtabellen

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

1 Einführung

2 Grundlagen

3 Graphentheorie

4 Kombinatorik

5 Logik

6 Grundlagen der Gruppen- und Zahlentheorie

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

2 Grundlagen

Grundbegriffe der Logik

Mengen

Folgen, Tupel und Wörter

Die natürlichen Zahlen, Induktionsbeweise

Relationen

Funktionen

Kardinalität: Vergleiche von unendlichen Mengen

Zusammenfassung

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Mehrdeutigkeit

⚠ Zentrales Ziel der Mathematik ist Eindeutigkeit (wohldefiniert, wohlgeformt, wohlgestellt, wohlfundiert), d.h. es gibt nur eine eindeutige Lesart (Interpretation) einer Aussage (Aussagesatzes):

- „Alleen sind nicht gefährlich – Rasen ist gefährlich“

  Mehrdeutige Semantik: „das Rasen“ vs. „der Rasen“?

- “I saw a man on the hill with a telescope”

  Mehrdeutige Syntax: Gruppierung der Wörter zu Satzteilen?

  (I saw (a man (on the hill) (with a telescope)))

  (I saw (a man (on the hill)) (with a telescope))

  (I saw (a man (on the hill (with a telescope))))

- Klammern „$(,),[,],\{,\},\langle,\rangle$“ (und übliche Satzzeichen) zur Vorgabe einer eindeutigen Gruppierung. (Siehe auch Dyck-Sprache, wohlgeformter Text.)
- Definitionen dienen zur Assoziation einer eindeutigen Bedeutung/Interpretation mit einem Wort/Begriff – zumindest in einem gewissen Kontext (Forschungsgebiet, Buch, Artikel, Debatte). (Siehe definire: „ab-/ein-/be-grenzen, erklären“.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Aussagen

- Mathematik basiert auf den Grundlagen der Logik.
- Zentrale Frage der Logik ist die Erfüllbarkeit und (Allgemein-)Gültigkeit einer Aussage (Behauptung, Feststellung, Zusicherung).
- Eine Aussage ist der Inhalt eines Deklarativsatzes:

  Man kann entweder zustimmen (wahr) oder ablehnen (falsch) (Wahrheitswert).

  Eine Aussage kann verschiedene **syn**onyme Formulierungen (konkreter Deklarativsatz/Formel/Code) besitzen.

  Der Wahrheitswert (wahr, falsch) einer Aussage hängt von der Situation (Kontext, Interpretation, „Welt“, Fall, „Perspektive“, Speicherinhalt, Buch, Autor, ...) ab.
- Beispiel: „0 ist eine natürliche Zahl“ vs „0 ist eine nichtnegative ganze Zahl“
- Eine Aussage ist erfüllbar, falls es eine Situation gibt, in der sie wahr ist.

  Eine Aussage ist (allgemein-)gültig, falls es keine Situation gibt, in der sie falsch ist („immer wahr“). (Eine gültige Aussage ist immer erfüllbar.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Bandersnatches, Junktoren, Quantoren

- Um zu entscheiden, ob eine Aussage erfüllbar oder sogar gültig ist, muss man sich auf die Bedeutung einiger Grundbegriffe einigen.
- Beispiel: „If all bandersnatches are borogoves, and all borogoves are slithy, then all bandersnatches are slithy“ (Prof. Esparza)
- Die Wörter „bandersnatches“, „borogoves“, „slithy“ stammen aus dem Gedicht Jabberwocky von Lewis Carroll.

  Die Wörter hatten (ursprünglich) keine Bedeutung (kein Eintrag in einem Wörterbuch).
- Bzgl. der üblichen Definitionen, wie man Junktoren und Quantoren in der Logik und Mathematik liest, ist obiger Satz dennoch gültig.

  (D.h. die konkrete Interpretation von „bandersnatch“ spielt keine Rolle.)

Def Junktoren: und, oder, nicht, falls-dann (wenn-dann), genau-dann-wenn (falls-und-nur-falls), entweder-oder

Def Quantoren: für alle, es existiert (es gibt, für einige), für kein (es gibt kein)

(Verschiedene Schriftarten/Fettschrift/Farben dienen häufig zur besseren Unterscheidung von z.B. „und“ in Aufzählungen und „und“ als logischer Junktor.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Junktoren

Situation A	Situation B	nicht A	A und B	A oder B	Falls A, dann B	A gdw B	Entweder A oder B
falsch	falsch	wahr	falsch	falsch	wahr	wahr	falsch
falsch	wahr	wahr	falsch	wahr	wahr	falsch	wahr
wahr	falsch	falsch	falsch	wahr	falsch	falsch	wahr
wahr	wahr	falsch	wahr	wahr	wahr	wahr	falsch

- „Klassifikation“ aller Situationen in vier Klassen bzgl. A, B.
- Fürs Erste: „**syn**“ kurz für „**syn**onym (gleichbedeutend)“ z.B. (überprüfen!):
  - „falls A, dann B“ **syn** „(nicht A) oder B“ **syn** „B oder nicht A“
    (Siehe Konditional: nur unter Bedingung, dass A wahr, ist B relevant.)
  - „A gdw B“ **syn** „(falls A, dann B) und (falls B, dann A)“
    („gdw“: „genau dann, wenn“; auch: „falls, und nur falls,“ vgl. iff)
  - „entweder A oder B“ **syn** „nicht (A gdw B)“
  - „weder A noch B“ **syn** „nicht (A oder B)“ **syn** „(nicht A) und (nicht B)“

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Junktoren

Situation A	Situation B	nicht A	A und B	A oder B	Falls A, dann B	A gdw B	Entweder A oder B
falsch	falsch	wahr	falsch	falsch	wahr	wahr	falsch
falsch	wahr	wahr	falsch	wahr	wahr	falsch	wahr
wahr	falsch	falsch	falsch	wahr	falsch	falsch	wahr
wahr	wahr	falsch	wahr	wahr	wahr	wahr	falsch

- Im strengen Sinn werden obige „Aussagen mit Variablen“ als Aussageformen bezeichnet:
  - Erst durch Ersetzen von A, B durch Aussagen (z.B. den trivialen Aussagen) kann man zustimmen oder ablehnen.
  - Man kann A, B auch durch andere Aussageformen oder mathematische Ausdrücke ersetzen, um komplexe Aussagen(-formen) zu bilden (z.B. ersetze A durch „$x = 2$“, B durch „$x^2 = 4$“).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

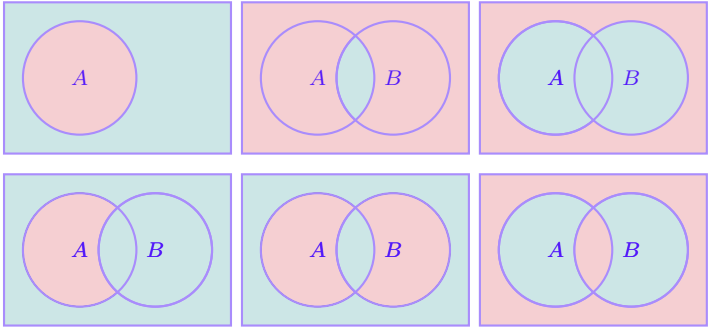
Situation A	Situation B	nicht A	A und B	A oder B	Falls A, dann B	A gdw B	Entweder A oder B
falsch	falsch	wahr	falsch	falsch	wahr	wahr	falsch
falsch	wahr	wahr	falsch	wahr	wahr	falsch	wahr
wahr	falsch	falsch	falsch	wahr	falsch	falsch	wahr
wahr	wahr	falsch	wahr	wahr	wahr	wahr	falsch

- Beispiele: Für welche Werte von $x \in \mathbb{R}$ (Situation) gilt
  - „$x = 2$ und $x^2 = 4$.“
  - „$x = 2$ oder nicht $x = 2$.“
  - „$x = 2$ und nicht $x = 2$.“
  - „Falls $x = 2$, dann $x^2 = 4$.“
  - „Falls $x^2 = 4$, dann $x = 2$.“
  - „$x^2 = 4$ genau dann, wenn $x = 2$.“
- Aussageformen können auch mittels Venn-Diagrammen veranschaulicht werden.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Junktoren: Wahrheitstabellen und Venn-Diagramme

Situation		nicht	A	A	Falls A,	A	Entweder A
		A	und	oder	dann B	gdw	oder B
A	B		B	B		B	
falsch	falsch	wahr	falsch	falsch	wahr	wahr	falsch
falsch	wahr	wahr	falsch	wahr	wahr	falsch	wahr
wahr	falsch	falsch	falsch	wahr	falsch	falsch	wahr
wahr	wahr	falsch	wahr	wahr	wahr	wahr	falsch

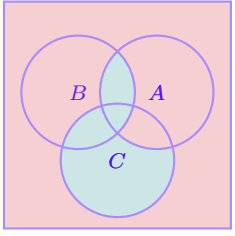


No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

If-then-else vs. Konditional vs. Bikonditional

- Entsprechend kann man Wahrheitstabellen und Venn-Diagramme auch für drei und mehr Aussagen(variablen) erstellen, z.B. „if-then-else (ITE)“.

Situation			Falls A dann B sonst C	Falls A dann B sonst wahr	Falls A dann B sonst nicht B	Falls A dann B und falls nicht A dann C
A	B	C				
falsch	falsch	falsch	falsch	wahr	wahr	falsch
falsch	falsch	wahr	wahr	wahr	wahr	wahr
falsch	wahr	falsch	falsch	wahr	falsch	falsch
falsch	wahr	wahr	wahr	wahr	falsch	wahr
wahr	falsch	falsch	falsch	falsch	falsch	falsch
wahr	falsch	wahr	falsch	falsch	falsch	falsch
wahr	wahr	falsch	wahr	wahr	wahr	wahr
wahr	wahr	wahr	wahr	wahr	wahr	wahr



If-then-else vs. Konditional vs. Bikonditional

- Entsprechend kann man Wahrheitstabellen und Venn-Diagramme auch für drei und mehr Aussagen(variablen) erstellen, z.B. „if-then-else (ITE)“.

Situation			Falls A dann B sonst C	Falls A dann B sonst wahr	Falls A dann B sonst nicht B	Falls A dann B und falls nicht A dann C
A	B	C				
falsch	falsch	falsch	falsch	wahr	wahr	falsch
falsch	falsch	wahr	wahr	wahr	wahr	wahr
falsch	wahr	falsch	falsch	wahr	falsch	falsch
falsch	wahr	wahr	wahr	wahr	falsch	wahr
wahr	falsch	falsch	falsch	falsch	falsch	falsch
wahr	falsch	wahr	falsch	falsch	falsch	falsch
wahr	wahr	falsch	wahr	wahr	wahr	wahr
wahr	wahr	wahr	wahr	wahr	wahr	wahr

- „falls-dann (wenn-dann)“ wird als Konditional bezeichnet: Der Wahrheitswert von B ist nur relevant unter der Bedingung (Annahme; im Fall), dass A wahr ist.
- „genau-dann-wenn (falls-und-nur-falls)“ wird als Bikonditional bezeichnet.

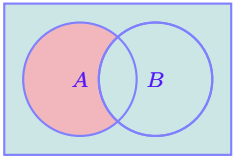
⚠ Kausalität oder zeitliche Abhängigkeiten spielen in der Aussagenlogik keine Rolle: „Falls Schweine fliegen können, ist π irrational.“

⚠ Programmiersprachen kennen meistens nur „if A, then do b, else do c“ für eine Aussage (Bedingung) A und Anweisungen b, c.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Hinreichend vs. notwendig, „impliziert“

A	B	falls A, dann B
falsch	falsch	wahr
falsch	wahr	wahr
wahr	falsch	falsch
wahr	wahr	wahr



- Im Allgemeinen ist „falls A, dann B“ nicht gültig.

  z.B. „Falls $x \in \mathbb{Z}$, dann $x/2 \in \mathbb{Z}$“ ist wahr für $x := 2$, aber falsch für $x := 1$.

  D.h. das Wissen, dass $x \in \mathbb{Z}$ wahr ist, ist nicht hinreichend (ausreichend), um $x/2 \in \mathbb{Z}$ zu folgern; bzw. $x/2 \in \mathbb{Z}$ ist nicht notwendig für $x \in \mathbb{Z}$.

- Für spezielle A, B kann „falls A, dann B“ gültig sein.

  z.B. „Falls $x/2 \in \mathbb{Z}$, dann $x \in \mathbb{Z}$“ ist gültig.

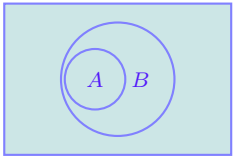
  D.h. das Wissen, dass $x/2 \in \mathbb{Z}$ wahr ist, ist hinreichend (ausreichend), um $x \in \mathbb{Z}$ zu folgern; bzw. $x \in \mathbb{Z}$ ist notwendig für $x/2 \in \mathbb{Z}$.

(Beachte, dass „$\mathbb{Z}$“ die Definition der ganzen Zahlen $\mathbb{Z}$ beinhaltet.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Hinreichend vs. notwendig, „impliziert“

A	B	falls A, dann B
falsch	falsch	wahr
falsch	wahr	wahr
~~wahr~~	~~falsch~~	~~falsch~~
wahr	wahr	wahr



- Ist „falls A, dann B“ (allgemein)gültig, dann sagt man auch
  - „A ist hinreichend für B“ bzw. „B ist notwendig für A“
  - häufig auch: „A impliziert B“, „B folgt (logisch) aus A“

  Ist „A gdw B“ allgemeingültig, dann ist A hinreichend und notwendig für B.

  (A gdw B **syn** falls A, dann B; und falls B, dann A.)
- Mathematische Sätze/Theoreme/Lemmata sind gültige Aussagen, welche häufig von der Form „Wenn (falls) A, dann B“ bzw. „A genau dann, wenn B“ sind.
  (Die Gültigkeit der Aussage muss bewiesen/argumentiert werden; siehe später.)
  Gültige „gdw“ werden manchmal hervorgehoben, z.B. „**gdw**“.

Bsp (A gdw B) **gdw** (B gdw A).

- „**gdw**“ ist ein Spezialfall von „**syn**“ für Aussageformen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Beschränkte Quantoren

- "If all bandersnatches are borogoves, and all borogoves are slithy, then all bandersnatches are slithy."
- Wie liest man „**Alle** (**einige**, **keine**) vom Typ 1 sind auch vom Typ 2"?
  („Typ": „Art", „Sorte", „mit der Eigenschaft")
- Formal beschränkte Quantoren:
  - „Alle x mit $R(x)$ sind/haben/erfüllen auch $P(x)$."
    „Für alle x, für die $R(x)$ wahr ist, ist auch $P(x)$ wahr."
    „Für jedes Objekt x gilt, falls $R(x)$ wahr ist, dann ist auch $P(x)$ wahr."
  - „Einige x mit $R(x)$ sind/haben/erfüllen auch $P(x)$."
    „Es existiert ein x, für welches $R(x)$ und zusätzlich auch $P(x)$ wahr ist."
    „Es gibt mind. ein Objekt x, das neben $R(x)$ auch $P(x)$ erfüllt."

  Dabei bezeichnen R, P Prädikate (parametrisierte Aussagen, die eine Eigenschaft „zusprechen"), z.B.:

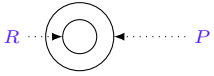
  - „x is a bandersnatch", „x is a borogove", „x is slithy"
    „$x = 2$", „$x^2 = 2$", „$x > y$", „$x \neq y$", „$x \in \mathbb{Q}$", „$x \in \mathbb{R}$"

  (Formal sind z.B. „is a bandersnatch" und „$>$ (größer)" die eigentlichen Prädikate.)

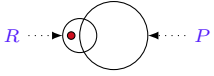
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Beschränkte Quantoren und Venn-Diagramme

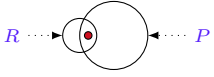
- „Alle x mit $R(x)$ sind $P(x)$“ **gdw** „Für alle x, falls $R(x)$, dann (auch) $P(x)$“



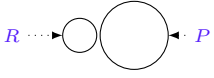
- „Nicht alle x mit $R(x)$ sind $P(x)$“ **gdw** „Es gibt ein x mit $R(x)$ und nicht $P(x)$“



- „Einige x mit $R(x)$ sind $P(x)$“ **gdw** „Es gibt x, so dass $R(x)$ und (auch) $P(x)$“



- „Kein x mit $R(x)$ ist $P(x)$“ **gdw** „Für alle x: Falls $R(x)$, dann nicht $P(x)$“



- Vergleiche: $R(x) =$„$x \in \mathbb{Q}$“ vs. $R(x) =$„$x \in \mathbb{R}$“ für $P(x) =$„$x^2 \neq 2$“

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Beschränkte Quantoren

- „Einige x mit $R(x)$ sind $P(x)$“ ist nur wahr, **falls** es ein konkretes Objekt d gibt, für welches $R(d)$ und $P(d)$ wahr sind (d ist dann ein Beispiel oder Zeuge).
  - Etwas formaler: „Es existiert x, so dass $R(x)$ und $P(x)$“
- „Alle x mit $R(x)$ sind $P(x)$“ ist nur falsch, **falls** es ein konkretes Objekt d gibt, für das $R(d)$ und nicht $P(d)$ wahr ist (d ist dann ein Gegenbeispiel).
  - Etwas formaler: „Für alle x, falls $R(x)$, dann $P(x)$“

Bsp Beispiele:

- „Es gibt eine rationale Zahl x mit $x^2 = 2$.“
  vs „Es gibt eine reelle Zahl x mit $x^2 = 2$.“
- „Für alle rationalen Zahlen x gilt $x^2 \neq 2$.“
  vs „Für alle reellen Zahlen x gilt $x^2 \neq 2$.“
- „Für alle reellen Zahlen x gilt, falls $x = \sqrt{2}$, dann $x^2 = 2$.“
  vs „Für alle reellen Zahlen x gilt, falls $x^2 = 2$, dann $x = \sqrt{2}$.“
  vs „Für alle reellen Zahlen x gilt: falls $x^2 = 2$, dann $x = -\sqrt{2}$ oder $x = \sqrt{2}$.“

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Beschränkte Quantoren

- „Einige x mit $R(x)$ sind $P(x)$“ ist nur wahr, **falls** es ein konkretes Objekt d gibt, für welches $R(d)$ und $P(d)$ wahr sind (d ist dann ein Beispiel oder Zeuge).
  - Etwas formaler: „Es existiert x, so dass $R(x)$ und $P(x)$“
- „Alle x mit $R(x)$ sind $P(x)$“ ist nur falsch, **falls** es ein konkretes Objekt d gibt, für das $R(d)$ und nicht $P(d)$ wahr ist (d ist dann ein Gegenbeispiel).
  - Etwas formaler: „Für alle x, falls $R(x)$, dann $P(x)$“
- Durch Negation verbunden: Existenz vs. Absenz eines (Gegen-)Beispiels
- „Nicht alle x mit $R(x)$ sind $P(x)$“ **gdw** „Einige x mit $R(x)$ sind nicht $P(x)$“
- „Kein (nicht einige) x mit $R(x)$ ist $P(x)$“ **gdw** „Alle x mit $R(x)$ sind nicht $P(x)$“

Bsp Beispiele:

- „Nicht für jede reelle Zahl x gilt $x^2 = -1$“ **gdw**
  „es gibt eine reelle Zahl x mit $x^2 \neq -1$.“
- „Es gibt keine reelle Zahl x mit $x^2 = -1$“ **gdw**
  „für jede reelle Zahl x gilt $x^2 \neq -1$.“

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Beweise

- Behauptung: „If all bandersnatches are borogoves, and all borogoves are slithy, then all bandersnatches are slithy.“ ist gültig.
- Die zu beweisende Aussage hat für geeignete Prädikate P, Q, R die Form:

If		(A)	for all x: if $P(x)$, then $Q(x)$,
	and	(B)	for all y: if $Q(y)$, then $R(y)$,
then		(C)	for all z: if $P(z)$, then $R(z)$.

- Beweis:
  - Betrachte eine Situation, in der (A) und (B) wahr sind (sonst fertig).
  - Wir müssen zeigen, dass dann stets auch (C) wahr ist, d.h. dass es kein Gegenbeispiel d gibt, für welches $P(d)$ wahr und $R(d)$ falsch ist.
  - Bezeichne d ein beliebiges Objekt, von dem wir nur wissen, dass $P(d)$ wahr ist. Dann gilt:
    - Da $P(d)$ und (A) wahr sind, muss auch $Q(d)$ wahr sein.
    - Da $Q(d)$ und (B) wahr sind, muss auch $R(d)$ wahr sein.

  Wann immer (A) und (B) wahr sind, hat also jedes Objekt mit Eigenschaft P auch die Eigenschaft R, d.h. (C) ist wahr unabhängig von der (subjektiven) Interpretation von P, Q, R (“bandersnatch”, “borogoves”, “slithy”).

No commercial use, ask for use in Discrete Structures (IN0015,INHN0004), JSW, TUM CIT Department of Computer Science

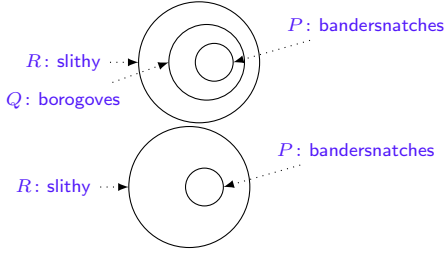
Beweise

If		Ⓐ	for all x: if $P(x)$, then $Q(x)$,
	and	Ⓑ	for all y: if $Q(y)$, then $R(y)$,
then		Ⓒ	for all z: if $P(z)$, then $R(z)$.

▷ Falls Ⓐ bzw. Ⓑ wahr:

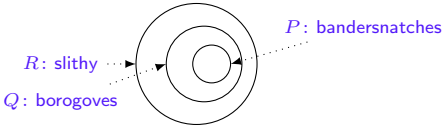


▷ Wann immer Ⓐ und Ⓑ wahr sind, ist insbesondere Ⓒ wahr:



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Argumente in Form von logischen Schlüssen (Inferenzen)



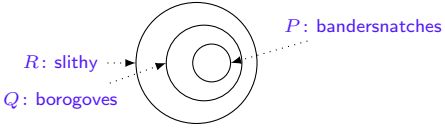
- Ein (logischer) Schluss ist ein Argument der Form

$$\frac{\begin{array}{lll}\text{(Falls)} & \text{A1} & \ldots, \\ \text{(und)} & \text{A2} & \ldots, \end{array}}{\begin{array}{lll}\text{(dann)} & \text{C} & \ldots. \end{array}}$$

Man bezeichnet

- A1 , A2 als Prämissen (Annahme, Antezedens).
  Im Allgemeinen eine beliebige (endliche) Anzahl „A1 und A2 und . . . und An “
- C als die eigentliche Konklusion (Konsequenz, Folgerung).

Argumente in Form von logischen Schlüssen (Inferenzen)



- Häufig kurz in der Form („A1 ; A2 ; daher C “)

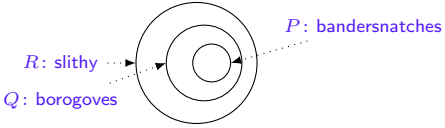


Der „Strich“ entspricht einem „daher“, „damit“, „somit“, „darauf folgt“, „deshalb“, „folglich“, „also“.

- Die auf Aristoteles zurückgehenden Syllogismen hatten die spezielle Form mit genau zwei Prämissen, wobei Prämissen und Konklusion von der Form „Alle/Einige/Keine x mit $P(x)$ sind $Q(x)$“ sind und die Prämissen ein gemeinsames Prädikat verwenden (z.B. „borogoves“).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Argumente in Form von logischen Schlüssen (Inferenzen)



- Prämissen sind üblicherweise bereits bewiesene Aussagen („Sätze, Theoreme“), Definitionen, Axiome, Postulate, Regeln, (unbewiesene) Annahmen, Hypothesen, Vermutungen, Gesetze, ...
- Ist das Argument (Schluss, Inferenz) gültig, dann sagt man, dass die Prämissen zusammen („A1 und A2 “) für die Konklusion hinreichend sind, während die Konklusion notwendig ist, damit alle Prämissen gleichzeitig wahr sein können.
- Die Konklusion kann dann als zusätzliche Prämisse für weitere Schlussfolgerungen verwendet werden.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

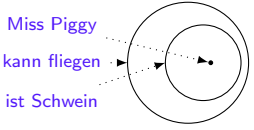
Syllogismen: Beispiele

Bsp „Wenn alle Schweine fliegen können und Miss Piggy ein Schwein ist, dann kann Miss Piggy fliegen“

(A) Alle Schweine können fliegen
(B) Miss Piggy ist ein Schwein.
(Es gibt ein Schwein namens „Miss Piggy“.)

(C) Miss Piggy kann fliegen.

▷ Gültiges Argument:



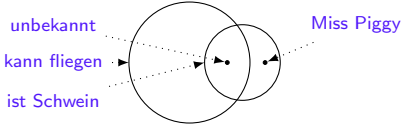
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Syllogismen: Beispiele

Bsp „Wenn einige Schweine fliegen können und Miss Piggy ein Schwein ist, dann kann Miss Piggy fliegen“

A	Einige Schweine können fliegen
B	Miss Piggy ist ein Schwein.
C	Miss Piggy kann fliegen.

- Ungültiges Argument:



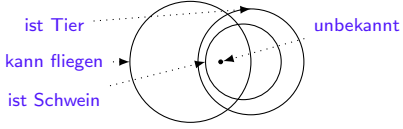
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Syllogismen: Beispiele

Bsp „Wenn einige Schweine fliegen können und alle Schweine Tiere sind, dann können einige Tiere fliegen“

A	Einige Schweine können fliegen.
B	Alle Schweine sind Tiere.
C	Einige Tiere können fliegen.

▷ Gültiges Argument:



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Syllogismen: Beispiele

▷ Welche der folgenden Argumente sind gültig?

(A) Alle, die sich benehmen, kriegen ein Geschenk.
(B) Alle Kinder benehmen sich.

(C) Alle Kinder kriegen ein Geschenk.

(A) Alle, die sich benehmen, kriegen ein Geschenk.
(B) Alle Kinder kriegen ein Geschenk.

(C) Alle Kinder benehmen sich.

(A) Alle, die sich benehmen, kriegen ein Geschenk.
(B) Einige Kinder kriegen kein Geschenk.

(C) Einige Kinder benehmen sich nicht.

(A) Alle, die ein Geschenk kriegen, benehmen sich.
(B) Alle Kinder kriegen ein Geschenk.

(C) Alle Kinder benehmen sich.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Typische Schlussfolgerungen: Lineares Gleichungssystem

▷ Betrachte das Gleichungssystem

$$3x + 4y = 0 \qquad -2x + 3y = 17$$

Wir nehmen an, dass es konkrete Zahlen $x, y \in \mathbb{R}$ gibt, für welche beide Gleichungen (und) gelten.

Dann folgt aufgrund der Definition von $\mathbb{R}$:

A $3x + 4y = 0$
B $-2x + 3y = 17$

(A nach y auflösen)

C $y = -0.75x$

(in B y durch die rechte Seite von C ersetzen und vereinfachen)

D $-2x - 2.25x = 17$

(D nach x auflösen)

E $x = -4$

(in C x durch die rechte Seite von E ersetzen und vereinfachen)

F $y = 3$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Einige typische (quantorenfreie) Schlussfolgerungen

falls A, dann B
A
...

B

falls A, dann B
nicht B
...
---
nicht A

A oder B
nicht B
...

A

falls A, dann B
falls B, dann C
...
---
falls A, dann C

falls A, dann C
falls B, dann C
A oder B
...

C

falls nicht B, dann nicht A
...
---
falls A, dann B

falls nicht A, dann B
falls nicht A, dann nicht B
...

A

- „Wenn man allen Aussagen über dem Strich zustimmt, dann muss man auch der Aussage darunter zustimmen.“
- Zusätzliche Prämissen schränken nur die Anwendbarkeit der Schlussfolgerungen weiter ein.
- Die Reihenfolge der Prämissen spielt keine Rolle.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Übliche Beweisstrategien

- direkter Beweis
  (z.B. für „Falls A, dann B“: Nimm A an und zeige, dass B wahr ist mit Hilfe von logischen Schlüssen)
- Kontraposition
  - „Falls A, dann B“ **gdw** „Falls nicht B, dann nicht A“.
  - „Alle x mit $P(x)$ sind $Q(x)$“ **gdw** „Alle x mit nicht $Q(x)$ sind nicht $P(x)$“
- Widerspruch
  - Man zeigt A, indem man „falls nicht A, dann falsch“ zeigt.
- Beweis für „A genau dann, wenn B“: zeige, dass „falls A, dann B“ und „falls B, dann A“. Oder benutze eine Aneinanderreihung von „genau dann, wenn“.
- Induktion (mehr dazu später)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Direkter Beweis

Bsp Falls $0 \leq x \leq 2$, dann $-x^3 + 4x + 1 > 0$.

- Wir nehmen an, dass $0 \leq x \leq 2$.
- Dann sind $x, (2 - x), (2 + x)$ alle nichtnegativ.
- Dann gilt $x(2 - x)(2 + x) \geq 0$.
- Wenn man zu einer nichtnegativen Zahl 1 addiert, ist die Summe positiv. Deswegen $x(2 - x)(2 + x) + 1 > 0$.
- Ausmultiplizieren zeigt $x(2 - x)(2 + x) + 1 = -x^3 + 4x + 1 > 0$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Kontraposition

Bsp Wenn $3n + 2$ ungerade ist, dann ist n ungerade.

- Wir benutzen folgende Tatsache: Für jede gerade Zahl m gibt es eine ganze Zahl k, sodass $m = 2k$.

  Beweis:

- Wir nehmen an, dass n gerade ist. („nicht B“)
- Das heißt, es gibt eine ganze Zahl k, sodass $n = 2k$.
- Dann gilt (einsetzen) $3n + 2 = 6k + 2 = 2(3k + 1)$.
- Das heißt, $3n + 2$ ist eine gerade Zahl. („nicht A“)

- Übung: Zeige, dass wenn $n = ab$ und a, b positive ganze Zahlen sind, dann gilt $a \leq \sqrt{n}$ oder $b \leq \sqrt{n}$.

Bsp Unter 22 beliebigen Tagen fallen mindestens 4 auf denselben Wochentag.

- Wir nehmen an, dass die Aussage nicht wahr ist, d.h. es gibt eine Menge von 22 Tagen, von denen keine 4 auf den gleichen Wochentag fallen. („nicht A“)
- Das heißt, jeder Wochentag kommt höchstens 3 mal vor.
- Da es 7 Wochentage gibt, kann die Menge daher höchstens aus $7 \cdot 3 = 21$ Tagen bestehen.
- Dies widerspricht der Annahme, dass wir 22 Tage gewählt haben. („falsch“)

Wir benutzen hier das „Schubfachprinzip“ – mehr dazu später.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Bsp $\sqrt{2}$ ist nicht rational.

- Nach Definition bezeichnet $\sqrt{2}$ die (einzige) positive reelle Zahl x mit $x^2 = 2$.
- Behauptung: Für jede rationale Zahl x gilt $x^2 \neq 2$.
- Beweis durch Widerspruch:
  - Nimm an, dass x_0 eine rationale Zahl ist mit $x_0^2 = 2$.
  - Dann gibt es ganze Zahlen m, n mit $n \neq 0$ und $x_0 = \frac{m}{n}$.
  - OBdA ist der Bruch gekürzt, d.h. $\mathsf{ggT}(m, n) = 1$ (sonst kürze durch $\mathsf{ggT}(m, n)$).
  - Einsetzen in die ursprüngliche Annahme ($x_0^2 = 2$) ergibt $m^2 = 2n^2$.
  - Daher ist 2 ein Teiler von m^2.
  - Daher ist 2 ein Teiler von m (Lemma von Euklid).
  - Daher gilt $m = 2k$ und damit auch $2k^2 = n^2$.
  - Daher ist 2 ein Teiler von n^2 und somit auch von n.
  - Folglich ist 1 nicht der einzige positive gemeinsame Teiler von m, n.
  - Das ist ein Widerspruch.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Bsp Falls n^2 positiv ist, dann ist n positiv.

- ▷ Wir nehmen an, dass n^2 positiv ist.
- ▷ Da die Aussage „Falls n positiv ist, dann ist n^2 positiv“ gilt, können wir schließen, dass n positiv ist.

Bsp Falls n^2 gerade ist, dann ist n gerade.

- ▷ Wir nehmen an, dass n^2 gerade ist.
- ▷ Das heißt, $n^2 = 2k$ für ein $k \in \mathbb{Z}$.
- ▷ Sei $n = 2t$ für ein $t \in \mathbb{Z}$.
- ▷ Daher ist n gerade.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Mehr Beweise, aber falsche!!

Zeige „falls A, dann B“.

Bsp Falls n^2 positiv ist, dann ist n positiv. (Aussage falsch, Beweis falsch)

- ▷ Wir nehmen an, dass n^2 positiv ist. Wir nehmen A an.
- ▷ Da die Aussage „Falls n positiv ist, dann ist n^2 positiv“ gilt, können wir schließen, dass n positiv ist.

Aus „Falls B, dann A“ folgt nicht „falls A, dann B“!

Bsp Wenn n^2 gerade ist, dann ist n gerade. (Aussage richtig, Beweis falsch)

- ▷ Wir nehmen an, dass n^2 gerade ist. Wir nehmen A an.
- ▷ Das heißt, $n^2 = 2k$ für ein $k \in \mathbb{Z}$.
- ▷ Sei $n = 2t$ für ein $t \in \mathbb{Z}$. Wir nehmen B an, anstatt es zu beweisen!
- ▷ Daher ist n gerade.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Common proof techniques

Proof by intimidation Trivial!

Proof by cumbersome notation The theorem follows immediately from the fact that $\left|\bigoplus_{k\in S}\left(\mathfrak{R}^{\mathbb{F}^\alpha(i)}\right)_{i\in\mathcal{U}_k}\right| \preccurlyeq \aleph_1$ when $[\mathfrak{H}]_{\mathcal{W}} \cap \mathbb{F}^\alpha(\mathbb{N}) \neq \emptyset$.

Proof by inaccessible literature The theorem is an easy corollary of a result proven in a hand-written note handed out during a lecture by the Yugoslavian Mathematical Society in 1973.

Proof by ghost reference The proof my be found on page 478 in a textbook which turns out to have 396 pages.

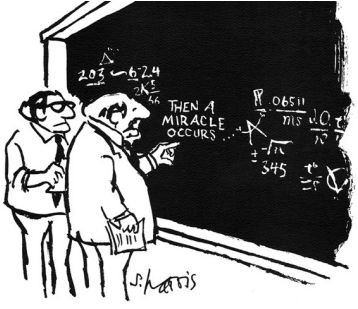
Circular argument Proposition 5.18 in [BL] is an easy corollary of Theorem 7.18 in [C], which is again based on Corollary 2.14 in [K]. This, on the other hand, is derived with reference to Proposition 5.18 in [BL].

Proof by authority My good colleague Andrew said he thought he might have come up with a proof of this a few years ago...

Internet reference For those interested, the result is shown on the web page of this book. Which unfortunately doesn't exist any more.

Proof by avoidance *Chapter 3:* The proof of this is delayed until Chapter 7 when we have developed the theory even further. *Chapter 7:* To make things easy, we only prove it for the case $z = 0$, but the general case in handled in Appendix C. *Appendix C:* The formal proof is beyond the scope of this book, but of course, our intuition knows this to be true.

facebook.com/Mathematicx



"I THINK YOU SHOULD BE MORE EXPLICIT HERE IN STEP TWO."

Proof by resource limits: Due to lack of space, we omit this part of the proof...

Theorem, Lemma, Korollar, Definition, ...

- Mit „Proposition“, „Lemma (Pl.: Lemmata)“, „Theorem“, „Satz“, „Korollar“ und manchmal „Fakt“ weist man auf bewiesene Aussagen hin, die eine besondere Relevanz entweder für folgende Beweise oder im Allgemeinen haben.
- Die genaue Verwendung hängt vom Autor ab, im Weiteren grob folgende Unterscheidung:

  Thm kurz für Theorem (Satz): wichtiges, häufig verwendetes und/oder nicht offensichtliches Resultat

  Lem kurz für Lemma: weniger wichtig oder Hilfsresultat für ein Theorem

  Kor kurz für Korollar: einfach zu beweisende Konsequenz eines Theorems/Lemmas

  Fkt kurz für Fakt: offensichtliches Ergebnis (proof by intimidation ...)

  und

  Def kurz für Definition: eindeutige Begriffsabgrenzung/erklärung (ggf. Eindeutigkeit/Wohldefiniertheit zu beweisen).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Notation für Junktoren und Quantoren

	formale Logik (DS)		Python (C++)	C, C++, Java
wahr	(triviale Tautologie)	wahr, $\top$	`True`	`true`
falsch	(triviale Kontradiktion)	falsch, $\bot$	`False`	`false`
nicht	Negation	$\neg A$	`not A`	`!A`
oder	Disjunktion	$(A \vee B)$	`A or B`	`A||B`
und	Konjunktion	$(A \wedge B)$	`A and B`	`A&&B`
falls/wenn-dann	Konditional, Subjunktion	$(A \to B)$	`not A or B`	`!A||B`
genau-dann-wenn	Bikonditional	$(A \leftrightarrow B)$	`A==B`	`A==B`
entweder-oder	exklusives Oder, (e)XOR	$(A \oplus B)$	`A!=B`	`A!=B`
alle	Allquantor	$\forall x F$	`all`	
einige	Existenzquantor	$\exists x F$	`any`	

- Die korrekte Verwendung von $\wedge, \vee, \neg, \to, \exists, \forall$ benötigt die Definition einer formalen Sprache einschließlich deren Semantik.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

2 Grundlagen

Grundbegriffe der Logik

Mengen

Grundlegende Konzepte und Notation

Russellsche Antinomie

Teilmengen, Gleichheit, Mengenoperationen

Mengen von Mengen (verschachtelte Mengen)

Visualisierung von Mengen: Venn- und KV-Diagramme

Mengenterme

Folgen, Tupel und Wörter

Die natürlichen Zahlen, Induktionsbeweise

Relationen

Funktionen

Kardinalität: Vergleiche von unendlichen Mengen

Zusammenfassung

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science



[Quelle]

§ 1.

Der Mächtigkeitsbegriff oder die Cardinalzahl.

Unter einer ‚Menge' verstehen wir jede Zusammenfassung M von bestimmten wohlunterschiedenen Objecten m unsrer Anschauung oder unseres Denkens (welche die ‚Elemente' von M genannt werden) zu einem Ganzen.

[Quelle]

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Grundkonzept der Menge

- Der Begriff „Menge“ steht für das Konzept, dass man wohlunterschiedene Objekte z.B. auf Grund einer gemeinsamen Eigenschaft zusammenfasst.
  - Autos wohlunterscheidbar anhand der Fahrzeug-Identifizierungsnummer.
  - Zusammenfassung von Autos desselben Herstellers.
- Die Reihenfolge der Objekte, mehrfache Nennung (Mehrfachheiten) von Objekten, etc. spielen keine Rolle.
  - So kennt eine Menge kein erstes, zweites, usw. Objekt; ebenso ist z.B. „zweimal enthalten“ für eine Menge nicht definiert (nur „nicht enthalten“ oder „enthalten“).
- Daneben sind noch die Basisoperationen wichtig, um aus gegebenen Mengen neue Mengen zu bilden: Vereinigung, Schnitt, Differenz, ...

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Basisdefinitionen

Def Gegeben ein Objekt x und eine Menge M kann nur einer von zwei komplementären Fällen eintreten:

- ▷ das Objekt x ist in der Menge M enthalten, kurz $x \in M$.
- ▷ das Objekt x ist nicht in der Menge M enthalten, kurz $x \notin M$.
- ▷ Man liest $x \in M$ ($x \notin M$) auch als „x ist (kein) Element von M“

Def Die leere Menge wird mit $\{\}$ oder $\emptyset$ bezeichnet:

Man definiert, dass für jedes Objekt x stets $x \notin \emptyset$ gilt.

(Mit anderen Worten: Es gibt kein Objekt x mit $x \in \emptyset$.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Unendliche Zahlenmengen, die aus der Schule bekannt sind

Fkt Folgende Mengen von Zahlen (in Dezimaldarstellung) sind wohldefiniert:

- $\mathbb{N}_0 = \{0, 1, 2, \ldots\}$ Menge der natürlichen Zahlen inkl. 0.
  $\mathbb{N} = \{1, 2, 3, \ldots\}$ Menge der positiven natürlichen/ganzen Zahlen.
- $\mathbb{P} = \{2, 3, 5, 7, 11, \ldots\}$ Menge der Primzahlen.
- $\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$ Menge der ganzen Zahlen.
- $\mathbb{Q} = \{p/q \mid p \in \mathbb{Z}, q \in \mathbb{N}\}$ Menge der rationalen Zahlen.
- $\mathbb{R}$ Menge der reellen Zahlen, dargestellt als unendliche Dezimalbrüche $\pm n{,}d_1 d_2 d_3 \ldots$ mit $n \in \mathbb{N}_0$ und $d_i \in \{0, 1, \ldots, 9\}$ für alle $i \in \mathbb{N}$ (dabei z.B. $0{,}\overline{9} = 1{,}\overline{0}$).

(Formal fordert das Unendlichkeitsaxiom die Existenz von $\mathbb{N}_0$; aus $\mathbb{N}_0$ lassen sich dann $\mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}$ konstruieren.)

- Es gibt verschiedene Möglichkeiten, weitere Mengen zu definieren:
  - Mittels expliziter (endlicher) Auflistung.
  - Mittels Auswahl (Aussonderungen).
  - Mittels mengentheoretischer Operationen (Vereinigung, Schnitt, ...; später).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Mengen: Definition durch Auflistung (Aufzählung)

Def $x \in \{x_1, \ldots, x_k\}$ **:gdw** x kommt in $x_1, \ldots, x_k$ vor.

($x \notin \{x_1, \ldots, x_k\}$ **gdw** x kommt nicht in $x_1, \ldots, x_k$ vor.)

- Die öffnende bzw. schließende Mengenklammer $\{$, $\}$ signalisieren dabei den Beginn bzw. das Ende der Mengendefinition, „,“ dient als Trennzeichen.
- Im strikten Sinn muss die Liste vollständig ausgeschrieben werden, häufig wird aber eine angedeutete Auflistung zzgl. erklärendem Text akzeptiert:
  - $\{\mathbb{N}, \mathbb{N}_0\}$ und $\{\mathbb{N}, \mathbb{N}_0, \mathbb{Z}\}$ sind Mengen.
  - {A,B,...,Z,a,b,...,z} Menge der Buchstaben des lateinischen Alphabets.
  - $\{!, ", \#, \ldots, \sim\}$ Menge der darstellbaren ASCII-Zeichen.

Def $[n] := \{1, 2, \ldots, n\}$ für jedes $n \in \mathbb{N}_0$ mit $[0] := \emptyset$.

$\mathbb{Z}_n := \{0, 1, \ldots, n-1\}$ für jedes $n \in \mathbb{N}_0$ mit $\mathbb{Z}_0 := \emptyset$.

⚠ „**:gdw**“ und „:=“ zur Betonung von Definitionen.

(Der Text auf der linken Seite steht für Text auf der rechten Seite.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Mengen: Definition durch Auswahl (Aussonderung) und Ersetzung

Def $x \in \{y \in A \mid P(y)\}$ **:gdw** $x \in A$ und $P(x)$ sind wahr.

($x \notin \{y \in A \mid P(y)\}$ **gdw** $x \notin A$ oder $P(x)$ falsch.)

Def $x \in \{f(y) \mid y \in A\}$ **:gdw** $x = f(y)$ für ein $y \in A$.

($x \notin \{f(y) \mid y \in A\}$ **gdw** $x \neq f(y)$ für alle $y \in A$.)

- Es ist wichtig, dass A bereits wohldefiniert ist und für jedes Element $x \in A$ eindeutig bestimmt ist, ob es die Eigenschaft (Prädikat) P besitzt oder nicht, bzw. dass die Konstruktion (Funktion) f eindeutig definiert ist (siehe Russell).
- Statt „$\mid$“ wird auch „$:$“ verwendet (z.B. $\{x \in \mathbb{R}\colon |x| < 1\}$).
- Lies „$\{x \in A \mid P(x), Q(x)\}$“ als „$\{x \in A \mid P(x)$ und $Q(x)\}$“.
- Statt „$\{x \in A \mid P(x)\}$“ auch „$\{x \mid x \in A, P(x)\}$“.
- $x \in \{x \in A \mid P(x)\}$ **gdw** $x \in \{y \in A \mid P(y)\}$ (gebundene Umbenennung).

Bsp $A := \{0, 1, 2, 3, 4\}$

$B := \{x \in A \mid x \leq 2\}$ oder auch $B := \{x \mid x \in A, x \leq 2\}$

$C := \{y \in A\colon y \text{ gerade}\}$ oder auch $C := \{y\colon y \in A, y \text{ gerade}\}$

$D := \{2x \mid x \in A\}$, $E := \{\{x\} \mid x \in A, x \leq 2\}$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

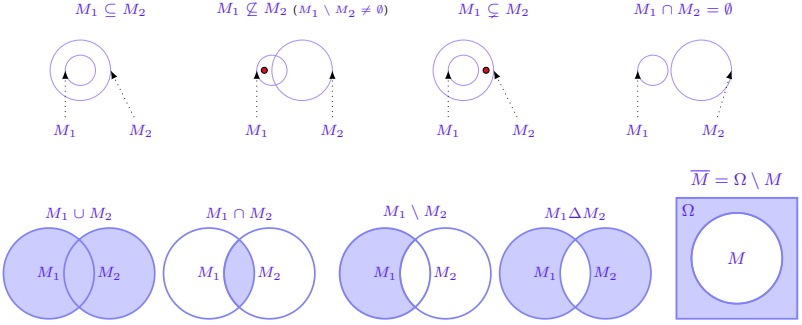
⚠ Da Mengen Objekte sind, können sie selbst Elemente von Mengen sein, z.B.

$$\{\, 1, \{1,2\}, \{1,\{\}\}, \{\{1\}, \{\{2\}\}, 3\} \,\}$$

- Betrachte die Menge $R := \{x \mid x \notin x\}$.
- Frage: Gilt $R \in R$?
- Problem: zyklische Abhängigkeit, Selbstreferenz.
- Analog: Der Dorfbarbier, der genau die Männer des Dorfes rasiert, die sich nicht selbst rasieren. (Einen solchen Barbier kann es nicht geben.)
- Naive Mengenlehre ist widersprüchlich.
  - Ausweg: Axiomatische Mengenlehre; z.B. Mengen dürfen nur aus einer bereits gegebenen Menge A ausgesondert werden, $\{x \in A \mid P(x)\}$ (Aussonderungsaxiom) und keine Menge M enthält sich selbst, $M \notin M$ (folgt aus dem Fundierungsaxiom).
  - Damit gibt es keine „Menge aller Mengen“, insbesondere ist $\{x \mid x \notin x\}$ keine wohldefinierte Menge.
  - Siehe Zermelo-Fraenkel-Mengenlehre und Typentheorie.

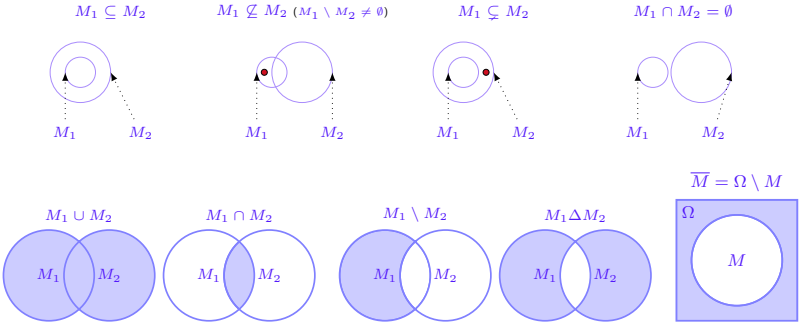
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Mengenoperationen und Beziehungen zwischen Mengen



- Venn-Diagramme wie zuvor: Mengen werden als Flächen dargestellt, typischerweise als Inneres geschlossener Kurven (Kreise, Rechtecke, Ellipsen, ...).
- Die Definitionen der grundlegenden Mengenoperationen und -beziehungen beruhen auf den Definitionen der Junktoren und Quantoren.
- Das umgebende Rechteck stellt das „Universum (Grundmenge)“ Ω dar.
  Wird nur für das Komplement benötigt.

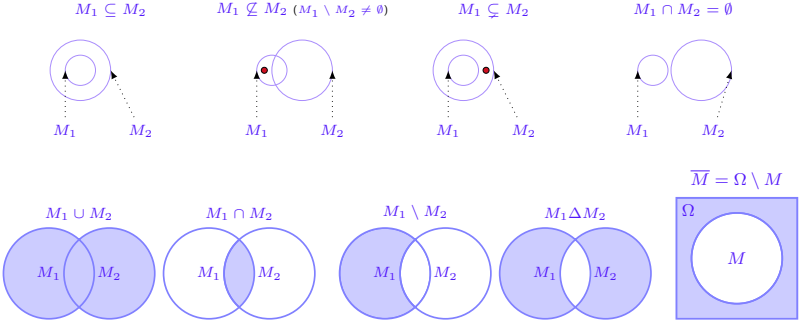
Mengenoperationen und Beziehungen zwischen Mengen



Def Seien M_1, M_2 beliebige (wohldefinierte) Mengen. Dann:

- $M_1 \subseteq M_2$ **:gdw** für jedes $x \in M_1$ gilt $x \in M_2$ (M_1 ist eine Teilmenge von M_2).
  $M_1 \not\subseteq M_2$ **:gdw** es gibt ein $x \in M_1$ mit $x \notin M_2$ (M_1 ist keine Teilmenge von M_2).
- $M_1 \subsetneq M_2$ **:gdw** $M_1 \subseteq M_2$ und $M_2 \not\subseteq M_1$ (M_1 ist eine echte Teilmenge von M_2).
- $M_1 = M_2$ **:gdw** $M_1 \subseteq M_2$ und $M_2 \subseteq M_1$ (M_1, M_2 sind gleich).
  $M_1 \neq M_2$ **:gdw** $M_1 \not\subseteq M_2$ oder $M_2 \not\subseteq M_1$ (M_1, M_2 sind ungleich).

Mengenoperationen und Beziehungen zwischen Mengen



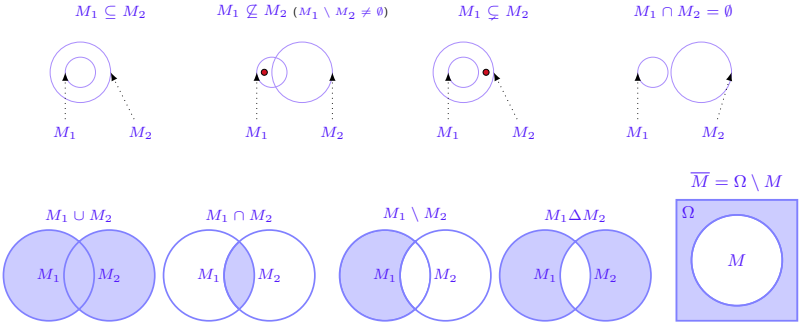
Def Seien M_1, M_2 beliebige (wohldefinierte) Mengen. Dann:

- $x \in M_1 \cup M_2$ **:gdw** $x \in M_1$ oder $x \in M_2$ ($M_1 \cup M_2$ ist die Vereinigung v. M_1, M_2).
- $x \in M_1 \cap M_2$ **:gdw** $x \in M_1$ und $x \in M_2$ ($M_1 \cap M_2$ ist der Schnitt von M_1, M_2).
- $x \in M_1 \setminus M_2$ **:gdw** $x \in M_1$ und $x \notin M_2$ ($M_1 \setminus M_2$ ist die Differenz „M_1 ohne M_2“).
- $x \in M_1 \Delta M_2$ **:gdw** entweder $x \in M_1$ oder $x \in M_2$ ($M_1 \Delta M_2$ ist ihre symmetrische Differenz).

(Für eine Menge M ist nur relevant, ob $x \in M$ bzw. $x \notin M$ gilt.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Mengenoperationen und Beziehungen zwischen Mengen

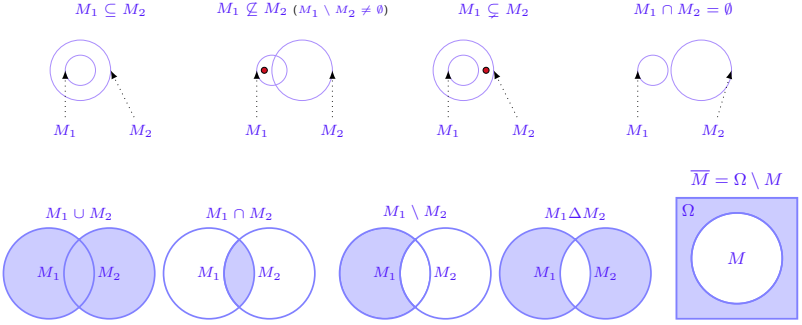


- Schnitt, Differenz und symmetrische Differenz lassen sich auch mittels Auswahl (Aussonderung) und Vereinigung definieren:
  - $M_1 \setminus M_2 = \{x \in M_1 \mid x \notin M_2\}$
  - $M_1 \Delta M_2 = (M_1 \setminus M_2) \cup (M_2 \setminus M_1) = \{x \in M_1 \cup M_2 \mid x \notin M_1 \text{ oder } x \notin M_2\}$.
  - $M_1 \cap M_2 = \{x \in M_1 \mid x \in M_2\} = \{x \in M_2 \mid x \in M_1\} = (M_1 \cup M_2) \setminus (M_1 \Delta M_2)$

Diese Operationen sind nur „syntaktischer Zucker“: Sie vereinfachen die Definition von Mengen, benötigt werden nur Aussonderung und Vereinigung.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Mengenoperationen und Beziehungen zwischen Mengen



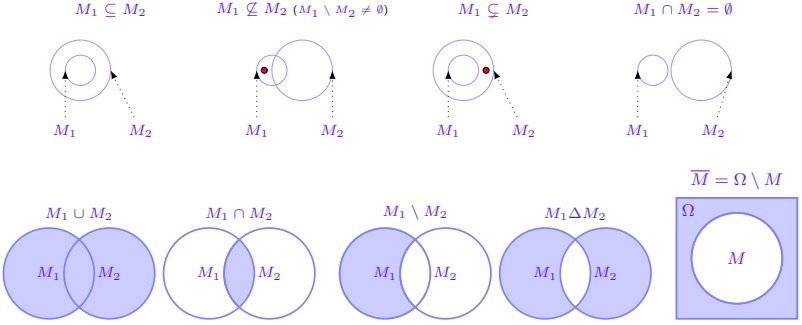
Def Abgeleitete Definitionen:

- $M_1 \supseteq M_2$ **:gdw** $M_2 \subseteq M_1$ (M_1 ist eine Obermenge von M_2).
  (Analog $M_1 \not\supseteq M_2$ und $M_1 \supsetneq M_2$.)
- M_1, M_2 sind disjunkt **:gdw** $M_1 \cap M_2 = \emptyset$.
- Ist eine Menge Ω (das „Universum“, die „Grundmenge“) fixiert, dann ist $\overline{M} := \Omega \setminus M$ das Komplement von M (bzgl. Ω).

⚠ $\overline{M}$ erfordert ein fixiertes Universum, sonst ist es undefiniert.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Mengenoperationen und Beziehungen zwischen Mengen



- Bemerkungen:
  - ⚠ Manche verwenden $\subset$ im Sinne von $\subsetneq$ (vgl.: $<, \leq$), andere im Sinne von $\subseteq$.
  - Statt „$M_1 \setminus M_2$“ wird häufig auch „$M_1 - M_2$“ geschrieben.

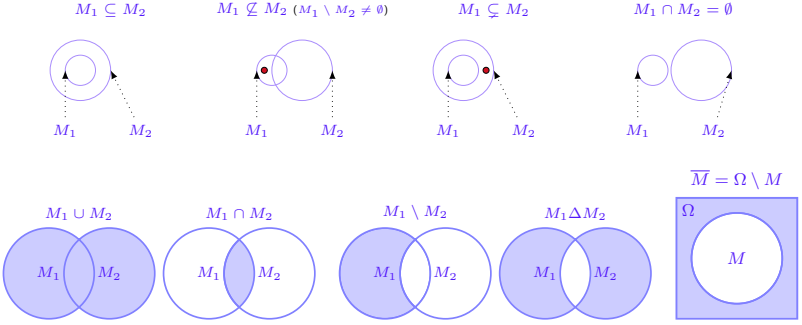
Fkt $M_1 \subseteq M_2$ **gdw** $M_1 \cap M_2 = M_1$ **gdw** $M_1 \cup M_2 = M_2$ **gdw** $M_1 \setminus M_2 = \emptyset$.

(Vgl.: $a \leq b$ **gdw** $\min(a,b) = a$ **gdw** $\max(a,b) = b$ **gdw** $a - b \leq 0$)

Fkt $M_1 = M_2$ **gdw** $M_1 \Delta M_2 = \emptyset$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Mengenoperationen und Beziehungen zwischen Mengen

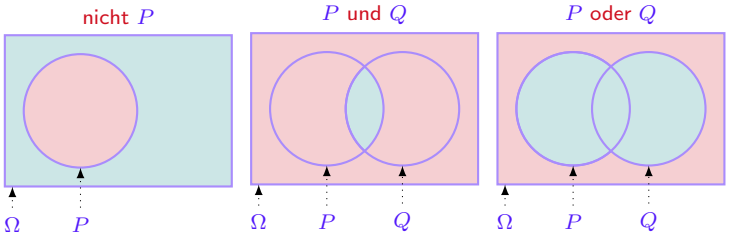


Bsp Seien $M_1 = \{a, b, c\}$ und $M_2 = \{b, c, d\}$ (mit $i = 1, 2$):

- $M_1 = \{b, a, c\} = \{a, a, b, b, c, c\} = M_1 \cup M_1 = M_1 \cap M_1 = \ldots$
- $M_1 \cup M_2 = M_2 \cup M_1 = \{a, b, c, d\}$, $M_i \subsetneq M_1 \cup M_2 \not\subseteq M_i$
- $M_1 \cap M_2 = M_2 \cap M_1 = \{b, c\}$, $M_1 \cap M_2 \subsetneq M_i \not\subseteq M_1 \cap M_2$
- $M_1 \setminus M_2 = \{a\} \neq \{d\} = M_2 \setminus M_1$
- $M_1 \Delta M_2 = M_2 \Delta M_1 = \{a, d\}$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

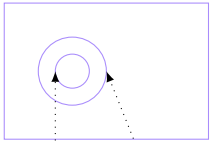
Vergleich grundlegender Konzepte von Mengen und Logik



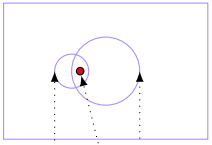
- Fixiere nicht leeres Universum Ω. Seien P, Q Prädikate, die Teilmengen definieren.
  - $\{x \in \Omega \mid \text{nicht } P(x)\} = \Omega \backslash \{x \in \Omega \mid P(x)\}$
  - $\{x \in \Omega \mid P(x) \text{ und } Q(x)\} = \{x \in \Omega \mid P(x)\} \cap \{x \in \Omega \mid Q(x)\}$
  - $\{x \in \Omega \mid P(x) \text{ oder } Q(x)\} = \{x \in \Omega \mid P(x)\} \cup \{x \in \Omega \mid Q(x)\}$

Vergleich grundlegender Konzepte von Mengen und Logik

„Alle P sind Q“ **gdw**

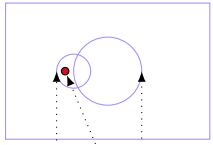


„Einige P sind Q“ **gdw**



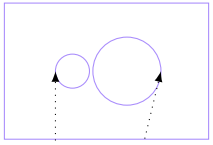
Beispiel

„Nicht alle P sind Q“ **gdw**



Gegenbeispiel

„Kein P ist Q“ **gdw**



- Fixiere nicht leeres Universum Ω. Seien P, Q Prädikate, die Teilmengen definieren.
  - „Alle P sind Q“ **gdw** $\{x \in \Omega \mid P(x)\} \subseteq \{x \in \Omega \mid Q(x)\}$
    (Immer wahr, falls $\{x \in \Omega \mid P(x)\} = \emptyset$.)
  - „Einige P sind Q“ **gdw** $\{x \in \Omega \mid P(x)\} \cap \{x \in \Omega \mid Q(x)\} \neq \emptyset$
  - „Nicht alle P sind Q“ **gdw** $\{x \in \Omega \mid P(x)\} \cap \{x \in \Omega \mid \text{nicht } Q(x)\} \neq \emptyset$
    gdw „Einige P sind nicht Q“
  - „Kein P ist Q“ **gdw** $\{x \in \Omega \mid P(x)\} \cap \{x \in \Omega \mid Q(x)\} = \emptyset$
    gdw $\{x \in \Omega \mid P(x)\} \subseteq \{x \in \Omega \mid \text{nicht } Q(x)\}$
    gdw „Alle P sind nicht Q“

Endliche Mengen, Kardinalität

Def Eine Menge M ist endlich **:gdw** es gibt ein $k \in \mathbb{N}_0$ und Objekte $x_1, x_2, \ldots, x_k$ mit $M = \{x_1, x_2, \ldots, x_k\}$.

- ▷ Übliche Konvention: $\{x_1, x_2, \ldots, x_k\} = \emptyset$ für $k = 0$.
- ▷ „unendlich“ kurz für „nicht endlich“.

Def Ist M endlich, so bezeichnet man mit $|M|$ die Anzahl der (paarweise) verschiedenen Objekte, welche Elemente von M sind.

D.h. $|M|$ ist die kleinste Zahl $k \in \mathbb{N}_0$, für welche es Objekte $x_1, x_2, \ldots, x_k$ mit $M = \{x_1, x_2, \ldots, x_k\}$ gibt.

- ▷ Bsp.: $|\{a, b, c\}| = |\{a, b, c, a, b, c\}| = |\{c, a, b, b, c, c, c\}| = |\{a, b\} \cup \{b, c\}| = 3$.

Def Man bezeichnet $|M|$ als die Kardinalität/Mächtigkeit von M.

- ▷ Häufig schreibt man kurz „$|M| < \infty$“ bzw. „$|M| = \infty$“, falls M endlich bzw. unendlich ist. Allerdings haben nicht alle unendlichen Mengen dieselbe Kardinalität.
- ▷ Die allgemeine Definition der Kardinalität von unendlichen Mengen benötigt den Begriff der injektiven Funktion (später).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Mengen von Mengen: Potenzmenge

Def Für jede Menge M ist auch die Zusammenfassung $\mathcal{P}(M)$ genau aller Teilmengen von M wieder eine Menge: $A \in \mathcal{P}(M)$ **:gdw** $A \subseteq M$.

⚠ Häufig auch „2^M“ statt „$\mathcal{P}(M)$“.

▷ Lies „$\mathcal{P}(M)$“ bzw. „2^M“ als „Potenzmenge von M“.

Bsp Beispiele:

$$
\begin{array}{lcll}
\mathcal{P}(\{1,2\}) & = & 2^{\{1,2\}} & = \{\emptyset, \{1\}, \{2\}, \{1,2\}\} \\
\mathcal{P}(\{1,2,3\}) & = & 2^{\{1,2,3\}} & = \{\emptyset, \{1\}, \{2\}, \{1,2\}\} \\
 & & & \cup \ \{\{3\}, \{1,3\}, \{2,3\}, \{1,2,3\}\} \\
\mathcal{P}(\emptyset) & = & 2^{\{\}} & = \{\emptyset\} \\
\mathcal{P}(\{\emptyset, \{\emptyset\}\}) & = & 2^{\{\emptyset, \{\emptyset\}\}} & = \{\emptyset, \{\emptyset\}, \{\{\emptyset\}\}, \{\emptyset, \{\emptyset\}\}\}
\end{array}
$$

Fkt Sei $M \neq \emptyset$. Fixiere beliebiges $x \in M$.
Dann gilt: $2^M = 2^{M-\{x\}} \cup \{K \cup \{x\} \mid K \in 2^{M-\{x\}}\}$.

Fkt Ist $M = \{x_1, \ldots, x_k\}$ endlich mit $|M| = k$, dann gilt $|2^M| = 2^{|M|} = 2^k$:
$A \subseteq M$ durch k Prädikate ($x_i \in A$ bzw. $x_i \notin A$) eindeutig bestimmt.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Für eine Menge S, deren Elemente selbst Mengen sind, definiert man:

$$\bigcup S := \bigcup_{M\in S} M := \{x \mid \text{es gibt ein } M \in S \text{ mit } x \in M\}$$
$$\bigcap S := \bigcap_{M\in S} M := \{x \mid \text{für jedes } M \in S \text{ gilt } x \in M\}$$

Gilt $S = \{M_1, \ldots, M_k\}$ für ein $k \in \mathbb{N}$, dann schreibt man auch

$$\bigcup_{i=1}^{k} M_i := \bigcup S \qquad \bigcap_{i=1}^{k} M_i := \bigcap S$$

Weiterhin gilt: $\bigcup \emptyset = \emptyset$, aber $\bigcap \emptyset$ ist undefiniert.

▷ Damit: $M_1 \cap M_2 = \bigcap\{M_1, M_2\}$ und $M_1 \cup M_2 = \bigcup\{M_1, M_2\}$
und für $S \neq \emptyset$ auch $\bigcap S = \{x \in \bigcup S \mid \text{für jedes } M \in S\colon x \in M\}$

Bsp Beispiele:

$$\bigcup\Big\{\{1,4\},\{1,3,2\},\{1,3\}\Big\} = \{1,4\}\cup\{1,3,2\}\cup\{1,3\} = \{1,2,3,4\}$$
$$\bigcap\Big\{\{1,4\},\{1,3,2\},\{1,3\}\Big\} = \{1,4\}\cap\{1,3,2\}\cap\{1,3\} = \{1\}$$

$$\bigcup\{\{x\} \mid x \in \mathbb{N}_0\} = \mathbb{N}_0 \qquad \bigcap\Big\{\{x \in \mathbb{R}_{>0} \mid 2 - \frac{1}{k} < x^2 \leq 2\} \mid k \in \mathbb{N}\Big\} = \{\sqrt{2}\}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Mengen von Mengen: Partitionen

Def Eine Partition einer Menge M ist eine Menge $P \subseteq 2^M$ von disjunkten, nicht leeren Teilmengen von M, deren Vereinigung M ergibt:

- Für alle $C, C' \in P$: entweder $C = C'$ oder $C \cap C' = \emptyset$ (paarweise disjunkt).
- Für alle $C \in P$: $C \neq \emptyset$ (nicht leer).
- $M = \bigcup P$ (die Elemente von P „überdecken“ M).

- Beispiel: Menge aller Partitionen von $\{1, 2, 3\}$

$$\Big\{\big\{\{1,2,3\}\big\}, \big\{\{1\},\{2,3\}\big\}, \big\{\{1,2\},\{3\}\big\}, \big\{\{1,3\},\{2\}\big\}, \big\{\{1\},\{2\},\{3\}\big\}\Big\}$$

- Einige mögliche Partitionen von $\mathbb{Z}$:

$$\{\mathbb{Z}\}, \quad \{\{x\} \mid x \in \mathbb{Z}\}, \quad \{2\mathbb{Z}, 1 + 2\mathbb{Z}\}, \quad \{3\mathbb{Z}, 1 + 3\mathbb{Z}, 2 + 3\mathbb{Z}\}$$

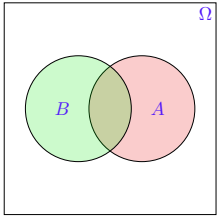
mit $r + n\mathbb{Z} := \{r + nz \mid z \in \mathbb{Z}\}$. Im Allgemeinen für $n \in \mathbb{N}$:

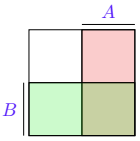
$$\{r + n\mathbb{Z} \mid r \in \mathbb{Z}_n\}$$

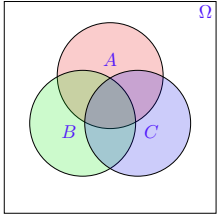
ist die Partition von $\mathbb{Z}$ nach dem Rest bei Division durch n.

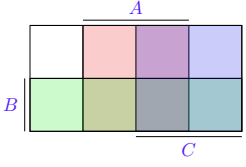
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Venn- und Karnaugh-Veitch-Diagramme als geometrische Darstellung von Wahrheitstabellen bzgl. $x \in A, x \in B, \ldots$ bei fixiertem Universum Ω.









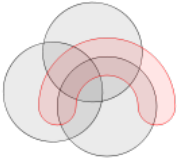
⚠ Allgemeine Lage aller Mengen bei Venn-Diagrammen für alle 2^n Schnitte/Fälle!

(Partition von Ω in 2^n Klassen.)

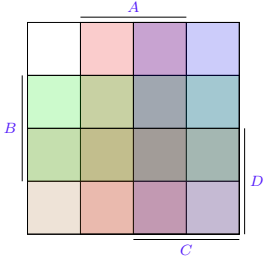
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

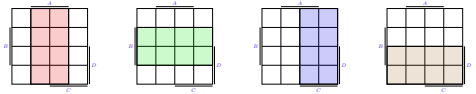
Visualisierung aller Fälle bei 4 allgemeinen Mengen

- Venn-Diagramme und KV-Diagramme für beliebige $A, B, C, D \subseteq \Omega$:



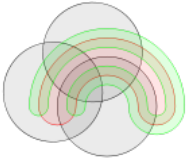
(Bildquelle: [wikipedia])



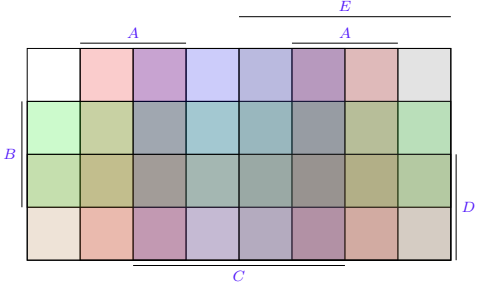


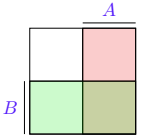
Visualisierung aller Fälle bei 5 allgemeinen Mengen

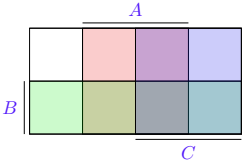
- Venn-Diagramme und KV-Diagramme für beliebige $A, B, C, D, E \subseteq \Omega$:

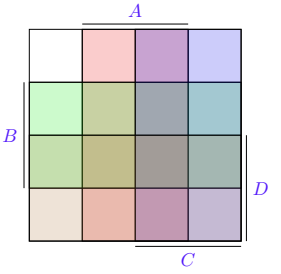


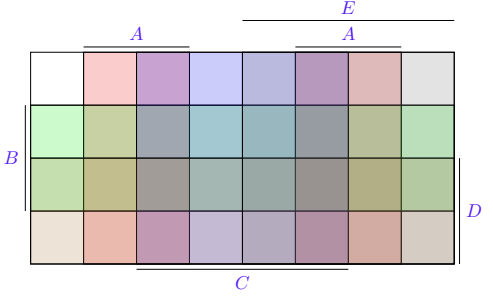
(Bildquelle: [wikipedia])











No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Mengenterme und Identitäten

- ▷ Mengentheoretische Ausdrücke (Terme) analog zu arithmetischen Ausdrücken:

  (Arithm.) $x \cdot (y + z), x + 5, \ldots$ (Mengen) $A \cap (B \cup C), A \cup \{5\}, \ldots$

  „Platzhalter für Mengen“ (Variablen) und konkrete Mengen (Konstanten) kombiniert mittels $\cap, \cup, \Delta, \setminus, 2^{\cdot}$ und bei gegebenem Universum auch $\overline{}$ (zzgl. Klammern für Auswertungsreihenfolge).

- ⚠ Problem: Mehrdeutigkeit von „$=$“ für Terme:

  1. Definition: „Es gelte $X = Y \cup Z$.“ ($X := Y \cup Z$)
  2. Beide Terme sind identisch/dieselben: $A \cup B = A \cup B$
  3. Gleichheit gilt nach Auswertung für vorgegebene Werte: $\{5\} \cup \{6\} = \{5, 6\}$
  4. Existenz einer Lösung der Gleichung: $X \cup \{5\} = \{5, 6\}$
  5. Identität(sgleichung) (semantische Äquivalenz): $X \cup Y = Y \cup X$

     Beide Seiten werten sich für alle konkreten Mengen stets zur selben Menge aus.

     In diesem Fall auch „$\equiv$“ statt „$=$“, um die universelle Gleichheit zu betonen.

     Z.B. $A \cap (B \cup C) \equiv (A \cap B) \cup (A \cap C)$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Identitäten (semantische Äquivalenzen)

Lem Identitäten: ($\equiv$: selbes Ergebnis für alle Mengen A, B, C)

$$\begin{aligned} A &\equiv A \cup A \\ A &\equiv A \cap A \\ A &\equiv A \cup \emptyset \\ \emptyset &\equiv A \cap \emptyset \\ A \cup B &\equiv B \cup A \\ A \cap B &\equiv B \cap A \\ A \Delta B &\equiv (A \setminus B) \cup (B \setminus A) \end{aligned}$$

$$\begin{aligned} A &\equiv A \cup (A \cap B) \\ A &\equiv A \cap (A \cup B) \\ A \cup (B \cup C) &\equiv (A \cup B) \cup C \\ A \cap (B \cap C) &\equiv (A \cap B) \cap C \\ A \cup (B \cap C) &\equiv (A \cup B) \cap (A \cup C) \\ A \cap (B \cup C) &\equiv (A \cap B) \cup (A \cap C) \\ A \setminus (B \cap C) &\equiv (A \setminus B) \cup (A \setminus C) \\ A \setminus (B \cup C) &\equiv (A \setminus B) \cap (A \setminus C) \end{aligned}$$

Lem Mit definiertem Universum Ω:

$$\begin{aligned} A \cap \overline{A} &\equiv \emptyset \\ A \cup \overline{A} &\equiv \Omega \\ A \setminus B &\equiv A \cap \overline{B} \end{aligned}$$

$$\begin{aligned} \overline{\overline{A}} &\equiv A \\ \overline{A \cup B} &\equiv \overline{A} \cap \overline{B} \\ \overline{A \cap B} &\equiv \overline{A} \cup \overline{B} \end{aligned}$$

⚠ Da die Identitäten für alle konkreten Mengen A, B, C gelten, darf man auch A, B, C durch beliebige Mengenterme ersetzen.

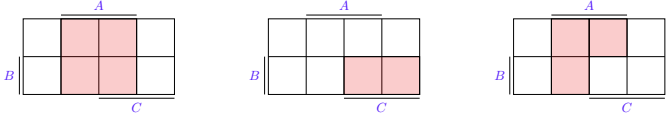
z.B.: $\underbrace{(A \cup B)}_{A'} \cap (\underbrace{(A \cup B)}_{A'} \cup B) \equiv A \cup B$ da $A' \equiv A' \cap (A' \cup B)$

Identitäten für Mengenoperationen

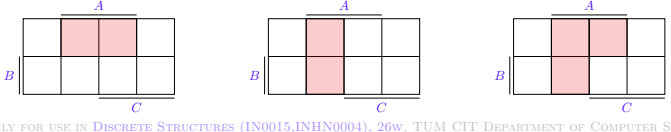
- Überprüfen/Beweisen z.B. durch Aufstellen des KV-Diagramms einmal für die linke und einmal für die rechte Seite einer Identität.
- Implizit ein Universum, das zumindest die Vereinigung aller betrachteten Mengen umfasst.

Bsp $A \setminus (B \cap C) \equiv (A \setminus B) \cup (A \setminus C)$

- Linke Seite: v.l.n.r. A, $B \cap C$, $A \setminus (B \cap C)$



- Rechte Seite: v.l.n.r. $A \setminus B$, $A \setminus C$, $(A \setminus B) \cup (A \setminus C)$



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

2 Grundlagen

Grundbegriffe der Logik

Mengen

Folgen, Tupel und Wörter

Die natürlichen Zahlen, Induktionsbeweise

Relationen

Funktionen

Kardinalität: Vergleiche von unendlichen Mengen

Zusammenfassung

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Mengen vs. Folgen

- Menge: Zusammenfassung ohne Beachtung von Anordnung oder Vielfachheiten:

$$\{a, b, \emptyset, \{b, a\}, a\} = \{\emptyset, \{a, b\}, a, b\}$$

- Folge: Zusammenfassung unter Beachtung der Anordnung und Vielfachheiten:

$$(a, b, \emptyset, \{b, a\}, a) \neq (\emptyset, \{a, b\}, a, b)$$

Runde Klammern („(“, „)“) zur Definition von Folgen.

Endliche Folgen werden auch Tupel genannt.

⚠ Viele Programmiersprachen unterscheiden zwischen Listen/Arrays und Tupeln:

- Tupel sind i.A. „unveränderlich (immutable, const)“, nur Umdefinition möglich.
- Listen und Arrays sind „veränderliche (mutable) Tupel“,

  je nach Programmiersprache weitere Unterschiede (Implementierung, Laufzeit, Speicher, . . .)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Folgen: Definition

Def Sei $I \subseteq \mathbb{N}_0$ („Indexmenge“), zunächst entweder $I = \mathbb{N}$ oder $I = \mathbb{N}_0$ oder $I = [n] = \{1, 2, \ldots, n\}$ oder $I = \mathbb{Z}_n = \{0, 1, \ldots, n-1\}$ (für $n \in \mathbb{N}_0$).

Ist für jeden Index $i \in I$ ein Objekt a_i definiert, dann schreibt man $(a_i)_{i \in I}$ für die Auflistung der Objekte a_i nach aufsteigendem Index i, z.B.

$$\Big(i\Big)_{i \in \mathbb{N}_0} = \Big(0, 1, 2, 3, 4, \ldots\Big) \qquad \Big(\frac{1}{i}\Big)_{i \in \mathbb{N}} = \Big(1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \ldots\Big)$$

und nennt $(a_i)_{i \in I}$ eine Folge (auch: Sequenz)

- Auch $(a_i | i \in I)$ für $(a_i)_{i \in I}$.
- a_i ist die/der/das i-te Komponente/Eintrag/Folgenglied.
- $|(a_i)_{i \in I}| := |I|$ ist die Länge der Folge; die Folge ist endlich, falls $|I| < \infty$.
- Für $I = [n]$ oder $I = \mathbb{Z}_n$ spricht man auch von „Tupel der Länge n (n-Tupel)“
  Dann: $(a_i)_{i \in [n]} = (a_1, \ldots, a_n)$ bzw. $(a_i)_{i \in \mathbb{Z}_n} = (a_0, a_1, \ldots, a_{n-1})$.
- Beachte: Für $I = \emptyset$ ist $(a_i)_{i \in I} = ()$ das leere Tupel mit Länge 0.

⚠ $(a_i)_{i \in I} = (b_j)_{j \in J}$ **gdw** $I = J$ und $a_i = b_i$ für alle $i \in I$:

$$(a, (b, c)) \neq (a, (c, b)) \neq (a, \{b, c\}) = (a, \{c, b\}) \neq () \neq (a, b, c)$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Wichtige Folgen

- Geometrische Folge: $a_i := cq^i$ für feste $c, q \in \mathbb{R}$.
  Z.B. für $c = 1$ und $q = -1/2$:

$$1, -\frac{1}{2}, \frac{1}{4}, -\frac{1}{8}, \frac{1}{16}, -\frac{1}{32}, \dots$$

- Geometrische Reihe: $a_0 := 0$ und $a_{i+1} := a_i + cq^i$ für feste $c, q \in \mathbb{R}$.
  Z.B. für $c = 1$ und $q = -1/2$:

$$0, 1, \frac{1}{2}, \frac{3}{4}, \frac{5}{8}, \frac{11}{16}, \frac{21}{32}, \dots$$

- Fibonacci-Zahlen: $F_0 := 0$, $F_1 := 1$, $F_{n+2} := F_{n+1} + F_n$

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \dots$$

(Induktive Definitionen, siehe natürliche Zahlen/Peano-Axiome.)

- Approximation von e: $a_i = (1 + \frac{1}{i})^i$ für $i \in \mathbb{N}$:

$$2, \frac{9}{4}, \frac{64}{27}, \frac{625}{256}, \frac{7776}{3125}, \dots$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Kartesisches Produkt, geordnetes Paar

Def Sind A, B Mengen, so definiert man die Menge $A \times B$ durch

$$A \times B := \{(a, b) \mid a \in A, b \in B\}$$

Man nennt $A \times B$ das kartesische Produkt von A und B; es umfasst genau alle 2-Tupel (auch: (geordnetes) Paar), deren erste Komponente ein Element aus A und deren zweite Komponente ein Element aus B ist.

$A \times B$	b_1	b_2	b_3
a_1	(a_1, b_1)	(a_1, b_2)	(a_1, b_3)
a_2	(a_2, b_1)	(a_2, b_2)	(a_2, b_3)

Def Entsprechend definiert man für eine endliche Folge $A_1, \ldots, A_k$ von Mengen:

$$A_1 \times \ldots \times A_k := \bigtimes_{i=1}^{k} A_i := \{(a_i)_{i \in [k]} \mid a_i \in A_i \text{ für alle } i \in [k]\}$$

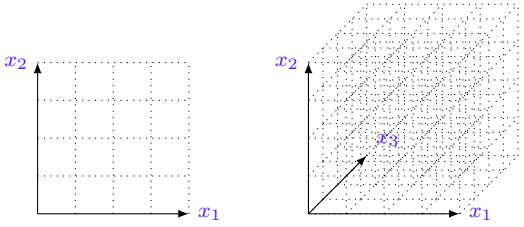
Für $k = 0$ definiert man $\bigtimes_{i=1}^{0} A_i = \{ () \}$.

Gilt $A = A_1 = \ldots = A_k$, dann kurz A^k für $A_1 \times \ldots \times A_k$.

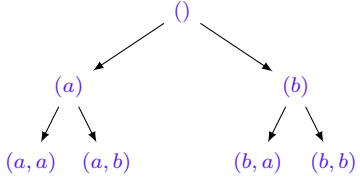
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Kartesisches Produkt

Bsp $\mathbb{R}^2$ und $\mathbb{R}^3$



- Das kartesische Produkt A^0, A^1, A^2, ... für eine gegebene endliche Menge A kann man als (gewurzelten) Baum visualisieren. Z.B. für $A = \{a, b\}$:



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Kartesisches Produkt: Eigenschaften

Lem Einige Eigenschaften des kartesischen Produkts: (Annahme: alle Mengen nicht leer)

- Distributiv für $\diamond \in \{\cap, \cup, \setminus\}$:

$$A \times (B \diamond C) \equiv (A \times B) \diamond (A \times C) \text{ und } (A \diamond B) \times C \equiv (A \times C) \diamond (B \times C)$$

- Nicht kommutativ für $A \neq B$: $A \times B \neq B \times A$
- Nicht assoziativ: $(A \times B) \times C \neq A \times B \times C \neq A \times (B \times C)$:
  - $A \times B \times C = \{(a, b, c) \mid a \in A, b \in B, c \in C\}$
  - $(A \times B) \times C = \{((a, b), c) \mid a \in A, b \in B, c \in C\}$
  - $A \times (B \times C) = \{(a, (b, c)) \mid a \in A, b \in B, c \in C\}$
- $A_1 \times B_1 \subseteq A_2 \times B_2$ **gdw** $A_1 \subseteq A_2$ und $B_1 \subseteq B_2$
  $A_1 \times B_1 = A_2 \times B_2$ **gdw** $A_1 = A_2$ und $B_1 = B_2$
- Bzgl. eines Universums $\Omega \supseteq A, B$:
  $(\Omega \times \Omega) \setminus (A \times B) = (A \times \overline{B}) \cup (\overline{A} \times B) \cup (\overline{A} \times \overline{B})$
- Kardinalität: $|A \times B| = |A| \cdot |B|$ (auch falls unendlich, siehe Kardinalität später)

(Beweise zur Übung; Illustration mittels Tabelle/Raster.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Alphabete und Wörter

Def Eine nicht leere Menge Σ heißt Alphabet, wenn ihre Elemente als unteilbare Zeichen aufgefasst werden (meist ist Σ endlich).

Ein Tupel $(a_1, \ldots, a_n) \in \Sigma^n$ schreibt man dann als Wort $a_1 a_2 \ldots a_n$. Dafür müssen die Zeichen so notiert sein, dass man das Tupel aus dem Wort eindeutig rekonstruieren kann.

Das leere Tupel $()$ wird durch das leere Wort ε (auch: λ) dargestellt.

- ▷ Alphabete (Zeichenvorrat/-menge) werden häufig mit „Σ“ (für „Symbol“) oder „Γ“ (für „Glyphe, Graphem“) bezeichnet.
- ▷ Alphabetszeichen sollten üblicherweise selbst nicht weiter zerlegbar sein im Gegensatz zu einem Code.

Bsp aba bzgl. $\Sigma_1 = \{a, b\}$: nur (a, b, a) möglich;
aba bzgl. $\Sigma_2 = \{a, b, ba\}$: sowohl (a, b, a) als auch (a, ba) möglich.

Bsp Vergleiche $\{0, 1, \ldots, 15\}$ und $\{0, 1, \ldots, 9, A, \ldots, F\}$ für Zahlen zur Basis 16.
(Natürliche Zahlen werden als Wörter von Ziffern Σ bzgl. der Basis $|\Sigma|$ dargestellt.)

Bsp Ein binäres Alphabet wie $\{0, 1\}$ kann verwendet werden, um komplexere Alphabete zu kodieren, siehe z.B. ISO 8859-1, UTF-8.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Wörter und Sprachen

Def Sei Σ ein Alphabet.

Dann verwendet man Σ^k direkt für die Menge der Wörter (nicht Tupel) der Länge k und Σ^* für die Menge aller endlichen Wörter, d.h.

$$\Sigma^k := \begin{cases} \{a_1 \dots a_k \mid a_i \in \Sigma \text{ für alle } i \in [k]\} & \text{für } k \geq 1 \\ \{\varepsilon\} & \text{für } k = 0 \end{cases} \qquad \Sigma^* = \bigcup_{k \in \mathbb{N}_0} \Sigma^k$$

Die Elemente von $\Sigma^* = \bigcup_{k \in \mathbb{N}_0} \Sigma^k$ bezeichnet man als (endliche) Wörter über Σ.

Eine Teilmenge $L \subseteq \Sigma^*$ wird als Sprache über Σ bezeichnet.

Ist $u = u_1 u_2 \dots u_k \in \Sigma^k$ ein Wort, so schreibt man wieder $|u|$ für seine Länge.

(Die Anzahl der Zeichen, aus denen u besteht; seine Länge als Tupel.)

Bsp $\Sigma = \{a, b\}$, $L = \{w \in \Sigma^* \mid w \text{ endet auf } a\} = \{a, aa, ba, \dots\}$

- ▷ Ein Programm ist selbst ein Wort einer Programmiersprache.

  In Programmiersprachen dienen Anführungszeichen dazu, (z.B.) den Compiler anzuweisen, das darin eingeschlossene Wort (Zeichenkette, String) nicht zu interpretieren. Das leere Wort (leerer String) wird dann zu “”.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Konkatenation

Def Konkatenation uv zweier Wörter bzw. Tupel u, v (mit $u_i, v_j \in \Sigma$):

$uv := u_1 \dots u_k v_1 \dots v_l$ für Wörter $u = u_1 \dots u_k$, $v = v_1 \dots v_l$.

$uv := (u_1, \dots, u_k, v_1, \dots, v_l)$ für Tupel $u = (u_1, \dots, u_k)$, $v = (v_1, \dots, v_l)$.

- ▷ Häufig $u \circ v$, falls uv mehrdeutig; aber $\circ$ wird später für Funktionen verwendet.
- ▷ In Programmiersprachen häufig „$+$“, obwohl $uv \neq vu$ i.A. nicht kommutativ.
- ▷ In Datenbanken verwendet man häufig eine „bedingte Konkatenation“, was je nach Bedingung auf verschiedene Formen eines “join ($\bowtie$)” führt.

Fkt (i) $u\varepsilon = \varepsilon u = u$, (ii) $u(vw) = uvw = (uv)w$ und (iii) $|uv| = |u| + |v|$.

Def Konkatenation uv zweier Sprachen $L, L' \subseteq \Sigma^*$:

$$LL' := \{uv \mid u \in L, v \in L'\}$$

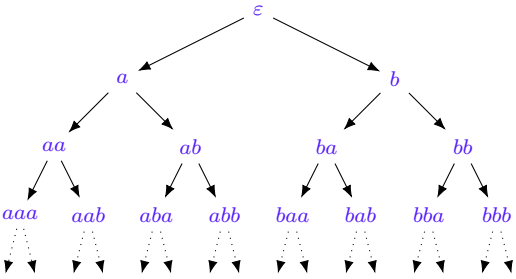
(D.h. bilde $L \times L'$ und lass dann wieder die äußeren Klammern und das Komma fallen.)

- ▷ Beispiel: Es gelte $L = \{aa, ab\}$ und $L' := \{ba, bb\}$.
  Dann $LL' = \{aaba, aabb, abba, abbb\} \neq L'L = \{baaa, baab, bbaa, bbab\}$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Σ^* und Bäume

- Für ein endliches Alphabet Σ kann man Σ^* als (unendlichen) Baum auffassen:
  - Die k-te Ebene besteht aus den Wörtern Σ^k
  - Für jedes Zeichen $x \in \Sigma$ gibt es eine Kante („Pfeil“) vom Wort $u \in \Sigma^k$ zum Wort $ux \in \Sigma^{k+1}$ (siehe auch Hasse-Diagramm der Präfixordnung).
- Z.B. ergibt sich für $\Sigma = \{a, b\}$ der unendliche Binärbaum:

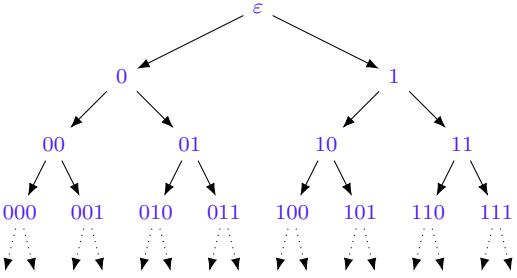


No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Σ^* und Bäume

- Für ein endliches Alphabet Σ kann man Σ^* als (unendlichen) Baum auffassen:
  - Die k-te Ebene besteht aus den Wörtern Σ^k
  - Für jedes Zeichen $x \in \Sigma$ gibt es eine Kante („Pfeil“) vom Wort $u \in \Sigma^k$ zum Wort $ux \in \Sigma^{k+1}$ (siehe auch Hasse-Diagramm der Präfixordnung).

Bsp Wenn man a durch 0 und b durch 1 ersetzt, erhält man eine binäre Repräsentation von $\mathbb{N}_0$ (beginnend mit dem höchstwertigen Bit, mit führenden Nullen).



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

2 Grundlagen

Grundbegriffe der Logik

Mengen

Folgen, Tupel und Wörter

Die natürlichen Zahlen, Induktionsbeweise

Relationen

Funktionen

Kardinalität: Vergleiche von unendlichen Mengen

Zusammenfassung

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

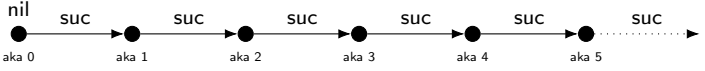
Die natürlichen Zahlen

- Die natürlichen Zahlen $\mathbb{N}_0$ (inkl. 0) sind die erste unendliche Menge:
  - Der Begriff der Abzählbarkeit („diskret“) ist direkt an $\mathbb{N}_0$ gebunden.
  - Auch lassen sich $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$ und $\mathbb{C}$ schrittweise aus $\mathbb{N}_0$ konstruieren.
  - Welche Mengen können durch einen Computer aufgezählt werden?
- Entsprechend ist man an einer möglichst präzisen Definition interessiert:
  Die konkrete Darstellung von $\mathbb{N}_0$ sollte dabei unwichtig sein.
  - $\{\varepsilon, |, ||, |||, ||||, |||||, \ldots\}$ unäre Strichlistendarstellung
  - $\{0, 1, 2, 3, 4, 5, 6, \ldots, 10, 11, \ldots\} = \mathbb{N}_0$ kanonische Dezimaldarstellung
  - $\{0, 1, 01, 11, 001, \ldots\}$ LSBF-Binärdarstellung
    $\{0, 1, 10, 11, 100, \ldots\}$ MSBF-Binärdarstellung
    $\{0, 1, 11, 10, 110, \ldots\}$ Gray-Code
  - $\{\emptyset, \{\emptyset\}, \{\emptyset, \{\emptyset\}\}, \ldots\}$ nach John von Neumann (Unendlichkeitsaxiom)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Peano-Axiome: Informell

- Charakterisierung der natürlichen Zahlen nach Peano:
  1. Es gibt eine ausgezeichnete natürliche Zahl, die mit nil (nichts, null) bezeichnet wird.
  2. Jede natürliche Zahl hat einen eindeutig definierten Nachfolger („eins mehr“), der selbst eine natürliche Zahl ist.
  3. Jede natürliche Zahl hat einen von nil verschiedenen Nachfolger.
  4. Zwei natürliche Zahlen mit gleichem Nachfolger sind gleich.
  5. Falls eine Menge X sowohl nil als auch mit jeder natürlichen Zahl (in X) deren Nachfolger enthält, dann enthält X bereits jede natürliche Zahl.
- Die Peano-Axiome beschreiben (genau!) einen „einseitig unendlichen Pfad, der nie im Kreis läuft“ („einseitigen Zahlenstrahl“):



(3 : „keine Schleifen“, 4 : „kein Zusammenfluss“, 5 : „kein weiterer Zahlenstrahl“.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Induktionsaxiom

⚠ Nach den Peano-Axiomen gilt für jede Menge X (mit kanonisch $\mathbb{N}_0 = \{0, 1, 2, \ldots\}$):

$$\mathbb{N}_0 \subseteq X \textbf{ gdw } \text{(IB)}\ 0 \in X \text{ und } \text{(IS)} \text{ für alle } n \in X \cap \mathbb{N}_0 \text{ gilt auch } n + 1 \in X.$$

- Diese Aussage gilt aufgrund der Definition von $\mathbb{N}_0$.
- Negation:

$$\mathbb{N}_0 \not\subseteq X \textbf{ gdw } 0 \notin X \text{ oder es gibt ein } n \in X \cap \mathbb{N}_0 \text{ mit } n + 1 \notin X.$$

Vereinfacht: (benötigt Wohlordnung von $\mathbb{N}_0$)

$$\mathbb{N}_0 \not\subseteq X \textbf{ gdw es gibt ein kleinstes } n_0 \in \mathbb{N}_0 \text{ mit } n_0 \notin X$$

n_0 ist dann das kleinste/erste Gegenbeispiel, d.h. für alle $n \in \mathbb{N}_0$ mit $n < n_0$ gilt noch $n \in X$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Induktionsaxiom

⚠ Nach den Peano-Axiomen gilt für jede Menge X (mit kanonisch $\mathbb{N}_0 = \{0, 1, 2, \ldots\}$):

$\mathbb{N}_0 \subseteq X$ **gdw** (IB) $0 \in X$ und (IS) für alle $n \in X \cap \mathbb{N}_0$ gilt auch $n + 1 \in X$.

▷ Induktive (rekursive) Definition: Um $(a_n)_{n \in \mathbb{N}_0}$ zu definieren, definiere

- (IB) explizit $a_0, \ldots, a_l$ ($l \in \mathbb{N}_0$) und
- (IS) für alle $n \in \mathbb{N}_0$, wie das nächste Element a_{l+n+1} aus den bereits definierten Elementen $a_0, \ldots, a_{l+n}$ hervorgeht (konstruiert, berechnet wird).
  - ⚠ Der zentrale Punkt ist, dass die Definition von a_{l+n+1} sich nur auf bereits definierte Objekte bezieht (kein Russell).

(X ist die Menge der Indizes $l + n$, für welche a_{l+n} (wohl) definiert ist.)

Bsp Für jedes $n \in \mathbb{N}_0$ sei $a_n \in \mathbb{N}_0$ definiert durch

$$a_n := \begin{cases} 0 & \text{falls } n = 0 \\ 2a_{n-1} + 1 & \text{falls } n > 0 \end{cases}$$

z.B.

$$a_0 = 0 \quad a_1 = 1 \quad a_2 = 3 \quad a_3 = 7 \quad a_4 = 15 \quad \ldots$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Induktionsaxiom

⚠ Nach den Peano-Axiomen gilt für jede Menge X (mit kanonisch $\mathbb{N}_0 = \{0, 1, 2, \ldots\}$):

$\mathbb{N}_0 \subseteq X$ **gdw** (IB) $0 \in X$ und (IS) für alle $n \in X \cap \mathbb{N}_0$ gilt auch $n + 1 \in X$.

▷ Induktionsbeweis: Sei P ein Prädikat, das für alle $n \in \mathbb{N}_0$ definiert ist.
Um zu zeigen, dass $P(n)$ für alle $n \in \mathbb{N}_0$ wahr ist, zeige

- (IB) $P(0)$ ist wahr (ohne Bezug zu $P(n)$) und
- (IS) sobald $P(n)$ für ein beliebiges (aber festes) $n \in \mathbb{N}_0$ wahr ist, ist auch $P(n+1)$ wahr.

($X = \{n \in \mathbb{N}_0 \mid P(n)\}$ ist die Menge aller $n \in \mathbb{N}_0$, für welche P wahr ist.)

Bsp Für jedes $n \in \mathbb{N}_0$ sei $a_n \in \mathbb{N}_0$ definiert durch

$$a_n := \begin{cases} 0 & \text{falls } n = 0 \\ 2a_{n-1} + 1 & \text{falls } n > 0 \end{cases}$$

Dann gilt $a_n = 2^n - 1$ für alle $n \in \mathbb{N}_0$ (mit $P(n)$ das Prädikat „$a_n = 2^n - 1$“; a_n ist bereits definiert).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Beispiel: Für jedes $n \in \mathbb{N}_0$ definieren wir die Zahl $a_n \in \mathbb{N}_0$ wie folgt:

$$a_n := \begin{cases} 0 & \text{falls } n = 0 \\ 2a_{n-1} + 1 & \text{falls } n > 0 \end{cases}$$

z.B.

$$a_0 = 0 \quad a_1 = 1 \quad a_2 = 3 \quad a_3 = 7 \quad a_4 = 15 \quad \ldots$$

- Vermutung: $a_n = 2^n - 1$ für alle $n \in \mathbb{N}_0$, d.h. $\{n \in \mathbb{N}_0 \mid a_n = 2^n - 1\} = \mathbb{N}_0$.
- Wir zeigen, dass für die Menge $X := \{n \in \mathbb{N}_0 \mid a_n = 2^n - 1\}$ gilt:
  - $0 \in X$ (Induktionsbasis)
  - Für jedes $n \in X \cap \mathbb{N}_0$ gilt auch $n + 1 \in X$ (Induktionsschritt)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Beispiel: Für jedes $n \in \mathbb{N}_0$ definieren wir die Zahl $a_n \in \mathbb{N}_0$ wie folgt:

$$a_n := \begin{cases} 0 & \text{falls } n = 0 \\ 2a_{n-1} + 1 & \text{falls } n > 0 \end{cases}$$

  z.B.

$$a_0 = 0 \quad a_1 = 1 \quad a_2 = 3 \quad a_3 = 7 \quad a_4 = 15 \quad \ldots$$

- Vermutung: $a_n = 2^n - 1$ für alle $n \in \mathbb{N}_0$, d.h. $\{n \in \mathbb{N}_0 \mid a_n = 2^n - 1\} = \mathbb{N}_0$.
- Induktionsbasis: Wir zeigen, dass $0 \in X$.

  D.h. es ist $a_0 = 2^0 - 1$ zu zeigen.

  - Linke Seite (l.S.): $a_0 = 0$ nach Definition.
  - Rechte Seite (r.S.): $2^0 - 1 = 1 - 1 = 0$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Beispiel: Für jedes $n \in \mathbb{N}_0$ definieren wir die Zahl $a_n \in \mathbb{N}_0$ wie folgt:

$$a_n := \begin{cases} 0 & \text{falls } n = 0 \\ 2a_{n-1} + 1 & \text{falls } n > 0 \end{cases}$$

z.B.

$$a_0 = 0 \quad a_1 = 1 \quad a_2 = 3 \quad a_3 = 7 \quad a_4 = 15 \quad \ldots$$

- Vermutung: $a_n = 2^n - 1$ für alle $n \in \mathbb{N}_0$, d.h. $\{n \in \mathbb{N}_0 \mid a_n = 2^n - 1\} = \mathbb{N}_0$.
- Induktionsschritt: Wir zeigen, für jedes $n \in X \cap \mathbb{N}_0$ gilt auch $n + 1 \in X$.

  Sei $n \in \mathbb{N}_0$ beliebig gewählt, aber im Weiteren fest („fixiert“).
  - Induktionsannahme: $n \in X$, d.h. wir nehmen $a_n = 2^n - 1$ an.

    (Statt Induktionsannahme auch Induktionshypothese.)
  - Induktionsbehauptung: $n + 1 \in X$, d.h. es ist $a_{n+1} = 2^{n+1} - 1$ zu zeigen.
  - Beweis der Induktionsbehauptung:

$$a_{n+1} \overset{\text{Def.}}{=} 2a_n + 1 \overset{\text{I.A.}}{=} 2(2^n - 1) + 1 = 2^{n+1} - 2 + 1 = 2^{n+1} - 1$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Beispiel: Für jedes $n \in \mathbb{N}_0$ definieren wir die Zahl $a_n \in \mathbb{N}_0$ wie folgt:

$$a_n := \begin{cases} 0 & \text{falls } n = 0 \\ 2a_{n-1} + 1 & \text{falls } n > 0 \end{cases}$$

  z.B.

$$a_0 = 0 \quad a_1 = 1 \quad a_2 = 3 \quad a_3 = 7 \quad a_4 = 15 \quad \ldots$$

- Vermutung: $a_n = 2^n - 1$ für alle $n \in \mathbb{N}_0$, d.h. $\{n \in \mathbb{N}_0 \mid a_n = 2^n - 1\} = \mathbb{N}_0$.
- Wir zeigen, dass für die Menge $X := \{n \in \mathbb{N}_0 \mid a_n = 2^n - 1\}$ gilt:
  - $0 \in X$ (Induktionsbasis) ✓
  - Für jedes $n \in X \cap \mathbb{N}_0$ gilt auch $n + 1 \in X$ (Induktionsschritt) ✓
- Nach dem Induktionsaxiom gilt somit $\mathbb{N}_0 \subseteq X$, damit auch $X = \mathbb{N}_0$.
- Somit gilt $a_n = 2^n - 1$ für alle $n \in \mathbb{N}_0$.
- Induktionsbeweise überall, z.B. Korrektheit von rekursiven Programmen oder Programmen mit while-Schleifen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Struktur eines Induktionsbeweises

⚠ Um „für alle $n \in \mathbb{N}_0$: $P(n)$“ mittels Induktion nach n zu zeigen:

- ▷ Induktionsbasis (I.B.): Beweise $P(0)$ (bzgl. der bereits getroffenen Annahmen).
- ▷ Induktionsschritt (I.S.): Fixiere ein beliebiges $n \in \mathbb{N}_0$, über das bis jetzt noch keine Annahmen getroffen worden sind.
  - ▷ Induktionsannahme (I.A.):
    $P(n)$ gilt für das fixierte n (bzgl. der bereits getroffenen Annahmen).
  - ▷ Induktionsbehauptung (I.Beh.):
    $P(n+1)$ gilt für das fixierte n (bzgl. der bereits getroffenen Annahmen zzgl. $P(n)$).
  - ▷ Schließlich beweise $P(n+1)$ unter den bereits getroffenen Annahmen unter zusätzlichen Annahmen $P(n)$ für das fixierte n.

▷ Typische Varianten:

- ▷ „Für alle $n \in \mathbb{N}_0$ mit $n \geq n_0$: $P(n)$.“ ($n_0 \in \mathbb{N}_0$)
  I.B.: Zeige $P(n_0)$; I.S.: Fixiere bel. $n \geq n_0$.
- ▷ Starke Induktion:
  I.A.: $P(k)$ gilt für alle $n_0 \leq k \leq n$; I.Beh.: $P(n+1)$ gilt auch. ($n_0, k \in \mathbb{N}$)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

⚠ Geometrische Summe

Für alle $n \in \mathbb{N}_0$ und für alle $x \in \mathbb{R}$ mit $x \neq 1$:

$$\sum_{k=0}^{n} x^k = 1 + x + \ldots + x^n = \frac{1 - x^{n+1}}{1 - x}$$

▷ Beweis mittels Induktion nach $n \in \mathbb{N}_0$:

I.B.: Für $n = 0$ ergibt die LS $\sum_{k=0}^{0} x^k = 1$ für jedes $x \in \mathbb{R} \setminus \{1\}$.

Die RS ergibt $\frac{1-x^{0+1}}{1-x} = 1$ für jedes $x \in \mathbb{R} \setminus \{1\}$.

I.S.: Fixiere ein beliebiges $n \in \mathbb{N}_0$.

I.A.: Für das gewählte n gilt $\sum_{k=0}^{n} x^k = \frac{1-x^{n+1}}{1-x}$ für alle $x \in \mathbb{R} \setminus \{1\}$.

I.Beh.: Für das gewählte n gilt $\sum_{k=0}^{n+1} x^k = \frac{1-x^{n+2}}{1-x}$ für alle $x \in \mathbb{R} \setminus \{1\}$.

Beweis: Für jedes konkrete $x \in \mathbb{R} \setminus \{1\}$:

$$\text{(LS)} \sum_{k=0}^{n+1} x^k = x^{n+1} + \sum_{k=0}^{n} x^k \overset{\text{I.A.}}{=} x^{n+1} + \frac{1 - x^{n+1}}{1 - x} = \frac{1 - x^{n+2}}{1 - x} \text{ (RS)}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Beispiel: geometrische Summe, endliche Summen und Produkte

⚠ Geometrische Summe

Für alle $n \in \mathbb{N}_0$ und für alle $x \in \mathbb{R}$ mit $x \neq 1$:

$$\sum_{k=0}^{n} x^k = 1 + x + \ldots + x^n = \frac{1 - x^{n+1}}{1 - x}$$

Def Für $(a_i)_{i \in \mathbb{N}_0}$ eine reelle Folge ist $\sum_{i=0}^{n} a_i$ induktiv definiert durch

I.B.: Falls $n = 0$: $\sum_{i=0}^{0} a_i := a_0$.

I.S.: Falls $n > 0$: $\sum_{i=0}^{n} a_i := a_n + \sum_{i=0}^{n-1} a_i$.

Analog für $(a_i)_{i \in \mathbb{N}}$ mit $\sum_{i=1}^{0} a_i := 0$.

(Die Wahl des Summationsindex ist beliebig, solange nicht schon bereits verwendet.)

Entsprechend ist auch $\prod_{j=1}^{n} a_j$ induktiv definiert durch:

$\prod_{j=1}^{0} a_j := 1$ und $\prod_{j=1}^{n+1} a_j := a_{n+1} \cdot \prod_{j=1}^{n} a_j$.

(Die (Partial-)Summen und Produkte werden in der gegebenen Reihenfolge nach aufsteigendem Index berechnet. Für jedes gegebene $n \in \mathbb{N}$ spielt die Reihenfolge auf Grund der Kommutativität keine Rolle, aber für den Grenzwert der resultierenden Folgen.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Jede natürliche Zahl größer als 1 ist ein Produkt von Primzahlen.

- Induktionsbasis (I.B.): 2 ist Primzahl, also ist $P(2)$ wahr (Konvention).
- Induktionsschritt (I.S.): Fixiere $n \geq 2$. Dann:
  - I.A.: Für $2 \leq i \leq n$ ist i ein Produkt von Primzahlen.
  - I.Beh.: $n+1$ ist ein Produkt von Primzahlen.
  - **Fall 1:** $n+1$ ist eine Primzahl. Dann gilt $P(n+1)$
  - **Fall 2:** $n+1$ ist keine Primzahl.

    Dann gibt es $2 \leq k, m < n+1$ mit $n+1 = km$.

    Aus der I.A. folgt, dass k, m jeweils Produkte aus Primzahlen sind.

    Dann ist $km = n+1$ auch ein Produkt aus Primzahlen. D.h. $P(n+1)$ ist wahr.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Induktionsbeweise: Parkettierung (Satz von Golomb)

Für alle $n \in \mathbb{N}$ kann jedes Gitter der Größe $2^n \times 2^n$, in dem ein beliebiges Feld fehlt, mit L-förmigen Kacheln (bestehend aus jeweils 3 Feldern) parkettiert werden.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Beispiel: Fibonacci-Zahlen

Def Fibonacci-Zahlen:

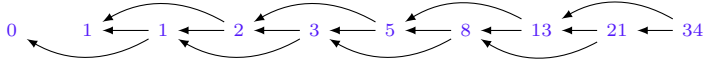
$$F_0 := 0 \quad F_1 := 1 \quad F_{n+2} := F_{n+1} + F_n$$

z.B.:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, \ldots$$

F_n: „Anzahl erwachsener Kaninchenpaare nach n Monaten“.

(Annahme: Jedes erw. Paar erzeugt je Monat ein Jungtierpaar, welches nach einem Monat erwachsen wird; Zuwachs an erw. Paaren in Monat $n+2$ daher $F_{n+2} - F_{n+1} = F_n$.)



Zeige, dass für alle $n \in \mathbb{N}_0$: (Übung)

$$1.\ F_n \leq 2^n \qquad 2.\ F_n \leq \frac{2 \cdot \Phi^n}{\sqrt{5}} \qquad 3.\ F_n = \frac{\Phi^n - (-\Phi)^{-n}}{\sqrt{5}}$$

wobei $\Phi := (1 + \sqrt{5})/2 \approx 1.61803\ldots$ (goldener Schnitt).

⚠ Beachte, dass F_{n+2} aus F_n und F_{n+1} hervorgeht, d.h. es werden Annahmen bzgl. F_n und F_{n+1} benötigt.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Beispiel: Untere Schranke für die Fibonacci-Zahlen

- Betrachte den Anfang der Folge:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, \ldots$$

Vergleiche mit $2^{n/2}$ (abgerundet auf 2 Nachkommastellen):

$$1.0, 1.41, 2.0, 2.82, 4.0, 5.65, 8.0, 11.31, 16.0, 22.62, \ldots$$

Rate/behaupte: $F_n \geq 2^{(n-2)/2}$ für alle $n \geq 1$.

- Beweis mittels starker Induktion für alle $n \geq 1$:

I.B.: $F_1 \overset{\text{Def.}}{=} 1 \geq 2^{-1/2} \approx 0,71$

$F_2 \overset{\text{Def.}}{=} 0 + 1 = 1 \geq 2^{0/2} = 1$

I.S.: Fixiere beliebiges $n \geq 1$.

I.A.: Für das fixierte n gilt $F_k \geq 2^{(k-2)/2}$ für alle $k = 1, \ldots, n+1$.

I.Beh.: Für das fixierte n gilt auch $F_{n+2} \geq 2^{n/2}$.

Beweis: Wegen $n \geq 1$ sind F_n und F_{n+1} durch die I.A. abgedeckt, also

$$F_{n+2} \overset{\text{Def.}}{=} F_{n+1} + F_n \overset{\text{I.A.}}{\geq} 2^{(n-1)/2} + 2^{(n-2)/2} \geq 2^{n/2} \underbrace{(2^{-1/2} + 2^{-1})}_{\geq 1} \geq 2^{n/2}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Mittels Induktion zeigt man sehr einfach (Übung):

Lem Für jede endliche Menge A gilt $|2^A| = 2^{|A|}$.

- Skizze des Induktionsschritts:
  Fixiere beliebiges $a \in A$. Dann partitioniere 2^A mittels a:

$$P_a := \{B \subseteq A \mid a \in B\} \qquad P_{-a} := \{B \subseteq A \mid a \notin B\}$$

  Dann $|P_a| = |P_{-a}|$ und $|P_{-a}| = |2^{A-\{a\}}| \overset{\text{I.A.}}{=} 2^{|A|-1}$.

Lem Für jedes $n \in \mathbb{N}$ und alle nicht-leeren, endlichen Mengen $A_1, \ldots, A_n$ gilt

$$|A_1 \times \ldots \times A_n| = |A_1| \cdots |A_n|$$

Insbesondere $|A^n| = |A|^n$ für $A := A_1 = \ldots = A_n$ (auch für $n = 0$).

- Skizze des Induktionsschritts:
  Für jedes $a \in A_{n+1}$ sei

$$B_a := \{(a_1, \ldots, a_n, a) \mid (a_1, \ldots, a_n) \in A_1 \times \ldots \times A_n\}$$

  Dann $|B_a| = |A_1 \times \ldots \times A_n| \overset{\text{I.A.}}{=} |A_1| \cdots |A_n|$ für jedes $a \in A_{n+1}$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

In jeder Menge von $n \geq 1$ Pferden haben alle Pferde die gleiche Farbe.

- I.B.: $P(1)$ ist wahr, da ein Pferd eine Farbe hat.
- I.S.: Annahme: $P(n)$ ist wahr.
  - Betrachte eine Menge $h_1, \ldots h_n, h_{n+1}$ von $n+1$ Pferden.
  - Nach der I.A. haben die ersten n Pferde $h_1, \ldots, h_n$ die gleiche Farbe.
  - Nach der I.A. haben auch $h_2, \ldots, h_{n+1}$ die gleiche Farbe.
  - Da $\{h_1, \ldots, h_n\} \cap \{h_2, \ldots, h_{n+1}\} \neq \emptyset$ haben alle $n+1$ Pferde die gleiche Farbe.
    (Insbesondere gilt $h_2 \in \{h_1, \ldots, h_n\} \cap \{h_2, \ldots, h_{n+1}\}$.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

2 Grundlagen

Grundbegriffe der Logik

Mengen

Folgen, Tupel und Wörter

Die natürlichen Zahlen, Induktionsbeweise

Relationen

Binäre Relationen

Binäre Relationen auf einer Menge (Endorelationen)

Reflexiv-transitiver Abschluss

Eigenschaften von Endorelationen

Äquivalenzrelationen

Ordnungsrelationen

Funktionen

Kardinalität: Vergleiche von unendlichen Mengen

Zusammenfassung

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Gegeben Mengen $A_1, A_2, \ldots, A_k$ nennt man $R \subseteq A_1 \times A_2 \times \ldots \times A_k$ eine (k-stellige) Relation oder eine Relation der Stelligkeit/Arität k.

Gilt $(a_1, a_2, \ldots, a_k) \in R$, dann sagt man, dass $a_1, a_2, \ldots, a_k$ bzgl. R in Relation/Beziehung stehen.

- Konzeptuelle Grundlage für z.B. relationale Datenbanken:

A_{Id}	A_{Nachname}	A_{Vorname}
1	Man	Spider
2	Brot	Bernd
3	Woman	Wonder
4	Gaga	Lady

R

A_{Id}	$A_{\text{Matrikelnummer}}$
1	3141
2	271828
3	1701
4	3694

S

A_{Id}	A_{Beruf}
1	h
2	b
3	h
4	a

T

Zusätzliche Operationen: Join (bedingte Konkat.) $\bowtie$ und Projektion π:

- $R \bowtie_{i_0=j_0} S := \{(r_1, \ldots, r_k, s_1, \ldots, s_l) \mid (r_i)_{i\in[k]} \in R, (s_j)_{j\in[l]} \in S, r_{i_0} = s_{j_0}\}$
- $\pi_{i_1,\ldots,i_l}(R) := \{(r_{i_1}, \ldots, r_{i_l}) \mid (r_i)_{i\in[k]} \in R\}$
- Bsp.: $\pi_3(R \bowtie_{1=1} (T \bowtie_{2=1} \{(\text{h})\})) = \{(\text{Wonder}), (\text{Spider})\}$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Binäre Relationen

Def Im Weiteren: binäre (2-stellige) Relationen $R \subseteq A \times B$ mit $A \neq \emptyset \neq B$.

- ▷ Grundlage für verzeigerte Datenstrukturen wie z.B. Listen und Bäume.
- ▷ Falls $A = B$, dann „binäre Relation auf A“ oder „Endorelation“.
- ▷ $aR := \{b \in B \mid (a,b) \in R\}$: Nachfolger von $a \in A$ bzgl. R
  $Rb := \{a \in A \mid (a,b) \in R\}$: Vorgänger von $b \in B$ bzgl. R

▷ Beispiele:

- ▷ „Vater-von“, „Elternteil-von“, „Kind-von“, „Nachkomme-von“, „Vorfahre-von“
  „notwendig-für“, „hinreichend-für“, „abhängig-von“, „input-output“, „zeigt-auf“
  $\{(x,y) \in A \times B \mid P(x,y)\}$ für ein binäres Prädikat P.
- ▷ Relationen auf $\mathbb{R}$ (und $\mathbb{N}_0, \mathbb{Z}, \mathbb{Q}$):
  „kleiner-gleich“ $\leq$, „(strikt) kleiner“ $<$, „gleich“ $=$, „ungleich“ $\neq$
- ⚠ Notationsproblem: $= \; = \{(x,x) \mid x \in \mathbb{R}\} \subseteq \mathbb{R} \times \mathbb{R}$ (?!)
  Daher: $\leq_{\mathbb{R}}$ „kleiner-gleich auf $\mathbb{R}$“ bzw. $=_{\mathbb{N}}$ „Identität auf $\mathbb{N}$“ usw.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Binäre Relationen

Def Im Weiteren: binäre (2-stellige) Relationen $R \subseteq A \times B$ mit $A \neq \emptyset \neq B$.

- ▷ Grundlage für verzeigerte Datenstrukturen wie z.B. Listen und Bäume.
- ▷ Falls $A = B$, dann „binäre Relation auf A“ oder „Endorelation“.
- ▷ $aR := \{b \in B \mid (a,b) \in R\}$: Nachfolger von $a \in A$ bzgl. R
  $Rb := \{a \in A \mid (a,b) \in R\}$: Vorgänger von $b \in B$ bzgl. R

Def Identität(srelation) auf einer Menge A: $\mathsf{Id}_A := \{(a,a) \mid a \in A\}$

Def Infixnotation: „$a\ R\ b$“ für „$(a,b) \in R$“; und „$a\ \not{R}\ b$“ für „$(a,b) \notin R$“

- ▷ z.B.: $3 \leq_{\mathbb{N}} 5$ statt $(3,5) \in \leq_{\mathbb{N}}$.

Def Transponierte (konverse, inverse, Umkehr-) Relation $R^\top := \{(b,a) \mid (a,b) \in R\}$.

- ▷ z.B.: $\leq_{\mathbb{R}}^\top = \geq_{\mathbb{R}}$; „hinreichend-für$^\top$ = notwendig-für“.

Def Für später: Sei $R \subseteq A \times B$ eine binäre Relation. R ist eine ...

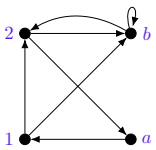
... partielle Funktion, falls jedes $a \in A$ höchstens einen Nachfolger hat ($|aR| \leq 1$).

... (totale) Funktion, falls jedes $a \in A$ genau einen Nachfolger hat ($|aR| = 1$).

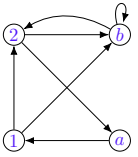
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Binäre Relationen: Graphen

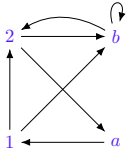
Bsp $E = \{(1,b),(2,b),(2,a),(1,2),(b,b),(b,2),(a,1)\}$ mit $V = \{1,2,a,b\}$



oder



oder

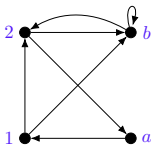


- Veranschaulichung von binären Relationen als gerichtete Graphen.
- Formale Definition von Graph später in DS, für jetzt:
  Gegeben (endliche) Relation $E \subseteq A \times B$ setze $V = A \cup B$:
  - Elemente von V auch „Knoten“ (engl.: „node/vertex“) genannt.
  - Elemente von E auch „Kanten“ (engl.: „edge“) genannt.
  - „Male“ für jeden Knoten $v \in V$ einen „Punkt mit Namen v“.
  - „Male“ für jede Kante $(s,t) \in E$ einen Pfeil vom „Punkt s“ zu dem „Punkt t“.
    Falls $s = t$, „male Schleife bei s“. (s für „source“, t für „target“.)

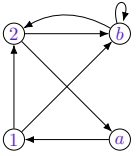
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Binäre Relationen: Graphen

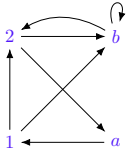
Bsp $E = \{(1,b),(2,b),(2,a),(1,2),(b,b),(b,2),(a,1)\}$ mit $V = \{1,2,a,b\}$



oder



oder



Def Eine Folge von Knoten $(v_0,\ldots,v_l) \in V^{l+1}$ ist ein

(i) Pfad (der Länge l) **:gdw** $(v_{i-1},v_i) \in E$ für alle $i \in [l]$.

(ii) Zyklus (Kreislauf) **:gdw** $(v_0,\ldots,v_l)$ ist Pfad mit $v_0 = v_l, l > 0$.

- ▷ v_i: Position zur Zeit i; in l Schritten entlang der Kanten von v_0 nach v_l.
- ▷ Länge entspricht Anzahl Schritten/traversierten Kanten.
- ▷ Häufig auch nur $v_0,\ldots,v_l$ oder $v_0 \ldots v_l$, falls nicht mehrdeutig.

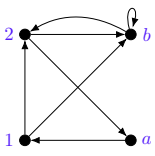
▷ Z.B.: $(2,a,1,b)$ Pfad der Länge 3; $(1,b,2,a,1)$ Pfad (Zyklus) der Länge 4; (b,b) Pfad (Zyklus) der Länge 1; (1) Pfad der Länge 0 (kein Zyklus); $(1,b,2,1)$ ist kein Pfad.

▷ s kann t erreichen, falls es einen Pfad endlicher Länge von s nach t gibt.

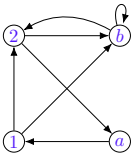
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Binäre Relationen: Graphen

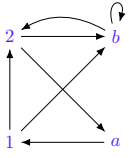
Bsp $E = \{(1,b),(2,b),(2,a),(1,2),(b,b),(b,2),(a,1)\}$ mit $V = \{1,2,a,b\}$



oder



oder



⚠ Graphen/Relationen haben i.A. keine geometrische Bedeutung:

- ▷ Positionierung der Knoten ist daher prinzipiell irrelevant, und Kanten dürfen auch gekrümmt sein. Aber möglichst keine Überdeckungen, -lappung, -schneidungen (siehe planare Graphen später).

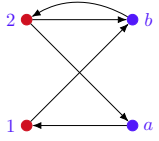
  Siehe auch Graphisomorphismus später.

- ▷ Genaue Darstellung von Anwendung abhängig.

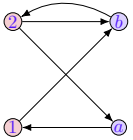
▷ Lassen sich die Knoten so mit zwei Farben einfärben, dass alle Kanten nur Knoten unterschiedlicher Farbe verbinden, dann positioniere alle Knoten der einen Farbe links, die anderen rechts (bipartiter Graph).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

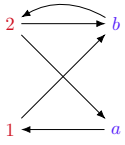
- Bsp.: $E = \{(1,b),(2,b),(2,a),(b,2),(a,1)\}$ mit $V = \{1,2,a,b\}$



oder



oder



⚠ Graphen/Relationen haben i.A. keine geometrische Bedeutung:

- Positionierung der Knoten ist daher prinzipiell irrelevant, und Kanten dürfen auch gekrümmt sein. Aber möglichst keine Überdeckungen, -lappung, -schneidungen (siehe planare Graphen später).

  Siehe auch Graphisomorphismus später.

- Genaue Darstellung von Anwendung abhängig.

- Lassen sich die Knoten so mit zwei Farben einfärben, dass alle Kanten nur Knoten unterschiedlicher Farbe verbinden, dann positioniere alle Knoten der einen Farbe links, die anderen rechts (bipartiter Graph).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

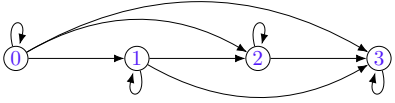
Kleiner-gleich

Def $\leq_{\mathbb{N}_0} := \{(a, a+b) \mid a, b \in \mathbb{N}_0\}$

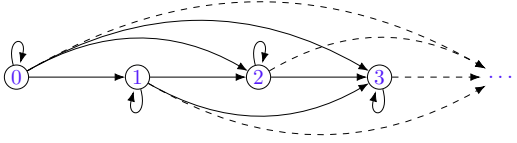
$\leq_{\mathbb{Z}} := \{(a, a+b) \mid a \in \mathbb{Z}, b \in \mathbb{N}_0\}$

$\leq_{\mathbb{Q}} := \{(\frac{a}{b}, \frac{c}{d}) \mid a, c \in \mathbb{Z}, b, d \in \mathbb{N}, ad \leq_{\mathbb{Z}} bc\}$

- Beispiel: $\{(a, b) \in \mathbb{Z}_4^2 \mid a \leq_{\mathbb{N}_0} b\}$ ($\leq_{\mathbb{N}_0}$ eingeschränkt auf $\mathbb{Z}_4$)



- Beispiel: Skizze zu $\leq_{\mathbb{N}_0} \subseteq \mathbb{N}_0 \times \mathbb{N}_0$



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Sei $R \subseteq A \times B$ mit $A = \{a_1, \ldots, a_m\}$ und $B = \{b_1, \ldots, b_n\}$ ($m = |A|, n = |B|$).
Die Matrix-Darstellung von R bzgl. der gegebenen Aufzählung ist:

$$M_R = (r_{i,j})_{i \in [m], j \in [n]} = \begin{pmatrix} r_{1,1} & \ldots & r_{1,j} & \ldots & r_{1,n} \\ \vdots & & \vdots & & \vdots \\ r_{i,1} & \ldots & r_{i,j} & \ldots & r_{i,n} \\ \vdots & & \vdots & & \vdots \\ r_{m,1} & \ldots & r_{m,j} & \ldots & r_{m,n} \end{pmatrix}$$

mit $r_{i,j} := 1$ falls $(a_i, b_j) \in R$ sonst $r_{i,j} := 0$.

▷ Bsp: $A = \{1, 2, 3\}$, $B = \{a, b, c\}$, $R = \{(1, a), (1, b), (2, c), (3, a)\}$

$$M_R = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} \qquad M_R^\top := M_{R^\top} = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

⚠ Matrix hängt von den fixierten Aufzählungen $(a_i)_{i \in [m]}$ und $(b_j)_{j \in [n]}$ ab.
Verschiedene Aufzählungen (Anordnungen) ergeben verschiedene Matrixdarstellungen derselben Relation R! (Wir nehmen an, dass jedes $a \in A$ genau einmal in der Aufzählung $a_1, \ldots, a_m$ vorkommt; entsprechend für B.)

No commercial use, sale or duplication, http://www.logic.at/staff/scheucher/discmath.html, 2018, Institute of Computer Science

Relationales Produkt (Komposition von Relationen)

Def Sind $R \subseteq A \times B$ sowie $S \subseteq C \times D$ binäre Relationen, dann ist ihr relationales/relatives Produkt $RS \subseteq A \times D$ definiert durch

$$RS = \{(a, d) \mid a \in A, x \in B \cap C, d \in D, (a, x) \in R, (x, d) \in S\}$$

(Mittels Join und Projektion: $RS = \pi_{1,4}(R \bowtie_{2=1} S)$.)

⚠ Anmerkungen:

- Die Operation selbst wird als relationale/relative Komposition bezeichnet, das eigentliche Ergebnis als relationales/relatives Produkt.

  (Auch: „relative Multiplikation“ für die Operation; manchmal „;“ als Operatorsymbol wie in Programmiersprachen, manchmal $\circ$ mit R, S vertauscht, d.h. $RS = R; S = S \circ R$.)

- Im Weiteren einfach „relationales Produkt“ zur Unterscheidung zur verwandten Komposition von Funktionen. (Funktionen sind binäre Relationen; $f \circ g = gf$).

- Unterscheide RS von $R \times S = \{((a, b), (c, d)) \mid (a, b) \in R, (c, d) \in S\}$.

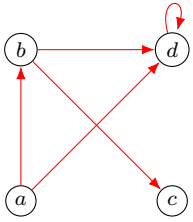
  R^k wird umdefiniert zu „k-faches relationales Produkt von R mit sich selbst“.

- Es ist assoziativ, aber nicht kommutativ (siehe folgende Folien).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

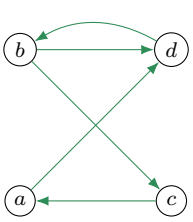
Relationales Produkt

R



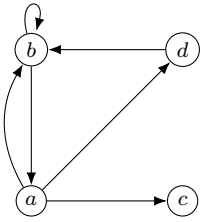
	a	b	c	d
a	0	1	0	1
b	0	0	1	1
c	0	0	0	0
d	0	0	0	1

S



	a	b	c	d
a	0	0	0	1
b	0	0	1	1
c	1	0	0	0
d	0	1	0	0

RS



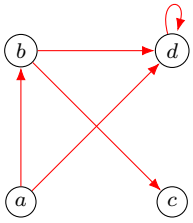
	a	b	c	d
a	0	1	1	1
b	1	1	0	0
c	0	0	0	0
d	0	1	0	0

$$M_X = \left(x_{i,j}\right)_{i,j} \quad M_Y = \left(y_{i,j}\right)_{i,j} \quad M_{XY} = \left(\max_u \left(x_{s,u} \cdot y_{u,t}\right)\right)_{s,t} =: M_X \cdot M_Y$$

- ▷ Vergleiche Zeile des Startknotens s in M_X mit Spalte des Zielknotens t in M_Y.

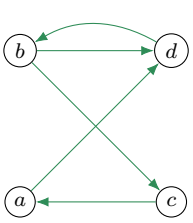
Relationales Produkt

R



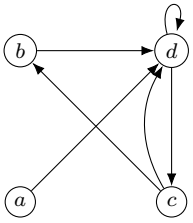
$$\begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \end{array}$$

S



$$\begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \end{array}$$

SR



$$\begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix} \end{array}$$

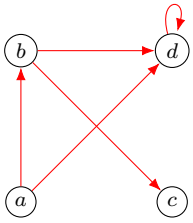
$$M_X = \left(x_{i,j}\right)_{i,j} \quad M_Y = \left(y_{i,j}\right)_{i,j} \quad M_{XY} = \left(\max_u \left(x_{s,u} \cdot y_{u,t}\right)\right)_{s,t} =: M_X \cdot M_Y$$

- Vergleiche Zeile des Startknotens s in M_X mit Spalte des Zielknotens t in M_Y.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

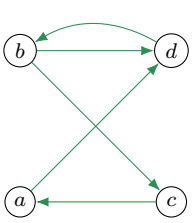
Relationales Produkt

R



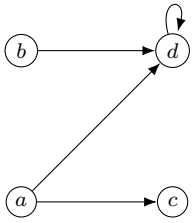
$$\begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \end{array}$$

S



$$\begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \end{array}$$

RR



$$\begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \end{array}$$

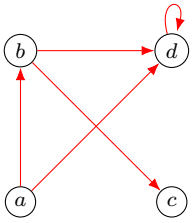
$$M_X = \left(x_{i,j}\right)_{i,j} \quad M_Y = \left(y_{i,j}\right)_{i,j} \quad M_{XY} = \left(\max_u \left(x_{s,u} \cdot y_{u,t}\right)\right)_{s,t} =: M_X \cdot M_Y$$

- Vergleiche Zeile des Startknotens s in M_X mit Spalte des Zielknotens t in M_Y.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

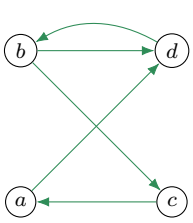
Relationales Produkt

R



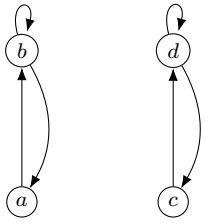
$$\begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \end{array}$$

S



$$\begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \end{array}$$

SS



$$\begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix} \end{array}$$

$$M_X = \left(x_{i,j}\right)_{i,j} \quad M_Y = \left(y_{i,j}\right)_{i,j} \quad M_{XY} = \left(\max_u \left(x_{s,u} \cdot y_{u,t}\right)\right)_{s,t} =: M_X \cdot M_Y$$

- Vergleiche Zeile des Startknotens s in M_X mit Spalte des Zielknotens t in M_Y.

Relationales Produkt: Rechenregeln

Def Für eine Menge X sei $\mathsf{Id}_X := \{(x, x) \mid x \in X\}$ (Identitätsrelation).

Lem Für alle $R, R_1, R_2 \subseteq A \times B$, $S, S_1, S_2 \subseteq B \times C$, $T \subseteq C \times D$ gilt:

- Assoziativ: $RST := R(ST) \equiv (RS)T$
- Links-/Rechtsneutrale Elemente: $\mathsf{Id}_A R \equiv R \equiv R\mathsf{Id}_B$
- Distributiv über $\cup$:

$$(R_1 \cup R_2)S \equiv (R_1 S) \cup (R_2 S) \text{ und } R(S_1 \cup S_2) \equiv (RS_1) \cup (RS_2)$$

- „Null“ (Annihilator): $\emptyset R = \emptyset = R\emptyset$
- Monotonie: Falls $R_1 \subseteq R_2$ und $S_1 \subseteq S_2$, dann $R_1 S_1 \subseteq R_2 S_2$.

(Beweise zur Übung.)

⚠ Im Allgemeinen jedoch

- nicht distributiv über $\cap$ z.B. $R = \{(0, 1), (0, 2)\}$, $S_1 = \{(1, 3)\}$, $S_2 = \{(2, 3)\}$.
- nicht kommutativ z.B. $R = \{(0, 1)\}$, $S = \{(1, 2)\}$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Endorelationen

Def Im Spezialfall $R \subseteq A \times A$ ($A \neq \emptyset$):

- ▷ „R auf A“, „Endorelation“ (auch „homogene Relation“)
- ▷ Falls $R \subseteq A \times B$, auch $R \subseteq (A \cup B) \times (A \cup B)$.

k-faches relationales Produkt von R mit sich selbst (induktive Definition):

$$R^k := \begin{cases} \mathsf{Id}_A := \{(a,a) \mid a \in A\} & \text{falls } k = 0 \\ R^{k-1}R & \text{falls } k > 0 \end{cases}$$

⚠ Notation „überladen“: R^k für $R \subseteq A \times A$ **vs** $A^k = \{(a_1, \ldots, a_k) \mid a_i \in A\}$ für allgemeine Menge A **vs** $\Sigma^k = \{a_1 \ldots a_k \mid a_i \in \Sigma\}$ für Alphabet Σ.

Lem $(a,b) \in R^k$ gdw es gibt einen Pfad der Länge k von a nach b (Induktion).

Lies „$(a,b) \in R^k$“ als „b ist von a in (genau) k Schritten erreichbar“.

Def $R^{\leq k} := \bigcup_{i=0}^{k} R^i$ („höchstens k Schritte“); $R^{<k} := \bigcup_{i=0}^{k-1} R^i$ („weniger als k Schritte“)

Fkt $R^{\leq k} = (\mathsf{Id}_A \cup R)^k$ („auf der Stelle treten“).

Lem Falls $n = |A|$, dann gilt $R^{<n} = R^{<k}$ für alle $k \geq n$ (Induktion).

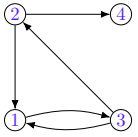
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Erreichbarkeit und Pfade

Lem $(a, b) \in R^k$ gdw es gibt einen Pfad der Länge k von a nach b

Lem Falls $n = |A|$, dann gilt $R^{<n} = R^{<k}$ für alle $k \geq n$ (Induktion).

Bsp Sei R gegeben durch:



$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \end{array}$$

Pfade vs. Erreichbarkeit:

- ▷ Pfade der Länge 0: $\{(1), (2), (3), (4)\} = A^1$

vs $R^0 = \{(1,1), (2,2), (3,3), (4,4)\} = \mathsf{Id}_A$

- ▷ Pfade der Länge 1: $\{(1,3), (2,1), (2,4), (3,1), (3,2)\} = R$

vs $R^1 = \{(1,3), (2,1), (2,4), (3,1), (3,2)\} = R$

- ▷ Pfade der Länge 2: $\{(1,3,1), (1,3,2), (2,1,3), (3,1,3), (3,2,1), (3,2,4)\}$

vs $R^2 = \{(1,1), (1,2), (2,3), (3,3), (3,1), (3,4)\}$

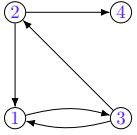
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Erreichbarkeit und Pfade

Lem $(a, b) \in R^k$ gdw es gibt einen Pfad der Länge k von a nach b

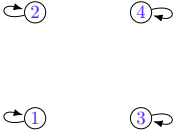
Lem Falls $n = |A|$, dann gilt $R^{<n} = R^{<k}$ für alle $k \geq n$ (Induktion).

Bsp Sei R gegeben durch:

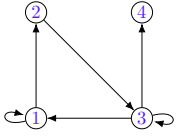


$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \end{array}$$

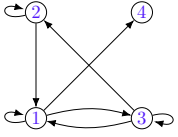
v.l.n.r.: R^0, R^2, R^3



$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



$$\begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$



$$\begin{pmatrix} 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

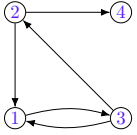
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Erreichbarkeit und Pfade

Lem $(a, b) \in R^k$ gdw es gibt einen Pfad der Länge k von a nach b

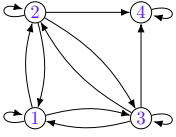
Lem Falls $n = |A|$, dann gilt $R^{<n} = R^{<k}$ für alle $k \geq n$ (Induktion).

Bsp Sei R gegeben durch:

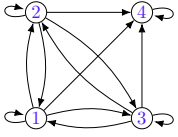


$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \end{array}$$

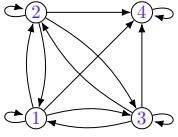
v.l.n.r.: $R^{\leq 2}$, $R^{\leq 3}$, $R^{\leq 4}$



$$\begin{pmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



$$\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



$$\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Lem Für alle $R \subseteq A \times A$, für alle $a, b \in A$ und für alle $k \in \mathbb{N}_0$:
$(a, b) \in R^k$ gdw es gibt einen Pfad der Länge k von a nach b (bzgl. R).
(„erreichbar in genau k Schritten“ vs. „Pfad bestehend aus genau k Schritten“)

- Beweis mittels Induktion nach $k \in \mathbb{N}_0$:
  - I.B.: Für $k = 0$
    LS: $R^0 = \mathsf{Id}_A = \{(a, a) \mid a \in A\}$.
    RS: Für jedes $a \in A$ ist (a) ein Pfad der Länge 0.
  - I.S.: Sei $k \in \mathbb{N}_0$ beliebig fixiert.
    I.A.: $R^k = \{(a_0, a_k) \mid \text{Es gibt einen Pfad } (a_0, \ldots, a_k) \text{ der Länge } k\}$.
    I.Beh.: $R^{k+1} = \{(a_0, a_{k+1}) \mid \text{Es gibt Pfad } (a_0, \ldots, a_{k+1}) \text{ der Länge } k+1\}$.
    Beweis: $(a_0, a_{k+1}) \in R^{k+1}$
    gdw es gibt $a_k \in A$ mit $(a_0, a_k) \in R^k$ und $(a_k, a_{k+1}) \in R$ ($R^{k+1} = R^k R$)
    gdw es gibt einen Pfad $(a_0, \ldots, a_k)$ der Länge k und $(a_k, a_{k+1}) \in R$ (I.A.)
    gdw es gibt einen Pfad $(a_0, \ldots, a_k, a_{k+1})$ der Länge $k+1$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Erreichbarkeit im endlichen Fall

Lem Falls $n = |A|$, dann gilt $R^{<n} = R^{<k}$ für alle $k \geq n$ ($A \neq \emptyset$).

(„Alles, was erreichbar ist, ist bereits in $< n$ Schritten erreichbar.“)

▷ Beweis: Wir zeigen, es gibt kein kleinstes Gegenbeispiel:

▷ Sei $k_0 \geq n$ minimal mit $R^{<n} \neq R^{<k_0}$.

Dann muss immer noch $n < k_0$ und damit $n \leq k_0 - 1$ gelten.

Da k_0 das kleinste Gegenbeispiel ist, gilt $R^{<n} = R^{<k_0-1}$.

Daher $R^{<n} \subsetneq R^{<k_0} = R^{<k_0-1} \cup R^{k_0-1} = R^{<n} \cup R^{k_0-1}$.

Fixiere ein bel. $(a_0, a_{k_0-1}) \in R^{k_0-1} \setminus R^{<n}$.

Dann gibt es einen Pfad $(a_0, a_1, \ldots, a_{k_0-1})$ bestehend aus $k_0 - 1$ Schritten.

Dieser Pfad besucht $(k_0 - 1) + 1 > n = |A|$ Knoten/Objekte:

d.h. es gibt „Zeitpunkte“ $0 \leq i < j \leq k_0 - 1$ mit $a_i = a_j$,

d.h. der „Kreislauf“ $(a_i, a_{i+1}, \ldots, a_j)$ ist unnötig.

Daher ist $(a_0, \ldots, a_i, a_{j+1}, \ldots, a_{k_0-1})$ ein Pfad der Länge $< k_0 - 1$.

(Wir entfernen mindestens einen Schritt, da $i < j$.)

Somit gilt bereits $(a_0, a_{k_0-1}) \in R^{<k_0-1} = R^{<n}$. Widerspruch.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Einschub: Starke Induktion+Widerspruch

- Ziel: Zeige, dass $P(n)$ wahr ist für alle $n \in \mathbb{N}_0$.

  Zeige stattdessen, dass es kein kleinstes Gegenbeispiel gibt.

  - (IB) Zeige direkt, dass $P(n)$ wahr ist für alle $n \leq n_0$ für ein konkretes $n_0 \in \mathbb{N}_0$.
    (D.h. zeige, dass $k > n_0$ für jedes Gegenbeispiel k gelten muss.)
  - (IS) Nimm an, es gibt ein $k \in \mathbb{N}_0$, für welches $P(k)$ falsch ist.
    Dann gibt es ein kleinstes $k_0 = \min\{k \in \mathbb{N}_0 \mid \text{ nicht } P(k)\}$.
    (Auf Grund der Wohlordnung von $\mathbb{N}_0$; siehe Ordnungsrelationen.)
    Zeige einen Widerspruch basierend auf den Annahmen, dass $P(k_0)$ falsch ist, aber ((IA)) $P(n)$ wahr für alle $n < k_0$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Reflexiv-transitiver Abschluss

Def Sei $R \subseteq A \times A$ eine Endorelation.

R ist reflexiv **:gdw** $\mathsf{Id}_A \subseteq R$ gilt. (D.h. jeder Knoten hat eine Schleife.)

R ist transitiv **:gdw** $RR \subseteq R$ gilt.

(D.h. wo man in zwei Schritten hinkommt, kommt man schon in einem hin.)

Der transitive Abschluss von R ist

$$R^+ := \bigcup_{k \in \mathbb{N}} R^k = R \cup RR \cup RRR \cup \ldots$$

(D.h. man ergänzt eine Kante für jeden Pfad positiver endlicher Länge.)

Der reflexiv-transitive Abschluss von R ist:

$$R^* := \mathsf{Id}_A \cup R^+ = \bigcup_{k \in \mathbb{N}_0} R^k = \mathsf{Id}_A \cup R \cup RR \cup RRR \cup \ldots$$

(D.h. man ergänzt eine Kante für jeden Pfad endlicher Länge.)

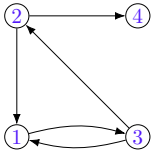
Lies „$(a, b) \in R^*$“ als „b ist von a aus erreichbar (in endlich vielen Schritten)“.

⚠ Unterscheide Σ^* für ein Alphabet Σ von R^* für $R \subseteq A \times A$.

(Siehe auch Kleene-Algebren nach partiellen Ordnungen.)

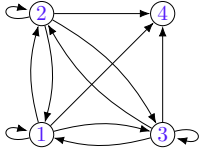
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 2017, TUM CIT Department of Computer Science

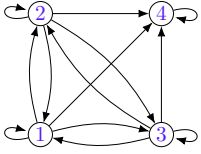
- Sei R gegeben durch:



$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \end{array}$$

- v.l.n.r. R^+, R^*:





Da $|A| = 4$, gilt $R^+ = R \cup R^2 \cup R^3 \cup R^4$ und $R^* = \mathsf{Id} \cup R^+ = \mathsf{Id} \cup R \cup R^2 \cup R^3$.

Schnellere Berechnung von R^*:

$$\left((\mathsf{Id} \cup R)^2\right)^2 = \left(\mathsf{Id} \cup R \cup R^2\right)^2 = \mathsf{Id} \cup R \cup R^2 \cup R^3 \cup R^4 = R^*$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Reflexiv-transitiver Abschluss: Eigenschaften

Lem Für alle $R, S, X \subseteq A \times A$ gilt:

- ⚠ Falls $|A| = n$ endlich, dann $R^* = R^{<n} = R^{\leq n-1} = (\mathsf{Id} \cup R)^{n-1} = (\mathsf{Id} \cup R)^n$.
- ⚠ Falls $\mathsf{Id}_A \subseteq X$ (reflexiv), $XX \subseteq X$ (transitiv) und $R \subseteq X$, dann $R^* \subseteq X^* = X$.

  $R^* = \bigcap\{X \subseteq A \times A \mid \mathsf{Id}_A \subseteq X, XX \subseteq X, R \subseteq X\}$.
- ⚠ Falls $XX \subseteq X$ (transitiv) und $R \subseteq X$, dann $R^+ \subseteq X$.

  $R^+ = \bigcap\{X \subseteq A \times A \mid XX \subseteq X, R \subseteq X\}$.
- ▷ $(R^\top)^* = (R^*)^\top$ und $(R^+)^\top = (R^\top)^+$.
- ▷ $(\mathsf{Id} \cup R)^+ = (\mathsf{Id} \cup R)^* = R^*$; falls R reflexiv: $R^+ = R^*$.

  $R^* = R^* R^* = (R^*)^* = (R^+)^* = (R^*)^+$

  $R^+ = R\,R^* = R^*\,R = R^*\,R^+ = R^+\,R^* = (R^+)^+$
- ▷ Falls $R \subseteq S$, dann $R^* \subseteq S^*$ und $R^+ \subseteq S^+$.
- ▷ Falls $RX = XS$, dann $R^*X = XS^*$.
- ▷ $(R \cup S)^* = R^*(SR^*)^* = (R^*S)^*R^*$

(Beweise zur Übung/siehe Übungen.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Reflexiv-transitiver Abschluss: MU-Puzzle

- $\Sigma = \{\mathrm{I}, \mathrm{M}, \mathrm{U}\}$, $A = \Sigma^*$, $R \subseteq A \times A$

$$R := \{\underbrace{(x\mathrm{I}, x\mathrm{IU})}_{(1)}, \underbrace{(\mathrm{M}x, \mathrm{M}xx)}_{(2)}, \underbrace{(x\mathrm{III}y, x\mathrm{U}y)}_{(3)}, \underbrace{(x\mathrm{UU}y, xy)}_{(4)} \mid x, y \in \Sigma^*\}$$

- Bsp.: $\mathrm{MI}\, R^+ \mathrm{MI}$:

$$\mathrm{MI} \xrightarrow{(2)} \mathrm{MII} \xrightarrow{(2)} \mathrm{MIIII} \xrightarrow{(1)} \mathrm{MIIIIU} \xrightarrow{(3)} \mathrm{MIUU} \xrightarrow{(4)} \mathrm{MI}$$

- Frage: Gilt $\mathrm{MI}\, R^* \,\mathrm{MU}$?
- Übung: Betrachte

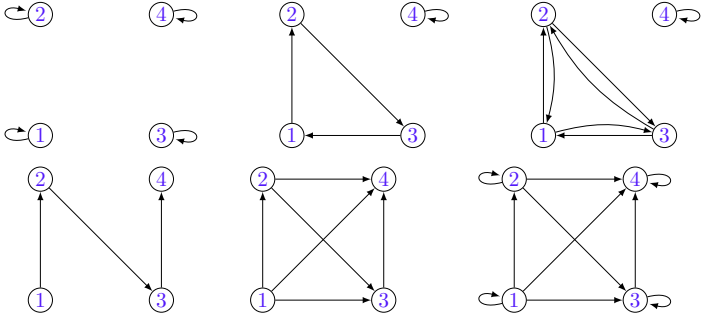
$$X := \{(x, y) \in \Sigma^* \times \Sigma^* \colon |x|_\mathrm{I} \text{ durch 3 teilbar } \mathbf{gdw} \ |y|_\mathrm{I} \text{ durch 3 teilbar}\}$$

  mit $|x|_\mathrm{I}$ die Anzahl der Vorkommen von I in $x \in \Sigma^*$.
  Zeige $R^* \subseteq X$ mittels $\mathsf{Id}_A \subseteq X$, $XX \subseteq X$, $R \subseteq X$.

- Anwendung: R^* „Bedeutung“ einer while-Schleife mit Schleifenrumpf R.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Endorelationen: Eigenschaften

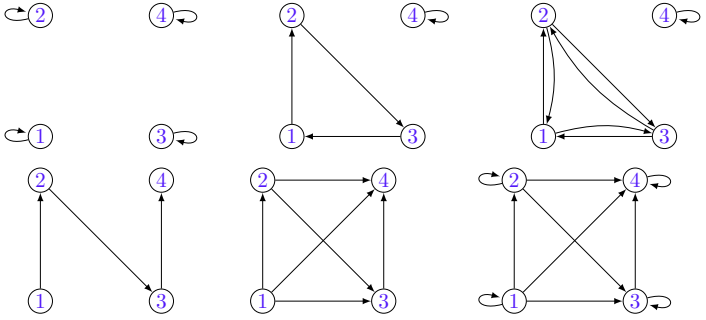


Def Eine Endorelation $R \subseteq A \times A$ ist

- reflexiv **:gdw** „für alle $a \in A$: $(a, a) \in R$“ ($\mathsf{Id}_A \subseteq R$)
  („Jeder Knoten hat eine Schleife“)
- irreflexiv **:gdw** „für alle $a \in A$: $(a, a) \notin R$“ ($\mathsf{Id}_A \cap R = \emptyset$)
  („Kein Knoten hat eine Schleife“)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Endorelationen: Eigenschaften



Def Eine Endorelation $R \subseteq A \times A$ ist

- symmetrisch **:gdw** „für alle $(s,t) \in R$: $(t,s) \in R$“ ($R^\top = R$)

  („Zwischen je zwei Knoten entweder beide Kanten oder gar keine Kante“)

- antisymmetrisch **:gdw** „für alle $(s,t) \in R$: falls $(t,s) \in R$, dann $s = t$“ ($R \cap R^\top \subseteq \mathsf{Id}_A$)

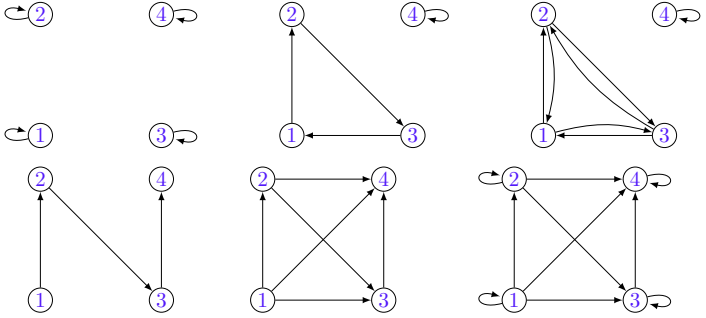
  („Zwischen zwei verschiedenen Knoten existiert höchstens eine Kante“)

- asymmetrisch **:gdw** „für alle $(s,t) \in R$: $(t,s) \notin R$“ ($R \cap R^\top = \emptyset$)

  (irreflexiv und antisymmetrisch; vgl. $\leq_{\mathbb{N}_0}$ vs. $\leq_{\mathbb{N}_0}$.)

No commercial use, but feel free to use it. Discrete Structures (KMU-SS-INFO-005), 2018 © MI CIT Department of Computer Science

Endorelationen: Eigenschaften



Def Eine Endorelation $R \subseteq A \times A$ ist

- ▷ transitiv **:gdw** „für alle $(s,t),(t,u) \in R$: $(s,u) \in R$“
  ($RR := \{(s,u) \mid (s,t),(t,u) \in R\} \subseteq R$)

  („Kommt man mit genau zwei Schritten von s nach u, dann auch mit genau einem.“)

- Weitere Beispiele:
  - $=_{\mathbb{Z}}$: reflexiv; symmetrisch, antisymmetrisch, transitiv
  - $\leq_{\mathbb{Z}}$: reflexiv, antisymmetrisch, transitiv
  - $<_{\mathbb{Z}}$: asymmetrisch (irreflexiv und antisymmetrisch), transitiv
  - $\neq_{\mathbb{Z}}$: irreflexiv, symmetrisch, nicht transitiv
  - $|_{\mathbb{Z}} \subseteq \mathbb{Z} \times \mathbb{Z}$ mit „$a|b$“ definiert durch „$\frac{b}{a} \in \mathbb{Z}$“ („$a$ teilt b“)
    nur transitiv
    Alternativ: „$a|b$“ definiert durch „$b = ka$ für ein $k \in \mathbb{Z}$“. (Eigenschaften?)
  - $|_{\mathbb{N}} \subseteq \mathbb{N} \times \mathbb{N}$ (Einschränkung von „teilt“ auf positive ganze Zahlen)
    reflexiv, antisymmetrisch, transitiv (beide Definitionen stimmen auf $\mathbb{N}$ überein)
  - $\equiv_m \subseteq \mathbb{Z} \times \mathbb{Z}$ mit „$a \equiv_m b$“ definiert durch „$m|(a-b)$“ für festes $m \in \mathbb{N}$
    reflexiv, symmetrisch, transitiv
  - $\subseteq$ auf $\mathcal{P}(\mathbb{Z})$: reflexiv, antisymmetrisch, transitiv
  - Kongruenzbegriff in der Geometrie: reflexiv, symmetrisch, transitiv

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Wichtige Endorelationen auf Wörtern (Zeichenketten, strings)

- Für Wörter $u, v \in \Sigma^*$
  - u ist ein Präfix von v (kurz: $u \preceq_p v$),
    falls es ein $w \in \Sigma^*$ mit $uw = v$ gibt.
    $\preceq_p$: reflexiv, antisymmetrisch, transitiv
  - u ist ein Suffix von v (kurz: $u \preceq_s v$),
    falls es ein $w \in \Sigma^*$ mit $wu = v$ gibt.
    $\preceq_s$: reflexiv, antisymmetrisch, transitiv
  - u ist ein Infix (Faktor) von v (kurz: $u \preceq_i v$),
    falls es $w, w' \in \Sigma^*$ mit $wuw' = v$ gibt.
    $\preceq_i$: reflexiv, antisymmetrisch, transitiv
  - u und v sind konjugiert (kurz: $u \cong_c v$),
    falls es $w, w' \in \Sigma^*$ mit $u = ww'$ und $v = w'w$ gibt.
    $\cong_c$: reflexiv, symmetrisch, transitiv

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Ordnungsrelationen vs. Äquivalenzrelationen

Def $R \subseteq A \times A$ ist eine (partielle) Ordnung auf A, falls

reflexiv, antisymmetrisch, transitiv ($\mathsf{Id}_A \cup RR \subseteq R$, $R \cap R^\top \subseteq \mathsf{Id}_A$).

- ▷ Z.B.: $\leq_{\mathbb{Z}}, |_{\mathbb{N}}, \subseteq, \preceq_p, \preceq_s, \preceq_i$.
- ▷ Formalisierung von „a vor b“:

  Z.B. um Daten anwendungsabhängig zu sortieren („(sort) key“).

Def $R \subseteq A \times A$ ist eine Äquivalenzrelation auf A, falls

reflexiv, symmetrisch, transitiv ($\mathsf{Id}_A \cup RR \subseteq R$, $R \cap R^\top = R$).

- ▷ Z.B.: $=_{\mathbb{Z}}, \equiv_m, \cong_c$, „Kongruenz von Dreiecken“.
- ▷ Formalisierung von „a und b sind austauschbar/gleichwertig“:

  Z.B. um „redundante“ Einträge in einer Datenbank zu entfernen.

- ▷ Erinnerung: Falls R reflexiv ($\mathsf{Id}_A \subseteq R$) und transitiv ($RR \subseteq R$), dann $R = R^*$.

⚠ Die genaue Bedeutung von „a und b sind gleichwertig“ und „a kommt vor b“ ist üblicherweise abhängig von der Anwendung/dem Kontext.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Äquivalenzrelationen

Def Sei $A \neq \emptyset$. $R \subseteq A \times A$ ist eine Äquivalenzrelation, falls

R reflexiv ($\mathrm{Id}_A \subseteq R$), symmetrisch ($R = R^\top$) und transitiv ($RR \subseteq R$) ist.

- ▷ $a, b \in A$ äquivalent/gleichwertig („gleichfarbig“) bzgl. R, falls $(a, b) \in R$.
- ▷ R partitioniert A in Teilmengen äquivalenter Objekte (Äquivalenzklassen).

▷ Beispiele: $=_{\mathbb{Z}}$ (identisch), $\equiv_m$ (kongruent modulo m), $\cong_c$ (konjugiert)

⚠ Insbesondere $\equiv_m \subseteq \mathbb{Z} \times \mathbb{Z}$ wichtig z.B. für Kryptographie.

- ▷ $a \equiv_m b$: „a und b haben denselben Rest bei Division durch m“

  Jedes $a \in \mathbb{Z}$ ist bzgl. $\equiv_m$ zu jedem $b \in a + m\mathbb{Z} := \{a + mz \mid z \in \mathbb{Z}\}$ äquivalent.

  Partition von $\mathbb{Z}$ bzgl. $\equiv_m$ in die Restklassen $\{a + m\mathbb{Z} \mid a \in \mathbb{Z}_m\}$

  „Gröbste“ Partition von $\mathbb{Z}$ bzgl. $\equiv_1$: $\{\mathbb{Z}\}$
- ▷ $a =_{\mathbb{Z}} b$: „a und b dieselbe/identische Zahl“

  Jedes $a \in \mathbb{Z}$ ist bzgl. $=_{\mathbb{Z}}$ nur zu sich selbst äquivalent.

  „Feinste“ Partition von $\mathbb{Z}$ bzgl. $=_{\mathbb{Z}}$: $\{\{x\} \mid x \in \mathbb{Z}\}$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Äquivalenzrelationen

Def Sei $R \subseteq A \times A$ eine Äquivalenzrelation. Dann:

$[a]_R := aR = \{a' \in A \mid (a, a') \in R\} = Ra$ Äquivalenzklasse von $a \in A$ bzgl. R.

$A/R := \{[a]_R \mid a \in A\}$ Quotient von A bzgl. R.

$X_R \subseteq A$ ist ein Repräsentantensystem bzgl. R
:gdw für jedes $a \in A$ gibt es genau ein $r \in X_R$ mit $r \in [a]_R$.

Fkt A/R ist eine Partition von A mit $R = \bigcup\{K \times K \mid K \in A/R\}$.

Fkt Jede Partition P von A definiert Äquivalenzrelation $\bigcup\{K \times K \mid K \in P\}$.

- Beweis, dass A/R Partition:
  - Aus $\mathrm{Id}_A \subseteq R$ folgt $a \in [a]_R$, also $[a]_R \neq \emptyset$.
  - Es gelte $a \in [b]_R = bR$, d.h. bRa, und wegen $R = R^\top$ auch aRb.
    Aus $RR \subseteq R$ folgt $[a]_R = aR \subseteq (bR)R = bR = [b]_R$.
    Aus $R = R^\top$ folgt $[b]_R \subseteq [a]_R$ und damit $[a]_R = [b]_R$.
    Damit: Falls $c \in [a]_R \cap [b]_R$, sofort $[c]_R = [a]_R = [b]_R$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Äquivalenzrelationen

Def Sei $R \subseteq A \times A$ eine Äquivalenzrelation. Dann:

$[a]_R := aR = \{a' \in A \mid (a, a') \in R\} = Ra$ Äquivalenzklasse von $a \in A$ bzgl. R.

$A/R := \{[a]_R \mid a \in A\}$ Quotient von A bzgl. R.

$X_R \subseteq A$ ist ein Repräsentantensystem bzgl. R
:gdw für jedes $a \in A$ gibt es genau ein $r \in X_R$ mit $r \in [a]_R$.

Fkt A/R ist eine Partition von A mit $R = \bigcup\{K \times K \mid K \in A/R\}$.

Fkt Jede Partition P von A definiert Äquivalenzrelation $\bigcup\{K \times K \mid K \in P\}$.

- Beispiel: $=_{\mathbb{Z}}$ **vs** $\equiv_3$ auf $\mathbb{Z}$:
  - $[1]_{=_{\mathbb{Z}}} = \{1\}$ **vs** $[1]_{\equiv_3} = 1 + 3\mathbb{Z} = \{\ldots, -5, -2, 1, 4, \ldots\}$
  - $\mathbb{Z}/=_{\mathbb{Z}} = \{\{x\} \mid x \in \mathbb{Z}\}$ **vs** $\mathbb{Z}/\equiv_3 = \{3\mathbb{Z}, 3\mathbb{Z} + 1, 3\mathbb{Z} + 2\}$
  - Repräsentantensystem: nur $\mathbb{Z}$ **vs** $\mathbb{Z}_3$ oder $[3]$ oder $\{-1, 0, 1\}$ oder ...
- Beispiel: Signed vs. unsigned n-bit integers verwenden beide $\equiv_{2^n}$, unterscheiden sich nur in den Repräsentanten: $\{-2^{n-1}, \ldots, -1, 0, 1, \ldots, 2^{n-1} - 1\}$ vs. $\mathbb{Z}_{2^n}$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Äquivalenzrelationen

Def Sei $R \subseteq A \times A$ eine Äquivalenzrelation. Dann:

$[a]_R := aR = \{a' \in A \mid (a, a') \in R\} = Ra$ Äquivalenzklasse von $a \in A$ bzgl. R.

$A/R := \{[a]_R \mid a \in A\}$ Quotient von A bzgl. R.

$X_R \subseteq A$ ist ein Repräsentantensystem bzgl. R
:gdw für jedes $a \in A$ gibt es genau ein $r \in X_R$ mit $r \in [a]_R$.

Fkt A/R ist eine Partition von A mit $R = \bigcup\{K \times K \mid K \in A/R\}$.

Fkt Jede Partition P von A definiert Äquivalenzrelation $\bigcup\{K \times K \mid K \in P\}$.

⚠ Eine ÄqRel R entspricht der Partition A/R von A in ÄqKlassen.

Eine ÄqKlasse $[a]_R$ entspricht einer „Farbe“, ihr Teilgraph ist vollständig:

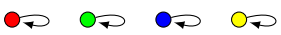
Alle Kanten zwischen Knoten derselben Farbe; ansonsten keine Kanten.

(Die ÄqRel R sieht nur die Farbe $[a]_R$, aber abstrahiert („vergisst“) von allen anderen Eigenschaften von $a \in A$. Der Wechsel von A nach A/R bedeutet, dass man alle Objekte in $[a]_R$ als identisch betrachtet (identifiziert), geometrisch entspricht das einer Projektion.)

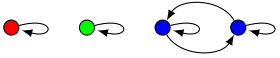
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Äquivalenzrelationen als Partitionen/Färbungen

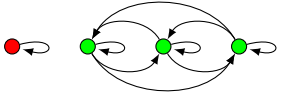
- Mögliche ÄqRel bzgl. $A = \{a, b, c, d\}$:



$A/R = \{\{a\}, \{b\}, \{c\}, \{d\}\}$



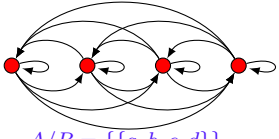
$A/R = \{\{a\}, \{b\}, \{c, d\}\}$



$A/R = \{\{a\}, \{b, c, d\}\}$



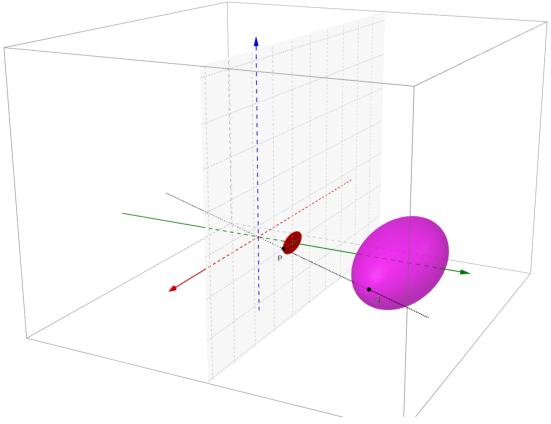
$A/R = \{\{a, b\}, \{c, d\}\}$



$A/R = \{\{a, b, c, d\}\}$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Definiere $\sim$ auf $\mathbb{R}^2 \times \mathbb{R}_{>0}$ mittels „$x \sim y$ **:gdw** $x = \lambda y$ für ein $\lambda \in \mathbb{R} \setminus \{0\}$“
  - Punkt (x_1, x_2, x_3) wird durch Projektion $(\frac{x_1}{x_3}, \frac{x_2}{x_3}, 1)$ repräsentiert.
  - Spezialfall: $\mathbb{Q} := (\mathbb{Z} \times \{0\} \times \mathbb{N})/\sim$ mit $(a, 0, b)$ repräsentiert durch $(a/b, 0, 1)$.
  - Siehe auch: reelle projektive Ebene, 3d-Projektion



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Weitere wichtige Äquivalenzrelationen:

  Äquivalenzen auf Termen/Formeln/Programmen

- $(x+y)^2$ und $x^2+2xy+y^2$ verschiedene Zeichenketten/arithmetische Terme, aber werten sich für alle reellen Zahlen $x, y \in \mathbb{R}$ stets zu demselben Wert aus, weswegen man (ungenau) einfach $(x+y)^2 = x^2+2xy+y^2$ schreibt.
- Entsprechend $(A \cap \overline{C}) \setminus B$ und $(A \cap \overline{(B \cup C)})$ im mengentheoretischen Sinn äquivalent, aber verschiedene Terme.
- Wichtig für die Informatik: Entscheiden, ob zwei (als Zeichenkette) verschiedene Programme dasselbe tun.

  Z.B. prototypische Implementierung vs. optimierte Implementierung.
- Ausblick: Im Allgemeinen kann man kein Programm schreiben, das die Äquivalenz von zwei anderen Programmen entscheidet.

  Siehe Einführung in die theoretische Informatik.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Äquivalenzrelationen aus gegebenen Endorelationen

Fkt $R \subseteq A \times A$ ist eine Äquivalenzrelation gdw. $R^* \subseteq R$ und $R^\top \subseteq R$.

Fkt Für alle $S \subseteq A \times A$ sind

- $(S \cup S^\top)^* = S^*(S^\top S^*)^*$ (ungerichteter u-w-Pfad)
- $S^* \cap (S^\top)^* = S^* \cap (S^*)^\top$ (u-w-Pfad und w-u-Pfad)

immer Äquivalenzrelationen mit

- Für jede Äquivalenzrelation $R \subseteq A \times A$ mit $S \subseteq R$ gilt $(S \cup S^\top)^* \subseteq R$.
- Für jede Äquivalenzrelation $R \subseteq A \times A$ mit $R \subseteq S^*$ gilt $R \subseteq S^* \cap (S^\top)^*$.
- $S^* \cap (S^\top)^* \subseteq S^* \subseteq (S \cup S^\top)^*$
- Falls $S = S^\top$, dann $(S \cup S^\top)^* = S^* = S^* \cap (S^\top)^*$.

Fkt Seien $R, S \subseteq A \times A$ beides Äquivalenzrelationen.

- Dann auch $R \cap S$ mit $[a]_{R \cap S} = [a]_R \cap [a]_S$.
- $R \subseteq S$ **gdw** $[a]_R \subseteq [a]_S$ für alle $a \in A$.

(Beweise zur Übung. $R \cup S$ ist i.A. nicht transitiv; $(R \cup S)^*$ ist wieder ÄqRel.)

Bsp (i) Falls $n|m$, dann $\equiv_m \subseteq \equiv_n$; (ii) $\equiv_n \cap \equiv_m = \equiv_{\mathsf{kgV}(m,n)}$.

NO COMMERCIAL USE, ONLY FOR USE IN DISCRETE STRUCTURES (LV.0015,1HHN0001), 26W, TU GRAZ, DEPARTMENT OF COMPUTER SCIENCE

Ordnungsrelationen

Def $R \subseteq A \times A$ ist eine (partielle) Ordnung(srelation) **:gdw**
R reflexiv ($\mathsf{Id}_A \subseteq R$), antisym. ($R^\top \cap R \subseteq \mathsf{Id}_A$) und transitiv ($RR \subseteq R$).

- „reflexiv und antisymmetrisch“: $R^\top \cap R = \mathsf{Id}_A$
- $a, b \in A$ bzgl. R vergleichbar, falls $(a, b) \in R$ oder $(b, a) \in R$,
  ansonsten sind a, b bzgl. R unvergleichbar.
- Falls a, b vergleichbar, dann Trichotomie („Dreiteilung“) wegen Antisym.:
  entweder ❶ $a = b$ oder ❷ $a \neq b$ und aRb oder ❸ $a \neq b$ und bRa.
- Ordnen vergleichbare Objekte an, unvergleichbare Objekte jedoch nicht.

Fkt Falls R partielle Ordnung, dann auch $R^\top$ (duale Ordnung).

- Beispiele: $\leq_{\mathbb{Z}}$, $|_{\mathbb{N}}$, $\subseteq$, $\preceq_p$, $\preceq_s$, $\preceq_i$
  - Für zwei beliebige $a, b \in \mathbb{Z}$ gilt mindestens $a \leq_{\mathbb{Z}} b$ oder $b \leq_{\mathbb{Z}} a$,
    d.h. bzgl. $\leq_{\mathbb{Z}}$ sind alle ganzen Zahlen vergleichbar.
  - Es gibt $a, b \in \mathbb{N}$, so dass weder $a|_{\mathbb{N}} b$ noch $b|_{\mathbb{N}} a$ gilt,
    z.B. sind alle Primzahlen bzgl. $|_{\mathbb{N}}$ unvergleichbar.
  - $\leq_{\mathbb{Z}}^\top = \geq_{\mathbb{Z}}$ und $a|_{\mathbb{N}}^\top b$ **gdw** „a Vielfaches von b“.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Ordnungsrelationen: linear/total, strikt

Def $R \subseteq A \times A$ totale/lineare Ordnung (auch: Totalordnung) **:gdw**
R partielle Ordnung und alle a, b bzgl. R vergleichbar ($A \times A = R \cup R^\top$),

- ▷ Z.B. $\leq, \geq$ auf $\mathbb{N}_0, \mathbb{Z}, \mathbb{Q}, \mathbb{R}$, aber nicht $|_{\mathbb{N}}$, $\subseteq$, $\preceq_p$.
- ▷ Eine totale Ordnung R ordnet alle Objekte „auf einer Linie“ an:
  Falls xRy und xRz, dann $y = z$ oder yRz oder zRy (Trichotomie)
  Vgl.: $x = y$ oder $x < y$ oder $x > y$ auf $\mathbb{R}$.

Def $R \subseteq A \times A$ ist strikte/starke/strenge Ordnung (auch: Striktordnung) **:gdw**
R asymmetrisch ($R^\top \cap R = \emptyset$) und transitiv ($RR \subseteq R$) ist.

- ▷ Erinnerung: „irreflexiv und antisymmetrisch = asymmetrisch“
- ▷ D.h. ist R eine (partielle) Ordnung, dann ist $R \setminus \mathrm{Id}_A$ eine Striktordnung.
  Bsp.: $<_{\mathbb{Z}} = \leq_{\mathbb{Z}} \setminus \mathrm{Id}_{\mathbb{Z}}$

Fkt Falls $R, S \subseteq A \times A$ beides partielle Ordnungen, dann auch $R \cap S$.
(Falls R PO. und S ÄqRel, dann ist $R \cap S$ eine PO.)
I.A. ist $R \cup S$ keine partielle Ordnung mehr (vgl. Äquivalenzrelationen).

Def Strikte schwache Ordnung: strikte Ordnung, Unvergleichbarkeit ist ÄqRel.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

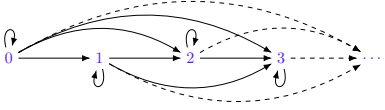
Ordnungsrelationen: Hasse-Diagramm

▷ Visualisierung einer (partiellen) Ordnung $R \subseteq A \times A$:
Man stellt nur eine möglichst „kleine“ Relation $S \subseteq R$ dar,
für die $S^* = R$ gilt. (S ist stets irreflexiv.)

Def Falls S irreflexiv ($S \cap \mathsf{Id}_A = \emptyset$) ist, keine transitiven Kanten enthält ($S^+ S \cap S = \emptyset$), und $S^* = R$ erfüllt, dann ergibt S ein Hasse-Diagramm von R.

▷ I.A. muss es kein Hasse-Diagramm geben (z.B. $(\mathbb{Q}, \leq)$, siehe Übungen).
Falls aber z.B. R endlich, dann berechne $S := R_s \setminus (R_s R_s)$ mit $R_s := R \setminus \mathsf{Id}_A$.
(D.h. entferne zuerst alle Schleifen, dann alle transitiven Kanten; beachte: R ist transitiv.)

Bsp Statt $\leq_{\mathbb{N}_0} = \{(i, i+j) \mid i, j \in \mathbb{N}_0\}$



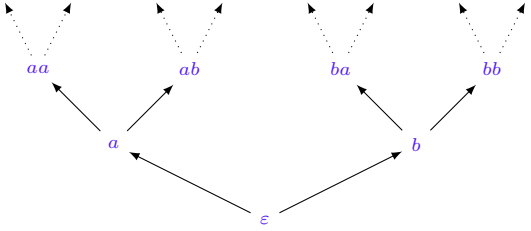
nur $S = \{(i, i+1) \mid i \in \mathbb{N}_0\}$ („kleinster Nachfolger“; siehe Peano)



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Hasse-Diagramm der Präfixordnung auf $\{a, b\}^*$:

  „Nachfolgerrelation“ auf Wörtern $\{(u, ua), (u, ub) \mid u \in \Sigma^*\}$



Bsp Sei $\leq$ totale Ordnung auf Σ. Die lexikographische Ordnung bzgl. $\leq$ ist dann:

$$\leq_{\text{lex}} = \underbrace{\{(u, uw) \mid u, w \in \Sigma^*\}}_{\preceq_p} \cup \underbrace{\{(uxv, uyw) \mid u, v, w \in \Sigma^*, x, y \in \Sigma, x < y\}}_{\text{“tie breaker”}}$$

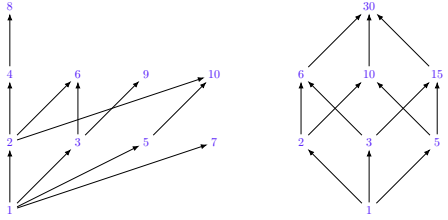
$\leq_{\text{lex}}$ ist daher eine totale Ordnung auf Σ^*.

(Vgl. mit $\leq_{\mathbb{N}_0}$ bzgl. LSBF-Darstellung von $\mathbb{N}_0$; siehe später Präfixbäume und geordnete Bäume.)

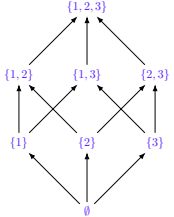
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Ordnungsrelationen: Hasse-Diagramm

- Hasse-Diagramm zu $|$ eingeschränkt auf $[10]$ bzw. $\{d \in \mathbb{N} \colon d|30\}$



- Hasse-Diagramm zu $\subseteq$ beschränkt auf $\mathcal{P}([3])$ („selbe Struktur“ (isomorph) wie $|$ auf $\{d \in \mathbb{N} \colon d|30\}$; siehe „Verband“ später):

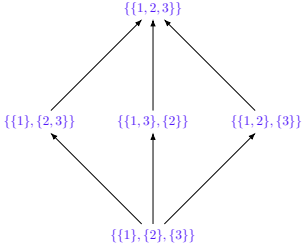


No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Hasse-Diagramm zu $\subseteq$ auf Äquivalenzrelationen über $\{1,2,3\}$,

$$\{\sim \subseteq [3] \times [3] \mid \mathsf{Id}_{[3]} \subseteq \sim, \sim^{\top} \subseteq \sim, \sim\sim \subseteq \sim\}$$

wobei $\sim$ durch $[3]/\sim$ dargestellt wird.

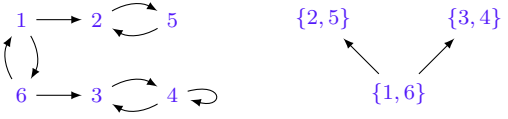


⚠ $\sim_1 \subseteq \sim_2$ **gdw** $[a]_{\sim_1} \subseteq [a]_{\sim_2}$ für alle a.
$\sim_1 \cap \sim_2$ wieder Äquivalenzrelation mit Klassen $[a]_{\sim_1} \cap [a]_{\sim_2}$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Sei $R \subseteq A \times A$ eine Endorelation (z.B. a hängt von b ab).
- Dann ist $S := R^* \cap (R^\top)^*$ die größte Äquivalenzrelation, welche eine Teilmenge von R^* ist. (aSb **gdw** a, b hängen transitiv von einander ab.)

  Und $T := \{([a]_S, [b]_S) \mid (a,b) \in R^*\}$ ist eine partielle Ordnung auf dem Quotienten A/S, welche für die topologische Sortierung verwendet wird.
- Falls R^* partielle Ordnung (damit antisymmetrisch), dann $S = \mathsf{Id}_A$ und $T \cong R^*$.
- Beispiel: R links, Hasse-Diagramm von T rechts: (In der Graphentheorie heißen die Klassen $[a]_S$ starke Zusammenhangskomponenten (SCCs).)



Bsp Setze $A = \mathbb{N}_0^2$ und „$(a,b)R(c,d)$ **:gdw** $a + d \leq b + c$“.

Dann „$(a,b)S(c,d)$ **gdw** $a + d = b + c$“ und $(A/S, T)$ entspricht $(\mathbb{Z}, \leq_{\mathbb{Z}})$.

(Zeige: R ist transitiv; $(a,b) \in \mathbb{N}_0^2$ entspricht z.B. $a - b \in \mathbb{Z}$ (Balkenwaage).)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Ordnungsrelationen: maximales vs. größtes Element

Def Sei $R \subseteq A \times A$ eine (partielle) Ordnung.

$m \in A$ ist ein maximales Element bzgl. R, **:gdw**
Für jedes $a \in A$ mit mRa gilt auch aRm und damit $a = m$.

- „Kein Element oberhalb."
- Es kann mehrere (unvergleichbare) maximale Elemente geben!

$m \in A$ ist das größte Element bzgl. R **:gdw** aRm für jedes $a \in A$ gilt.

- „Oberhalb jedes anderen Elements"

⚠ Höchstens ein größtes Element (mit allen vergleichbar).

- Ist R total, dann ist ein maximales Element sofort auch das größte Element.

Def Entsprechend definiert man ein minimales und das kleinste Element bzgl. R:

- $m \in A$ ist ein minimales Element bzgl. R, **:gdw**
  Für jedes $a \in A$ mit aRm gilt auch mRa und damit $a = m$.
- $m \in A$ ist das kleinste Element bzgl. R **:gdw** mRa für alle $a \in A$ gilt.

Fkt maximal/größtes bzgl. R dual zu minimal/kleinstes bzgl. $R^\top$.

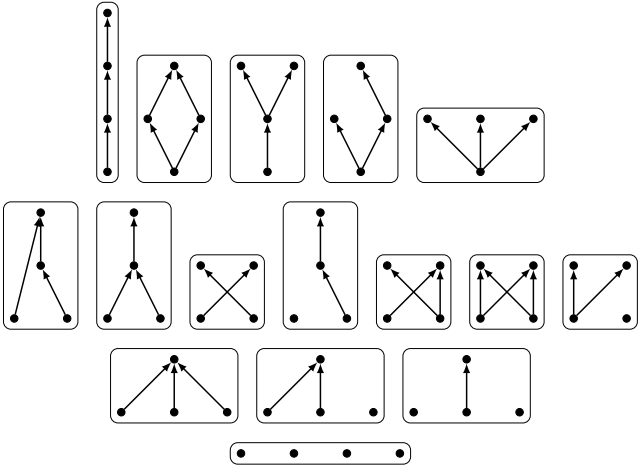
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Maximale und minimale Elemente: Beispiele

- Bzgl. $|_{\mathbb{N}}$ ist 1 das kleinste Element; es gibt keine maximalen Elemente.
- Bzgl. $|_{\mathbb{N}}^{\top}$ („ist Vielfaches“) ist 1 das größte Element.
- Bzgl. $|_{\mathbb{N}\setminus\{1\}}$ sind die Primzahlen die minimalen Elemente.
- Bzgl. $\leq_{\mathbb{Z}}$ gibt es keine minimalen/maximalen Elemente
- Bzgl. $\leq_{\mathbb{N}_0}$ ist 0 das kleinste Element
- Bzgl. $\geq_{\mathbb{N}_0}$ ist 0 das größte Element

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Hasse-Diagramme aller partiellen Ordnungen auf $|A| = 4$ „modulo Isomorphie“.
  (isomorph für jetzt „strukturgleich“; konkretes A unwichtig; später mehr.)
  Alle transitiven Kanten und Schleifen implizit.



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Maximum, Minimum, Supremum, Infimum

Def Sei $R \subseteq A \times A$ eine (partielle) Ordnung und $X \subseteq A$. Bzgl. A und R:

$a \in A$ ist obere bzw. untere Schranke von X **:gdw** $X \subseteq Ra$ bzw. $X \subseteq aR$.

(D.h. xRa bzw. aRx für alle $x \in X$; im Hasse-Diagramm: alle $x \in X$ können a erreichen bzw. können von a aus erreicht werden; nach Annahme $R = R^*$ und $R \cap R^\top = \mathrm{Id}_A$.)

$x = \max X$ (Maximum von X) **:gdw** $x \in X$ und $X \subseteq Rx$.

$x = \min X$ (Minimum von X) **:gdw** $x \in X$ und $X \subseteq xR$.

($\max X$ ist das größte, $\min X$ das kleinste Element bzgl. $R|_X := R \cap (X \times X)$.)

$a = \sup X$ (Supremum von X) **:gdw** $a \in A$ und $a = \min\{u \in A \mid X \subseteq Ru\}$.

$a = \inf X$ (Infimum von X) **:gdw** $a \in A$ und $a = \max\{l \in A \mid X \subseteq lR\}$.

($\sup X$: „kleinste obere Schranke“; $\inf X$: „größte untere Schranke“.)

⚠ Manchmal $\max^R$, $\min^R$, $\sup^R$, $\inf^R$, um Ordnung anzuzeigen.

Fkt Wechselt man von R nach $R^\top$, werden $\sup, \max$ zu $\inf, \min$ (u.u.).

⚠ $x = \max X$ **gdw** $x = \sup X$ und $x \in X$.

$\sup X$ kann existieren, obwohl $\max X$ undefiniert ist: z.B. $\max \emptyset$ existiert nie, aber $\sup \emptyset = \min A$ existiert **gdw** A hat ein kleinstes Element bzgl. R.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Wohlordnung der natürlichen Zahlen und Induktion

Lem Existiert $\inf\{a,b\}$ ($\min\{a,b\}, \sup\{a,b\}, \max\{a,b\}$) für alle $a,b \in A$ (bzgl. R), dann existiert $\inf X$ ($\min X, \sup X, \max X$) für alle endlichen, nicht leeren $X \subseteq A$.

▷ Beweis mittels starker Induktion nach $n = |X| \in \mathbb{N}$:

I.B.: Falls $X = \{x_1\}$, dann $x_1 = \inf X = \min X$.

I.S.: Sei $n \in \mathbb{N}$ beliebig fixiert.

I.A.: Für alle $X = \{x_1, \ldots, x_n\} \subseteq A$ existiert $\inf X$ (bzgl. R).

I.Beh.: Für alle $X = \{x_1, \ldots, x_n, x_{n+1}\} \subseteq A$ existiert $\inf X$ (bzgl. R).

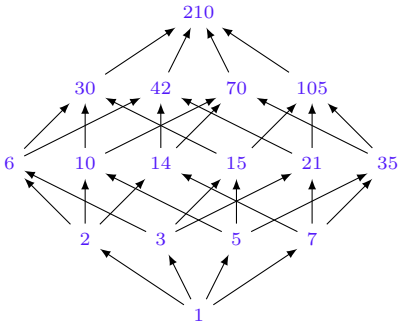
da: Betrachte bel. $X = \{x_1, \ldots, x_n, x_{n+1}\} \subseteq A$; setze $X' = \{x_1, \ldots, x_n\}$.
Dann existiert $i' := \inf X'$ nach I.A.; setze $i := \inf\{x_{n+1}, i'\}$.
Sei $l \in A$ eine beliebige untere Schranke von X, d.h. $X \subseteq lR = lR^*$.
Dann gilt auch lRx_{n+1} und lRi', da l auch untere Schranke von X'.
Damit folgt lRi und $i = \inf X$, da l beliebige untere Schranke.

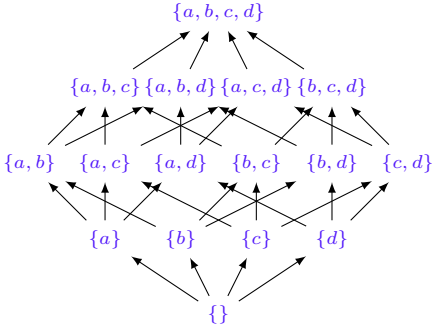
Fkt R ist total **gdw** $\min\{a,b\}$ ($\max\{a,b\}$) existiert für alle $a,b \in A$.

Thm $\mathbb{N}_0$ ist bzgl. $\leq_{\mathbb{N}_0}$ wohlgeordnet: $\min X$ existiert für alle $X \subseteq \mathbb{N}_0$ mit $X \neq \emptyset$.
(Für bel. $m \in X$ setze $X' = \{x \in X \mid x \leq m\}$. Dann: $\min X' = \min X$.)

⚠ Starke Induktion+Widerspruch: $\min\{x \in \mathbb{N}_0 \mid \text{ nicht } P(x)\}$ existiert nicht.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

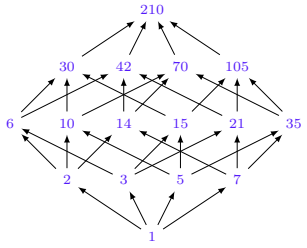


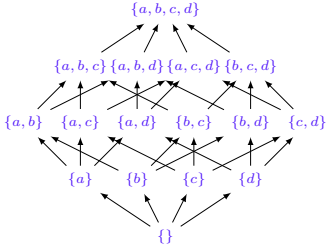


($a := 2, b := 3, c := 5, d := 7$: natürliche Zahlen sind (Multi)mengen von Primzahlen.)

inf: größte untere Schranke z.B. ggT, $\bigcap$
sup: kleinste obere Schranke z.B. kgV, $\bigcup$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science





Def (A, R) ist ein Verband **:gdw** $R \subseteq A \times A$ ist eine partielle Ordnung auf A und für alle $a, b \in A$ existieren (bzgl. R) $\inf\{a, b\}$ und $\sup\{a, b\}$.

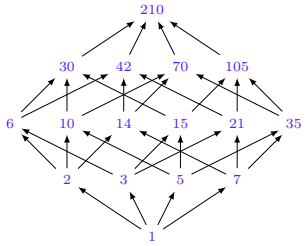
Fkt Ist R eine totale Ordnung, dann ist (A, R) ein Verband mit $\inf\{a, b\} = \min\{a, b\}$ und $\sup\{a, b\} = \max\{a, b\}$ stets.

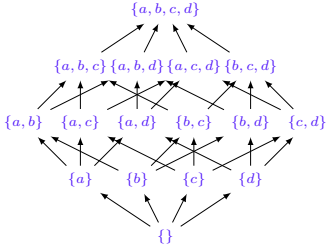
Def (A, R) ist ein vollständiger Verband **:gdw** (A, R) ist ein Verband und für alle $X \subseteq A$ existieren $\sup X$ und $\inf X$.

Fkt In einem vollständigen Verband ist $\sup \emptyset = \min A$ das kleinste und $\inf \emptyset = \max A$ das größte Element.

Fkt (A, R) ist ein (vollständiger) Verband **gdw** $(A, R^\top)$ ist ein (vollständiger) Verband.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science





- Beispiele: Verbände (nicht vollständig)
  - $(\mathbb{N}_0, \leq_{\mathbb{N}_0})$, $(\mathbb{Z}, \leq_{\mathbb{Z}})$, $(\mathbb{Q}, \leq_{\mathbb{Q}})$, $(\mathbb{R}, \leq_{\mathbb{R}})$, $(\mathbb{N}, |)$, $(\Sigma^*, \leq_{\text{lex}})$.
  - $\max X = \sup X$ existiert in $(\mathbb{Z}, \leq_{\mathbb{Z}})$ **gdw** X nicht leer ist und eine obere Schranke in $\mathbb{Z}$ hat.
  - $(\mathbb{R}, \leq_{\mathbb{R}})$ entsteht aus $(\mathbb{Q}, \leq_{\mathbb{Q}})$, indem man $\sup X$ für jedes nicht leere $X \subseteq \mathbb{Q}$ ergänzt, welches eine obere Schranke in $\mathbb{Q}$ hat; z.B. $\sqrt{2} = \sup\{x \in \mathbb{Q}_{\geq 0} \mid x^2 \leq 2\} \in \mathbb{R} \setminus \mathbb{Q}$.
- Beispiel: Vollständige Verbände
  - $([n], \leq_{\mathbb{N}_0})$, $(\mathbb{N}_0, |)$ (mit $0|0$), $(D_n, |)$ (mit $D_n = \{d \in \mathbb{N} \colon d|n\}$), $(2^A, \subseteq)$
  - Jede endliche total geordnete Menge

No commercial use, only for use in Discrete Structures (IN0015, INHN0004), 26w, TUM CIT Department of Computer Science

2 Grundlagen
Grundbegriffe der Logik
Mengen
Folgen, Tupel und Wörter
Die natürlichen Zahlen, Induktionsbeweise
Relationen
Funktionen
Grundlagen
Injektivität, Surjektivität und Bijektivität
Bilder, Urbilder, gleich-wertig
Kardinalität: Vergleiche von unendlichen Mengen
Zusammenfassung

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Funktionen

Def Eine Relation $R \subseteq A \times B$ ist eine (totale) Funktion/Abbildung
:gdw es gibt für jedes $a \in A$ genau ein $b \in B$ mit $(a, b) \in R$.

- d.h. für alle $a \in A$ gilt $|aR| = 1$. (Erinnerung: $aR := \{b \in B \mid (a, b) \in R\}$.)
- R ist eine partielle Funktion **:gdw** $|aR| \leq 1$ für alle $a \in A$.

- Z.B. vergleiche

$$
\begin{aligned}
R_1 &= \{(x, y) \in \mathbb{R}^2 \mid x^2 + y^2 = 1\} \\
R_2 &= \{(x, y) \in \mathbb{R}^2 \mid y \geq 0, x^2 + y^2 = 1\} \\
R_3 &= \{(x, y) \in [-1, 1] \times \mathbb{R} \mid y \geq 0, x^2 + y^2 = 1\}
\end{aligned}
$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Funktionen

Def Eine Relation $R \subseteq A \times B$ ist eine (totale) Funktion/Abbildung
:gdw es gibt für jedes $a \in A$ genau ein $b \in B$ mit $(a, b) \in R$.

- ▷ d.h. für alle $a \in A$ gilt $|aR| = 1$. (Erinnerung: $aR := \{b \in B \mid (a, b) \in R\}$.)
- ▷ R ist eine partielle Funktion **:gdw** $|aR| \leq 1$ für alle $a \in A$.

▷ Konventionen:

- ▷ „$f \colon A \to B$“ um zu sagen, dass $f \subseteq A \times B$ eine Funktion von A nach B ist.
  A ist dann die Definitionsmenge, B die Zielmenge von f.
- ▷ „$f(a)$“ bezeichnet das eindeutige Bild (Nachfolger) von a bzgl. $f \colon A \to B$:

$$af = \{b \in B \mid (a, b) \in f\} \overset{|af|=1}{=} \{f(a)\}$$

  Aber $b \in B$ kann kein, ein oder mehrere Urbilder (Vorgänger) $fb \in 2^A$ besitzen.

⚠ afb **gdw** $b = f(a)$ (a, b tauschen Plätze).

- ▷ Definition von Funktionen mittels „$\mapsto$“ z.B.:

$f \colon \mathbb{R} \setminus \{0\} \to \mathbb{R}, x \mapsto x^{-1}$ oder $f \colon \begin{cases} \mathbb{R} \setminus \{0\} \to \mathbb{R} \\ x \mapsto x^{-1} \end{cases}$ oder $f \colon \mathbb{R} \setminus \{0\} \to \mathbb{R}$ mit $f(x) := x^{-1}$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Funktionen

Def Eine Relation $R \subseteq A \times B$ ist eine (totale) Funktion/Abbildung
:gdw es gibt für jedes $a \in A$ genau ein $b \in B$ mit $(a, b) \in R$.

- ▷ d.h. für alle $a \in A$ gilt $|aR| = 1$. (Erinnerung: $aR := \{b \in B \mid (a, b) \in R\}$.)
- ▷ R ist eine partielle Funktion **:gdw** $|aR| \leq 1$ für alle $a \in A$.

⚠ Eine Funktion benötigt eine Definitionsmenge A und eine Zielmenge B.

- ▷ Die unterschiedlichen Ausdrücke $(1 - x)^{-1}$ und $\sum_{i=0}^{\infty} x^i$ beschreiben dieselbe Funktion von $(-1, 1)$ nach $\mathbb{R}$, aber nicht von $\mathbb{R}_{>1}$ nach $\mathbb{R} \cup \{\infty\}$.

  Entsprechend können unterschiedliche Programme ("source code") dieselbe Funktion beschreiben.

- ▷ Eine partielle Funktion $f \subseteq A \times B$ kann zu einer totalen Funktion

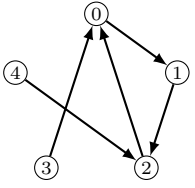
$$f_\perp := f \cup \{(a, \perp) \mid a \in A, af = \emptyset\}$$

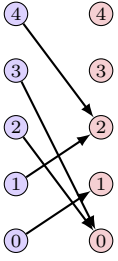
  erweitert werden, wobei $\perp \notin B$ ein soweit unbenutztes Symbol ist (vgl. „Exceptions“).

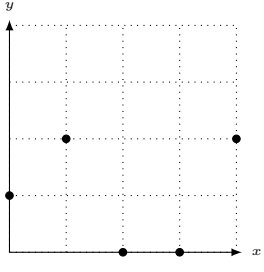
Graphische Darstellung von Funktionen

- Mögliche Darstellung von $f\colon A \to B$:
  1. Als Relation/Graph wie bisher: Ein Knoten für jedes $x \in A \cup B$.
  2. Als bipartiter Graph: A (links) und B (rechts) (potentiell zwei Knoten je Objekt).
  3. Als Punkte in $\mathbb{R}^{n+m}$, falls $A \subseteq \mathbb{R}^n, B \subseteq \mathbb{R}^m$ (zB. $\vec{x}(t)$, $h(\vec{x})$).

Bsp $f\colon \mathbb{Z}_5 \to \mathbb{Z}_5, x \mapsto (x^2+1) \bmod 5$ (Rest der Division durch 5)



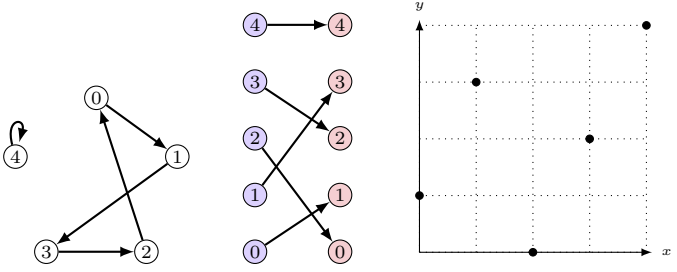




Graphische Darstellung von Funktionen

- Mögliche Darstellung von $f\colon A \to B$:
  1. Als Relation/Graph wie bisher: Ein Knoten für jedes $x \in A \cup B$.
  2. Als bipartiter Graph: A (links) und B (rechts) (potentiell zwei Knoten je Objekt).
  3. Als Punkte in $\mathbb{R}^{n+m}$, falls $A \subseteq \mathbb{R}^n, B \subseteq \mathbb{R}^m$ (zB. $\vec{x}(t)$, $h(\vec{x})$).

Bsp $f\colon \mathbb{Z}_5 \to \mathbb{Z}_5, x \mapsto (2x+1) \bmod 5$ (Rest der Division durch 5)

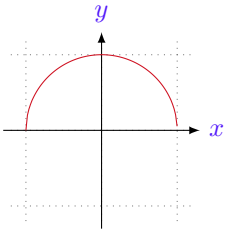


($(\mathbb{Z}_5, f)$ zerfällt in gerichtete Kreise, da f bijektiv ist. Siehe Zykelnotation.)

Graphische Darstellung von Funktionen

- Mögliche Darstellung von $f\colon A \to B$:
  1. Als Relation/Graph wie bisher: Ein Knoten für jedes $x \in A \cup B$.
  2. Als bipartiter Graph: A (links) und B (rechts) (potentiell zwei Knoten je Objekt).
  3. Als Punkte in $\mathbb{R}^{n+m}$, falls $A \subseteq \mathbb{R}^n, B \subseteq \mathbb{R}^m$ (zB. $\vec{x}(t)$, $h(\vec{x})$).

Bsp $f = \{(x, y) \in [-1, 1] \times \mathbb{R} \mid x^2 + y^2 = 1, y \geq 0\}$

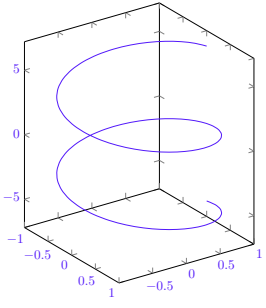


No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Graphische Darstellung von Funktionen

- Mögliche Darstellung von $f\colon A \to B$:
  1. Als Relation/Graph wie bisher: Ein Knoten für jedes $x \in A \cup B$.
  2. Als bipartiter Graph: A (links) und B (rechts) (potentiell zwei Knoten je Objekt).
  3. Als Punkte in $\mathbb{R}^{n+m}$, falls $A \subseteq \mathbb{R}^n, B \subseteq \mathbb{R}^m$ (zB. $\vec{x}(t)$, $h(\vec{x})$).

Bsp $f = \{(t, (\sin(\frac{2\pi}{6}t), \cos(\frac{2\pi}{6}t))) \in \mathbb{R} \times \mathbb{R}^2 \mid t \in \mathbb{R}\}$ (t: vertikale Achse)



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Tupel, Selbstabbildungen, Mengen, Multimengen, Stelligkeit (Arität)

Def $B^A := \{f \colon A \to B\}$ (Menge aller Funktionen von A nach B.)

- $f \in A^{[n]}$ kann durch $(f(1), ..., f(n)) \in A^n$ eindeutig beschrieben werden.
  Falls A, B endlich: $|B^A| = |B|^{|A|}$ Funktionen von A nach B (auch allgemein).
- $f \in A^A$ nennt man auch eine Selbstabbildung von A.
  $\mathrm{Id}_A \in A^A$ wird Identität(sfunktion) auf A genannt.
- $\{0,1\}^A$ Menge aller charakteristischen Funktionen bzgl. A (vgl. 2^A):
  $T \subseteq A$ entspricht $\delta_T \colon A \to \{0,1\}$ mit „$\delta_T(x) = 1$ **gdw** $x \in T$“.
- $f \in \mathbb{N}_0^A$ kann als Multimenge aufgefasst werden.
  ($f(a)$ gibt gerade an, wie häufig $a \in A$ „in f“ vorkommt (mehr später).)

Falls $f \colon A_1 \times \ldots \times A_k \to B$, statt $f((a_1, \ldots, a_k))$ nur $f(a_1, \ldots, a_k)$.

- k: Stelligkeit (Arität) von f: nullär, unär, binär, ternär,...
  (Formal ist $f \subseteq (A_1 \times \ldots \times A_k) \times B$ aber eine binäre Relation.)
  Falls $A_1 = \ldots = A_k = B$, auch k-stellige Operation auf A genannt.
- Spezialfall nulläre Funktion: $f \colon \{()\} \to B$.
  (Nur für „kein Argument“ definiert: $f() = b \in B$; vgl. Programmiersprachen.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Operationen

Def $f\colon A^k \to A$ wird auch als k-stellige Operation auf A bezeichnet.

- $f \subseteq A^k \times A$ ist binäre Relation, aber k-stellige Operation.
- Häufig $\diamond, \oplus, \odot, \otimes, \boxdot$ u.ä. Symbole als Bezeichner für binäre Operationen.
  Üblicherweise $(a \diamond b) := \diamond(a, b)$ (Infix- vs. Präfixnotation; siehe z.B. Python, C++).
  Falls eindeutig statt $(a \diamond b)$ nur $a \diamond b$.
- Standardbeispiele: kanonische Addition und Multiplikation auf $\mathbb{R}$.

Def Für $\diamond\colon A \times A \to A$:

- $\diamond$ ist assoziativ **:gdw** $(a \diamond b) \diamond c = a \diamond (b \diamond c)$ für alle $a, b, c \in A$.
  Falls $\diamond$ assoziativ: $(a_1 \diamond a_2 \diamond \ldots \diamond a_n) := (\ldots((a_1 \diamond a_2) \diamond \ldots) \diamond a_n)$ (induktiv).
- $\diamond$ ist kommutativ **:gdw** $a \diamond b = b \diamond a$ für alle $a, b \in A$.
- $\diamond$ ist idempotent **:gdw** $a \diamond a = a$ für alle $a \in A$.
- e_l ist linksneutral bzgl. $\diamond$ **:gdw** $e_l \diamond a = a$ für alle $a \in A$.
  e_r ist rechtsneutral bzgl. $\diamond$ **:gdw** $a \diamond e_r = a$ für alle $a \in A$.

- Z.B. $\cap, \cup\colon 2^A \times 2^A \to 2^A$ sind assoziativ, kommutativ, idempotent.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Komposition von Funktionen

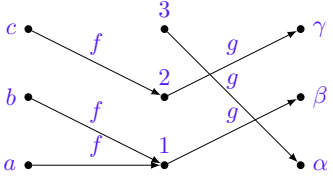
Def Komposition (Nacheinanderausführung) von Funktionen:

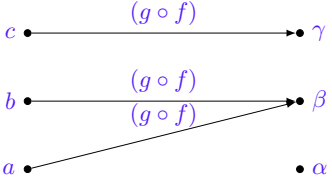
Für $f\colon A \to B$ und $g\colon B \to C$ ist $(g \circ f)$ („g nach f“) definiert durch

$$(g \circ f)\colon A \to C, a \mapsto g(f(a))$$

⚠ Beachte umgekehrte Lese/Schreibrichtung im Vergleich zu Relationen:

als Relationen		als Funktionen
fg	$= \{(a, g(f(a))) \mid a \in A\} =$	$(g \circ f)$





No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- ▷ Da $g \circ f = fg$, folgt:

Fkt Die Funktionskomposition ist assoziativ,

$$\begin{aligned}(h \circ (g \circ f))(a) &= h((g \circ f)(a)) \\ &= h(g(f(a))) \\ &= (h \circ g)(f(a)) \quad = \quad ((h \circ g) \circ f)(a)\end{aligned}$$

aber nicht kommutativ.

- ▷ Z.B. betrachte $f \circ g$ und $g \circ f$ für $f, g \in \mathbb{R}^{\mathbb{R}}$ mit $f(x) := x + 1$ und $g(x) := x^2$.

Fkt Id_A ist rechtsneutral, Id_B ist linksneutral: $(f \circ \mathsf{Id}_A) = f = (\mathsf{Id}_B \circ f)$.

(Spezialfall von $\mathsf{Id}_A R = R = R\mathsf{Id}_B$ für $R \subseteq A \times B$.)

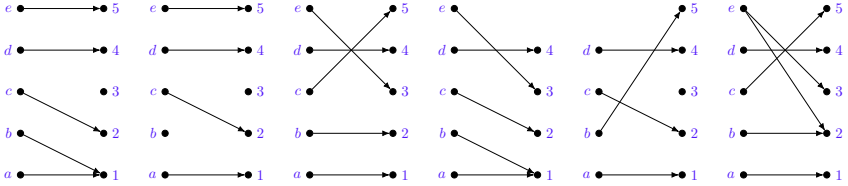
Def Falls $f \colon A \to A$, definiert man f^n induktiv für alle $n \in \mathbb{N}_0$:

$$f^0 := \mathsf{Id}_A \qquad f^{n+1} := f \circ f^n$$

(Spezialfall von R^n für $R \subseteq A \times A$: $f^n(a) = b$ **gdw** $af^n = \{b\}$)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Injektive, surjektive und bijektive Funktionen



Def Eine Funktion $f\colon A \to B$ ist

- injektiv **:gdw** $|fb| \leq 1$ für jedes $b \in B$ (höchstens ein Urbild/Vorgänger).

  d.h. für alle $a, a' \in A$ gilt: falls $f(a) = f(a')$, dann $a = a'$.

- surjektiv **:gdw** $|fb| \geq 1$ für jedes $b \in B$ (mindestens ein Urbild/Vorgänger).

  d.h. für jedes $b \in B$ gibt es ein $a \in A$ mit $f(a) = b$.

- bijektiv **:gdw** $|fb| = 1$ für jedes $b \in B$ (genau ein Urbild/Vorgänger).

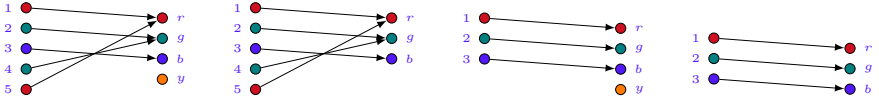
  d.h. f ist sowohl injektiv als auch surjektiv.

Fkt Es gelte $f\colon A \to B$ und $g\colon B \to C$.

- Sind f, g beide injektiv (surjektiv), dann ist $(g \circ f)$ injektiv (surjektiv).
- Ist $(g \circ f)$ injektiv/surjektiv, dann ist f injektiv/g surjektiv.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Umkehrfunktion



Fkt Sei $f\colon A \to B$ mit $f^\top = \{(b,a) \mid (a,b) \in f\}$.

- f ist injektiv **gdw** $|bf^\top| \leq 1$ für alle $b \in B$.
- f ist surjektiv **gdw** $|bf^\top| \geq 1$ für alle $b \in B$.
- Daher: f ist bijektiv **gdw** $f^\top$ ist eine Funktion von B nach A.

Def Gilt $f\colon A \xrightarrow{\text{bij}} B$, dann ist $f^{-1} := f^\top$ die Umkehrfunktion von f.

Kor Ist f bijektiv, dann auch f^{-1}, da $(f^{-1})^\top = (f^\top)^\top = f$.
Insbesondere: $f = (f^{-1})^{-1}$ und $f^{-1} \circ f = \mathsf{Id}_A$ und $f \circ f^{-1} = \mathsf{Id}_B$.

Fkt Ist $f\colon A \to A$ injektiv oder surjektiv und A endlich, dann ist f bijektiv.

⚠ Die Notation „f^{-1}“ ist „überladen“ (siehe Bilder und Urbilder):

Def Falls f nicht bijektiv: $f^{-1}\colon B \to 2^A$ mit $f^{-1}(b) := fb = \{a \in A \mid f(a) = b\}$.
Und allgemein: $f^{-1}\colon 2^B \to 2^A$ mit $f^{-1}(Y) := \bigcup\{fb \mid b \in Y\}$.

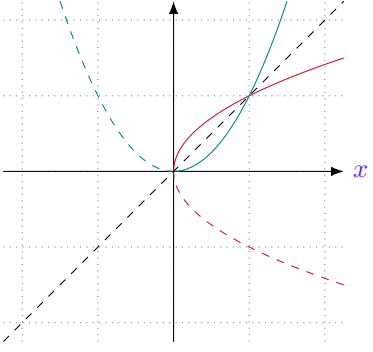
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Umkehrfunktion: Spiegelung, Monotonie

Fkt Falls $f\colon A \to B$ mit $(A, \leq_A)$ und $(B, \leq_B)$ total geordnet und für alle $a <_A a'$ stets $f(a) <_B f(a')$, dann ist f injektiv und $f\colon A \to f(A)$ bijektiv.

⚠ Falls $f\colon \mathbb{R} \to \mathbb{R}$, übliche Darstellung von f als Punkte $(x, f(x))$ in der Ebene; dann ist $f^\top = \{(f(x), x) \mid x \in \mathbb{R}\}$ die Spiegelung an $\{(x, x) \mid x \in \mathbb{R}\}$.

▷ Beispiel: $f\colon \mathbb{R} \to \mathbb{R}, x \mapsto x^2$ ist nicht bijektiv, also $f^\top$ keine Funktion.
Aber $g\colon \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}, x \mapsto x^2$ ist bijektiv mit $g^{-1}\colon \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}, x \mapsto \sqrt{x}$.



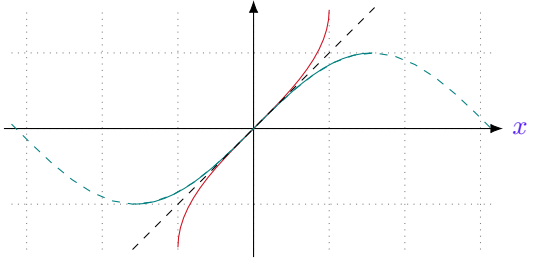
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Umkehrfunktion: Spiegelung, Monotonie

Fkt Falls $f\colon A \to B$ mit $(A, \leq_A)$ und $(B, \leq_B)$ total geordnet und für alle $a <_A a'$ stets $f(a) <_B f(a')$, dann ist f injektiv und $f\colon A \to f(A)$ bijektiv.

⚠ Falls $f\colon \mathbb{R} \to \mathbb{R}$, übliche Darstellung von f als Punkte $(x, f(x))$ in der Ebene; dann ist $f^\top = \{(f(x), x) \mid x \in \mathbb{R}\}$ die Spiegelung an $\{(x, x) \mid x \in \mathbb{R}\}$.

▷ Beispiel: Auch $f\colon \mathbb{R} \to \mathbb{R}, x \mapsto \sin(x)$ ist nicht bijektiv, also nicht umkehrbar. Aber $g\colon [-\frac{\pi}{2}, \frac{\pi}{2}] \to [-1, 1], x \mapsto \sin(x)$ ist bijektiv mit $g^{-1} = \arcsin$.



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Fkt Sei $f\colon A \xrightarrow{\text{bij}} B$. Dann gilt für jedes $g\colon B \to A$:
Falls $(g \circ f) = \mathsf{Id}_A$ oder $(f \circ g) = \mathsf{Id}_B$, dann bereits $g = f^{-1}$.
($f^{-1} = f^{-1} \circ \mathsf{Id}_B = f^{-1} \circ (f \circ g) = (f^{-1} \circ f) \circ g = \mathsf{Id}_A \circ g = g$.)

Fkt Für $f\colon A \to B$ und $g\colon B \to A$ gelte $(g \circ f) = \mathsf{Id}_A$ und $(f \circ g) = \mathsf{Id}_B$.
Dann sind f, g bijektiv mit $f^{-1} = g$ und $g^{-1} = f$.
(Da dann sind $(g \circ f)$ und $(f \circ g)$ beide bijektiv und damit auch f, g.)

Fkt Falls $f\colon A \xrightarrow{\text{bij}} B$ und $g\colon B \xrightarrow{\text{bij}} C$, dann $(g \circ f)^{-1} = (f^{-1} \circ g^{-1})$.
($(g \circ f) \circ (f^{-1} \circ g^{-1}) = (g \circ (f \circ f^{-1}) \circ g^{-1})$.)

Def Eine Permutation von A ist eine bijektive Selbstabbildung $f\colon A \xrightarrow{\text{bij}} A$.
$\mathcal{S}_A := \{f\colon A \xrightarrow{\text{bij}} A\}$ ist die Menge aller Permutationen von A.

Kor Mit $f, g \in \mathcal{S}_A$ gilt auch $\mathsf{Id}_A, f^{-1}, f \circ g \in \mathcal{S}_A$.
($\mathcal{S}_A$ bildet bzgl. der Komposition die Symmetrische Gruppe auf A.)

- ▷ Permutationen werden in Kombinatorik, Graphentheorie und Gruppentheorie verwendet.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Selbstabbildung als gerichtete Graphen und Zykelnotation

Fkt Für $f\colon A \to A$ betrachte den Graph $G_f = (A, \{(a, f(a)) \mid a \in A\})$:

- ▷ Orbit (Trajektorie) von $a \in A$: eindeutig bestimmter unendlicher Pfad

$$\left(f^i(a)\right)_{i \in \mathbb{N}_0} = (a, f(a), f^2(a), \ldots)$$

  Gibt es $i < j$ mit $f^i(a) = f^j(a)$, dann wird die Trajektorie zu einem „Lasso“ und a durchläuft für immer den Zyklus/Kreis. $f^i(a), f^{i+1}(a), \ldots, f^j(a)$.

- ▷ f ist injektiv **gdw** G_f zerfällt in Zyklen ($\mathbb{Z}_n$) und „Zahlengeraden“ ($\mathbb{N}_0, \mathbb{Z}$).
- ▷ Falls A endlich: (siehe Zykelnotation und $R^* = R^{<|A|}$)
  f bijektiv **gdw** f injektiv **gdw** f surjektiv **gdw** G_f besteht nur aus Zyklen.

Def Gilt $a = f(a)$, dann nennt man a einen Fixpunkt von f.

Bsp Vergleiche

- ▷ $f_1\colon \mathbb{Z} \to \mathbb{Z}, x \mapsto x + 2$
- ▷ $f_2\colon \mathbb{Z} \to \mathbb{Z}, x \mapsto x + \mathsf{sign}(x)$
- ▷ $f_3\colon \mathbb{Z}_6 \to \mathbb{Z}_6, x \mapsto (x + 2) \bmod 6$
- ▷ $c\colon \mathbb{N} \to \mathbb{N}$ mit $c(n) := n/2$, falls n gerade, sonst $c(n) := 3n + 1$ (nächste Folie)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Selbstabbildung als gerichtete Graphen und Zykelnotation

Fkt Für $f\colon A \to A$ betrachte den Graph $G_f = (A, \{(a, f(a)) \mid a \in A\})$:

- ▷ Orbit (Trajektorie) von $a \in A$: eindeutig bestimmter unendlicher Pfad

$$\left(f^i(a)\right)_{i \in \mathbb{N}_0} = (a, f(a), f^2(a), \ldots)$$

  Gibt es $i < j$ mit $f^i(a) = f^j(a)$, dann wird die Trajektorie zu einem „Lasso“ und a durchläuft für immer den Zyklus/Kreis. $f^i(a), f^{i+1}(a), \ldots, f^j(a)$.

- ▷ f ist injektiv **gdw** G_f zerfällt in Zyklen ($\mathbb{Z}_n$) und „Zahlengeraden“ ($\mathbb{N}_0, \mathbb{Z}$).
- ▷ Falls A endlich: (siehe Zykelnotation und $R^* = R^{<|A|}$)
  f bijektiv **gdw** f injektiv **gdw** f surjektiv **gdw** G_f besteht nur aus Zyklen.

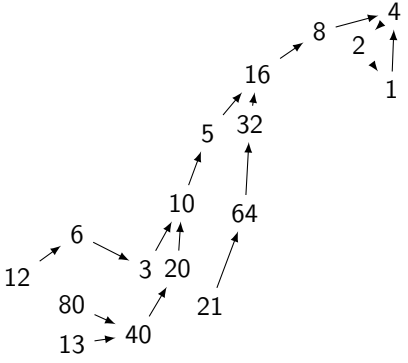
Def Gilt $a = f(a)$, dann nennt man a einen Fixpunkt von f.

- ▷ Anwendungen: Zykelnotation, Satz von Lagrange, Ordnung eines Elements, Gruppenexponent (s. Gruppentheorie), Pollard's ρ-Methode.
- ▷ Selbstabbildungen dienen zur Beschreibung von Programmen und allgemein dynamischer Systeme.

  (Siehe z.B. Wiederkehrsatz und Pumping-Lemma – oder sogar Nietzsche.)

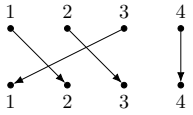
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

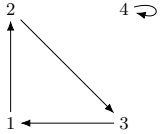
$$c\colon \mathbb{N} \to \mathbb{N}, n \mapsto \begin{cases} n/2 & n \in 2\mathbb{N} \\ 3n+1 & n \in 2\mathbb{N}+1 \end{cases}$$



Collatz-Problem: Enden alle Pfade im Kreis $(4, 2, 1)$ (nur eine CC)?

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science





Def Für $f \in B^A$ mit $|A| = n \in \mathbb{N}$

- Zweizeilenform („Wertetabelle“):

$$f = \begin{pmatrix} a_1 & a_2 & \dots & a_n \\ f(a_1) & f(a_2) & \dots & f(a_n) \end{pmatrix} \overset{\text{im Bsp}}{=} \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 3 & 1 & 4 \end{pmatrix}$$

- Tupelschreibweise für fixierte Aufzählung $A = \{a_1, a_2, \dots, a_n\}$:

$$f = (f(a_1), f(a_2), \dots, f(a_n)) \overset{\text{im Bsp}}{=} (2, 3, 1, 4)$$

  Damit kann B^A mit B^n identifiziert werden

- Zyklen-/Zykelschreibweise für Permutationen $f \colon A \xrightarrow{\text{bij}} A$:

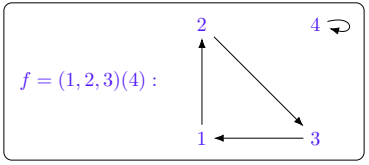
$$f \overset{\text{im Bsp}}{=} (1, 2, 3)(4) = (4)(3, 1, 2) = \dots$$

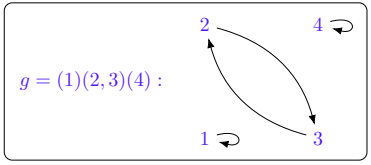
  Ein Tupel $(a, f(a), \dots, f^i(a))$ je Kreis (d.h. $f^{i+1}(a) = a$) im Graph (A, f).

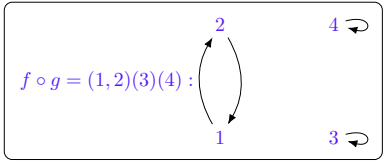
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

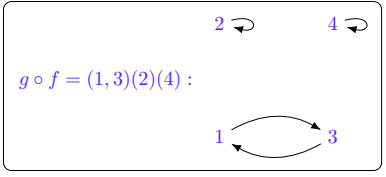
Zyklen-/Zykelschreibweise: Beispiele

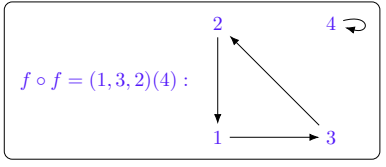
- Beispiel: $f = (1,2,3)(4)$, $g = (1)(2,3)(4)$

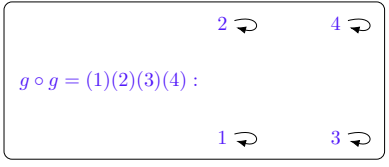




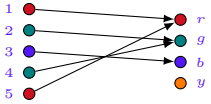








No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science



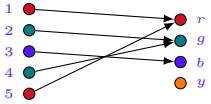
Def Sei $f\colon A \to B$ eine Funktion.

- $f(X) := \{f(a) \mid a \in X\} = \bigcup\{af \mid a \in X\}$ (Bilder von $X \subseteq A$).
- $\mathsf{im}(f) := f(A) = \{f(a) \mid a \in A\} \subseteq B$ (Bildmenge von f).
- $f^{-1}(Y) := \{a \in A \mid f(a) \in Y\} = \bigcup\{fb \mid b \in Y\}$ (Urbilder von $Y \subseteq B$).

⚠ Anmerkungen:

- f wird so zu einer Funktion von 2^A nach 2^B erweitert.
- f^{-1} ist so stets als Funktion von 2^B nach 2^A definiert, selbst wenn f nicht bijektiv (d.h. $f^\top$ keine Funktion) ist; dann üblicherweise $f^{-1}(b) := f^{-1}(\{b\}) = fb$.
- Alle $a' \in f^{-1}(f(a)) = aff^\top$ haben dieselbe „Farbe“ $f(a)$.
- $\mathsf{im}(f)$ ist i.A. unbekannt! Z.B. ist die Goldbachsche Vermutung gerade $\mathsf{im}(g) = 2\mathbb{N}_{>2}$ für $g\colon \mathbb{P}^2_{>2} \to \mathbb{N}, (p,q) \mapsto p+q$; siehe auch semi-entscheidbar. ($\mathbb{P}_{>2} = \{3,5,\ldots\}$.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science



Def Sei $f\colon A \to B$ eine Funktion.

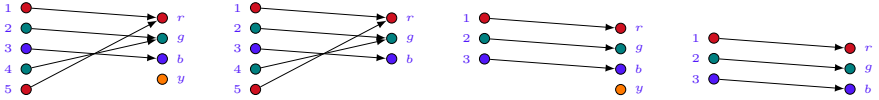
- $f(X) := \{f(a) \mid a \in X\} = \bigcup\{af \mid a \in X\}$ (Bilder von $X \subseteq A$).
- $\mathsf{im}(f) := f(A) = \{f(a) \mid a \in A\} \subseteq B$ (Bildmenge von f).
- $f^{-1}(Y) := \{a \in A \mid f(a) \in Y\} = \bigcup\{fb \mid b \in Y\}$ (Urbilder von $Y \subseteq B$).

Lem Sei $f\colon A \to B$ eine Funktion. Dann gilt für alle $X, X' \subseteq A$, $Y, Y' \subseteq B$:

- $f^{-1}(B) = A$ (da f total), aber i.A. nur $\mathsf{im}(f) = f(A) \subseteq B$.
- $f(f^{-1}(Y)) \subseteq Y$; und $X \subseteq f^{-1}(f(X))$; und $f(X) \subseteq Y$ **gdw** $X \subseteq f^{-1}(Y)$.
- Falls $X \subseteq X'$, dann $f(X) \subseteq f(X')$; und falls $Y \subseteq Y'$, dann $f^{-1}(Y) \subseteq f^{-1}(Y')$.
- $f^{-1}(Y \diamond Y') = f^{-1}(Y) \diamond f^{-1}(Y')$ für $\diamond \in \{\cap, \cup, \setminus\}$.
  (Z.B.: $x \in f^{-1}(Y \setminus Y')$ **gdw** $f(x) \in Y$ und $f(x) \notin Y'$ **gdw** $x \in f^{-1}(Y) \setminus f^{-1}(Y')$.)
- $f(X \cup X') = f(X) \cup f(X')$,
  aber i.A. nur $f(X \cap X') \subseteq f(X) \cap f(X')$ und $f(X \setminus X') \supseteq f(X) \setminus f(X')$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Verträglichkeit mit Mengenoperationen



Lem Sei $f\colon A \to B$ eine Funktion. Dann gilt

- f ist surjektiv **gdw** $\mathrm{im}(f) = f(A) = B$ **gdw** $Y = f(f^{-1}(Y))$ für alle $Y \subseteq B$.
- f ist injektiv **gdw** $X = f^{-1}(f(X))$ für alle $X \subseteq A$
  gdw $f(X \cap X') = f(X) \cap f(X')$ für alle $X, X' \subseteq A$
  gdw $f(X \setminus X') = f(X) \setminus f(X')$ für alle $X, X' \subseteq A$
  gdw $f(A \setminus X) = f(A) \setminus f(X)$ für alle $X \subseteq A$.
- f ist bijektiv **gdw** $f(A \setminus X) = B \setminus f(X)$ für alle $X \subseteq A$.

⚠ Falls $f\colon A \to B$ injektiv ist, kann man $a \in A$ mit $f(a) \in B$ identifizieren und A als Teilmenge von B auffassen; ist f sogar bijektiv, dann kann man A und B bis auf Umbenennung als dieselbe Menge auffassen; siehe Kardinalität später.

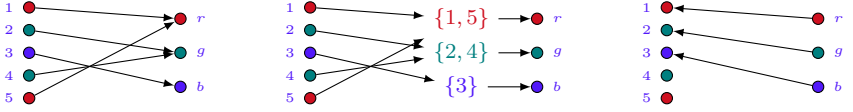
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Funktionen und Äquivalenzrelationen

Def Für $f\colon A \to B$ sei $\equiv_f := \{(a, a') \in A \times A \mid f(a) = f(a')\}$ („gleichwertig bzgl. f“).

Fkt $\equiv_f$ ist stets eine Äquivalenzrelation mit:

- Die Äquivalenzklasse von $a \in A$ ist $[a]_f = f^{-1}(f(a))$ („selber Wert wie a“).
  Der Quotient ist $A/\equiv_f = \{[a]_f \mid a \in A\} = \{f^{-1}(b) \mid b \in f(A)\}$.
- $[\cdot]_f\colon A \to A/\equiv_f, a \mapsto [a]_f$ ist stets surjektiv (gruppiere nach Wert bzgl. f).
- $\tilde{f}\colon A/\equiv_f \to B, [a]_f \mapsto f(a)$ ist stets injektiv mit $f(a) = \tilde{f}([a]_f)$.
- Ist f injektiv, dann ist $[\cdot]_f$ bijektiv. Ist f surjektiv, dann ist $\tilde{f}$ bijektiv.



($f(a) \in B$ entspricht der Farbe (Rang, Label, Wert, Bedeutung, ...) von $a \in A$. Dann entspricht f der „gefärbten Partition“ $\tilde{f}\colon A/\equiv_f \xrightarrow{\text{inj}} B$. Falls $A \subseteq \mathbb{N}_0$, kann man $r(a) := \min[a]_f$ als Repräsentant wählen; und falls f surjektiv, dann definiert $g(b) := \min f^{-1}(b)$ eine injektive Abbildung $g\colon B \to A$ mit $b = f(g(b))$ (Rechtsinverses) zu erhalten; i.A. benötigt man das Auswahlaxiom.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

2 Grundlagen

Grundbegriffe der Logik

Mengen

Folgen, Tupel und Wörter

Die natürlichen Zahlen, Induktionsbeweise

Relationen

Funktionen

Kardinalität: Vergleiche von unendlichen Mengen

Zusammenfassung

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Kardinalität von Mengen

- $f\colon A \xrightarrow{\text{inj}} B$ erlaubt es jedes Element aus $a \in A$ eindeutig mit $f(a)$ zu identifizieren: „B hat mindestens so viele Elemente wie A.“

  (Erinnerung: $f(X \cap X') \equiv f(X) \cap f(X')$ **gdw** f injektiv.)

- $f\colon A \xrightarrow{\text{bij}} B$ erlaubt jedes $a \in A$ mit $f(a)$ und jedes $b \in B$ mit $f^{-1}(b)$ eindeutig zu identifizieren: „B hat genauso viele Elemente wie A.“

- Formal „mächtig“ statt „so viele Elemente“:

Def $|A| \leq |B|$ **:gdw** es gibt eine injektive Funktion $f\colon A \to B$

(„A höchstens gleichmächtig zu B.“)

$|A| = |B|$ **:gdw** es gibt eine bijektive Funktion $f\colon A \to B$

(„A, B gleichmächtig.“)

- $|A| \geq |B|$ **:gdw** $|B| \leq |A|$.
  $|A| < |B|$ **:gdw** $|A| \leq |B|$ und nicht $|A| = |B|$.
- $|A| = n$ **:gdw** $|A| = |[n]|$
  $|A| \leq n$ **:gdw** $|A| \leq |[n]|$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Kardinalität von Mengen

Fkt Für alle Mengen A, B, C gilt.

- ▷ Falls $A \subseteq B$, dann $|A| \leq |B|$ (mittels $f\colon A \to B, a \mapsto a$).
- ▷ $|A| \leq |A|$ und $|A| = |A|$ (mittels Id_A).
- ▷ Falls $|A| = |B|$, dann $|A| \leq |B|$ und $|B| \leq |A|$ (mittels f, f^{-1}).
- ▷ Falls $|A| \leq |B|$ und $|B| \leq |C|$, dann auch $|A| \leq |C|$.
  Falls $|A| = |B|$ und $|B| = |C|$, dann auch $|A| = |C|$.
  (Da $(f \circ g)$ injektiv (bijektiv), falls f und g beide injektiv (bijektiv).)

Thm Satz von Cantor-Bernstein-Schröder:
$|A| = |B|$ **gdw** $|A| \leq |B|$ und $|B| \leq |A|$.

(D.h. $f_1\colon A \xrightarrow{\text{inj}} B$ und $f_2\colon B \xrightarrow{\text{inj}} A$ können stets zu $f\colon A \xrightarrow{\text{bij}} B$ kombiniert werden.)

- ▷ „$|A| = |B|$“ definiert daher eine Äquivalenzrelation auf der Klasse aller Mengen.
  „$|A|$“ (auch: „$\#A$“) steht für die Äquivalenzklasse von A.
  Man nennt „$|A|$“ die Kardinalzahl (Kardinalität, Mächtigkeit) von A.
- ▷ „$|A| \leq |B|$“ ist dann eine partielle Ordnung auf den Kardinalzahlen.

(Nur unter zusätzlicher Annahme z.B. des Wohlordnungssatzes eine lineare Ordnung.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Abzählbare Menge

Def Eine Menge A ist abzählbar **:gdw** $|A| \leq |\mathbb{N}|$ („überabzählbar“ für „nicht abzählbar“).

Fkt $|\mathbb{N}| = |2\mathbb{N}|$ mittels $2\mathbb{N} \xrightarrow{\text{bij.}} \mathbb{N}, n \mapsto n/2$ oder

mittels $\mathbb{N} \xrightarrow{\text{inj.}} 2\mathbb{N}, n \mapsto 2n$ und $2\mathbb{N} \xrightarrow{\text{inj.}} \mathbb{N}, n \mapsto n$

(siehe Hilberts Hotel)

Fkt $|\mathbb{N}| = |\mathbb{Z}|$ mittels der Bijektion

$$f\colon \mathbb{Z} \xrightarrow{\text{bij}} \mathbb{N}, z \mapsto 2z+1 \text{ falls } z \geq 0 \text{ sonst } -2z$$

Fkt $|\mathbb{N}| \leq |\mathbb{N}_0| \leq |\mathbb{Z}| \leq |\mathbb{N}|$, also $\mathbb{N}, \mathbb{N}_0, \mathbb{Z}$ gleichmächtig.

(Falls $A \subseteq B$, dann $|A| \leq |B|$ mittels $f\colon A \to B, a \mapsto a$.)

⚠ Satz von Euklid: $|\mathbb{P}| = |\mathbb{N}|$ (mehr dazu später)

Fkt Falls $|A| \leq |\mathbb{N}|$, entweder $|A| = |[n]| = n$ für ein $n \in \mathbb{N}_0$ oder $|A| = |\mathbb{N}|$.

(Sei $f\colon A \xrightarrow{\text{inj}} \mathbb{N}$. Falls $m := \max f(A)$ existiert, $|A| = |f(A)| \leq |[m]|$; sonst entweder $A = \emptyset$ oder definiere $a\colon \mathbb{N} \xrightarrow{\text{inj}} A$ induktiv: für jedes $i \in \mathbb{N}$ muss es ein $a(i) \in A$ mit $f(a(i)) > \max\{f(a(1)), \ldots, f(a(i-1))\}$ geben.)

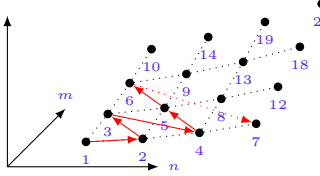
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Abzählbare Menge

Def Eine Menge A ist abzählbar **:gdw** $|A| \leq |\mathbb{N}|$ („überabzählbar“ für „nicht abzählbar“).

Fkt $|\mathbb{N}| = |\mathbb{N} \times \mathbb{N}|$ mittels der Cantorschen Paarungsfunktion:

$$f\colon \mathbb{N} \times \mathbb{N} \xrightarrow{\text{bij}} \mathbb{N}, (n,m) \mapsto m + \sum_{i=1}^{m+n-2} i = m + \frac{(m+n-1)(m+n-2)}{2}$$



(n,m)	1	2	3	4	...
1	1	3	6	10	...
2	2	5	9	14	...
3	4	8	13	19	...
4	7	12	18	25	...
⋮	⋮	⋮	⋮	⋮	⋱

Kor Durch Iteration $|\mathbb{N}^k| = |\mathbb{N}|$ für jedes $k \in \mathbb{N}$ und auch $|\bigcup_{k\in\mathbb{N}} \mathbb{N}^k| = |\mathbb{N}|$.

- ▷ Alternativ nach Gödel mittels Fundamentalsatz der Arithmetik:
  $(n_1, \ldots, n_k) \mapsto 2^{n_1} 3^{n_2} \cdots p_k^{n_k}$ für alle $k \in \mathbb{N}_0$ und $(n_1, \ldots, n_k) \in \mathbb{N}^k$.

Kor $|\mathbb{N}| \leq |\mathbb{Q}| \leq |\mathbb{Z} \times \mathbb{N}| \leq |\mathbb{N} \times \mathbb{N}| \leq |\bigcup_{k\in\mathbb{N}} \mathbb{N}^k| \leq |\mathbb{N}| = |\{1\}\{0,1\}^*| = |\{0,1\}^*|$.

(Auch die Menge aller algebraischen Zahlen ist abzählbar.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Satz von Cantor

Thm Für jede Menge A gilt $|A| < \left|2^A\right|$.

- Beweis:
  - Behauptung: $|A| \leq |2^A|$. ($\mathcal{P}(A) = 2^A$.)

    Die Abbildung $f\colon A \to 2^A, a \mapsto \{a\}$ ist injektiv.
  - Behauptung: Es gibt keine Bijektion von A nach 2^A.

    Cantors Diagonalargument:

    Wir nehmen an, dass es doch $f\colon A \xrightarrow{\text{bijektiv}} 2^A$ gibt.

    Setze $D := \{a \in A \mid a \notin f(a)\} \in 2^A$.

    (D ist Menge, da A Menge und f Funktion.)

    Da f bijektiv nach Annahme, gibt es ein $d \in A$ mit $f(d) = D$.

    Damit ergibt sich der Widerspruch: $d \in D$ **gdw** $d \notin f(d) = D$.

    (Vergleiche auch Russellsche Antinomie.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Satz von Cantor

Thm Für jede Menge A gilt $|A| < \left|2^A\right|$.

- Spezialfall: $A = \mathbb{N}$ (analog auch für $A = [n]$).
  - Es gelte $f\colon A \xrightarrow{\text{bij}} 2^A$ mit $A = \mathbb{N}$.
    Stelle $f(a) \subseteq A$ als charakteristische Funktion/Sequenz dar.
    Dann kann f selbst als Tabelle dargestellt werden mit $f(a)$ als a.te Reihe, z.B.:

a	$f(a)$	1	2	3	4	…
1	$\mathbb{P}$	0	1	1	0	…
2	$\mathbb{N}$	1	1	1	1	…
3	$2\mathbb{N}$	0	1	0	1	…
4	$2\mathbb{N}+1$	1	0	1	0	…
⋮	⋮					

    D ist dann die Menge, die durch die negierte Diagonale beschrieben wird.
    Im Beispiel: $D = \{a \in A \mid a \notin f(a)\} = \{1, 3, 4, \ldots\} \subseteq A$.
    Nach Konstruktion unterscheidet sich D von jeder Reihe ($a \in D$ **gdw** $a \notin f(a)$) und damit gibt es kein $a \in A$ mit $f(a) = D$.
    Daher wird Cantors Argument auch als Diagonalargument bezeichnet.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Überabzählbare vs. abzählbare Mengen

Thm $|\{0,1\}^*| = |\{1\}\{0,1\}^*| = |\mathbb{N}|$ und $\left|\{0,1\}^{\mathbb{N}}\right| = |[0,1]|$ (Binärdarstellung).

Lem $|(0,1)| \leq |[0,1]| \leq |\mathbb{R}| = |(0,1)|$ mittels $f\colon \mathbb{R} \to (0,1), x \mapsto \frac{e^x}{1+e^x}$.

Kor $|\mathbb{N}| < \left|2^{\mathbb{N}}\right| = \left|\{0,1\}^{\mathbb{N}}\right| = |[0,1]| = |\mathbb{R}|$

Kor Da $|\Sigma^*| = |\mathbb{N}|$ für jedes nicht leere, abzählbare Alphabet Σ (Gödel oder Cantor):

- ▷ Die Menge der C/Java/Python-Programme ist abzählbar.
- ▷ Nicht jede abzählbare Menge kann durch ein Programm aufgezählt werden.
  (Nicht jedes $f \in \{0,1\}^{\mathbb{N}}$ bzw. $x \in \mathbb{R}$ ist berechenbar (mit bel. Genauigkeit).)

Def $|A| \cdot |B| := |A \times B|$ und $|B|^{|A|} := |B^A|$.

Lem $(|C|^{|B|})^{|A|} = |(C^B)^A| = |C^{A\times B}| = |C|^{|A|\cdot|B|} = |C^{B\times A}| = |(C^A)^B|$ (Currying)
(Vgl. $C^{A\times B}$, $(C^B)^A$, C^{B^A} für $A = [n], B = [m], C = [k]$; $f \in A^{B\times C}$ als Tabelle/Matrix.)

Kor $|\mathbb{R}| = \left|2^{\mathbb{N}}\right| \leq \left|\mathbb{N}^{\mathbb{N}}\right| \leq \left|\mathbb{R}^{\mathbb{N}}\right| = \left|(2^{\mathbb{N}})^{\mathbb{N}}\right| = \left|2^{\mathbb{N}\times\mathbb{N}}\right| = \left|2^{\mathbb{N}}\right|$.

Kor $|\mathbb{R}| < \left|2^{\mathbb{R}}\right| = \left|\{0,1\}^{\mathbb{R}}\right| \leq \left|\mathbb{R}^{\mathbb{R}}\right| = \left|(2^{\mathbb{N}})^{\mathbb{R}}\right| = \left|2^{\mathbb{N}\times\mathbb{R}}\right| \leq \left|2^{\mathbb{R}^2}\right| \leq \left|2^{\mathbb{R}^{\mathbb{N}}}\right| \leq \left|2^{\mathbb{R}}\right|$

Kor $|\mathbb{N}| < ??? < \left|2^{\mathbb{N}}\right| = |\mathbb{R}| < \left|2^{2^{\mathbb{N}}}\right| = \left|2^{\mathbb{R}}\right| < \ldots$ (siehe Kontinuumshypothese.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

2 Grundlagen

Grundbegriffe der Logik

Mengen

Folgen, Tupel und Wörter

Die natürlichen Zahlen, Induktionsbeweise

Relationen

Funktionen

Kardinalität: Vergleiche von unendlichen Mengen

Zusammenfassung

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Grundlagen – Zusammenfassung

- Mengen:
  - Grundlage für alle anderen Zusammenfassungen, Relationen, Funktionen
  - Operationen: Schnitt, Vereinigung, Differenz, sym. Differenz, Komplement (bzgl. Grundmenge), Potenzmenge
  - Visualisierung: Venn-Diagramme, KV-Diagramme
- Relationen: $R \subseteq A_1 \times A_2 \times \ldots \times A_k$
  - Grundlage für relationale Datenbanken, Funktionen, Ordnungen
  - Operationen (neben Mengenoperationen): Join, Projektion, Konkatenation, relationales Produkt, (reflexiv-)transitiver Abschluss
  - Visualisierung: Digraph

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Grundlagen – Zusammenfassung

- Ordnungen: $R \subseteq A \times A$
  - R reflexiv, antisymmetrisch und transitiv
  - Grundlage für Optimierung/Maximierung/Minimierung
  - Visualisierung: Hasse-Diagramm
- Äquivalenzrelationen: $R \subseteq A \times A$
  - R reflexiv, symmetrisch und transitiv
  - Grundlage für Abstraktion
  - Visualisierung: Digraph, Äquivalenzklassen

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Grundlagen – Zusammenfassung

- Funktionen: $\{f\colon A \to B\} = B^A \subseteq 2^{A\times B}$
  - Grundlage für Beschreibung von Eingabe-Ausgabe-Verhalten von Programmen
  - Visualisierung: Funktionsgraph (manchmal Digraph als Relation)
- Kardinalität:
  - Vergleich von Mengen bzgl. der Anzahl der zusammengefassten Objekte
  - $|A| \leq |B|$, falls $f\colon A \xrightarrow{\text{injektiv}} B$ existiert.
  - $|A| = |B|$, falls $|A| \leq |B|$ und $|B| \leq |A|$,

    oder falls $f\colon A \xrightarrow{\text{bijektiv}} B$ existiert.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

1 Einführung

2 Grundlagen

3 Graphentheorie

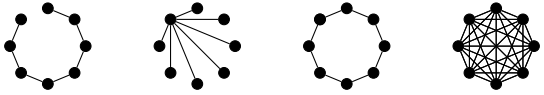
4 Kombinatorik

5 Logik

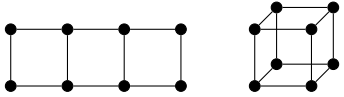
6 Grundlagen der Gruppen- und Zahlentheorie

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Bisher Graphen zur Darstellung von Endorelationen $E \subseteq V \times V$, z.B. Hasse-Diagramm einer partiellen Ordnung.
- Graphentheorie ist nicht an den konkreten Objekten V, sondern an den Eigenschaften von E (z.B. E^*, $E^* \cap (E^\top)^*$, $(E \cup E^\top)^*$) interessiert (s. Topologie).
- Beispiel: Netzwerktopologien von 8 Servern mit bidirektionaler Verbindung



(Pfad, Stern, Kreis/Ring, vollständig)



(Gitter, (Hyper)Würfel)

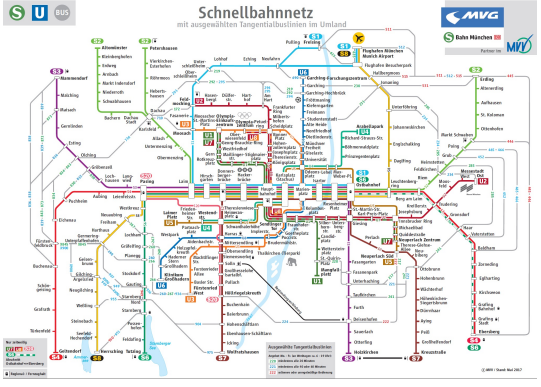
(Längster kürzester Pfad (Durchmesser)? Wie viele Verbindungen (Kanten) können ausfallen, bis ein Server unerreichbar ist? Anzahl an Kanten (Kosten)?)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Verkehrsnetze:

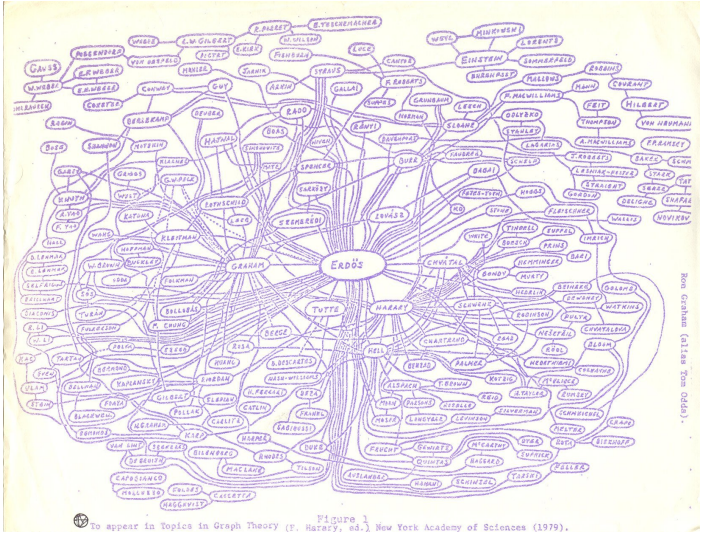
  Wegfindung/Navigation, billigste/schnellste/umweltfreundlichste Verbindung, alternative Routen bei Staus/Sperrungen, ...

- Bsp.: MVV-Netz



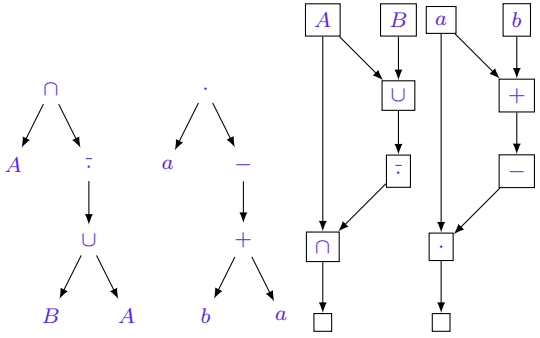
[Quelle]

- Erdős-Zahl:



[Quelle]

- Syntax von Termen/Formeln, Schaltkreise:
- Bsp.: $A \cap \overline{(B \cup A)}$ und $a \cdot (-(b + a))$ als Syntaxbäume und als „Schaltkreise“ (Subtermsharing)



- Programmiersprachen:

  Syntax, Semantik, Compiler, Programmanalyse, Clone, Refactoring, UML, ...

- Bsp.: Syntax (abstrakter Syntaxbaum)

```
def gcd(a,b):
  while b != 0:
    if a > b:
      a = a - b
    else:
      b = b - a
  return a
```



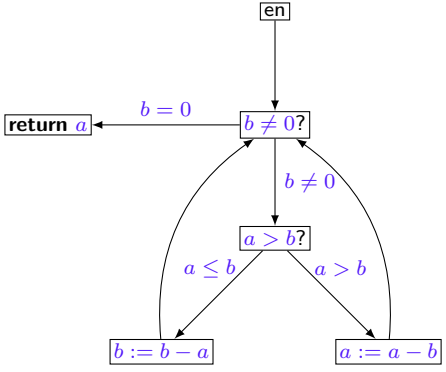
[Quelle]

- Programmiersprachen:

  Syntax, Semantik, Compiler, Programmanalyse, Clone, Refactoring, UML, ...

- Bsp.: Kontrollfluss

```
def gcd(a,b):
  while b != 0:
    if a > b:
      a = a - b
    else:
      b = b - a
  return a
```

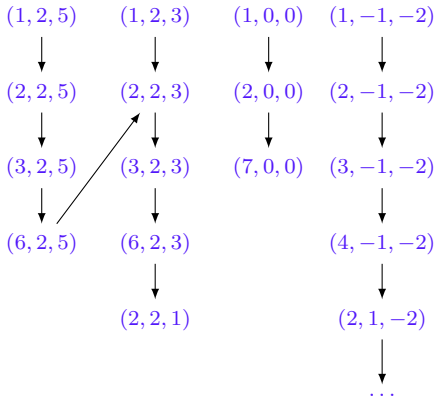


- Programmiersprachen:

  Syntax, Semantik, Compiler, Programmanalyse, Clone, Refactoring, UML, ...

- Bsp.: Semantik (Programmzustand: (pc,a,b))

```
def gcd(a,b):                1
  while b != 0:              2
    if a > b:                3
      a = a - b              4
    else:                    5
      b = b - a              6
  return a                   7
```

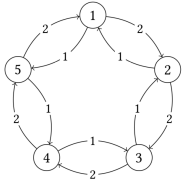


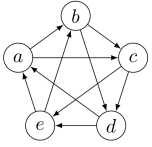
Beispielanwendungen von Graphen

- Weitere Beispiele aus zwei Master-Vorlesungen, die ich regelmäßig halte (Algorithmic Game Theory und Computational Social Choice):

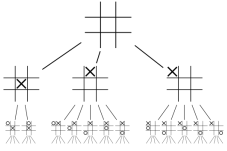
	R	P	S
R	0	-1	$+1$
P	$+1$	0	-1
S	-1	$+1$	0

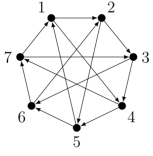






R_1	R_2	R_3
a	d	c
b	e	e
c	a	b
d	b	d
e	c	a





3. Graphentheorie
   - Grundlagen (Wiederholung)
   - Einfache Graphen (ungerichtet, schlingenfrei)
   - Matchings

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

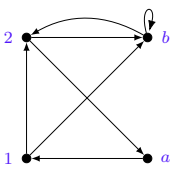
Graphen (Erinnerung, siehe Relationen)

Def Ein (gerichteter) Graph $G = (V, E)$ (auch Digraph) besteht aus

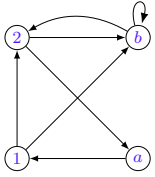
- einer Menge V von Knoten (auch Ecken, Vertices; üblicherweise $V \neq \emptyset$).
- einer Menge (Relation) $E \subseteq V \times V$ von Kanten (engl.: edges).
  - Nachfolger von $u \in V$: $uE = \{v \in V \mid uEv\}$ (Nachfahren: uE^+)
  - Vorgänger von $v \in V$: $Ev = \{u \in V \mid uEv\}$ (Vorfahren: E^+u)
  - $(u, u) \in E$ nennt man eine Schlinge oder Schleife.

$G' = (V', E')$ ist ein Teilgraph von G **:gdw** $V' \subseteq V$ und $E' \subseteq E$.
$G[V'] := (V', E \cap (V' \times V'))$ ist der durch $V' \subseteq V$ induzierte Teilgraph.

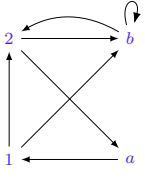
- Verschiedene graphische Darstellungen z.B.:



oder



oder



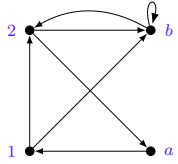
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Adjazenzmatrix (Wiederholung)

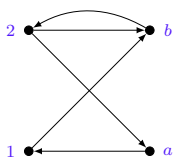
Def Adjazenzmatrix $(a_{i,j})_{i,j\in[n]}$ von $G = (V, E)$ für eine fixierte Aufzählung $V = \{v_1, \ldots, v_n\}$ ($|V| = n$) mit $a_{i,j} := 1$, falls $v_i E v_j$, sonst $a_{i,j} := 0$

- D.h. die Matrixdarstellung von E bzgl. $V = \{v_1, \ldots, v_n\}$.
- Jede Matrix $(m_{i,j})_{i,j\in[n]}$ „induziert“ auch einen Graphen (iEj **gdw** $m_{i,j} \neq 0$).

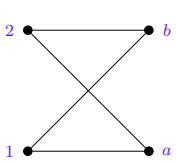
- Beispiel für $V = \{1, 2, a, b\}$



$$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$



$$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$



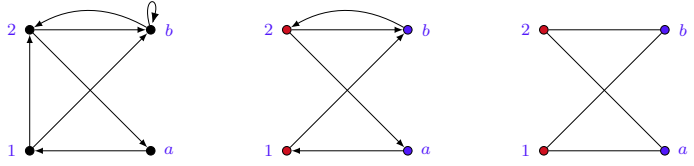
$$\begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

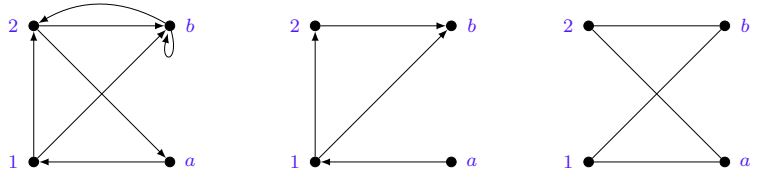
(Matrizen sind unbeschriftete Tabellen/2d-Tupel; sowohl runde als auch eckige Klammern werden für Matrizen verwendet.)

Def Ein (gerichteter) Graph $G = (V, E)$ ist

- endlich **:gdw** $|V| < \infty$.
- bipartit **:gdw** $V = A \cup B$ mit $A \cap B = \emptyset$ und $E \subseteq (A \times B) \cup (B \times A)$.
  (D.h. nur Kanten zwischen A und B; siehe 2-färbbar/Knotenfärbung später.)
  Häufig kurz „$V = A \uplus B$“ als Hinweis, dass A, B disjunkt sind.
- ungerichtet **:gdw** E ist symmetrisch ($E = E^\top$).
- schleifenfrei **:gdw** E ist irreflexiv ($\mathrm{Id}_V \cap E = \emptyset$).
- einfach **:gdw** E ist symmetrisch und irreflexiv.
  Statt der gerichteten Kanten $(s, t), (t, s)$ kurz eine ungerichtete Kante $\{s, t\}$.

Zunächst allgemeine (gerichtete) Graphen, später (einfache) Graphen.





Def Eine Folge $(v_0, \ldots, v_l) \in V^{l+1}$ ist ein (gerichteter) Pfad der Länge l in $G = (V, E)$ **:gdw** jeder „Schritt“ $(v_i, v_{i+1}) \in E$ führt entlang einer Kante.

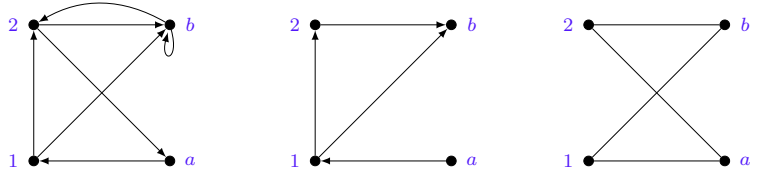
- Häufig auch ohne äußere Klammern $v_0, v_1, \ldots, v_l$ oder als Wort $v_0 v_1 \ldots v_l$.
- „s-t-Pfad“: $v_0 = s$ und $t = v_l$.
- Man kommt mittels der l Schritte $(v_0, v_1), \ldots, (v_{l-1}, v_l) \in E$ von v_0 nach v_l.
  „Länge“ gleichbedeutend mit „Anzahl Schritte“ (da keine Gewichte).
- (v_0) ist ein Pfad der Länge $l = 0$ („für alle $i \in \emptyset$: …“ immer wahr).
- Erinnerung: $(s, t) \in E^l$ **gdw** es gibt einen s-t-Pfad der Länge l.

Def Ein Pfad ist einfach **:gdw** alle v_i sind paarweise verschieden.

(D.h. kein Knoten kommt mehrmals in der Folge vor/wird mehrmals besucht.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gerichtete Pfade und Distanz



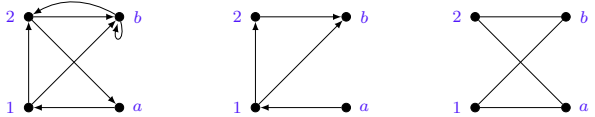
Def Eine Folge $(v_0, \ldots, v_l) \in V^{l+1}$ ist ein (gerichteter) Pfad der Länge l in $G = (V, E)$ **:gdw** jeder „Schritt“ $(v_i, v_{i+1}) \in E$ führt entlang einer Kante.

Def Distanz $d(u, v)$: minimale Schrittzahl, um von u nach v zu gelangen.

- $d(u, v) := \min\left(\{k \in \mathbb{N}_0 \colon (u, v) \in E^k\} \cup \{\infty\}\right)$.
- $d(u, v) = \infty$ **gdw** es gibt keinen Pfad von u nach v.
- Im Allgemeinen: $d(u, v) \neq d(v, u)$; falls $E = E^\top$, dann stets $d(u, v) = d(v, u)$.
  (Z.B. in mittlerem Graph: $d(a, b) = 2$ und $d(b, a) = \infty$.)

⚠ Erinnerung: Falls $|V| = n$, dann $E^* = E^{<n}$ (nur einfache Pfade relevant).
Daher: Falls $|V| = n$, dann entweder $d(u, v) < n$ oder $d(u, v) = \infty$.
(Für GAD: Gilt für gewichtete Graphen i.A. nicht mehr.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science



Def Ein Zyklus (Kreis(-lauf)) ist ein Pfad $v_0, \ldots, v_{l-1}, v_l$ mit $l \geq 1$ und $v_0 = v_l$.

(Geschlossener Pfad mit mindestens einem Schritt.)

Der Zyklus ist einfach **:gdw** $v_0, \ldots, v_{l-1}$ ist ein einfacher Pfad.

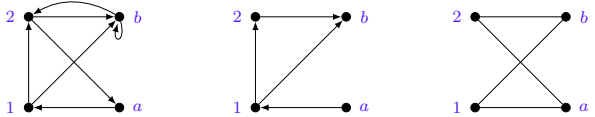
- Mit $v_0, \ldots, v_i, \ldots, v_l$ ist auch $v_i, \ldots, v_l, v_1, \ldots, v_i$ ein Zyklus.
- l ist wieder die Länge (Anzahl Schritte) des Zyklus.
- Ein Zyklus der Länge 1 ist immer eine Schleife.
- Soweit nicht explizit angegeben, besteht jeder Zyklus aus mindestens einer Kante (d.h. $l \geq 1$).

  (Manchmal ist es einfacher, den leeren Zyklus explizit zuzulassen.)

⚠ Für ungerichtete Graphen wird noch verlangt, dass jede Kante höchstens einmal verwendet wird. (Sonst ergibt eine ungerichtete Kante schon einen Zyklus).

Dann hat ein Zyklus mindestens Länge $l \geq 3$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science



Def Ein Zyklus (Kreis(-lauf)) ist ein Pfad $v_0, \ldots, v_{l-1}, v_l$ mit $l \geq 1$ und $v_0 = v_l$.

(Geschlossener Pfad mit mindestens einem Schritt.)

Der Zyklus ist einfach **:gdw** $v_0, \ldots, v_{l-1}$ ist ein einfacher Pfad.

Fkt Jeder Zyklus enthält einen einfachen Zyklus (Kreis).

(Unter allen Paaren (i, j) mit $i < j$ und $v_i = v_j$ wähle ein Paar mit $j - i$ minimal.)

Def Ein Graph ist azyklisch **:gdw** er enthält keinen (einfachen) Zyklus.

- „DAG“ kurz für „directed acyclic graph“.
- Das Hasse-Diagramm einer partiellen Ordnung ist immer ein DAG.
  Falls $G = (V, E)$ DAG, dann ist E^* partielle Ordnung auf V.
  (Ein DAG muss noch kein Hasse-Diagramm sein.)
- Siehe gewurzelte Bäume später (Hasse-Diagramme bzgl. Präfixordnung).

Zusammenhangskomponenten (Erinnerung)

Fkt Für einen Graphen $G = (V, E)$ definieren $E^* \cap (E^*)^\top$ und $(E \cup E^\top)^*$ Äquivalenzrelationen auf V, welche spezielle Teilgraphen induzieren.

Def Die Äquivalenzklassen bzgl.

- $E^* \cap (E^*)^\top$ heißen starke Zusammenhangskomponenten (SCC) von G.

  (u, v sind in derselben SCC **gdw** es gibt einen u-v-Pfad und einen v-u-Pfad.)

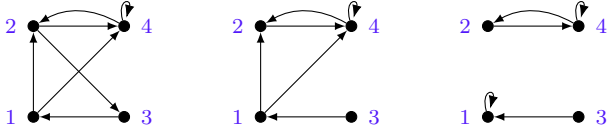
  G ist stark zusammenhängend, falls V eine (die einzige) SCC ist.

- $(E \cup E^\top)^*$ heißen Zusammenhangskomponenten (CC) von G.

  (u, v sind in derselben CC **gdw** es gibt einen ungerichteten u-v-Pfad in $(V, E \cup E^\top)$.)

  G ist zusammenhängend, falls V eine (die einzige) CC ist.

- Beispiele:



Zusammenhangskomponenten (Erinnerung)

Fkt Für einen Graphen $G = (V, E)$ definieren $E^* \cap (E^*)^\top$ und $(E \cup E^\top)^*$ Äquivalenzrelationen auf V, welche spezielle Teilgraphen induzieren.

Def Die Äquivalenzklassen bzgl.

- ▷ $E^* \cap (E^*)^\top$ heißen starke Zusammenhangskomponenten (SCC) von G.

  (u, v sind in derselben SCC **gdw** es gibt einen u-v-Pfad und einen v-u-Pfad.)

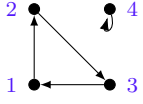
  G ist stark zusammenhängend, falls V eine (die einzige) SCC ist.

- ▷ $(E \cup E^\top)^*$ heißen Zusammenhangskomponenten (CC) von G.

  (u, v sind in derselben CC **gdw** es gibt einen ungerichteten u-v-Pfad in $(V, E \cup E^\top)$.)

  G ist zusammenhängend, falls V eine (die einzige) CC ist.

- ▷ Zykelnotation: Für $f\colon A \xrightarrow{\text{bij.}} A$ mit $|A| < \infty$ sind die SCCs von $G_f = (A, f)$ einfache Zyklen. Je SCC ein einfacher Pfad notiert: $(1, 2, 3)(4)$



Zusammenhangskomponenten (Erinnerung)

Fkt Für einen Graphen $G = (V, E)$ definieren $E^* \cap (E^*)^\top$ und $(E \cup E^\top)^*$ Äquivalenzrelationen auf V, welche spezielle Teilgraphen induzieren.

Def Die Äquivalenzklassen bzgl.

- ▷ $E^* \cap (E^*)^\top$ heißen starke Zusammenhangskomponenten (SCC) von G.

  (u, v sind in derselben SCC **gdw** es gibt einen u-v-Pfad und einen v-u-Pfad.)

  G ist stark zusammenhängend, falls V eine (die einzige) SCC ist.

- ▷ $(E \cup E^\top)^*$ heißen Zusammenhangskomponenten (CC) von G.

  (u, v sind in derselben CC **gdw** es gibt einen ungerichteten u-v-Pfad in $(V, E \cup E^\top)$.)

  G ist zusammenhängend, falls V eine (die einzige) CC ist.

⚠ $E^* \cap (E^\top)^* \subseteq E^* \subseteq (E \cup E^\top)^*$ mit Gleichheit für $E = E^\top$.

- ▷ Für einen ungerichteten Graphen sind CCs und SCCs dasselbe.
- ▷ Spezialfall: Falls E Äquivalenzrelation, dann sind die (S)CCs gerade die Äquivalenzklassen von E, und jede SCC ist ein „vollständiger Graph“.
- ▷ SCCs können effizient z.B. mittels des Algorithmus von Tarjan berechnet werden.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Quotientengraphen

Def Sei $G = (V, E)$ ein Graph und $\sim$ eine Äquivalenzrelation auf V mit $[v]_\sim$ der Äquivalenzklasse von v. Dann ist der Quotient $G/\sim$ von G bzgl. $\sim$ definiert durch

$$G/\sim := (V/\sim, \{([s]_\sim, [t]_\sim) \mid (s,t) \in E\})$$

(Fasse $[s]_\sim$ als Farbe auf: Kante von Farbe c nach c' in $G/\sim$ **gdw** mindestens eine Kante von einem Knoten der Farbe c zu einem Knoten der Farbe c' in G; „Abstraktion“.)

Bsp links: $G = (V, E)$; rechts: $G/\sim$ für $\sim := (E^* \cap (E^*)^\top)$:



(Jede SCC wird zu einem einzelnen Knoten zusammengefasst; siehe topologisches Sortieren.)

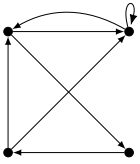
Bsp links: $G = (\mathbb{Z}, \{(x, x+1) \mid x \in \mathbb{Z}\})$; rechts: $G/\equiv_3$:



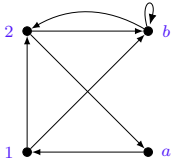
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Graphisomorphie

- Die Graphentheorie beschäftigt sich mit den Eigenschaften der Kantenrelation E (wie Zusammenhang, Erreichbarkeit), nicht aber mit den konkreten Objekten V.
- Häufig gibt man daher in der graphischen Darstellung keine konkreten Bezeichner an.
- Z.B. für $V = \{1, 2, a, b\}$ und $E = \{(1,2), (1,b), (2,a), (2,b), (a,1), (b,2), (b,b)\}$



statt



- Graphen haben zunächst keine geometrische Bedeutung; das Layout sollte der Lesbarkeit dienen.
- Man identifiziert daher Graphen, die „bis auf Knotenumbenennung“ dieselbe Kantenrelation besitzen, und nennt solche Graphen isomorph.

  („isos-morphe“: „gleiche Form/Gestalt“; „bis auf/modulo Isomorphie derselbe Graph“.)

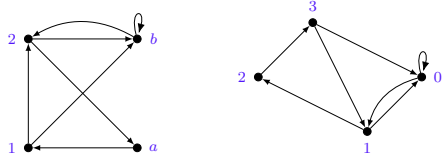
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Graphisomorphie

Def Zwei Graphen $G = (V_G, E_G), H = (V_H, E_H)$ sind isomorph ($G \cong H$) **:gdw** es gibt $\beta\colon V_G \xrightarrow{\text{bij}} V_H$ mit: $(u, v) \in E_G$ **gdw** $(\beta(u), \beta(v)) \in E_H$ für alle $u, v \in V_G$.

- β ist dann ein Graphisomorphismus; falls $G = H$, auch Graphautomorphismus.
- β benennt die Knoten so um, dass die Kanten respektiert werden, d.h. für alle $k \in \mathbb{N}_0$ gilt induktiv dann auch: $(u, v) \in E_G^k$ **gdw** $(\beta(u), \beta(v)) \in E_H^k$.

- Beispiel:



Fkt $\cong$ ist eine Äquivalenzrelation.
Für den Rest der Graphentheorie werden isomorphe (Di)Graphen identifiziert, soweit nicht explizit anders angegeben.
(Es ist kein Algorithmus bekannt, der in polynomieller Zeit entscheiden kann, ob zwei Graphen isomorph sind. Dennoch geht man davon aus, dass dieses Problem nicht NP-vollständig ist.)

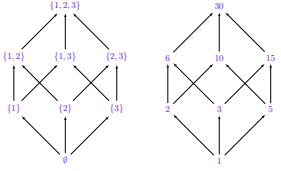
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Graphisomorphie

Def Zwei Graphen $G = (V_G, E_G), H = (V_H, E_H)$ sind isomorph ($G \cong H$) **:gdw** es gibt $\beta\colon V_G \xrightarrow{\text{bij}} V_H$ mit: $(u, v) \in E_G$ **gdw** $(\beta(u), \beta(v)) \in E_H$ für alle $u, v \in V_G$.

- β ist dann ein Graphisomorphismus; falls $G = H$, auch Graphautomorphismus.
- β benennt die Knoten so um, dass die Kanten respektiert werden, d.h. für alle $k \in \mathbb{N}_0$ gilt induktiv dann auch: $(u, v) \in E_G^k$ **gdw** $(\beta(u), \beta(v)) \in E_H^k$.

- Beispiel: Hasse-Diagramme (Teilmengen und Teiler)



Fkt $\cong$ ist eine Äquivalenzrelation.

Für den Rest der Graphentheorie werden isomorphe (Di)Graphen identifiziert, soweit nicht explizit anders angegeben.

(Es ist kein Algorithmus bekannt, der in polynomieller Zeit entscheiden kann, ob zwei Graphen isomorph sind. Dennoch geht man davon aus, dass dieses Problem nicht NP-vollständig ist.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Direktes Produkt $G_1 \times G_2$ zweier Graphen $G_i = (V_i, E_i)$:

$$G_1 \times G_2 := \big(V_1 \times V_2, \{((s_1, s_2), (t_1, t_2)) \mid (s_1, t_1) \in E_1, (s_2, t_2) \in E_2\}\big)$$

Kartesisches Produkt $G_1 \square G_2$ zweier Graphen $G_i = (V_i, E_i)$:

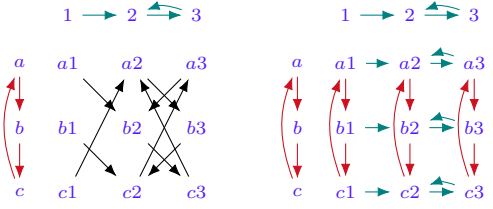
$$G_1 \square G_2 := \Big(V_1 \times V_2, \{((v_1, s_2), (v_1, t_2)) \mid v_1 \in V_1, (s_2, t_2) \in E_2\} \cup \{((s_1, v_2), (t_1, v_2)) \mid (s_1, t_1) \in E_1, v_2 \in V_2\}\Big)$$

(Idee: synchron (im Gleichschritt) vs. verschränkt (interleaving) parallel.)

Fkt Sind E_1, E_2 beide reflexiv, dann ist $G_1 \square G_2$ ein Teilgraph von $G_1 \times G_2$.

Fkt $G_1 \diamond G_2 \cong G_2 \diamond G_1$ und $(G_1 \diamond G_2) \diamond G_3 \cong G_1 \diamond (G_2 \diamond G_3)$ für $\diamond \in \{\square, \times\}$.

Bsp (Paare als Wörter)



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

3 Graphentheorie

Grundlagen (Wiederholung)

Einfache Graphen (ungerichtet, schlingenfrei)

Spezielle Familien von einfachen Graphen

Bäume: Spannbaum, Eigenschaften

Gradfolge

Euler- und Hamiltonkreis

Planare Graphen und Knotenfärbungen

Matchings

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Einfache Graphen (Erinnerung)

Def Einfacher Graph $G = (V, E)$: keine Schleifen, Kanten ungerichtet
Eine ungerichtete Kante $\{u, w\}$ statt zwei gerichteter Kanten $(u, w), (w, u)$.

- ▷ $E \subseteq \binom{V}{2}$ mit $\binom{V}{2} := \{\{u, w\} \subseteq V \mid u \neq w\}$ ($E = E^\top$, $E \cap \mathsf{Id}_V = \emptyset$)
- ▷ $(v_0, v_1, \ldots, v_l) \in V^{l+1}$ Pfad, falls: $\{v_{i-1}, v_i\} \in E$ für alle $i \in [l]$.
  $(v_0, v_1, \ldots, v_l) \in V^{l+1}$ Zyklus, falls: Pfad mit $v_l = v_0$, der jede Kante $\{v_{i-1}, v_i\}$ höchstens einmal (in höchstens eine Richtung; bedingt $l \geq 3$) benutzt.
- ▷ β Isomorphismus, falls: $\{u, v\} \in E_G$ **gdw** $\{\beta(u), \beta(v)\} \in E_H$.
- ▷ Nachbarschaft: $\Gamma(u) := \{w \in V \mid \{u, w\} \in E\}$ (statt Nachfolger/Vorgänger).
  (Knoten-)Grad: $\deg(u) := |\Gamma(u)|$.

- ▷ Aber hier „Graph“ kurz für „endlicher einfacher Graph“ (außer anders angegeben).
- ▷ Angepasste Definition von „Zyklus“, damit nicht schon eine einzelne Kante $\{u, w\}$ zu einem Kreis (u, w, u) führt.
- ▷ SCCs und CCs gleichbedeutend, da $E^* \cap (E^\top)^* = E^* = (E \cup E^\top)^*$.
- ▷ Zunächst einige wichtige Familien von einfachen Graphen.

(Bis auf Isomorphie: Angegebene Knotenmengen und Kantenrelationen nur zur Präzisierung.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Charakterisierung von bipartiten Graphen

- Satz: Ein einfacher Graph ist bipartit genau dann, wenn er keinen Kreis ungerader Länge enthält.
- Beweis: Ein bipartiter Graph enthält keinen Kreis ungerader Länge.
  - Nehmen wir zum Widerspruch an, dass $G = (A \uplus B, E)$ ein bipartiter Graph ist, der einen Kreis $v_0, v_1, \ldots, v_l$ mit l ungerade enthält.
  - OBdA („Ohne Beschränkung der Allgemeinheit“) sei $v_0 \in A$ (andernfalls vertausche A und B).
  - Dann muss gelten, dass $v_1 \in B$, $v_2 \in A$, usw. Schließlich erhalten wir, dass $v_l \in B$ weil l ungerade ist.
  - Dies kann jedoch nicht sein, da $v_l = v_0 \in A$.

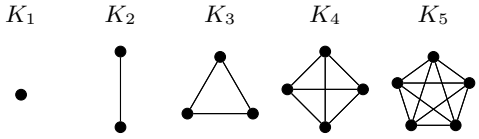
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Satz: Ein einfacher Graph ist bipartit genau dann, wenn er keinen Kreis ungerader Länge enthält.
- Beweis: Ein Graph, der keinen Kreis ungerader Länge enthält, ist bipartit.
  - Sei $G = (V, E)$ ein Graph, der keinen Kreis ungerader Länge enthält. OBdA sei G zshgd. (ansonsten betrachte jede Zshgskomponente getrennt).
  - Nimm einen beliebigen Knoten $x \in V$. Definiere $A, B \subseteq V$, so dass $x \in A$ und für alle weiteren Knoten $v \in V \setminus \{x\}$:
    $v \in A$ wenn der kürzeste Pfad zwischen x und v gerade Länge hat und
    $v \in B$ wenn der kürzeste Pfad zwischen x und v ungerade Länge hat.
  - Betrachte zwei Knoten $v_1, v_2 \in A$ oder $v_1, v_2 \in B$.
  - Aufgrund der Konstruktion gibt es zwei kürzeste Pfade $v_1, \ldots, x$ und $v_2, \ldots, x$, die entweder beide gerade oder beide ungerade Länge haben.
  - Sei x' der erste gemeinsame Knoten dieser beiden Pfade. Da die Reststücke $x', \ldots, x$ beide Länge $d(x', x)$ haben, haben $v_1, \ldots, x'$ und $v_2, \ldots, x'$ gleiche Parität und $v_1, \ldots, x', \ldots, v_2$ somit gerade Länge.
  - Es kann also nicht sein, dass $\{v_1, v_2\} \in E$, da der Kreis $v_1, \ldots, x', \ldots, v_2, v_1$ sonst ungerade Länge hätte. Es folgt, dass G bipartit ist.

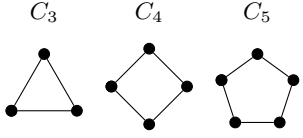
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Vollständige, Kreis- und Pfadgraphen

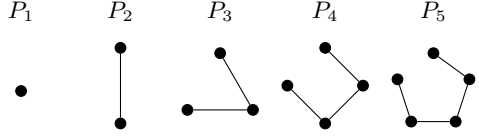
Def Vollständiger Graph $K_n := \left([n], \binom{[n]}{2}\right)$



Def Kreisgraph $C_n := ([n], \{\{i, (i \bmod n) + 1\} \mid i \in [n]\})$ für $n \geq 3$



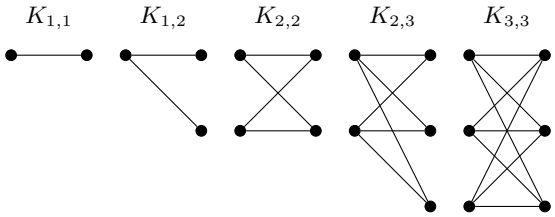
Def Pfadgraph $P_n := ([n], \{\{i, i+1\} \mid i \in [n-1]\})$



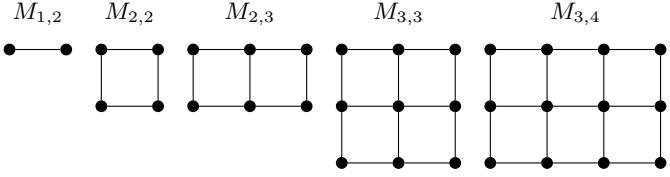
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Vollständige bipartite Graphen und Gittergraphen

Def Vollst. bipartiter Graph $K_{m,n} := ([m+n], \{\{i,j\} \mid i \in [m],\ j \in [m+n] \setminus [m]\})$



Def Gittergraph $M_{m,n} := ([m] \times [n], \{\{(i,j),(k,l)\} \mid |i-k| + |j-l| = 1\}) \cong P_m \square P_n$

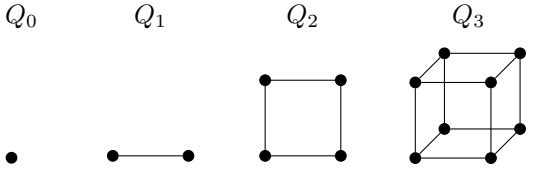


No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Hyperwürfel

Def Hyperwürfel der Dimension d: ($Q_0 \cong P_1$, $Q_{d+1} \cong P_2 \square Q_d$)

$$Q_d := (\{0,1\}^d, \{\{u,v\} \mid \sum_{i=1}^{d} |u_i - v_i| = 1\}) \quad \text{mit} \quad Q_0 := (\{\varepsilon\}, \emptyset)$$



$K_n, C_n, P_n, K_{m,n}, M_{m,n}, Q_d$: Vergleich

Def Ein Graph ist k-regulär **:gdw** $\deg(v) = k$ für alle $v \in V$.

- K_n: $|V| = n$, $|E| = \binom{n}{2} = \frac{n(n-1)}{2}$, $(n-1)$-regulär
  In höchstens einem Schritt von jedem Knoten zu jedem anderen.
- C_n: $|V| = n$, $|E| = n$, 2-regulär ($n \geq 3$)
  In höchstens $\lfloor \frac{n}{2} \rfloor$ Schritten von jedem Knoten zu jedem anderen.
- P_n: $|V| = n$, $|E| = n - 1$, regulär **gdw** $n \leq 2$
  In höchstens $n - 1$ Schritten von jedem Knoten zu jedem anderen.
- $K_{m,n}$: $|V| = m + n$, $|E| = mn$, regulär **gdw** $n = m$
  In höchstens 2 Schritten von jedem Knoten zu jedem anderen.
- $M_{m,n}$: $|V| = mn$, $|E| = (m-1)n + m(n-1)$, regulär **gdw** $m, n \leq 2$
  In höchstens $(m-1) + (n-1)$ Schritten von jedem Knoten zu jedem anderen.
- Q_d: $|V| = 2^d$, $|E| = 2^{d-1}d$, d-regulär
  In höchstens d Schritten von jedem Knoten zu jedem anderen.

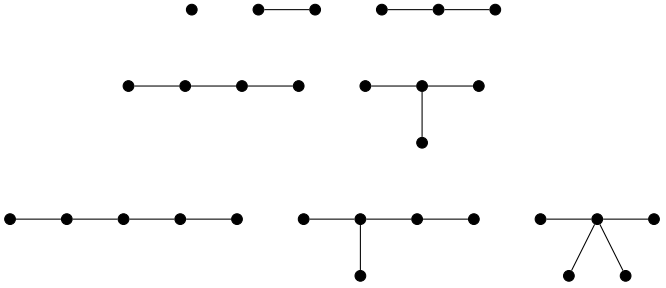
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Ein Graph $G = (V, E)$ ist ein Baum **:gdw** G ist einfach, zshgd. und azyklisch.

$u \in V$ ist ein Blatt, falls $\deg(u) \leq 1$, ansonsten ein innerer Knoten.

Def Ein Graph ist ein Wald, falls alle seine CCs Bäume sind.

▷ Beispiel: Ein Wald, dessen Zshgskomponenten (bis auf Isomorphie) gerade alle Bäume mit höchstens 5 Knoten sind

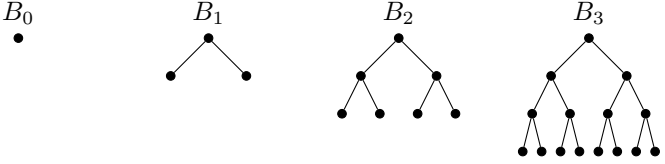


No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Perfekter Binärbaum (häufig auch vollständiger Binärbaum) der Höhe h

$$B_h := (\{0,1\}^{\leq h}, \{\{u, ux\} \mid u \in \{0,1\}^{<h}, x \in \{0,1\}\})$$

B_0 B_1 B_2 B_3



Bzgl. unserer Wahl $V = \{0,1\}^{\leq h}$:

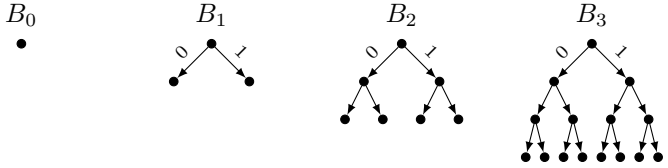
- u Blatt **gdw** $u \in \{0,1\}^h$.
- Daher gibt es 2^h Blätter und insgesamt $\sum_{i=0}^{h} 2^i = 2^{h+1} - 1$ Knoten.
- $\deg(\varepsilon) = 0$, falls $h = 0$, sonst $\deg(\varepsilon) = 2$.
- Für $h > 0$: $\deg(u) = 1$ **gdw** $u \in \{0,1\}^h$.

Perfekte Binärbäume

Def Perfekter Binärbaum (häufig auch vollständiger Binärbaum) der Höhe h

$$B_h := (\{0,1\}^{\leq h}, \{\{u, ux\} \mid u \in \{0,1\}^{<h}, x \in \{0,1\}\})$$

B_0 B_1 B_2 B_3

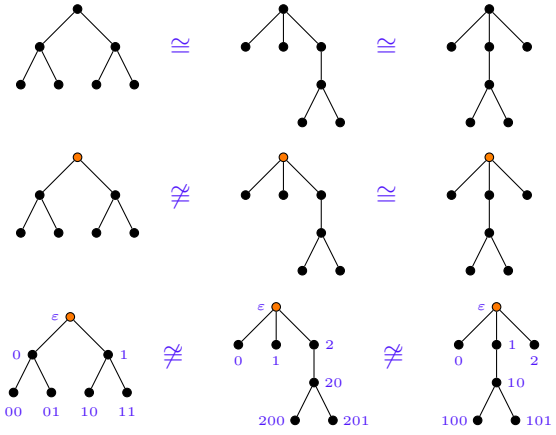


⚠ B_h wird häufig implizit als gewurzelter oder sogar geordneter Baum betrachtet:

- ε als Wurzel.
- Kante $\{u, ux\}$ wird als gerichtete Kante (u, ux) weg von der Wurzel gelesen. (Alternativ: hin zur Wurzel als (ux, u).)
- u ist dann das Elter seiner Kinder $u0$ (linkes Kind) und $u1$ (rechtes Kind).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Bäume: einfach, gewurzelt, geordnet



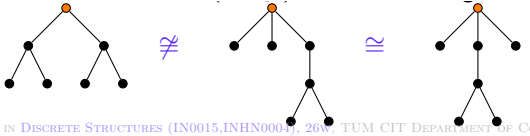
Def Ein Wurzelbaum (V, E, r) ist ein Baum (V, E), in dem ein Knoten $r \in V$ als Wurzel markiert ist.

Def Die Höhe von (V, E, r) ist $\max\{d(r, v) \mid v \in V\}$ (max. Wurzel-Blatt-Distanz).

⚠ Implizit sind alle Kanten von der Wurzel r weggerichtet:

- ▷ $d(r, \cdot)$ wird durch jede (implizit gerichtete) Kante um 1 erhöht.
- ▷ (V, E) ist das Hasse-Diagramm der partiellen Ordnung E^* auf V.
- ▷ r ist das kleinste Element mit $V = rE^*$.
- ▷ $u \in V$ ist der Elter der Kinder $uE = \{v \in \Gamma(u) \mid d(r, v) = d(r, u) + 1\}$.
- ▷ u ist das Minimum von uE^*.
- ▷ (V, E, r) wird üblicherweise so dargestellt, dass ein Elter u über seinen Nachfahren uE^+ liegt.

Bsp B_2 mit unterschiedlichen Wurzeln (orange). Die Höhe hängt von r ab.



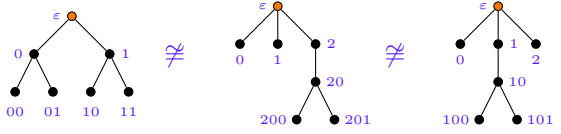
No commercial use, only for use in Discrete Structures (IN0015,INHN0005), 26W, TUM CIT Department of Computer Science

Def Ein Wurzelbaum (V, E, r) ist ein Baum (V, E), in dem ein Knoten $r \in V$ als Wurzel markiert ist.

Def Ein geordneter Baum ist ein Wurzelbaum, so dass für jeden Knoten u eine totale Ordnung auf dessen Kindern uE definiert ist (1., 2., ... Kind von links).

Def (V, E) ist ein Präfixbaum bezüglich des Alphabets Σ **:gdw** E^* ist die Präfixordnung auf Σ^* eingeschränkt auf $V \subseteq \Sigma^*$ und V ist präfixabgeschlossen (falls $uw \in V$, dann $u \in V$).

Bsp Eine totale Ordnung auf Σ (z.B. $\Sigma = \{0 < 1 < 2\}$) induziert auch einen Präfixbaum:

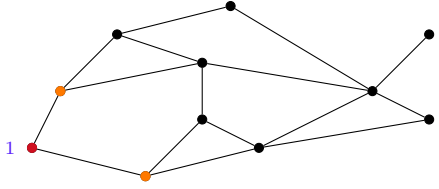


Fkt Man kann einen (endlichen, geordneten) Wurzelbaum (V, E, r) mit Hilfe eines ausreichend großen (geordneten) Alphabets $\Sigma = \{a_1, \ldots, a_n\}$ ($a_i < a_{i+1}$) so umbenennen, dass er ein Präfixbaum wird.

Def Ein Spannbaum $T = (V, E')$ eines einfachen Graphen $G = (V, E)$ ist ein Teilgraph ($E' \subseteq E$), der ein Baum ist, welcher alle Knoten überspannt.

Thm Jeder zshgd. Graph $G = (V, E)$ hat einen Spannbaum $T = (V, E')$ ($E' \subseteq E$).

- Wähle bel. Startknoten $s \in V$ und setze $V_1 := \{s\}$, $E_1 := \emptyset$, $i := 1$.
- Wiederhole bis $V_i = V$: Wähle bel. $e = \{u, v\} \in E$ mit $u \in V_i$ und $v \notin V_i$; dann setze $V_{i+1} := V_i \cup \{v\}$, $E_{i+1} := E_i \cup \{e\}$, $i := i + 1$.



⚠ Für je zwei Knoten genau ein einfacher Pfad in Baum. (Siehe z.B. STP)

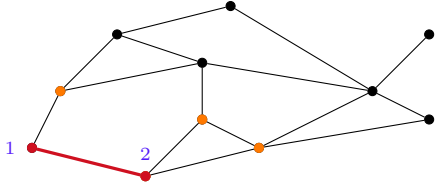
- Die Konstruktion eines Spannbaums, den man iterativ um neu „entdeckte“, aber bis jetzt noch nicht „erforschte“ Knoten ergänzt, ist eine natürliche Form des „knowledge retrievals“ (siehe auch web crawler), aber auch z.B. des maschinellen Schließens (siehe Resolution später).

Spannbaum (Erreichbarkeit) und Hamiltonpfade/-kreise

Def Ein Spannbaum $T = (V, E')$ eines einfachen Graphen $G = (V, E)$ ist ein Teilgraph ($E' \subseteq E$), der ein Baum ist, welcher alle Knoten überspannt.

Thm Jeder zshgd. Graph $G = (V, E)$ hat einen Spannbaum $T = (V, E')$ ($E' \subseteq E$).

- ▷ Wähle bel. Startknoten $s \in V$ und setze $V_1 := \{s\}$, $E_1 := \emptyset$, $i := 1$.
- ▷ Wiederhole bis $V_i = V$: Wähle bel. $e = \{u, v\} \in E$ mit $u \in V_i$ und $v \notin V_i$; dann setze $V_{i+1} := V_i \cup \{v\}$, $E_{i+1} := E_i \cup \{e\}$, $i := i + 1$.



⚠ Für je zwei Knoten genau ein einfacher Pfad in Baum. (Siehe z.B. STP)

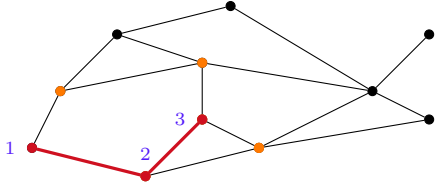
- ▷ Die Konstruktion eines Spannbaums, den man iterativ um neu „entdeckte“, aber bis jetzt noch nicht „erforschte“ Knoten ergänzt, ist eine natürliche Form des „knowledge retrievals“ (siehe auch web crawler), aber auch z.B. des maschinellen Schließens (siehe Resolution später).

Spannbaum (Erreichbarkeit) und Hamiltonpfade/-kreise

Def Ein Spannbaum $T = (V, E')$ eines einfachen Graphen $G = (V, E)$ ist ein Teilgraph ($E' \subseteq E$), der ein Baum ist, welcher alle Knoten überspannt.

Thm Jeder zshgd. Graph $G = (V, E)$ hat einen Spannbaum $T = (V, E')$ ($E' \subseteq E$).

- Wähle bel. Startknoten $s \in V$ und setze $V_1 := \{s\}$, $E_1 := \emptyset$, $i := 1$.
- Wiederhole bis $V_i = V$: Wähle bel. $e = \{u, v\} \in E$ mit $u \in V_i$ und $v \notin V_i$; dann setze $V_{i+1} := V_i \cup \{v\}$, $E_{i+1} := E_i \cup \{e\}$, $i := i + 1$.



⚠ Für je zwei Knoten genau ein einfacher Pfad in Baum. (Siehe z.B. STP)

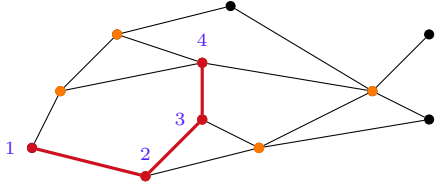
- Die Konstruktion eines Spannbaums, den man iterativ um neu „entdeckte“, aber bis jetzt noch nicht „erforschte“ Knoten ergänzt, ist eine natürliche Form des „knowledge retrievals“ (siehe auch web crawler), aber auch z.B. des maschinellen Schließens (siehe Resolution später).

Spannbaum (Erreichbarkeit) und Hamiltonpfade/-kreise

Def Ein Spannbaum $T = (V, E')$ eines einfachen Graphen $G = (V, E)$ ist ein Teilgraph ($E' \subseteq E$), der ein Baum ist, welcher alle Knoten überspannt.

Thm Jeder zshgd. Graph $G = (V, E)$ hat einen Spannbaum $T = (V, E')$ ($E' \subseteq E$).

- Wähle bel. Startknoten $s \in V$ und setze $V_1 := \{s\}$, $E_1 := \emptyset$, $i := 1$.
- Wiederhole bis $V_i = V$: Wähle bel. $e = \{u, v\} \in E$ mit $u \in V_i$ und $v \notin V_i$; dann setze $V_{i+1} := V_i \cup \{v\}$, $E_{i+1} := E_i \cup \{e\}$, $i := i + 1$.



⚠ Für je zwei Knoten genau ein einfacher Pfad in Baum. (Siehe z.B. STP)

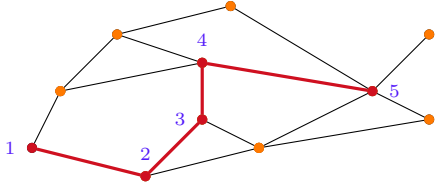
- Die Konstruktion eines Spannbaums, den man iterativ um neu „entdeckte“, aber bis jetzt noch nicht „erforschte“ Knoten ergänzt, ist eine natürliche Form des „knowledge retrievals“ (siehe auch web crawler), aber auch z.B. des maschinellen Schließens (siehe Resolution später).

Spannbaum (Erreichbarkeit) und Hamiltonpfade/-kreise

Def Ein Spannbaum $T = (V, E')$ eines einfachen Graphen $G = (V, E)$ ist ein Teilgraph ($E' \subseteq E$), der ein Baum ist, welcher alle Knoten überspannt.

Thm Jeder zshgd. Graph $G = (V, E)$ hat einen Spannbaum $T = (V, E')$ ($E' \subseteq E$).

- Wähle bel. Startknoten $s \in V$ und setze $V_1 := \{s\}$, $E_1 := \emptyset$, $i := 1$.
- Wiederhole bis $V_i = V$: Wähle bel. $e = \{u, v\} \in E$ mit $u \in V_i$ und $v \notin V_i$; dann setze $V_{i+1} := V_i \cup \{v\}$, $E_{i+1} := E_i \cup \{e\}$, $i := i + 1$.



⚠ Für je zwei Knoten genau ein einfacher Pfad in Baum. (Siehe z.B. STP)

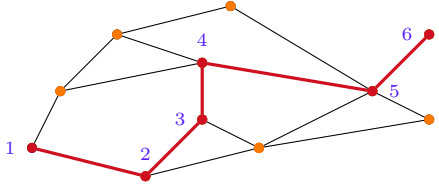
- Die Konstruktion eines Spannbaums, den man iterativ um neu „entdeckte“, aber bis jetzt noch nicht „erforschte“ Knoten ergänzt, ist eine natürliche Form des „knowledge retrievals“ (siehe auch web crawler), aber auch z.B. des maschinellen Schließens (siehe Resolution später).

Spannbaum (Erreichbarkeit) und Hamiltonpfade/-kreise

Def Ein Spannbaum $T = (V, E')$ eines einfachen Graphen $G = (V, E)$ ist ein Teilgraph ($E' \subseteq E$), der ein Baum ist, welcher alle Knoten überspannt.

Thm Jeder zshgd. Graph $G = (V, E)$ hat einen Spannbaum $T = (V, E')$ ($E' \subseteq E$).

- Wähle bel. Startknoten $s \in V$ und setze $V_1 := \{s\}$, $E_1 := \emptyset$, $i := 1$.
- Wiederhole bis $V_i = V$: Wähle bel. $e = \{u, v\} \in E$ mit $u \in V_i$ und $v \notin V_i$; dann setze $V_{i+1} := V_i \cup \{v\}$, $E_{i+1} := E_i \cup \{e\}$, $i := i + 1$.



⚠ Für je zwei Knoten genau ein einfacher Pfad in Baum. (Siehe z.B. STP)

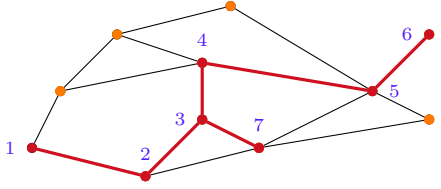
- Die Konstruktion eines Spannbaums, den man iterativ um neu „entdeckte“, aber bis jetzt noch nicht „erforschte“ Knoten ergänzt, ist eine natürliche Form des „knowledge retrievals“ (siehe auch web crawler), aber auch z.B. des maschinellen Schließens (siehe Resolution später).

Spannbaum (Erreichbarkeit) und Hamiltonpfade/-kreise

Def Ein Spannbaum $T = (V, E')$ eines einfachen Graphen $G = (V, E)$ ist ein Teilgraph ($E' \subseteq E$), der ein Baum ist, welcher alle Knoten überspannt.

Thm Jeder zshgd. Graph $G = (V, E)$ hat einen Spannbaum $T = (V, E')$ ($E' \subseteq E$).

- Wähle bel. Startknoten $s \in V$ und setze $V_1 := \{s\}$, $E_1 := \emptyset$, $i := 1$.
- Wiederhole bis $V_i = V$: Wähle bel. $e = \{u, v\} \in E$ mit $u \in V_i$ und $v \notin V_i$; dann setze $V_{i+1} := V_i \cup \{v\}$, $E_{i+1} := E_i \cup \{e\}$, $i := i + 1$.



⚠ Für je zwei Knoten genau ein einfacher Pfad in Baum. (Siehe z.B. STP)

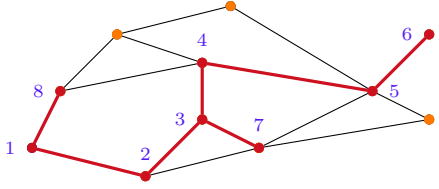
- Die Konstruktion eines Spannbaums, den man iterativ um neu „entdeckte“, aber bis jetzt noch nicht „erforschte“ Knoten ergänzt, ist eine natürliche Form des „knowledge retrievals“ (siehe auch web crawler), aber auch z.B. des maschinellen Schließens (siehe Resolution später).

Spannbaum (Erreichbarkeit) und Hamiltonpfade/-kreise

Def Ein Spannbaum $T = (V, E')$ eines einfachen Graphen $G = (V, E)$ ist ein Teilgraph ($E' \subseteq E$), der ein Baum ist, welcher alle Knoten überspannt.

Thm Jeder zshgd. Graph $G = (V, E)$ hat einen Spannbaum $T = (V, E')$ ($E' \subseteq E$).

- ▷ Wähle bel. Startknoten $s \in V$ und setze $V_1 := \{s\}$, $E_1 := \emptyset$, $i := 1$.
- ▷ Wiederhole bis $V_i = V$: Wähle bel. $e = \{u, v\} \in E$ mit $u \in V_i$ und $v \notin V_i$; dann setze $V_{i+1} := V_i \cup \{v\}$, $E_{i+1} := E_i \cup \{e\}$, $i := i + 1$.



⚠ Für je zwei Knoten genau ein einfacher Pfad in Baum. (Siehe z.B. STP)

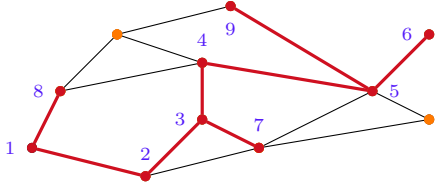
- ▷ Die Konstruktion eines Spannbaums, den man iterativ um neu „entdeckte“, aber bis jetzt noch nicht „erforschte“ Knoten ergänzt, ist eine natürliche Form des „knowledge retrievals“ (siehe auch web crawler), aber auch z.B. des maschinellen Schließens (siehe Resolution später).

Spannbaum (Erreichbarkeit) und Hamiltonpfade/-kreise

Def Ein Spannbaum $T = (V, E')$ eines einfachen Graphen $G = (V, E)$ ist ein Teilgraph ($E' \subseteq E$), der ein Baum ist, welcher alle Knoten überspannt.

Thm Jeder zshgd. Graph $G = (V, E)$ hat einen Spannbaum $T = (V, E')$ ($E' \subseteq E$).

- ▷ Wähle bel. Startknoten $s \in V$ und setze $V_1 := \{s\}$, $E_1 := \emptyset$, $i := 1$.
- ▷ Wiederhole bis $V_i = V$: Wähle bel. $e = \{u, v\} \in E$ mit $u \in V_i$ und $v \notin V_i$; dann setze $V_{i+1} := V_i \cup \{v\}$, $E_{i+1} := E_i \cup \{e\}$, $i := i + 1$.



⚠ Für je zwei Knoten genau ein einfacher Pfad in Baum. (Siehe z.B. STP)

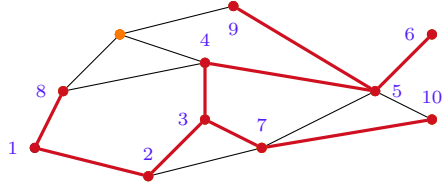
- ▷ Die Konstruktion eines Spannbaums, den man iterativ um neu „entdeckte“, aber bis jetzt noch nicht „erforschte“ Knoten ergänzt, ist eine natürliche Form des „knowledge retrievals“ (siehe auch web crawler), aber auch z.B. des maschinellen Schließens (siehe Resolution später).

Spannbaum (Erreichbarkeit) und Hamiltonpfade/-kreise

Def Ein Spannbaum $T = (V, E')$ eines einfachen Graphen $G = (V, E)$ ist ein Teilgraph ($E' \subseteq E$), der ein Baum ist, welcher alle Knoten überspannt.

Thm Jeder zshgd. Graph $G = (V, E)$ hat einen Spannbaum $T = (V, E')$ ($E' \subseteq E$).

- ▷ Wähle bel. Startknoten $s \in V$ und setze $V_1 := \{s\}$, $E_1 := \emptyset$, $i := 1$.
- ▷ Wiederhole bis $V_i = V$: Wähle bel. $e = \{u, v\} \in E$ mit $u \in V_i$ und $v \notin V_i$; dann setze $V_{i+1} := V_i \cup \{v\}$, $E_{i+1} := E_i \cup \{e\}$, $i := i + 1$.



⚠ Für je zwei Knoten genau ein einfacher Pfad in Baum. (Siehe z.B. STP)

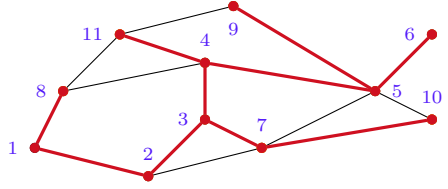
- ▷ Die Konstruktion eines Spannbaums, den man iterativ um neu „entdeckte“, aber bis jetzt noch nicht „erforschte“ Knoten ergänzt, ist eine natürliche Form des „knowledge retrievals“ (siehe auch web crawler), aber auch z.B. des maschinellen Schließens (siehe Resolution später).

Spannbaum (Erreichbarkeit) und Hamiltonpfade/-kreise

Def Ein Spannbaum $T = (V, E')$ eines einfachen Graphen $G = (V, E)$ ist ein Teilgraph ($E' \subseteq E$), der ein Baum ist, welcher alle Knoten überspannt.

Thm Jeder zshgd. Graph $G = (V, E)$ hat einen Spannbaum $T = (V, E')$ ($E' \subseteq E$).

- ▷ Wähle bel. Startknoten $s \in V$ und setze $V_1 := \{s\}$, $E_1 := \emptyset$, $i := 1$.
- ▷ Wiederhole bis $V_i = V$: Wähle bel. $e = \{u, v\} \in E$ mit $u \in V_i$ und $v \notin V_i$; dann setze $V_{i+1} := V_i \cup \{v\}$, $E_{i+1} := E_i \cup \{e\}$, $i := i + 1$.



⚠ Für je zwei Knoten genau ein einfacher Pfad in Baum. (Siehe z.B. STP)

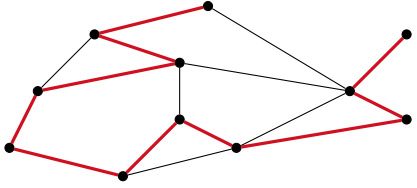
- ▷ Die Konstruktion eines Spannbaums, den man iterativ um neu „entdeckte“, aber bis jetzt noch nicht „erforschte“ Knoten ergänzt, ist eine natürliche Form des „knowledge retrievals“ (siehe auch web crawler), aber auch z.B. des maschinellen Schließens (siehe Resolution später).

Spannbaum (Erreichbarkeit) und Hamiltonpfade/-kreise

Def Ein Spannbaum $T = (V, E')$ eines einfachen Graphen $G = (V, E)$ ist ein Teilgraph ($E' \subseteq E$), der ein Baum ist, welcher alle Knoten überspannt.

Thm Jeder zshgd. Graph $G = (V, E)$ hat einen Spannbaum $T = (V, E')$ ($E' \subseteq E$).

- ▷ Wähle bel. Startknoten $s \in V$ und setze $V_1 := \{s\}$, $E_1 := \emptyset$, $i := 1$.
- ▷ Wiederhole bis $V_i = V$: Wähle bel. $e = \{u, v\} \in E$ mit $u \in V_i$ und $v \notin V_i$; dann setze $V_{i+1} := V_i \cup \{v\}$, $E_{i+1} := E_i \cup \{e\}$, $i := i + 1$.



Def Ein Hamiltonpfad ist ein Spannbaum, der ein Pfad ist. Ein Hamiltonkreis ist ein einfacher Kreis, der alle Knoten überspannt (ein geschlossener Hamiltonpfad).

(Nicht jeder zshgd. Graph hat einen Hamiltonpfad, nicht jeder Ham-Pfad ergibt einen Kreis.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Spannbaum (Beweis)

Thm Jeder zshgd. Graph $G = (V, E)$ hat einen Spannbaum $T = (V, E')$ ($E' \subseteq E$).

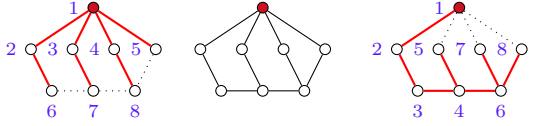
- ▷ Wähle bel. Startknoten $s \in V$ und setze $V_1 := \{s\}$, $E_1 := \emptyset$, $i := 1$.
- ▷ Wiederhole bis $V_i = V$: Wähle bel. $e = \{u, v\} \in E$ mit $u \in V_i$ und $v \notin V_i$;
  Aktualisiere $V_{i+1} := V_i \cup \{v\}$, $E_{i+1} := E_i \cup \{e\}$.

▷ Sei $G = (V, E)$ ein bel. Graph ($V \neq \emptyset$). Wir zeigen mittels Induktion nach $i \in \mathbb{N}$:
Für alle $i \leq |V|$ ist (V_i, E_i) ein Baum mit $|V_i| = |E_i| + 1 = i$.

- ▷ I.B.: $(V_1, E_1) = (\{s\}, \emptyset)$ ist ein Baum mit $|V_1| = 1 = |E_1| + 1$.
- ▷ I.S.: Fixiere $i \in \mathbb{N}$ beliebig.
  I.A.: Falls $i \leq |V|$, dann ist (V_i, E_i) Baum mit $|V_i| = |E_i| + 1 = i$.
  I.Beh.: Falls $i + 1 \leq |V|$, ist (V_{i+1}, E_{i+1}) Baum mit $|V_{i+1}| = i + 1 = |E_{i+1}| + 1$.
  Beweis: Wir nehmen $i + 1 \leq |V|$ an, so dass $i < |V|$.
  Nach I.A. ist (V_i, E_i) Baum mit $|V_i| = i$, $|E_i| = i - 1$, also $V \setminus V_i \neq \emptyset$.
  Wähle bel. $t \in V \setminus V_i$. Da G zshgd., gibt es einen s-t-Pfad π.
  Da $s \in V_i$ und $t \in V \setminus V_i$, enthält π eine Kante $\{u, v\}$ mit $u \in V_i$ und $v \in V \setminus V_i$. Es gibt also eine Kante zwischen V_i und $V \setminus V_i$. Jede solche Kante verbindet einen neuen Knoten mit (V_i, E_i), schließt daher keinen Kreis.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Unterschiedliche Heuristiken zur Auflösung der nichtdeterministischen Wahl:
  - Sei $u_1, u_2, \ldots, u_n$ die Reihenfolge, in der die Knoten hinzugefügt werden.
    Breadth-first (BFS): wähle $\{u_i, v\}$ mit i minimal (FIFO, queue).
    Depth-first (DFS): wähle $\{u_i, v\}$ mit i maximal (LIFO, stack).



(Zahlen: „Zeit des ersten Besuchs“.)

  - Algorithmus von Tarjan: erweitert DFS zur Bestimmung der SCCs.
  - Algorithmus von Dijkstra: wähle $\{u_i, v\}$, so dass v minimalen Abstand zu u_1 hat.
  - Algorithmus von Prim: wähle $\{u_i, v\}$ mit minimalen „Kosten“.
  - A*-Algorithmus: baut einen Spannbaum, bis ein Ziel erreicht ist, basierend auf einer Heuristik für den „besten nächsten Knoten“ (siehe auch „best-first search“).
- Für gerichtete Graphen bestimmt der Algorithmus aus dem Beweis einen gewurzelten Spannbaum der von s erreichbaren Knoten sE^*.

Spannbaum: Folgerungen

Kor Jeder zshgd. Graph $G = (V, E)$ muss $|E| \geq |V| - 1$ erfüllen.

(Spannbaum (V, E') hat schon $|E'| = |V| - 1$ Kanten.)

Kontraposition: Jeder Graph $G = (V, E)$ mit $|E| < |V| - 1$ ist nicht zshgd.

Kor Jeder zshgd. Graph $G = (V, E)$ mit $|E| \geq |V|$ enthält einen Kreis.

(Spannbaum (V, E') benutzt $|E'| = |V| - 1$ Kanten; jede weitere Kante ergibt Kreis in G.)

Kor Jeder Graph $G = (V, E)$ mit $|E| \geq |V|$ enthält einen Kreis.

(Schubfachprinzip: Mindestens eine CC $G_i = (V_i, E_i)$ muss $|E_i| \geq |V_i|$ erfüllen.)

Kontraposition: Jeder azyklische Graph $G = (V, E)$ muss $|E| \leq |V| - 1$ erfüllen.

Kor Für jeden Baum $G = (V, E)$ gilt $|E| = |V| - 1$.

(Zshgd: $|E| \geq |V| - 1$; azyklisch: $|E| \leq |V| - 1$.)

⚠ $P_n \oplus C_m$ erfüllt $|E| = |V| - 1$ ($|V| = m + n$, $|E| = m + n - 1$),
aber weder zshgd. noch azyklisch.

(„$\oplus$“: disjunkte Vereinigung von Graphen; Knoten notfalls umbenennen.)

Lem In jedem Baum gibt es zwischen je zwei Knoten genau einen einfachen Pfad.

(Zwei unterschiedliche einfache Pfade $u, \ldots, v, \ldots, w$ und $u, \ldots, v', \ldots, w$ (u, v, v', w paarweise verschieden) ergeben einen geschlossenen Pfad $u, \ldots, v, \ldots, w, \ldots, v', \ldots, u$, der einen Kreis enthält.)

Kor Ein Graph $G = (V, E)$ ist ein Baum **gdw** zwischen je zwei Knoten gibt es genau einen einfachen Pfad.

Kor Entfernt man eine beliebige Kante, dann zerfällt ein Baum in genau zwei Bäume.

(Sei $G = (V, E)$ Baum mit mind. einer Kante; wähle bel. Kante $e = \{u, w\} \in E$; setze $E' = E \setminus \{e\}$; jeder Knoten kann zumindest noch u oder w in (V, E') erreichen; da $\{u, w\}$ einziger einfacher u-w-Pfad in G, zerfällt (V, E') in genau zwei azyklische CCs/Bäume.)

Kor Für jeden Graph $G = (V, E)$ mit $|E| = |V| - 1$ gilt: zshgd. **gdw** azyklisch.

(Falls zshgd., dann muss G sein eigener Spannbaum sein; falls azyklisch, muss jede CC $|E_i| \leq |V_i| - 1$ erfüllen, womit es nur eine CC geben kann.)

Kor $G = (V, E)$ ist ein Baum **gdw** G ist zshgd. mit $|V| = |E| + 1$ **gdw** G ist azyklisch mit $|V| = |E| + 1$ **gdw** für alle $u, w \in V$ gibt es genau einen einfachen u-w-Pfad in G.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

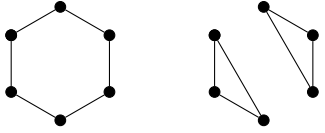
Gradfolge eines einfachen Graphen

- Kennt man $|V|$ und $|E|$, kann man manchmal bereits sagen, dass ein Graph $G = (V, E)$ mehrere CCs hat ($|E| < |V| - 1$) oder einen Kreis besitzt ($|E| \geq |V|$), ohne dass man E (die Struktur von G) selbst kennt.
- Die Gradfolge erhält etwas mehr Informationen von G, „abstrahiert“ aber dennoch von der konkreten Struktur:

Def Jedem (einfachen) Graphen $G = (V, E)$ können wir seine Gradfolge zuordnen: $(\deg(v_1), \deg(v_2), \ldots, \deg(v_n))$ für $V = \{v_1, v_2, \ldots, v_n\}$

  - Da wir Graphen nur bis auf Isomorphie unterscheiden, kann man stets annehmen, dass die Gradfolge aufsteigend sortiert ist (d.h. man betrachtet die Multimenge von Knotengraden).

- Beispiel: Bis auf Isomorphie die beiden einzigen (einfachen) Graphen mit der Gradfolge $(2, 2, 2, 2, 2, 2)$:



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

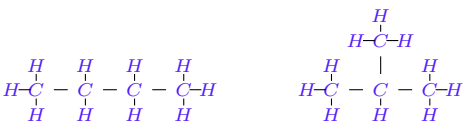
Gradfolge eines einfachen Graphen

- Kennt man $|V|$ und $|E|$, kann man manchmal bereits sagen, dass ein Graph $G = (V, E)$ mehrere CCs hat ($|E| < |V| - 1$) oder einen Kreis besitzt ($|E| \geq |V|$), ohne dass man E (die Struktur von G) selbst kennt.
- Die Gradfolge erhält etwas mehr Informationen von G, „abstrahiert“ aber dennoch von der konkreten Struktur:

Def Jedem (einfachen) Graphen $G = (V, E)$ können wir seine Gradfolge zuordnen: $(\deg(v_1), \deg(v_2), \ldots, \deg(v_n))$ für $V = \{v_1, v_2, \ldots, v_n\}$

- Da wir Graphen nur bis auf Isomorphie unterscheiden, kann man stets annehmen, dass die Gradfolge aufsteigend sortiert ist (d.h. man betrachtet die Multimenge von Knotengraden).

Bsp Strukturisomere des Butans C_4H_{10} (nicht isomorphe Realisierungen der Gradfolge):



- Anhand der Gradfolge lässt sich aber dennoch in manchen Fällen bereits entscheiden, ob ein Graph (bzw. sogar jeder Graph mit dieser Gradfolge) eine gewisse Eigenschaft (nicht) besitzt, ohne dass man hierzu die eigentliche Struktur analysieren muss.
- Beispiel:

  Ein Graph mit Gradfolge $(0, 2, 2, 2, 2)$ kann nicht zusammenhängend sein.
- Beispiel:

  Ein Graph mit Gradfolge $(1, 1, 1, 1)$ kann nicht zusammenhängend sein.
- Beispiel:

  Was gilt für den Zusammenhang eines Graphen mit Gradfolge $(1, 1, 2, 2, 2)$ bzw. $(1, 1, 1, 1, 2, 2, 2)$ bzw. $(1, 2, 3, 4, 5)$?

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Handschlaglemma (doppeltes Abzählen)

Lem „Handschlaglemma“

Für jeden (einfachen) Graphen $G = (V, E)$ gilt:

$$2\,|E| = \sum_{i \in [n]} \deg(v_i)$$

da jede Kante $\{u, v\} \in E$ doppelt gezählt, einmal durch $\deg(u)$ und einmal durch $\deg(v)$.

Kor Sei G ein (einfacher) Graph mit Gradfolge $(d_1, d_2, \ldots, d_n)$:

- $\sum_{i \in [n]} d_i$ ist gerade.
  Genauer: die Anzahl der Knoten von ungeradem Grad muss gerade sein.
- Falls $\sum_{i \in [n]} d_i < 2|V| - 2$, dann hat G mind. zwei CCs.
- Falls $\min d_i + \max d_i \geq |V| - 1$, dann ist G zshgd.
- Falls $\sum_{i \in [n]} d_i \geq 2|V|$, dann ist G zyklisch.

- Damit gibt es gar keinen Graphen mit Gradfolge $(1, 2, 3, 4, 5)$ und $(1, 1, 1, 1, 2, 2, 2)$ kann nicht zusammenhängend sein.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Lem „Handschlaglemma“

Für jeden (einfachen) Graphen $G = (V, E)$ gilt:

$$2\,|E| = \sum_{i \in [n]} \deg(v_i)$$

da jede Kante $\{u, v\} \in E$ doppelt gezählt, einmal durch $\deg(u)$ und einmal durch $\deg(v)$.

Lem Jeder Baum $G = (V, E)$ mit $|V| \geq 2$ hat mind. zwei Blätter.

▷ Beweis: Sei $G = (V, E)$ Baum mit Gradfolge $(d_1, d_2, \ldots, d_n)$.

Annahme: es gibt höchstens ein Blatt, d.h. $d_i \geq 2$ für $i \geq 2$.

Dann folgt $d_1 = 0$, da

$$2|V| - 2 = 2|E| = \sum_{i \in [n]} d_i \geq d_1 + 2(n-1) = d_1 + 2|V| - 2$$

womit G nicht zusammenhängend wäre.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Handschlaglemma (doppeltes Abzählen)

Lem „Handschlaglemma“

Für jeden (einfachen) Graphen $G = (V, E)$ gilt:

$$2\,|E| = \sum_{i \in [n]} \deg(v_i)$$

da jede Kante $\{u, v\} \in E$ doppelt gezählt, einmal durch $\deg(u)$ und einmal durch $\deg(v)$.

- Anmerkung: Allgemeines doppeltes Abzählen:
  - Sei $R \subseteq A \times B$ binäre Relation.
    Stelle R als Matrix M_R dar. Dann:
    Die Summe über die Zeile von $a \in A$ ergibt $|aR|$ (Anzahl Nachfolger).
    Die Summe über die Spalte von $b \in B$ ergibt $|Rb|$ (Anzahl Vorgänger).
    Die Summe über alle Einträge von M_R ergibt $|R|$.
    Summe aller Zeilensummen $= |R| =$ Summe über die Spaltensummen.
  - Für das Handschlaglemma betrachte die Inzidenzrelation $I = \{(v, e) \in V \times E \mid v \in e\}$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Algorithmus von Havel-Hakimi

Def Algorithmus von Havel-Hakimi:

- ▷ Eingabe: Aufsteigend sortierte Folge $(d_1, d_2, \ldots, d_n)$.
- ▷ Falls $d_1 < 0$ oder $d_n > n - 1$: Abbruch, es existiert kein entsprechender Graph.
- ▷ Falls $d_n = 0$: Gib $([n], \emptyset)$ zurück.
- ▷ $(d'_1, \ldots, d'_{n-1}) := \mathsf{sort}(d_1, \ldots, d_{n-d_n-1}, d_{n-d_n} - 1, \ldots, d_{n-1} - 1)$.

  Versuche rekursiv $G' = ([n-1], E')$ zu $(d'_1, \ldots, d'_{n-1})$ zu bestimmen.

  Dann gibt es $v_1, \ldots, v_{d_n} \in [n-1]$ mit $\deg(v_i) = d_{n-i} - 1$.

  Gib $G = ([n], E)$ mit $E = E' \cup \{\{n, v_i\} \mid i \in [d_n]\}$ zurück.

⚠ Algorithmus ist so noch nichtdeterministisch in Wahl der v_i.

▷ Beispiel: Eingabe $(1, 1, 2, 3, 4, 4, 5)$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Algorithmus von Havel-Hakimi:

- ▷ Eingabe: Aufsteigend sortierte Folge $(d_1, d_2, \ldots, d_n)$.
- ▷ Falls $d_1 < 0$ oder $d_n > n - 1$: Abbruch, es existiert kein entsprechender Graph.
- ▷ Falls $d_n = 0$: Gib $([n], \emptyset)$ zurück.
- ▷ $(d'_1, \ldots, d'_{n-1}) := \mathsf{sort}(d_1, \ldots, d_{n-d_n-1}, d_{n-d_n} - 1, \ldots, d_{n-1} - 1)$.

  Versuche rekursiv $G' = ([n-1], E')$ zu $(d'_1, \ldots, d'_{n-1})$ zu bestimmen.

  Dann gibt es $v_1, \ldots, v_{d_n} \in [n-1]$ mit $\deg(v_i) = d_{n-i} - 1$.

  Gib $G = ([n], E)$ mit $E = E' \cup \{\{n, v_i\} \mid i \in [d_n]\}$ zurück.

⚠ Algorithmus ist so noch nichtdeterministisch in Wahl der v_i.

▷ Beispiel: 1. Phase: top-down Reduktion der Folge

- ▷ $(1, 1, 2, 3, 4, 4, 5)$ führt zu $\mathsf{sort}(1, 0, 1, 2, 3, 3) = (0, 1, 1, 2, 3, 3)$
- ▷ $(0, 1, 1, 2, 3, 3)$ führt zu $\mathsf{sort}(0, 1, 0, 1, 2) = (0, 0, 1, 1, 2)$
- ▷ $(0, 0, 1, 1, 2)$ führt zu $(0, 0, 0, 0)$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Algorithmus von Havel-Hakimi:

- ▷ Eingabe: Aufsteigend sortierte Folge $(d_1, d_2, \ldots, d_n)$.
- ▷ Falls $d_1 < 0$ oder $d_n > n - 1$: Abbruch, es existiert kein entsprechender Graph.
- ▷ Falls $d_n = 0$: Gib $([n], \emptyset)$ zurück.
- ▷ $(d'_1, \ldots, d'_{n-1}) := \mathsf{sort}(d_1, \ldots, d_{n-d_n-1}, d_{n-d_n} - 1, \ldots, d_{n-1} - 1)$.

  Versuche rekursiv $G' = ([n-1], E')$ zu $(d'_1, \ldots, d'_{n-1})$ zu bestimmen.

  Dann gibt es $v_1, \ldots, v_{d_n} \in [n-1]$ mit $\deg(v_i) = d_{n-i} - 1$.

  Gib $G = ([n], E)$ mit $E = E' \cup \{\{n, v_i\} \mid i \in [d_n]\}$ zurück.

⚠ Algorithmus ist so noch nichtdeterministisch in Wahl der v_i.

▷ Beispiel: 2. Phase: bottom-up Konstruktion

- ▷ Graph zu $(0, 0, 0, 0)$

• • • •

Def Algorithmus von Havel-Hakimi:

- ▷ Eingabe: Aufsteigend sortierte Folge $(d_1, d_2, \ldots, d_n)$.
- ▷ Falls $d_1 < 0$ oder $d_n > n-1$: Abbruch, es existiert kein entsprechender Graph.
- ▷ Falls $d_n = 0$: Gib $([n], \emptyset)$ zurück.
- ▷ $(d'_1, \ldots, d'_{n-1}) := \mathsf{sort}(d_1, \ldots, d_{n-d_n-1}, d_{n-d_n} - 1, \ldots, d_{n-1} - 1)$.

  Versuche rekursiv $G' = ([n-1], E')$ zu $(d'_1, \ldots, d'_{n-1})$ zu bestimmen.

  Dann gibt es $v_1, \ldots, v_{d_n} \in [n-1]$ mit $\deg(v_i) = d_{n-i} - 1$.

  Gib $G = ([n], E)$ mit $E = E' \cup \{\{n, v_i\} \mid i \in [d_n]\}$ zurück.

⚠ Algorithmus ist so noch nichtdeterministisch in Wahl der v_i.

- ▷ Beispiel: 2. Phase: bottom-up Konstruktion
  - ▷ Graph zu $(0, 0, 1, 1, 2)$: Verbinde 5 mit Knoten vom Grad $0, 0$.



Def Algorithmus von Havel-Hakimi:

- ▷ Eingabe: Aufsteigend sortierte Folge $(d_1, d_2, \ldots, d_n)$.
- ▷ Falls $d_1 < 0$ oder $d_n > n - 1$: Abbruch, es existiert kein entsprechender Graph.
- ▷ Falls $d_n = 0$: Gib $([n], \emptyset)$ zurück.
- ▷ $(d'_1, \ldots, d'_{n-1}) := \mathsf{sort}(d_1, \ldots, d_{n-d_n-1}, d_{n-d_n} - 1, \ldots, d_{n-1} - 1)$.

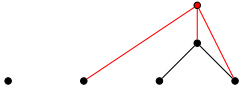
  Versuche rekursiv $G' = ([n-1], E')$ zu $(d'_1, \ldots, d'_{n-1})$ zu bestimmen.

  Dann gibt es $v_1, \ldots, v_{d_n} \in [n-1]$ mit $\deg(v_i) = d_{n-i} - 1$.

  Gib $G = ([n], E)$ mit $E = E' \cup \{\{n, v_i\} \mid i \in [d_n]\}$ zurück.

⚠ Algorithmus ist so noch nichtdeterministisch in Wahl der v_i.

- ▷ Beispiel: 2. Phase: bottom-up Konstruktion
  - ▷ Graph zu $(0, 1, 1, 2, 3, 3)$: Verbinde 6 mit Knoten vom Grad $2, 1, 0$



Algorithmus von Havel-Hakimi

Def Algorithmus von Havel-Hakimi:

- ▷ Eingabe: Aufsteigend sortierte Folge $(d_1, d_2, \ldots, d_n)$.
- ▷ Falls $d_1 < 0$ oder $d_n > n - 1$: Abbruch, es existiert kein entsprechender Graph.
- ▷ Falls $d_n = 0$: Gib $([n], \emptyset)$ zurück.
- ▷ $(d'_1, \ldots, d'_{n-1}) := \mathsf{sort}(d_1, \ldots, d_{n-d_n-1}, d_{n-d_n} - 1, \ldots, d_{n-1} - 1)$.

  Versuche rekursiv $G' = ([n-1], E')$ zu $(d'_1, \ldots, d'_{n-1})$ zu bestimmen.

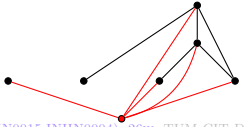
  Dann gibt es $v_1, \ldots, v_{d_n} \in [n-1]$ mit $\deg(v_i) = d_{n-i} - 1$.

  Gib $G = ([n], E)$ mit $E = E' \cup \{\{n, v_i\} \mid i \in [d_n]\}$ zurück.

⚠ Algorithmus ist so noch nichtdeterministisch in Wahl der v_i.

▷ Beispiel: 2. Phase: bottom-up Konstruktion

- ▷ Graph zu $(1, 1, 2, 3, 4, 4, 5)$: Verbinde 7 mit Knoten vom Grad $3, 3, 2, 1, 0$

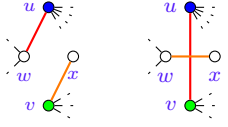


No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Lem Havel-Hakimi succeeds **iff** there is some simple graph with the given degree sequence.

▷ Proof sketch:

- ▷ If Havel-Hakimi succeeds, there is such a simple graph.
- ▷ Let $G = (V, E)$ be a simple graph with sorted degree seq. $d_1 \leq \ldots \leq d_n$.
  Let $\deg(u) = d_n$, $w \in \Gamma(u)$ and $v \notin \Gamma(u) \cup \{u\}$ with $\deg(v) > \deg(w)$.
  As v has more neighbors than w, it must have a neighbor $x \in \Gamma(v) \setminus (\Gamma(w) \cup \{w\})$ (and $x \neq u$ as $\{u, v\} \notin E$).
  We thus can delete $\{v, x\}$ and $\{u, w\}$ and add $\{w, x\}$ and $\{u, v\}$ instead so that now v is a neighbor of u and x a neighbor of w while preserving the node degrees.
  This way we can obtain a graph G' where a node of highest degree d_n is connected to nodes of degree $d_{n-1} \geq d_{n-2} \geq \ldots \geq d_{n-d_n}$ and then proceed recursively as in Havel-Hakimi.



Anmerkungen zu Gradfolgen

- ▷ Der Havel-Hakimi-Algorithmus kann i.A. nicht jede mögliche Realisierung finden. (Betrachte z.B. die Gradfolge $(3,3,3,3,3,3)$ des $K_{3,3}$.)

Def Eine Folge $(d_i)_{i\in[n]}$ ist graphisch **:gdw** $(d_i)_{i\in[n]}$ ist die Gradfolge eines einfachen Graphen.

Def Ist $G=(V,E)$ ein einfacher Graph, dann ist $\overline{G}=(V,\binom{V}{2}\setminus E)$ sein Komplementgraph.

Fkt $(d_1,\ldots,d_n)$ ist graphisch **gdw** $((n-1)-d_n,\ldots,(n-1)-d_1)$ ist graphisch.

Fkt Jede Gradfolge $(d_1,d_2,\ldots,d_n)\in\mathbb{N}_0^n$ (mit $d_i\le d_{i+1}$, $n\ge 2$),

welche $d_1=d_2=1$ und $\sum_{i\in[n]} d_i = 2n-2$ erfüllt,

kann durch einen Baum realisiert werden. (Beweis zur Übung.)

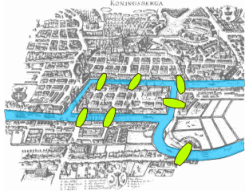
Lem Der Algorithmus von Havel-Hakimi kann auf gerichtete Graphen erweitert werden. Hierfür betrachtet man entsprechend $((|vE|,|Ev|))_{v\in[n]}$ (oBdA $V=[n]$).

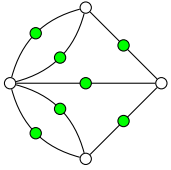
Siehe Kleitman-Wang-Algorithmus.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Eulerkreisproblem und Hamiltonkreisproblem

- Euler: Königsberger Brückenproblem



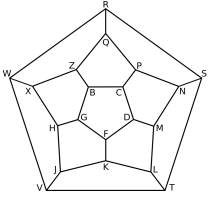


[Quelle]

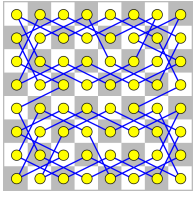
- Hamilton: The Icosian Game; siehe auch: Springerproblem



[Quelle]



[Quelle]



[Quelle]

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Eulerkreise und Hamiltonkreise: Definition

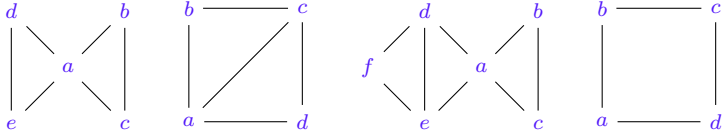
Def Sei $G = (V, E)$ ein einfacher Graph und $\zeta = (v_0, \ldots, v_l)$ ein Kreis ($v_l = v_0$) in G, der jeden Knoten (mind. einmal) besucht (d.h. $\{v_0, v_1, \ldots, v_l\} = V$).

- ▷ ζ ist ein Eulerkreis **:gdw** ζ benutzt jede Kante genau einmal:
  $l = |E|$ und $\{\{v_0, v_1\}, \{v_1, v_2\}, \ldots, \{v_{l-1}, v_l\}\} = E$.
- ▷ ζ ist ein Hamiltonkreis **:gdw** ζ besucht jeden Knoten genau einmal:
  $l = |V|$ und $\{v_0, v_1, \ldots, v_{l-1}\} = V$ (d.h. ζ ist einfach).

Im Fall $|V| = 1$ erlauben wir auch den leeren Kreis $()$.

(Erlaubt man $v_0 \neq v_l$, dann spricht man von einem Euler- bzw. Hamiltonpfad.)

▷ Beispiele: (vlnr) (1) nur Euler; (2) nur Hamilton; (3) keins; (4) beides



Eulerkreise und Hamiltonkreise: Definition

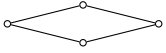
Def Sei $G = (V, E)$ ein einfacher Graph und $\zeta = (v_0, \ldots, v_l)$ ein Kreis ($v_l = v_0$) in G, der jeden Knoten (mind. einmal) besucht (d.h. $\{v_0, v_1, \ldots, v_l\} = V$).

- ζ ist ein Eulerkreis **:gdw** ζ benutzt jede Kante genau einmal:
  $l = |E|$ und $\{\{v_0, v_1\}, \{v_1, v_2\}, \ldots, \{v_{l-1}, v_l\}\} = E$.
- ζ ist ein Hamiltonkreis **:gdw** ζ besucht jeden Knoten genau einmal:
  $l = |V|$ und $\{v_0, v_1, \ldots, v_{l-1}\} = V$ (d.h. ζ ist einfach).

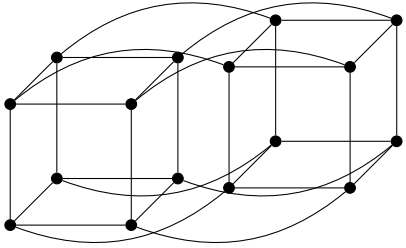
Im Fall $|V| = 1$ erlauben wir auch den leeren Kreis $()$.

(Erlaubt man $v_0 \neq v_l$, dann spricht man von einem Euler- bzw. Hamiltonpfad.)

- Anmerkungen:
  - Schleifen sind für beide Probleme uninteressant (warum?).
  - Für Hamiltonkreise sind Mehrfachkanten uninteressant (warum?).
  - Für Eulerkreise kann man Mehrfachkanten durch Hilfsknoten simulieren:

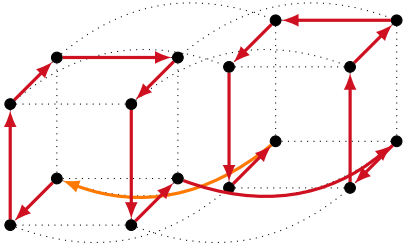

simuliert durch


Gray-Code: Hamiltonpfad/-kreis auf einem Hyperwürfel



$0000, 0001, 0011, 0010, 0110, 0111, 0101, 0100, 1100, 1101, 1111, 1110, 1010, 1011, 1001, 1000$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science



$0000, 0001, 0011, 0010, 0110, 0111, 0101, 0100, 1100, 1101, 1111, 1110, 1010, 1011, 1001, 1000$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Existenz von Euler- und Hamiltonkreisen

Thm Ein zusammenhängender einfacher Graph $G = (V, E)$ besitzt einen Eulerkreis **gdw** für alle $v \in V$ ist $\deg(v)$ gerade.

- ▷ Aus dem Beweis (nächste Folie) ergibt sich direkt ein effizienter Algorithmus für das Eulerkreisproblem.

Thm Das Hamiltonkreisproblem ist NP-vollständig.

- ▷ Informell: aktuell kennt man kein (signifikant) besseres Verfahren, als rekursiv alle einfachen Pfade durchzuprobieren (siehe „Einführung in die theo. Inf.“).
- ▷ Verallgemeinerung: Travelling salesman problem.

Lem Hinreichend nach Dirac: falls $2\deg(v) \geq |V| \geq 3$ für alle $v \in V$, lässt sich jeder längste einfache Pfad zu einem Hamiltonkreis umordnen.

Fkt Für $G = (V, E)$ mit $E \neq \emptyset$ setze $H := (E, \{\{e, e'\} \mid e, e' \in E, |e \cap e'| = 1\})$.
Dann: G hat einen Eulerkreis **gdw** G ist zshgd. und H hat einen Hamiltonkreis.
(D.h. man könnte das Eulerkreisproblem auf das Hamiltonkreisproblem reduzieren.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Beweis: Ein zusammenhängender einfacher Graph $G = (V, E)$, in dem jeder Knoten geraden Grad hat, besitzt eine Eulertour.

  Wir verwenden Induktion nach der Anzahl der Kanten $k := |E|$.

  - I.B.: Sei $k = 3$ (mit weniger Kanten gibt es mindestens einen Knoten mit ungeradem Grad). Jeder zshgd. Graph mit 3 Kanten ist isomorph zum C_3 und besitzt eine Eulertour.
  - I.S.: Sei $k \geq 3$ beliebig fixiert.
    - I.A.: Jeder zshgd. Graph $G = (V, E)$ mit $|E| \leq k$ und nur geraden Knotengraden besitzt eine Eulertour.
    - I.Beh.: Jeder zshgd. Graph $G = (V, E)$ mit $|E| = k + 1$ und nur geraden Knotengraden besitzt eine Eulertour.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Beweis: Ein zusammenhängender einfacher Graph $G = (V, E)$, in dem jeder Knoten geraden Grad hat, besitzt eine Eulertour.
  - I.S.: Sei $k \geq 3$ beliebig fixiert.
    - Beweis der I.Beh.: Sei $G = (V, E)$ zshgd. mit $|E| = k + 1$ und nur geraden Knotengraden.

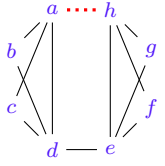
      Fixiere eine beliebige Kante $\{u, w\} \in E$. Sei $G' = (V, E \setminus \{\{u, w\}\})$.

      Wir behaupten: es gibt in G' einen Pfad von u nach w.

      Beweis der Behauptung: Die Zshgskomponente von u in G' enthält eine gerade Anzahl von Knoten mit ungeradem Grad (Handschlaglemma).

      Da u und w die einzigen Knoten mit ungeradem Grad sind, muss w zur Zshgskomponente von u gehören.

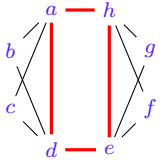
      Es folgt, dass mindestens ein Pfad von u nach w führt.



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Beweis: Ein zusammenhängender einfacher Graph $G = (V, E)$, in dem jeder Knoten geraden Grad hat, besitzt eine Eulertour.
  - I.S.: Sei $k \geq 3$ beliebig fixiert.
    - Beweis der I.Beh.: Sei $G = (V, E)$ zshgd. mit $|E| = k + 1$ und nur geraden Knotengraden.

      Es gibt somit in G' einen einfachen Pfad $v_0, \ldots, v_l$ von u nach w, zusammen mit $\{u, w\}$ somit einen Kreis $\kappa = v_0, v_1, \ldots, v_l, v_0$.

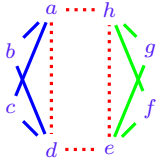


- Beweis: Ein zusammenhängender einfacher Graph $G = (V, E)$, in dem jeder Knoten geraden Grad hat, besitzt eine Eulertour.
  - I.S.: Sei $k \geq 3$ beliebig fixiert.
    - Beweis der I.Beh.: Sei $G = (V, E)$ zshgd. mit $|E| = k + 1$ und nur geraden Knotengraden.

      Sei G'' der Graph, der aus G durch Entfernen der Kanten von κ entsteht. G'' enthält höchstens k Kanten (tatsächlich sogar höchstens $k - 2$ Kanten).

      Falls G'' gar keine Kanten mehr enthält, haben wir bereits eine Eulertour (κ) gefunden. Andernfalls hat jeder Knoten in G'' geraden Grad.

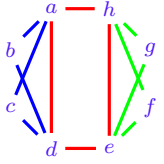
      Für jede der max. Zshgskomponenten von G'' finden wir daher eine Eulertour κ_i nach I.A.; wir dürfen annehmen, dass κ_i jeweils in einem Knoten von κ beginnt.



$\kappa = adeha$, $\kappa_1 = abdca$, $\kappa_2 = eghfe$

- Beweis: Ein zusammenhängender einfacher Graph $G = (V, E)$, in dem jeder Knoten geraden Grad hat, besitzt eine Eulertour.
  - I.S.: Sei $k \geq 3$ beliebig fixiert.
    - Beweis der I.Beh.: Sei $G = (V, E)$ zshgd. mit $|E| = k + 1$ und nur geraden Knotengraden.

      Wir erhalten eine Eulertour für ganz G, indem wir die Eulertouren κ_i anstelle ihrer Startknoten in κ substituieren.



$\kappa = adeha$, $\kappa_1 = abdca$, $\kappa_2 = eghfe$

Eulertour: $(abdca)d(eghfe)ha$

Hamiltonkreise: Hinreichende Bedingung

Lem (Dirac) Sei $G = (V, E)$ einfacher Graph.

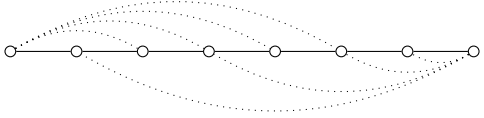
Falls ● $|V| \geq 3$ und $\deg(v) \geq \frac{|V|}{2}$ für alle $v \in V$,

dann besitzt G einen Hamiltonkreis.

▷ Beweis: Sei $G = (V, E)$ mit ● und $n := |V|$.

G ist zshgd., da $\min_{v \in V} \deg(v) + \max_{v \in V} \deg(v) \geq |V| - 1$ nach ● .

Sei $\pi = v_0, v_1, \ldots, v_m$ ein längster einfacher Pfad in G:



Alle Nachbarn von v_0 und v_m müssen auf π liegen, ansonsten wäre π nicht maximal.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Hamiltonkreise: Hinreichende Bedingung

Lem (Dirac) Sei $G = (V, E)$ einfacher Graph.

Falls • $|V| \geq 3$ und $\deg(v) \geq \frac{|V|}{2}$ für alle $v \in V$,

dann besitzt G einen Hamiltonkreis.

▷ Beweis: Sei $G = (V, E)$ mit • und $n := |V|$.

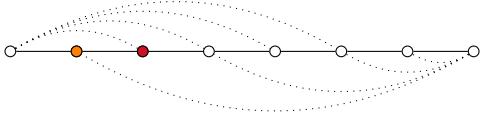
Seien $0 < i_1 < \ldots < i_k \leq m$ die Positionen der Nachbarn von v_0 bzgl. π:

▷ Nach Annahme $\deg(v_0) = k \geq \frac{n}{2}$.

Einer der Knoten $v_{i_1-1}, \ldots, v_{i_k-1}$ muss ein Nachbar von v_m sein:

▷ Ansonsten $\deg(v_m) \leq n - 1 - \frac{n}{2} < \frac{n}{2}$ entgegen Annahme • .

Es gibt also ein $j \in [k]$ mit $\{v_0, v_{i_j}\}, \{v_{i_j-1}, v_m\} \in E$.



Hamiltonkreise: Hinreichende Bedingung

Lem (Dirac) Sei $G = (V, E)$ einfacher Graph.

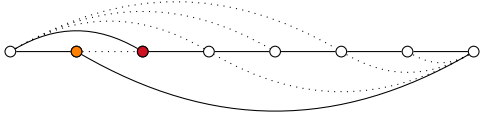
Falls ● $|V| \geq 3$ und $\deg(v) \geq \frac{|V|}{2}$ für alle $v \in V$,

dann besitzt G einen Hamiltonkreis.

▷ Beweis: Sei $G = (V, E)$ mit ● und $n := |V|$.

Damit ist $\kappa = v_0, v_1, \ldots, v_{i_j-1}, v_m, v_{m-1}, \ldots, v_{i_j}, v_0$ ein Kreis in G.

Da π einfach war, ist auch κ einfach.



Lem (Dirac) Sei $G = (V, E)$ einfacher Graph.

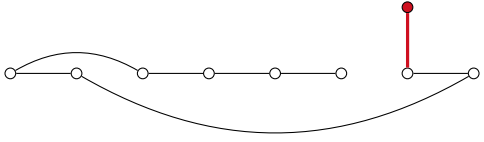
Falls ● $|V| \geq 3$ und $\deg(v) \geq \frac{|V|}{2}$ für alle $v \in V$,

dann besitzt G einen Hamiltonkreis.

▷ Beweis: Sei $G = (V, E)$ mit ● und $n := |V|$.

Alle Knoten liegen bereits auf κ, da:

- ▷ Ansonsten hat mind. ein v_i einen unbesuchten Nachbarn w, da G zshgd.
- ▷ Austausch von $\{v_{i-1}, v_i\}$ durch $\{w, v_i\}$ führt auf einfachen Pfad, der länger als π ist.



Lem (Ore) Falls $|V| \geq 3$ und $\deg(v) + \deg(w) \geq |V|$ für alle $\{v, w\} \in \binom{V}{2} \setminus E$, dann enthält G einen Hamiltonkreis.

(Beweis ist ähnlich zu Dirac.)

Planarität einfacher Graphen

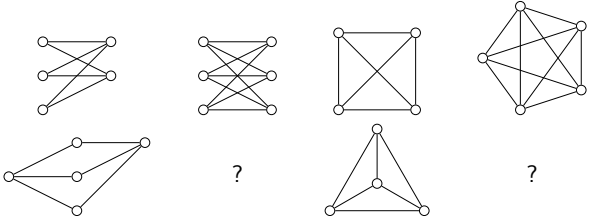
Def $G = (V, E)$ ist planar **:gdw** man ihn so in die euklidische Ebene zeichnen kann, dass Kanten stetige Kurven sind, die sich höchstens in gemeinsamen Knoten berühren. (Insbesondere dürfen keine zwei Knoten überlappen.)

Thm Satz von Wagner und Fáry (ohne Beweis)

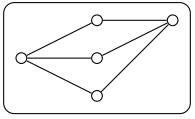
Für $A, B \in \mathbb{R}^2$ sei $[AB] := \{\lambda(B - A) + A \mid \lambda \in [0, 1]\}$ (Verbindungsstrecke).

$G = (V, E)$ ist planar **gdw** es gibt $P \colon V \to \mathbb{R}^2$, so dass für zwei unterschiedliche Kanten $\{u, w\}, \{u', w'\}$ stets gilt, dass $[P(u)P(w)]$ und $[P(u')P(w')]$ sich höchstens in einem Endpunkt berühren.

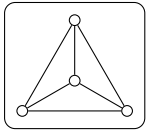
- ▷ Beispiel: Bäume, P_n, C_n, $M_{m,n}$ sind planar.
- ▷ Anwendungen: Topologie, Layout von Graphen, Diagrammen, Schaltkreise, ...



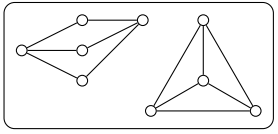
▷ Jede planare Darstellung unterteilt die Ebene in Flächen F:



$|F| = 3$



$|F| = 4$



$|F| = 6$

⚠ Die umschließende/äußere Fläche wird mitgezählt.

(Papier entlang Kanten zerschneiden; bzw. Graph auf Kugeloberfläche zeichnen.)

Der Rand jeder inneren Fläche ist ein einfacher Kreis in G.

Thm Eulersche Polyederformel (EPF):

Sei $G = (V, E)$ ein planarer Graph mit k CCs. Für die Anzahl $|F|$ der Flächen jeder planaren Darstellung von G gilt:

$$|F| - |E| + |V| = k + 1$$

Kor $|F|$ hängt somit nur von $|V|$, $|E|$ und der Anzahl der CCs ab, aber nicht von der konkreten planaren Darstellung.

Eulersche Polyederformel: Beweisskizze

Thm Sei $G = (V, E)$ ein planarer Graph mit k CCs. Für die Anzahl $|F|$ der Flächen jeder planaren Darstellung von G gilt:

$$|F| - |E| + |V| = k + 1$$

- Wir beweisen erst den Fall $k = 1$, d.h. G ist zshgd., so dass $|E| \geq |V| - 1$.

  Beweis mittels Induktion nach $n = |E| - |V| + 1 \in \mathbb{N}_0$.

- I.B.: Mit $n = 0$ gilt $|E| = |V| - 1$.

  Also ist G ein Baum, damit azyklisch, trennt also keine inneren Flächen ab.

  (Ganz präzise: mittels Induktion über $n = |V| = |E| + 1$.)

  Damit gibt es nur die umschließende Fläche, s.d. $|F| = 1$ und

  $$|F| - |E| + |V| = 1 - |V| + 1 + |V| = 2$$

  (Einen Baum kann man planar einbetten, indem man ihn als geordneten Präfixbaum liest, und Knoten als Darstellung von rationalen Zahlen, siehe Kardinalität: die Wurzel erhält die Koordinaten $(0, 0)$, und rekursiv weist man dem i-ten Kind eines Knotens u mit Koordinaten (x, y) die Koordinaten $(x + ib^{y-1}, y - 1)$ zu, wobei $b := \max\{|uE| : u \in V\}$ als Zahlenbasis dient. Siehe auch arithmetisches Kodieren als Anwendung.)

Eulersche Polyederformel: Beweisskizze

Thm Sei $G = (V, E)$ ein planarer Graph mit k CCs. Für die Anzahl $|F|$ der Flächen jeder planaren Darstellung von G gilt:

$$|F| - |E| + |V| = k + 1$$

- Wir beweisen erst den Fall $k = 1$, d.h. G ist zshgd., so dass $|E| \geq |V| - 1$.

  Beweis mittels Induktion nach $n = |E| - |V| + 1 \in \mathbb{N}_0$.

- I.S.: Sei $n \in \mathbb{N}_0$ beliebig fixiert.
  - I.A.: Die EPF gilt für jeden planaren Graphen $G = (V, E)$ mit $k = 1$ und $|E| - |V| + 1 = n$.
  - I.Beh.: Die EPF gilt für jeden planaren Graphen $G = (V, E)$ mit $k = 1$ und $|E| - |V| + 1 = n + 1$.

    (Die Induktion ergänzt einen Spannbaum von G um die verbleibenden $n = |E| - |V| + 1$ Kanten.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Eulersche Polyederformel: Beweisskizze

Thm Sei $G = (V, E)$ ein planarer Graph mit k CCs. Für die Anzahl $|F|$ der Flächen jeder planaren Darstellung von G gilt:

$$|F| - |E| + |V| = k + 1$$

- Wir beweisen erst den Fall $k = 1$, d.h. G ist zshgd., so dass $|E| \geq |V| - 1$.

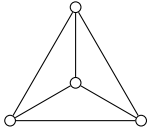
  Beweis mittels Induktion nach $n = |E| - |V| + 1 \in \mathbb{N}_0$.

- I.S.: Sei $n \in \mathbb{N}_0$ beliebig fixiert.

  Beweis der I.Beh.:

  Fixiere bel. $G = (V, E)$ mit: planar, $k = 1$, $|E| - |V| + 1 = n + 1$.

  Fixiere dann bel. planare Darstellung von G mit Flächen F.



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Eulersche Polyederformel: Beweisskizze

Thm Sei $G = (V, E)$ ein planarer Graph mit k CCs. Für die Anzahl $|F|$ der Flächen jeder planaren Darstellung von G gilt:

$$|F| - |E| + |V| = k + 1$$

- Wir beweisen erst den Fall $k = 1$, d.h. G ist zshgd., so dass $|E| \geq |V| - 1$.

  Beweis mittels Induktion nach $n = |E| - |V| + 1 \in \mathbb{N}_0$.

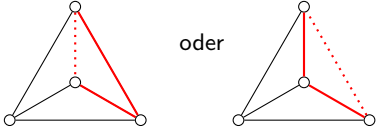
- I.S.: Sei $n \in \mathbb{N}_0$ beliebig fixiert.

  Beweis der I.Beh.:

  Da $|E| = |V| - 1 + n + 1 \geq |V|$, enthält G einen Kreis κ.

  Entfernen einer bel. Kante $\{u, w\}$ von κ führt auf $G' = (V, E \setminus \{\{u, w\}\})$.

  Dabei werden die durch $\{u, w\}$ getrennten Flächen zu einer Fläche vereinigt.



Eulersche Polyederformel: Beweisskizze

Thm Sei $G = (V, E)$ ein planarer Graph mit k CCs. Für die Anzahl $|F|$ der Flächen jeder planaren Darstellung von G gilt:

$$|F| - |E| + |V| = k + 1$$

- ▷ Wir beweisen erst den Fall $k = 1$, d.h. G ist zshgd., so dass $|E| \geq |V| - 1$.

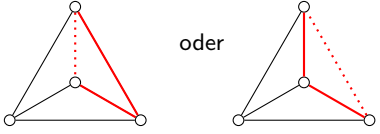
  Beweis mittels Induktion nach $n = |E| - |V| + 1 \in \mathbb{N}_0$.

- ▷ I.S.: Sei $n \in \mathbb{N}_0$ beliebig fixiert.

  Beweis der I.Beh.:

  G' ist noch zshgd. mit $V = V'$ und planar, hat eine Kante weniger als G (also $|E'| - |V'| + 1 = n$) und unterteilt die Ebene in $|F| - 1$ Flächen.

  Nach I.A. $(|F| - 1) - (|E| - 1) + |V| = 2$, also auch $|F| - |E| + |V| = 2$.



Eulersche Polyederformel: Beweisskizze

Thm Sei $G = (V, E)$ ein planarer Graph mit k CCs. Für die Anzahl $|F|$ der Flächen jeder planaren Darstellung von G gilt:

$$|F| - |E| + |V| = k + 1$$

▷ Falls $k > 1$, wende EPF auf jede CC (V_i, E_i) an:

$$|F_i| - |E_i| + |V_i| = 2$$

Dabei wird die umschließende Fläche für jede der k CCs gezählt, also $(k-1)$-mal zu oft, daher:

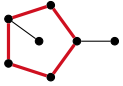
$$|F| - |E| + |V| = 1 + \sum_{i \in [k]} (|F_i| - 1 - |E_i| + |V_i|) = 1 + k$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Nichtplanarität von $K_{3,3}$ und K_5

Lem Für alle planaren Graphen $G = (V, E)$:
Ist jede Fläche durch $\geq d$ Kanten berandet, gilt $|E| \leq \frac{d}{d-2}(|V| - 2)$.

▷ Beweis: Für eine Fläche $f \in F$ sei $|f|$ die Anzahl der Kanten ihres Rands.



Sei $E' \subseteq E$ die Menge aller Randkanten (zwei angrenzende Flächen).

Für $G' = (V, E')$ gilt noch $F = F'$.

Nach Annahme gilt $|f| \geq d$ für jede (auch die äußere) Fläche $f \in F$.

Doppeltes Abzählen der Kanten in G' führt auf

$$2|E| \geq 2|E'| = \sum_{f \in F'} |f| \geq d|F'| = d|F| \quad \text{d.h.} \quad |F| \leq \frac{2}{d}|E|$$

und damit $2 \leq |F| - |E| + |V| \leq \frac{2}{d}|E| - |E| + |V|$

Auflösen nach $|E|$ ergibt: $|E| \leq \frac{d}{d-2}(|V| - 2)$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Nichtplanarität von $K_{3,3}$ und K_5

Lem Für alle planaren Graphen $G = (V, E)$:
Ist jede Fläche durch $\geq d$ Kanten berandet, gilt $|E| \leq \frac{d}{d-2}(|V| - 2)$.

Kor Für jeden planaren Graphen G mit $|V| \geq 3$ gilt $|E| \leq 3(|V| - 2)$.

- ▷ Entweder G ist ein Wald ($|F| = 1$), dann $|E| \leq |V| - 1 \leq 3(|V| - 2)$ ($|V| \geq 3$).
- ▷ Oder jede Fläche ist durch mind. $d = 3$ Kanten berandet.

Kor $K_{3,3}$ und K_5 sind nicht planar.

- ▷ Im K_5 gilt $|V| = 5$, $|E| = 10$, also $10 = |E| > 3|V| - 6 = 9$.
- ▷ Im $K_{3,3}$ besteht jeder Kreis aus $d \geq 4$ Kanten (da bipartit).

  Also $9 = |E| > 2(|V| - 2) = 8$.

Fkt Falls G planar ist, dann auch jeder Teilgraph von G.

⚠ Falls K_5 oder $K_{3,3}$ ein Teilgraph von G ist, dann ist G nicht planar.

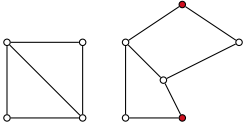
▷ Kuratowski/Wagner: G ist nicht planar **gdw** G „enthält“ $K_{3,3}$ oder K_5.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Charakterisierung planarer Graphen: Satz von Kuratowski

Def H ist Unterteilungsgraph von G, falls man H (bis auf Isomorphie) aus G mittels Kantenunterteilung erhalten kann.

- ▷ Unterteilen einer Kante $\{u, w\}$ bedeutet, dass man einen neuen Knoten x ergänzt und dann die Kante $\{u, w\}$ durch $\{u, x\}, \{w, x\}$ ersetzt.



⚠ Kantenunterteilung hat keinen Einfluss auf die Planarität. Daher:

Falls ein Unterteilungsgraph des $K_{3,3}$ oder des K_5 ein Teilgraph von G ist, dann ist G nicht planar.

Thm Satz von Kuratowski (1930): Ein einfacher Graph $G = (V, E)$ ist planar **gdw** kein Teilgraph von G ist ein Unterteilungsgraph des $K_{3,3}$ oder des K_5.

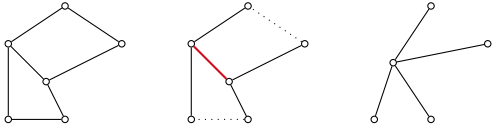
- ▷ Brute-Force-Planaritätstest: Teste alle Unterteilungen von K_5 und $K_{3,3}$ mit maximal $|V|$ Knoten. (Aber es gibt effizientere Algorithmen, um Planarität zu entscheiden.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Charakterisierung planarer Graphen: Satz von Wagner

Def Gegeben einfache Graphen H und G, dann ist H ein Minor von G, falls man H (bis auf Isomorphie) aus G mittels folgender Operationen erhalten kann:

- Entfernen von Kanten,
- Entfernen von Knoten vom Grad 0 und
- Kantenkontraktion (d.h. Quotient, der die Knoten einer Kante identifiziert).

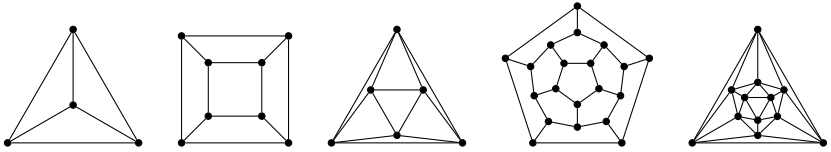


Thm Satz von Wagner (1937): Ein einfacher Graph $G = (V, E)$ ist planar **gdw** weder der $K_{3,3}$ noch der K_5 ist ein Minor von G.

- Brute-Force-Planaritätstest: alle Minoren von G testen (bessere Algorithmen).
- Siehe auch Verallgemeinerung: Satz von Robertson und Seymour (2004)

Eulersche Polyederformel: Platonische Körper

Kor Es gibt nur 5 zshgd. planare reguläre Graphen, bei denen jede Fläche durch dieselbe Anzahl von Kanten begrenzt wird (d.h. $\deg(v) = p \geq 3$, $|f| = q \geq 3$).



$$p|V| = 2|E| = q|F| \quad 3 \leq p \leq 5 \quad q \geq 3$$

$$2 = |F| - |E| + |V| = (\tfrac{2}{q} - 1 + \tfrac{2}{p})|E|$$

Tetrahedron	Cube	Octahedron	Dodecahedron	Icosahedron
Four faces	Six faces	Eight faces	Twelve faces	Twenty faces
				

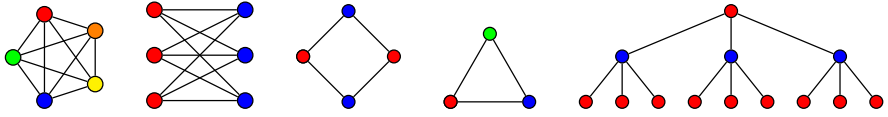
[Quelle]

Knotenfärbungen

Def Sei $G = (V, E)$ ein einfacher Graph. $c\colon V \to \mathbb{N}$ ist eine Knotenfärbung von G **:gdw** es gilt $c(u) \neq c(w)$ für jede Kante $\{u, w\} \in E$.

- Die Anzahl der von c verwendeten Farben ist $|c(V)|$.
- G ist k-färbbar, falls: es gibt eine Knotenfärbung c mit $|c(V)| \leq k$.
- 2-färbbare Graphen heißen auch bipartit.
- Die chromatische Zahl $\chi(G)$ von G ist die minimale Anzahl von Farben, für die es eine Knotenfärbung von G gibt, d.h.

$$\chi(G) := \min\Big\{|c(V)| : c \in \mathbb{N}^V \text{ Knotenfärbung von } G\Big\}$$

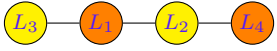


No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Konflikte zwischen (z.B.) Ressourcen führen häufig auf symmetrische, irreflexive Relationen und damit einfache Graphen G.

  $\chi(G)$ ist die minimale Anzahl an benötigten Ressourcen, um die Konflikte aufzulösen.

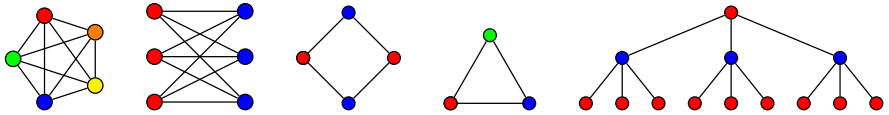
- Beispiel: Scheduling
  - Studiengang A muss Vorlesungen L_1, L_2 hören.
  - Studiengang B muss Vorlesungen L_1, L_3 hören.
  - Studiengang C muss Vorlesungen L_2, L_4 hören.



- Beispiel: Register-Allokation (vereinfacht)
  - Zwei Variablen stehen in Konflikt, falls beide gleichzeitig für eine Berechnung benötigt werden. Die chromatische Zahl entspricht der minimalen Anzahl an benötigten Registern.
- Zu entscheiden, ob $\chi(G) \leq 3$ gilt, ist i.A. so schwierig, wie die Berechnung eines Hamiltonkreises.

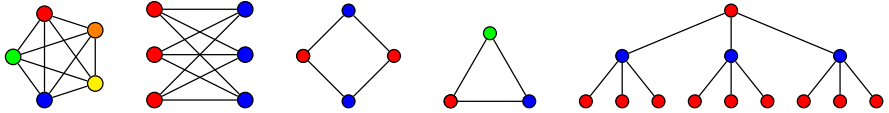
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Knotenfärbungen: Einfache Kriterien



- Fkt $\chi(K_n) = n$
- Fkt $\chi(K_{m,n}) = 2$
- Fkt $\chi(C_{2k}) = 2$ und $\chi(C_{2k+1}) = 3$.
- Fkt $\chi(G) \leq 2$ **gdw** G bipartit **gdw** jeder Kreis in G hat gerade Länge.
- Fkt $\chi(G) \geq 2$ **gdw** $E \neq \emptyset$.
- Fkt $\chi(G) \leq |V|$ für jeden einfachen Graphen $G = (V, E)$.
- Fkt $\chi(G) \leq 1 + \max_{v \in V} \deg(v)$.
- Fkt $\chi(H) \leq \chi(G)$, falls H Teilgraph von G.
- Fkt $\chi(G) \geq n$, falls K_n ein Teilgraph von G.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science



Lem Für jeden einfachen Graphen $G = (V, E)$ gilt

$$|E| \geq \binom{\chi(G)}{2} \qquad \chi(G) \leq \frac{1}{2} + \sqrt{2\,|E| + \frac{1}{4}}$$

- Beweis: Sei $c\colon V \to [\chi(G)]$ eine minimale Knotenfärbung.

  Identifiziere alle Knoten derselben Farbe:

  $$G' = \Big([\chi(G)], \{\{i, j\} \mid \{u, w\} \in E, c(u) = i, c(w) = j\}\Big)$$

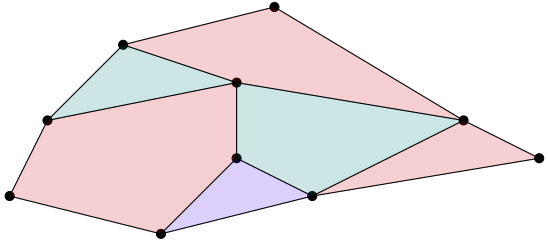
  Dann muss $G' \cong K_{\chi(G)}$ gelten:

  - Ansonsten gibt es Farben $i \neq j$, die durch keine Kante verbunden sind.

    Also könnte man die Farbe j einsparen.

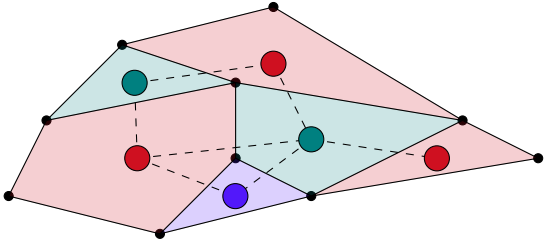
  Damit muss es mindestens $|E| \geq \frac{1}{2}\chi(G)(\chi(G) - 1)$ Kanten geben.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science



- Kann jede (kartographische) Karte mit höchstens vier Farben so gefärbt werden, dass Flächen mit gemeinsamer Kante (gemeinsame Knoten nicht relevant) unterschiedlich eingefärbt sind?

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science



- Kann jede (kartographische) Karte mit höchstens vier Farben so gefärbt werden, dass Flächen mit gemeinsamer Kante (gemeinsame Knoten nicht relevant) unterschiedlich eingefärbt sind?
- Kanten der Karte ergeben Konflikte zwischen Flächen.
- Im Fall einer Karte ist der Graph der Konfliktrelation planar.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Thm Vier-Farben-Satz:

Für jeden einfachen planaren Graphen $G = (V, E)$ gilt $\chi(G) \leq 4$.

- Einer der ersten computerunterstützten Beweise (1976):

  Reduktion „von Hand“ auf 1936 Fälle, welche durch einen Computer überprüft wurden; später Reduktion auf nur noch 633 Fälle (1996).

- 2005 erster vollständig formaler, durch einen Computer überprüfter Beweis.

⚠ Beachte: $\chi(K_{3,3}) = 2$, d.h. aus der Vier-Färbbarkeit folgt nicht die Planarität.

- Der schwächere Fünf-Farben-Satz

  Thm Für jeden einfachen planaren Graphen $G = (V, E)$ gilt $\chi(G) \leq 5$.

  lässt sich mittels eines fehlerhaften Beweisversuchs für den Vier-Farben-Satz beweisen.

▷ Beweis Fünf-Farben-Satz:

OBdA sei G zusammenhängend.

Annahme: Die Behauptung ist nicht wahr.

Dann gibt es einen einfachen planaren Graphen $G = (V, E)$ mit $\chi(G) > 5$ mit minimaler Knotenzahl $n = |V|$.

Da $\chi(G) \leq n$, muss $n \geq 6$ gelten.

Es gibt ein $u \in V$ mit $\deg(u) \leq 5$ (folgt aus der EPF: sonst $|E| \geq 3\,|V| > 3(|V| - 2)$).
Sei $H = G[V \setminus \{u\}]$.

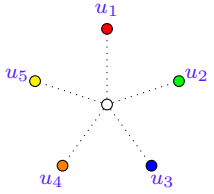
Da G ein Gegenbeispiel mit minimaler Knotenzahl ist, muss $\chi(H) \leq 5$ und damit aber auch $\deg(u) = 5$ in G gelten; insbesondere müssen alle fünf Farben für alle fünf Nachbarn von u in H verwendet werden.

Wir fixieren eine solche Färbung $c\colon V \setminus \{u\} \to [5]$ von H.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

▷ Beweis Fünf-Farben-Satz:

Wir zählen die Nachbarn von u in G (in einer Planardarstellung von G) im Uhrzeigersinn auf: $\Gamma(u) = \{u_1, \ldots, u_5\}$, wobei wir die Farben so benennen, dass $c(u_i) = i$ (1 rot, 2 grün, 3 blau, 4 orange, 5 gelb).

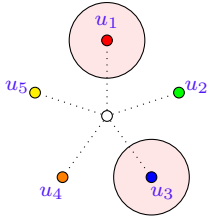


Wir unterscheiden nun die Fälle, ob es in H einen Pfad von u_1 nach u_3 gibt, auf dem sich nur die Farben Rot und Blau abwechseln, oder nicht.

Sei hierzu $H_{1,3} = H[c^{-1}(\{1,3\})]$, der durch die roten und blauen Knoten induzierte Teilgraph von H.

▷ Beweis Fünf-Farben-Satz:

Gibt es in H keinen Pfad zwischen u_1 und u_3, der nur die Farben Rot und Blau verwendet, dann liegen u_1 und u_3 in verschiedenen max. Zusammenhangskomponenten H_1, H_3 von $H_{1,3}$.



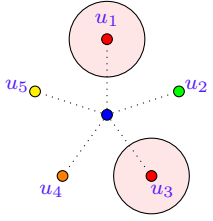
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

▷ Beweis Fünf-Farben-Satz:

Vertauschen wir in der Komponente H_3 von u_3 die Farben Rot und Blau

$$c'(v) := \begin{cases} c(v) & \text{falls } v \notin H_3 \\ 3 & \text{falls } c(v) = 1, v \in H_3 \\ 1 & \text{falls } c(v) = 3, v \in H_3 \end{cases}$$

dann wirkt sich dies nicht auf u_1 aus und wir erhalten eine korrekte Färbung c' von H mit:

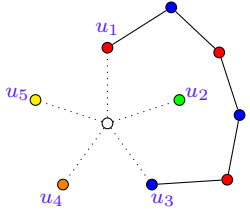


Mit $c'(u) := 3$ erhalten wir somit eine 5-Färbung von G obwohl $\chi(G) > 5$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

▷ Beweis Fünf-Farben-Satz:

Es muss somit in H einen Pfad π zwischen u_1 und u_3 geben, der nur die Farben Rot und Blau verwendet.

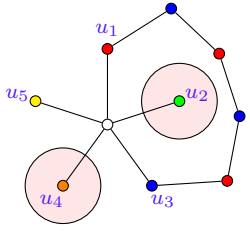


Sei nun $H_{2,4} = H[c^{-1}(\{2,4\})]$ analog zu gerade eben der Teilgraph von H, der von den grünen und orangefarbenen Knoten induziert wird.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

▷ Beweis Fünf-Farben-Satz:

Die max. Zusammenhangskomponente von u_2 bzgl. $H_{2,4}$ muss vollständig in dem durch den Kreis $u\pi u$ eingeschlossenen Gebiet liegen:



D.h. jeder Pfad von u_2 nach u_4 in H muss wegen der Planarität einen der Knoten aus π benutzen.

Damit müssen u_2 und u_4 bzgl. $H_{2,4}$ in verschiedenen Komponenten liegen; wir können somit die Farben Grün und Orange in der Komponente von u_4 vertauschen, womit wir u orange färben können.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

3 Graphentheorie

Grundlagen (Wiederholung)

Einfache Graphen (ungerichtet, schlingenfrei)

Matchings

Grundlagen und Heiratssatz

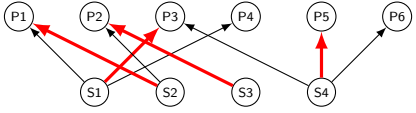
Matchings mit Präferenzen

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Problemstellung:

  Sei S eine Menge von Studierenden, P eine Menge von Praktikumsplätzen und $E \subseteq S \times P$ mit „sEp falls s Interesse an p hat“.

- E schränkt somit die möglichen Zuordnungen ein.



- Definition: Sei $G = (V, E)$ ein einfacher Graph. Ein Matching M ist eine Teilmenge $M \subseteq E$ der Kanten, so dass $|e \cap e'| \neq 1$ für alle $e, e' \in M$.
- Ein Matching M heißt perfekt, wenn es für alle $v \in V$ ein $m \in M$ gibt, so dass $v \in m$.
- Mit anderen Worten, ein Matching M ist perfekt, wenn $|M| = |V|/2$.

- Gegeben seien heiratswillige Damen und Herren. Jede Dame gibt an, mit welchen der Herren sie sich eventuell vermählen würde.
- Das Problem besteht nun darin, die Damen so zu verheiraten, dass jede Dame einen Herrn ihrer Wahl erhält, und dass selbstverständlich keine zwei Damen mit demselben Herrn verheiratet sind.



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Heiratssatz (Satz von Hall)

Thm Sei $G = (A \uplus B, E)$ ein einfacher bipartiter Graph mit (oBdA, sonst beide vertauschen) $|A| \leq |B|$.

Dann gibt es ein Matching $M \subseteq E$ mit $|M| = |A|$ genau dann, wenn jede Knotenteilmenge $X \subseteq A$ mindestens $|X|$ Nachbarn (in B) besitzt, d.h. falls

$$|\Gamma(X)| \geq |X| \text{ wobei } \Gamma(X) = \bigcup_{x \in X} \Gamma(x)$$

- Dass die Bedingung notwendig ist, sieht man sofort:

  Ist $M \subseteq E$ ein Matching mit $|M| = |A|$, dann ist $f_M = \{(a, b) \in A \times B \mid \{a, b\} \in M\}$ eine injektive Funktion mit $f_M(X) \subseteq \Gamma(X)$ und damit $|X| = |f_M(X)| \leq |\Gamma(X)|$.

- Wir zeigen, dass die Bedingung auch hinreichend ist.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Heiratssatz (Satz von Hall)

- Sei $G = (A \uplus B, E)$ ein einfacher bipartiter Graph mit der Eigenschaft

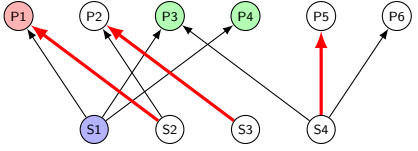
$$|\Gamma(X)| \geq |X| \text{ für alle } X \subseteq A \text{ wobei } \Gamma(X) = \bigcup_{x \in X} \Gamma(x)$$

- Sei $M \subseteq E$ ein Matching und $f_M \colon A \hookrightarrow B$ die durch M bestimmte partielle Funktion. Wir zeigen, wie man im Fall $|M| < |A|$ das Matching vergrößern kann.

  Sei also $a_0 \in A$ ein (bzgl. M) ungematchter Knoten. Setze $A_0 = \{a_0\}$ und $B_0 = \Gamma(A_0)$.

  Nach Annahme gilt $|B_0| \geq 1$. Gibt es ein $b \in B_0$, das ebenfalls ungematcht ist, können wir direkt M um $\{a_0, b\}$ erweitern. Beispiel:

$M = \{\{\text{S2,P1}\}, \{\text{S3,P2}\}, \{\text{S4,P5}\}\}$
$A_0 = \{\text{S1}\}, B_0 = \{\text{P1,P3,P4}\}$
$b =$ P3 oder P4



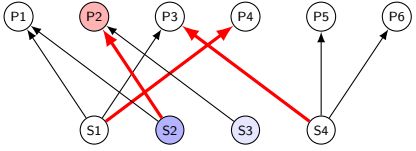
Heiratssatz (Satz von Hall)

- Sei $G = (A \uplus B, E)$ ein einfacher bipartiter Graph mit der Eigenschaft

$$|\Gamma(X)| \geq |X| \text{ für alle } X \subseteq A \text{ wobei } \Gamma(X) = \bigcup_{x \in X} \Gamma(x)$$

- Wir nehmen also an, dass alle Knoten in B_0 bereits gematcht sind. Setze $A_1 = A_0 \cup f_M^{-1}(B_0)$ und $B_1 = \Gamma(A_1)$. Beispiel:

$M = \{\{\text{S1,P4}\}, \{\text{S2,P2}\}, \{\text{S4,P3}\}\}$
$A_0 = \{\text{S3}\}, B_0 = \{\text{P2}\}$
$A_1 = \{\text{S3, S2}\}, B_1 = \{\text{P1, P2}\}$



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Heiratssatz (Satz von Hall)

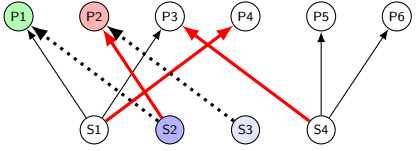
- Sei $G = (A \uplus B, E)$ ein einfacher bipartiter Graph mit der Eigenschaft

$$|\Gamma(X)| \geq |X| \text{ für alle } X \subseteq A \text{ wobei } \Gamma(X) = \bigcup_{x \in X} \Gamma(x)$$

- Hat ein $a_1 \in A_1$ einen Nachbarn b, der ungematcht ist, dann können wir a_1 mit b und a_0 mit $f_M(a_1)$ „ummatchen“ und damit M vergrößern.

Vor dem „Ummatchen“:

$M = \{\{\text{S1,P4}\}, \{\text{S2,P2}\}, \{\text{S4,P3}\}\}$
$A_0 = \{\text{S3}\}, B_0 = \{\text{P2}\}$
$A_1 = \{\text{S3}, \text{S2}\}, B_1 = \{\text{P1}, \text{P2}\}$
$a_1 = \text{S2}, b = \text{P1}$



Heiratssatz (Satz von Hall)

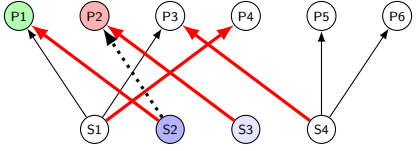
- Sei $G = (A \uplus B, E)$ ein einfacher bipartiter Graph mit der Eigenschaft

$$|\Gamma(X)| \geq |X| \text{ für alle } X \subseteq A \text{ wobei } \Gamma(X) = \bigcup_{x \in X} \Gamma(x)$$

- Hat ein $a_1 \in A_1$ einen Nachbarn b, der ungematcht ist, dann können wir a_1 mit b und a_0 mit $f_M(a_1)$ „ummatchen“ und damit M vergrößern.

  Nach dem „Ummatchen“:

$M' = \{\{\text{S1,P4}\}, \{\text{S2,P1}\}, \{\text{S3,P2}\}, \{\text{S4,P3}\}\}$
$A_0 = \{\text{S3}\}, B_0 = \{\text{P2}\}$
$A_1 = \{\text{S3, S2}\}, B_1 = \{\text{P1, P2}\}$
$a_1 = \text{S2}, b = \text{P1}$



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Heiratssatz (Satz von Hall)

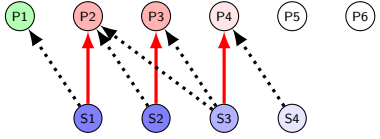
- Sei $G = (A \uplus B, E)$ ein einfacher bipartiter Graph mit der Eigenschaft

$$|\Gamma(X)| \geq |X| \text{ für alle } X \subseteq A \text{ wobei } \Gamma(X) = \bigcup_{x \in X} \Gamma(x)$$

- Wir wiederholen die Konstruktion $A_{i+1} = A_i \cup f_M^{-1}(B_i)$, $B_{i+1} = \Gamma(A_{i+1})$ so lange, bis wir ein $a_i \in A_i$ finden, das einen ungematchten Nachbarn $b \in \Gamma(a_i)$ besitzt, welcher uns erlaubt, das Matching um a_0 zu erweitern. Beispiel, in dem erst B_2 einen ungematchten Knoten enthält:

  Vor dem „Ummatchen“:

$M = \{\{\text{S1,P2}\}, \{\text{S2,P3}\}, \{\text{S3,P4}\}\}$
$A_0 = \{\text{S4}\}, B_0 = \{\text{P4}\}$
$A_1 = \{\text{S4, S3}\}, B_1 = \{\text{P2, P3, P4}\}$
$A_2 = \{\text{S4, S3, S2, S1}\}, B_2 = \{\text{P1, } \ldots, \text{ P4}\}$



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Heiratssatz (Satz von Hall)

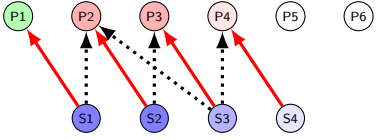
- Sei $G = (A \uplus B, E)$ ein einfacher bipartiter Graph mit der Eigenschaft

$$|\Gamma(X)| \geq |X| \text{ für alle } X \subseteq A \text{ wobei } \Gamma(X) = \bigcup_{x \in X} \Gamma(x)$$

- Wir wiederholen die Konstruktion $A_{i+1} = A_i \cup f_M^{-1}(B_i)$, $B_{i+1} = \Gamma(A_{i+1})$ so lange, bis wir ein $a_i \in A_i$ finden, das einen ungematchten Nachbarn $b \in \Gamma(a_i)$ besitzt, welcher uns erlaubt, das Matching um a_0 zu erweitern. Beispiel, in dem erst B_2 einen ungematchten Knoten enthält:

  Nach dem „Ummatchen“:

$M' = \{\{S1,P1\}, \{S2,P2\}, \{S3,P3\}, \{S4,P4\}\}$
$A_0 = \{S4\}, B_0 = \{P4\}$
$A_1 = \{S4, S3\}, B_1 = \{P2, P3, P4\}$
$A_2 = \{S4, S3, S2, S1\}, B_2 = \{P1, \ldots, P4\}$



Alle Kanten vom „Pfad“ S4, P4, S3, P3, S2, P2, S1, P1 werden „umgematcht“.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Heiratssatz (Satz von Hall)

- Sei $G = (A \uplus B, E)$ ein einfacher bipartiter Graph mit der Eigenschaft

$$|\Gamma(X)| \geq |X| \text{ für alle } X \subseteq A \text{ wobei } \Gamma(X) = \bigcup_{x \in X} \Gamma(x)$$

- Wir zeigen für alle $i \geq 0$: Wenn alle Knoten in B_i gematcht sind, dann gilt $A_i \subsetneq A_{i+1}$.

  Erinnerung: $A_{i+1} := A_i \cup f_M^{-1}(B_i)$ und $B_{i+1} := \Gamma(A_{i+1})$.

  (1) Es gilt $|f_M^{-1}(B_i)| = |B_i|$, denn alle Knoten in B_i sind gematcht und f_M ist injektiv.

  (2) Es gilt $a_0 \in A_i \setminus f_M^{-1}(B_i)$, denn $a_0 \in A_i$ ist nicht gematcht und alle Knoten in $f_M^{-1}(B_i)$ sind gematcht.

  Daraus folgt:

$$\begin{array}{rcll} |A_i| & \leq & |\Gamma(A_i)| & \text{(Annahme des Heiratssatzes)} \\ & = & |B_i| & \text{(Def. von } B_i\text{)} \\ & = & \left|f_M^{-1}(B_i)\right| & (1) \\ & < & \left|A_i \cup f_M^{-1}(B_i)\right| & (2) \\ & = & |A_{i+1}| & \text{(Def. von } A_{i+1}\text{)} \end{array}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Heiratssatz (Satz von Hall)

- Sei $G = (A \uplus B, E)$ ein einfacher bipartiter Graph mit der Eigenschaft

$$|\Gamma(X)| \geq |X| \text{ für alle } X \subseteq A \text{ wobei } \Gamma(X) = \bigcup_{x \in X} \Gamma(x)$$

- Wir zeigen: es gibt ein $i \geq 0$ so dass B_i einen ungematchten Knoten enthält.

  Da alle Mengen $A_0 \subseteq A_1 \subseteq A_2 \ldots$ Teilmengen der endlichen Menge A sind, gibt es einen Index $i \geq 0$ mit $A_i = A_{i+1}$. Für dieses i gilt dann, dass die Menge B_i einen ungematchten Knoten enthält.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Gegeben seien heiratswillige Damen und Herren in gleicher Anzahl.
- Jede Dame (jeder Herr) stellt eine Präferenzliste zur Verfügung, die alle Herren (alle Damen) enthält—in dieser Gesellschaft ist jede Heirat besser als keine Heirat.
- Das Problem besteht darin, die Damen und Herren so zu verheiraten, dass es keinen Anreiz für Seitensprünge gibt: Kein Paar würde lieber eine Beziehung miteinander anstatt mit dem jeweiligen Ehepartner eingehen.



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Formal:

  Gesucht ist eine bijektive Abbildung $f\colon A \to B$ (mit $|A| = |B| = n$).

  Der Einfachheit halber bezeichnen wir die Elemente von A als Frauen und die von B als Männer.

  Die Nebenbedingungen sind jetzt nicht mehr von der Form „a kann (nicht) mit b gematcht werden“.

  Stattdessen besitzt jede Frau $a \in A$ eine totale Präferenzordnung $\prec_a \subseteq B \times B$; und jeder Mann $b \in B$ entsprechend $\prec_b \subseteq A \times A$.

  Die Nebenbedingung ist nun, dass f stabil bzgl. aller Präferenzordnungen ist:

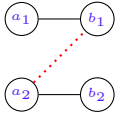
  - f ist instabil, falls es $a, a' \in A$ gibt mit

    $f(a) \prec_a f(a')$ („a bevorzugt den Partner von a'“) und

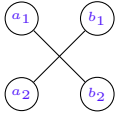
    $a' \prec_{f(a')} a$ („Der Partner von a' bevorzugt a.“)
  - a und $f(a')$ würden ihre aktuellen Partner verlassen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Beispiel:
  - Personen: $A = \{a_1, a_2\}$, $B = \{b_1, b_2\}$.
  - Präferenzen: $b_1 \prec_{a_1} b_2$, $b_2 \prec_{a_2} b_1$, $a_1 \prec_{b_1} a_2$, $a_1 \prec_{b_2} a_2$
  - Instabiles Matching („Ehe“): a_2 präferiert b_1, b_1 präferiert a_2.



  - Stabiles Matching („Ehe“): a_1 präferiert b_2, a_2 präferiert b_1; b_1 und b_2 präferieren a_2; b_1 hat seinen präferierten Partner, b_2 muss sich mit a_1 abfinden. Das Leben ist einfach nicht fair...



  - Stabil heißt nicht, dass jeder seinen am meisten präferierten Partner bekommt.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gale-Shapley-Algorithmus

- David Gale: wichtige Beiträge im Bereich Spieltheorie/Optimierung
- Lloyd S. Shapley: wichtige Beiträge im Bereich der Spieltheorie, ~~Mathematiknobelpreisträger~~ Preisträger für Wirtschaftswissenschaften der schwedischen Reichsbank im Gedenken an Alfred Nobel (2012).
- Gale und Shapley haben gezeigt, dass es für alle Kombinationen von Präferenzordnungen immer mind. ein stabiles Matching gibt.
- Gale-Shapley „Deferred Acceptance“ Algorithmus:
  - Gegeben: Mengen A, B und zugehörige Präferenzordnungen
  - Gesucht: Stabiles Matching $f\colon A \to B$
  - Solange es noch einen nicht verlobten Mann $b \notin \mathrm{im}(f)$ gibt:
    - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
    - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
    - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathsf{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

A					B				
a_1 :	b_4	$\succ$	b_3	$\succ$ b_1 $\succ$ b_2	b_1 :	a_1	$\succ$	a_3	$\succ$ a_4 $\succ$ a_2
a_2 :	b_2	$\succ$	b_4	$\succ$ b_1 $\succ$ b_3	b_2 :	a_1	$\succ$	a_3	$\succ$ a_2 $\succ$ a_4
a_3 :	b_4	$\succ$	b_1	$\succ$ b_2 $\succ$ b_3	b_3 :	a_2	$\succ$	a_1	$\succ$ a_3 $\succ$ a_4
a_4 :	b_3	$\succ$	b_2	$\succ$ b_1 $\succ$ b_4	b_4 :	a_2	$\succ$	a_4	$\succ$ a_1 $\succ$ a_3

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathsf{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

A					B				
a_1 :	b_4	$\succ$	b_3	$\succ$ b_1 $\succ$ b_2	b_1 :	a_1	$\succ$	a_3	$\succ$ a_4 $\succ$ a_2
a_2 :	b_2	$\succ$	b_4	$\succ$ b_1 $\succ$ b_3	b_2 :	a_1	$\succ$	a_3	$\succ$ a_2 $\succ$ a_4
a_3 :	b_4	$\succ$	b_1	$\succ$ b_2 $\succ$ b_3	b_3 :	a_2	$\succ$	a_1	$\succ$ a_3 $\succ$ a_4
a_4 :	b_3	$\succ$	b_2	$\succ$ b_1 $\succ$ b_4	b_4 :	a_2	$\succ$	a_4	$\succ$ a_1 $\succ$ a_3

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathrm{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

	A								B							
a_1 :	b_4	$\succ$	b_3	$\succ$	$\boxed{b_1}$	$\succ$	b_2		b_1 :	$\boxed{a_1}$	$\succ$	a_3	$\succ$	a_4	$\succ$	a_2
a_2 :	b_2	$\succ$	b_4	$\succ$	b_1	$\succ$	b_3		b_2 :	a_1	$\succ$	a_3	$\succ$	a_2	$\succ$	a_4
a_3 :	b_4	$\succ$	b_1	$\succ$	b_2	$\succ$	b_3		b_3 :	a_2	$\succ$	a_1	$\succ$	a_3	$\succ$	a_4
a_4 :	b_3	$\succ$	b_2	$\succ$	b_1	$\succ$	b_4		b_4 :	a_2	$\succ$	a_4	$\succ$	a_1	$\succ$	a_3

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathrm{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

A				
a_1 :	b_4	$\succ b_3 \succ$	$\boxed{b_1}$	$\succ b_2$
a_2 :	b_2	$\succ b_4 \succ b_1 \succ$		b_3
a_3 :	b_4	$\succ b_1 \succ b_2 \succ$		b_3
a_4 :	b_3	$\succ b_2 \succ b_1 \succ$		b_4

B				
b_1 :	$\boxed{a_1}$	$\succ a_3 \succ$	a_4	$\succ a_2$
b_2 :	a_1	$\succ a_3 \succ a_2 \succ$		a_4
b_3 :	a_2	$\succ a_1 \succ a_3 \succ$		a_4
b_4 :	a_2	$\succ a_4 \succ a_1 \succ$		a_3

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathrm{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

A				
a_1 :	b_4	$\succ b_3 \succ$	$\boxed{b_1}$	$\succ b_2$
a_2 :	b_2	$\succ b_4 \succ b_1 \succ$		b_3
a_3 :	b_4	$\succ b_1 \succ b_2 \succ$		b_3
a_4 :	b_3	$\succ b_2 \succ b_1 \succ$		b_4

B				
b_1 :	$\boxed{a_1}$	$\succ a_3 \succ$	a_4	$\succ a_2$
b_2 :	a_1	$\succ a_3 \succ$	a_2	$\succ a_4$
b_3 :	a_2	$\succ a_1 \succ$	a_3	$\succ a_4$
b_4 :	a_2	$\succ a_4 \succ$	a_1	$\succ a_3$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathrm{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

A								B							
a_1 :	b_4	$\succ$	b_3	$\succ$	$\boxed{b_1}$	$\succ$	b_2	b_1 :	$\boxed{a_1}$	$\succ$	a_3	$\succ$	a_4	$\succ$	a_2
a_2 :	b_2	$\succ$	b_4	$\succ$	b_1	$\succ$	b_3	b_2 :	a_1	$\succ$	a_3	$\succ$	a_2	$\succ$	a_4
a_3 :	b_4	$\succ$	b_1	$\succ$	b_2	$\succ$	b_3	b_3 :	a_2	$\succ$	a_1	$\succ$	a_3	$\succ$	a_4
a_4 :	b_3	$\succ$	b_2	$\succ$	b_1	$\succ$	b_4	b_4 :	a_2	$\succ$	a_4	$\succ$	a_1	$\succ$	a_3

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathrm{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

$$
\begin{array}{c|cc}
 & A & \\
a_1: & b_4 \succ b_3 \succ \boxed{b_1} \succ b_2 & \\
a_2: & b_2 \succ b_4 \succ b_1 \succ \boxed{b_3} & \\
a_3: & b_4 \succ b_1 \succ b_2 \succ b_3 & \\
a_4: & b_3 \succ b_2 \succ b_1 \succ b_4 &
\end{array}
\qquad
\begin{array}{cc|}
 & B \\
b_1: & \boxed{a_1} \succ a_3 \succ a_4 \succ a_2 \\
b_2: & a_1 \succ a_3 \succ a_2 \succ a_4 \\
b_3: & \boxed{a_2} \succ a_1 \succ a_3 \succ a_4 \\
b_4: & a_2 \succ a_4 \succ a_1 \succ a_3
\end{array}
$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathrm{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

A

$a_1 : \quad b_4 \succ b_3 \succ \boxed{b_1} \succ b_2$

$a_2 : \quad b_2 \succ b_4 \succ b_1 \succ \boxed{b_3}$

$a_3 : \quad b_4 \succ b_1 \succ b_2 \succ b_3$

$a_4 : \quad b_3 \succ b_2 \succ b_1 \succ b_4$

B

$b_1 : \quad \boxed{a_1} \succ a_3 \succ a_4 \succ a_2$

$b_2 : \quad a_1 \succ a_3 \succ a_2 \succ a_4$

$b_3 : \quad \boxed{a_2} \succ a_1 \succ a_3 \succ a_4$

$b_4 : \quad a_2 \succ a_4 \succ a_1 \succ a_3$

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathrm{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

A				
a_1 :	b_4	$\succ$ b_3	$\succ$ $\boxed{b_1}$	$\succ$ b_2
a_2 :	b_2	$\succ$ $\boxed{b_4}$	$\succ$ b_1	$\succ$ b_3
a_3 :	b_4	$\succ$ b_1	$\succ$ b_2	$\succ$ b_3
a_4 :	b_3	$\succ$ b_2	$\succ$ b_1	$\succ$ b_4

B				
b_1 :	$\boxed{a_1}$	$\succ$ a_3	$\succ$ a_4	$\succ$ a_2
b_2 :	a_1	$\succ$ a_3	$\succ$ a_2	$\succ$ a_4
b_3 :	a_2	$\succ$ a_1	$\succ$ a_3	$\succ$ a_4
b_4 :	$\boxed{a_2}$	$\succ$ a_4	$\succ$ a_1	$\succ$ a_3

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathrm{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

A				
a_1 :	b_4	$\succ$ b_3	$\succ$ $\boxed{b_1}$	$\succ$ b_2
a_2 :	b_2	$\succ$ $\boxed{b_4}$	$\succ$ b_1	$\succ$ b_3
a_3 :	b_4	$\succ$ b_1	$\succ$ b_2	$\succ$ b_3
a_4 :	b_3	$\succ$ b_2	$\succ$ b_1	$\succ$ b_4

B				
b_1 :	$\boxed{a_1}$	$\succ$ a_3	$\succ$ a_4	$\succ$ a_2
b_2 :	a_1	$\succ$ a_3	$\succ$ a_2	$\succ$ a_4
b_3 :	a_2	$\succ$ a_1	$\succ$ a_3	$\succ$ a_4
b_4 :	$\boxed{a_2}$	$\succ$ a_4	$\succ$ a_1	$\succ$ a_3

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathsf{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

$$
\begin{array}{c|cc}
 & A & \\
a_1: & b_4 \succ b_3 \succ \boxed{b_1} \succ & b_2 \\
a_2: & b_2 \succ \boxed{b_4} \succ b_1 \succ & b_3 \\
a_3: & b_4 \succ b_1 \succ \boxed{b_2} \succ & b_3 \\
a_4: & b_3 \succ b_2 \succ b_1 \succ & b_4
\end{array}
\qquad
\begin{array}{cc|}
 & B \\
b_1: & \boxed{a_1} \succ a_3 \succ a_4 \succ a_2 \\
b_2: & a_1 \succ \boxed{a_3} \succ a_2 \succ a_4 \\
b_3: & a_2 \succ a_1 \succ a_3 \succ a_4 \\
b_4: & \boxed{a_2} \succ a_4 \succ a_1 \succ a_3
\end{array}
$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathrm{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

A				
a_1 :	b_4	$\succ b_3$	$\succ \boxed{b_1}$	$\succ b_2$
a_2 :	b_2	$\succ \boxed{b_4}$	$\succ b_1$	$\succ b_3$
a_3 :	b_4	$\succ b_1$	$\succ \boxed{b_2}$	$\succ b_3$
a_4 :	b_3	$\succ b_2$	$\succ b_1$	$\succ b_4$

B				
b_1 :	$\boxed{a_1}$	$\succ a_3$	$\succ a_4$	$\succ a_2$
b_2 :	a_1	$\succ \boxed{a_3}$	$\succ a_2$	$\succ a_4$
b_3 :	a_2	$\succ a_1$	$\succ a_3$	$\succ a_4$
b_4 :	$\boxed{a_2}$	$\succ a_4$	$\succ a_1$	$\succ a_3$

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathsf{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

A				
a_1 :	b_4	$\succ$ **b_3** $\succ$	b_1	$\succ$ b_2
a_2 :	b_2	$\succ$ **b_4** $\succ$	b_1	$\succ$ b_3
a_3 :	b_4	$\succ$ b_1 $\succ$	**b_2**	$\succ$ b_3
a_4 :	b_3	$\succ$ b_2 $\succ$	b_1	$\succ$ b_4

B				
b_1 :	a_1	$\succ$ a_3 $\succ$	a_4	$\succ$ a_2
b_2 :	a_1	$\succ$ **a_3** $\succ$	a_2	$\succ$ a_4
b_3 :	a_2	$\succ$ **a_1** $\succ$	a_3	$\succ$ a_4
b_4 :	**a_2**	$\succ$ a_4 $\succ$	a_1	$\succ$ a_3

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathsf{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

A

$a_1 : \quad b_4 \succ \boxed{b_3} \succ b_1 \succ b_2$

$a_2 : \quad b_2 \succ \boxed{b_4} \succ b_1 \succ b_3$

$a_3 : \quad b_4 \succ b_1 \succ \boxed{b_2} \succ b_3$

$a_4 : \quad b_3 \succ b_2 \succ b_1 \succ b_4$

B

$b_1 : \quad a_1 \succ a_3 \succ a_4 \succ a_2$

$b_2 : \quad a_1 \succ \boxed{a_3} \succ a_2 \succ a_4$

$b_3 : \quad a_2 \succ \boxed{a_1} \succ a_3 \succ a_4$

$b_4 : \quad \boxed{a_2} \succ a_4 \succ a_1 \succ a_3$

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathsf{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

	A					B				
a_1 :	b_4	$\succ$	$\boxed{b_3}$	$\succ$	b_1 $\succ$ b_2	b_1 :	a_1	$\succ$	$\boxed{a_3}$	$\succ$ a_4 $\succ$ a_2
a_2 :	b_2	$\succ$	$\boxed{b_4}$	$\succ$	b_1 $\succ$ b_3	b_2 :	a_1	$\succ$	a_3	$\succ$ a_2 $\succ$ a_4
a_3 :	b_4	$\succ$	$\boxed{b_1}$	$\succ$	b_2 $\succ$ b_3	b_3 :	a_2	$\succ$	$\boxed{a_1}$	$\succ$ a_3 $\succ$ a_4
a_4 :	b_3	$\succ$	b_2	$\succ$	b_1 $\succ$ b_4	b_4 :	$\boxed{a_2}$	$\succ$	a_4	$\succ$ a_1 $\succ$ a_3

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathsf{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

	A			
a_1 :	b_4	$\succ$ **b_3**	$\succ$ b_1	$\succ$ b_2
a_2 :	b_2	$\succ$ **b_4**	$\succ$ b_1	$\succ$ b_3
a_3 :	b_4	$\succ$ **b_1**	$\succ$ b_2	$\succ$ b_3
a_4 :	b_3	$\succ$ b_2	$\succ$ b_1	$\succ$ b_4

	B			
b_1 :	a_1	$\succ$ **a_3**	$\succ$ a_4	$\succ$ a_2
b_2 :	a_1	$\succ$ a_3	$\succ$ a_2	$\succ$ a_4
b_3 :	a_2	$\succ$ **a_1**	$\succ$ a_3	$\succ$ a_4
b_4 :	**a_2**	$\succ$ a_4	$\succ$ a_1	$\succ$ a_3

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathsf{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

A

$a_1 : \quad b_4 \succ \boxed{b_3} \succ b_1 \succ b_2$

$a_2 : \quad \boxed{b_2} \succ b_4 \succ b_1 \succ b_3$

$a_3 : \quad b_4 \succ \boxed{b_1} \succ b_2 \succ b_3$

$a_4 : \quad b_3 \succ b_2 \succ b_1 \succ b_4$

B

$b_1 : \quad a_1 \succ \boxed{a_3} \succ a_4 \succ a_2$

$b_2 : \quad a_1 \succ a_3 \succ \boxed{a_2} \succ a_4$

$b_3 : \quad a_2 \succ \boxed{a_1} \succ a_3 \succ a_4$

$b_4 : \quad a_2 \succ a_4 \succ a_1 \succ a_3$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathrm{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

	A			
a_1 :	b_4	$\succ$ **b_3**	$\succ$ b_1	$\succ$ b_2
a_2 :	**b_2**	$\succ$ b_4	$\succ$ b_1	$\succ$ b_3
a_3 :	b_4	$\succ$ **b_1**	$\succ$ b_2	$\succ$ b_3
a_4 :	b_3	$\succ$ b_2	$\succ$ b_1	$\succ$ b_4

	B			
b_1 :	a_1	$\succ$ **a_3**	$\succ$ a_4	$\succ$ a_2
b_2 :	a_1	$\succ$ a_3	$\succ$ **a_2**	$\succ$ a_4
b_3 :	a_2	$\succ$ **a_1**	$\succ$ a_3	$\succ$ a_4
b_4 :	a_2	$\succ$ a_4	$\succ$ a_1	$\succ$ a_3

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gale-Shapley-Algorithmus

- Solange es noch einen nicht verlobten Mann $b \notin \mathsf{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.
- Beispiel: $A = \{a_1, a_2, a_3, a_4\}$ und $B = \{b_1, b_2, b_3, b_4\}$

A				
a_1 :	b_4	$\succ$ **b_3**	$\succ$ b_1	$\succ$ b_2
a_2 :	**b_2**	$\succ$ b_4	$\succ$ b_1	$\succ$ b_3
a_3 :	b_4	$\succ$ **b_1**	$\succ$ b_2	$\succ$ b_3
a_4 :	b_3	$\succ$ b_2	$\succ$ b_1	$\succ$ **b_4**

B				
b_1 :	a_1	$\succ$ **a_3**	$\succ$ a_4	$\succ$ a_2
b_2 :	a_1	$\succ$ a_3	$\succ$ **a_2**	$\succ$ a_4
b_3 :	a_2	$\succ$ **a_1**	$\succ$ a_3	$\succ$ a_4
b_4 :	a_2	$\succ$ **a_4**	$\succ$ a_1	$\succ$ a_3

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Solange es noch einen nicht verlobten Mann $b \notin \mathsf{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.

Einige Beobachtungen:

- In jeder Runde macht ein Mann einer Frau erstmals einen Antrag.
- Es gibt $n \cdot n$ verschiedene Möglichkeiten einen Antrag zu machen.
- Der Algorithmus terminiert also nach höchstens n^2 Runden.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Solange es noch einen nicht verlobten Mann $b \notin \mathrm{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.

Einige Beobachtungen:

- Keine zwei $a, a' \in A$ sind mit demselben $b \in B$ gematcht: Eine Frau verlobt sich nur mit einem alleinstehenden Mann.
- Die Zahl $|\mathrm{dom}(f)|$ der gematchten $a \in A$ kann nicht abnehmen: Sobald eine Frau verlobt ist, wird die Verlobung nur aufgelöst, wenn sie einen neuen Partner gefunden hat.
- Am Ende des Algorithmus ist $\mathrm{im}(f) = B$: Jede Frau erhält irgendwann einen Antrag.
- Nach Terminierung ist f ein perfektes Matching.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Solange es noch einen nicht verlobten Mann $b \notin \mathrm{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.

Behauptung: Nach Terminierung ist f ein stabiles Matching.

Annahme: Es gibt $a, a' \in A$ mit $b := f(a')$ und $f(a) \prec_a b$, $a' \prec_b a$.

- Nehmen wir an, dass a irgendwann einen Antrag von b erhalten hat.
- Wenn a den Antrag abgelehnt hat, hatte sie zu diesem (und jedem späteren) Zeitpunkt einen besseren Partner, d.h. $b \prec_a f(a)$.
- Wenn a und b sich verlobt haben, dann kann diese Verlobung nur aufgelöst worden sein, weil a sich verbessern konnte, d.h. ebenfalls $b \prec_a f(a)$.
- Also hat a nie einen Antrag von b bekommen. Das ist jedoch nur möglich, wenn $a \prec_b a'$ (da Männer immer der meistpräferierten Frau einen Antrag machen).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Solange es noch einen nicht verlobten Mann $b \notin \mathsf{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.

  Ohne Beweis: Nach Terminierung ist f optimal im folgenden Sinn:

  Ist e ein stabiles Matching, dann gilt $e^{-1}(b) \preceq_b f^{-1}(b)$ für jedes $b \in B$.
- D.h. jeder Mann erhält am Ende die am meisten präferierte Frau aus der Teilmenge aller Frauen, mit denen eine stabile Ehe überhaupt möglich ist.
- Konsequenz: Die konkrete Wahl eines ungematchten b in jeder Runde hat keinen Einfluss auf das Endresultat.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Solange es noch einen nicht verlobten Mann $b \notin \mathsf{im}(f)$ gibt:
  - b macht der Frau, die er unter denjenigen, die ihn noch nicht zurückgewiesen haben, am meisten begehrt (a) einen Antrag (größtes Element bzgl. $\prec_b$).
  - Wenn a noch nicht verlobt ist, verloben sich a und b ($f(a) := b$).
  - Wenn a bereits verlobt ist und sie b ihrem aktuellen Partner $f(a)$ bevorzugt ($f(a) \prec_a b$), wird die Verlobung mit $f(a)$ aufgelöst und sie verlobt sich mit b ($f(a) := b$). Ansonsten lehnt sie b's Antrag ab.

Weitere Bemerkungen:

- Während der Laufzeit des Algorithmus können sich Frauen nur verbessern und Männer nur verschlechtern.
- Wenn man die Rollen vertauscht und die Frauen die Anträge machen lässt, erhält man ein Matching, das für die Frauen optimal ist.
- Wenn der zu Grunde liegende Graph nicht bipartit ist, d.h. Matchings sind zwischen beliebigen Personen möglich und jeder hat eine vollständige Präferenzordnung über alle Personen, dann ist die Existenz eines stabilen Matchings nicht mehr garantiert.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

1 Einführung

2 Grundlagen

3 Graphentheorie

4 Kombinatorik

5 Logik

6 Grundlagen der Gruppen- und Zahlentheorie

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

4 Kombinatorik

Einführung

Kombinatorische Grundprinzipien

Das Urnenmodell: Ziehungen

Weitere Verteilungsprobleme

Zusammenfassung

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Kombinatorik beschäftigt sich mit dem (effizienten) Abzählen komplizierter Mengen.
- Entstanden aus der Untersuchung von Glücksspielen
  - Einfachster Wahrscheinlichkeitsbegriff $\frac{|A|}{|\Omega|}$ mit

    Ω Menge aller möglichen Spielverläufe („Elementarereignisse“),

    $A \subseteq \Omega$ Teilmenge aller Spielverläufe, an denen man interessiert ist.
  - Z.B.:

    Wahrscheinlichkeit, dass fünf beliebig gewählte Karten ein „Straight Flush“ sind.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Schwierigkeit bei Kombinatorik: Modellierung

  Natürlichsprachliche Beschreibung muss formalisiert werden.

  - Meistens mehrere äquivalente Formalisierungen möglich, manche einfacher abzuzählen als andere.

- Beispiel: Wie viele Poker-Hände sind möglich?
  - Karten $\mathcal{K} = \{2, 3, \ldots, 10, J, Q, K, A\}\{\heartsuit, \diamondsuit, \clubsuit, \spadesuit\}$
  - Anzahl Karten: $|\mathcal{K}| = 13 \cdot 4 = 52$.
  - Eine Poker-Hand besteht aus 5 Karten.
- Modellierung als 5-Tupel oder als 5-elementige Menge möglich.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Hand als 5-Tupel: $\Omega_g = \{(k_1, \ldots, k_5) \in \mathcal{K}^5 \mid |\{k_1, \ldots, k_5\}| = 5\}$.
  - „Geordnete Hand“
  - Für i-te Karte k_i nur noch $|\mathcal{K}| - (i-1)$ Karten verfügbar.
  - Insgesamt: $52 \cdot 51 \cdot 50 \cdot 49 \cdot 48$ Hände
- Hand als 5-elementige Menge: $\Omega_u = \{S \subseteq \mathcal{K} \mid |S| = 5\}$
  - „Ungeordnete Hand“
  - Jede ungeordnete Hand $\{k_1, \ldots, k_5\}$ lässt sich zu $5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 =: 5!$ geordneten Händen anordnen/auflisten.
  - D.h. je $5!$ geordnete Hände entsprechen einer ungeordneten Hand.
  - Also insgesamt: $\frac{52 \cdot 51 \cdot 50 \cdot 49 \cdot 48}{5!} = \frac{52!}{5! \cdot 47!} =: \binom{52}{5}$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Prinzipielles Vorgehen sobald Modellierung fixiert: Zählproblem schrittweise zurückführen auf bereits bekannte bzw. leicht abzuzählende Mengen.

Bsp Berechnung der Wahrscheinlichkeit eines „Straight Flush“.

- Ungeordnete Hand als 5-elementige Menge:

$$\Omega_u = \{H \subseteq \mathcal{K} \mid |H| = 5\}$$

„Straight Flush“ entspricht dann der Menge

$$\begin{aligned} A_u \quad = \quad & \{\{A\heartsuit, 2\heartsuit, 3\heartsuit, 4\heartsuit, 5\heartsuit\}, \{2\heartsuit, 3\heartsuit, 4\heartsuit, 5\heartsuit, 6\heartsuit\}, \\ & \ldots, \{9\spadesuit, 10\spadesuit, J\spadesuit, Q\spadesuit, K\spadesuit\}\} \end{aligned}$$

Jeder „Straight Flush“ (hier ohne Royal Flush) ist eindeutig durch die Farbe und die niedrigste Karte bestimmt, daher:

$$|A_u| = |\{A, 2, 3, \ldots, 9\} \times \{\heartsuit, \diamondsuit, \clubsuit, \spadesuit\}| = 36$$

Damit W'keit für „Straight Flush“ $\dfrac{|A_u|}{|\Omega_u|} = \dfrac{36}{\left(\frac{52 \cdot 51 \cdot \ldots \cdot 48}{5!}\right)}$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Prinzipielles Vorgehen sobald Modellierung fixiert: Zählproblem schrittweise zurückführen auf bereits bekannte bzw. leicht abzuzählende Mengen.

Bsp Beispiel: Berechnung der Wahrscheinlichkeit eines „Straight Flush“.

- Geordnete Hand als 5-Tupel mit 5 verschiedenen Einträgen:

$$\Omega_g = \{(k_1, \ldots, k_5) \in \mathcal{K}^5 \mid |\{k_1, \ldots, k_5\}| = 5\}$$

„Straight Flush“ entspricht dann der Menge

$$A_g \quad = \quad \{(k_1, \ldots, k_5) \mid \{k_1, \ldots, k_5\} \in A_u\}$$

Für jedes Element von A_u gibt es $5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 =: 5!$ Elemente in A_g. Daher:

$$|A_g| = |A_u| \cdot 5!$$

Damit W'keit für „Straight Flush“ $\dfrac{|A_g|}{|\Omega_g|} = \dfrac{36 \cdot 5!}{52 \cdot 51 \cdot \ldots \cdot 48}$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Jede endliche Menge A lässt sich mit $[|A|] = \{1, \ldots, |A|\}$ „identifizieren“, jede abzählbare Menge mit einer Teilmenge von $\mathbb{N}_0$.
- Damit lassen sich viele Abzählprobleme auf Teilmengen von $[n]^k$ oder $\mathbb{N}_0^k$ zurückführen, insbesondere (für $n, k \in \mathbb{N}_0$):
  - $\{(s_1, \ldots, s_k) \in [n]^k \mid |\{s_1, s_2, \ldots, s_k\}| = k\}$
  - $\{(s_1, \ldots, s_k) \in [n]^k \mid s_1 < s_2 < \ldots < s_k\}$
  - $\{(s_1, \ldots, s_k) \in [n]^k \mid s_1 \leq s_2 \leq \ldots \leq s_k\}$
  - $\{(s_1, \ldots, s_k) \in \mathbb{N}_0^k \mid s_1 + s_2 + \ldots + s_k = n\}$
  - $\{(s_1, \ldots, s_k) \in \mathbb{N}_0^k \mid s_1 + s_2 + \ldots + s_k \leq n\}$
- Wir bestimmen die Mächtigkeit dieser Mengen in Abhängigkeit von n, k.
- Bevor wir auf die einzelnen Mengen genauer eingehen, erwähnen wir einfache kombinatorische Grundprinzipien, die das Abzählen von Mengen vereinfachen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

4 Kombinatorik

Einführung

Kombinatorische Grundprinzipien

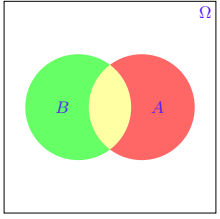
Das Urnenmodell: Ziehungen

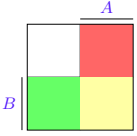
Weitere Verteilungsprobleme

Zusammenfassung

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Seien A, B beliebige Mengen.
  - Gilt $A \cap B = \emptyset$, dann auch $|A \cup B| = |A| + |B|$, da jedes Element von $A \cup B$ in genau einer der beiden Mengen vorkommt. D.h. die beiden Mengen können unabhängig voneinander abgezählt werden.





  - Analog im Fall einer Partition: wenn $X = \biguplus_{i \in \mathbb{N}_0} A_i$ dann $|X| = \sum_{i \in \mathbb{N}_0} |A_i|$.
  - Sind A, B endlich mit $A \cap B \neq \emptyset$, gilt $|A \cup B| < |A| + |B|$ und

$$|A \cup B| = |A| + |B| - |A \cap B|$$

  da $|A| + |B|$ jedes Element aus $A \cap B$ zweimal zählt.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Kombinatorische Grundprinzipien: Produktregel

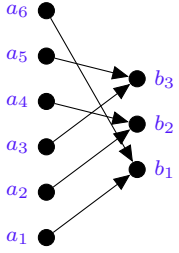
- Seien A, B beliebige Mengen.
  - Es gilt $|A \times B| = |A| \cdot |B|$, da wir für jedes der beiden Zeichen bzw. Komponenten unabhängig voneinander aus A bzw. B wählen dürfen.
  - Z.B. Anzahl Karten $\{2, 3, \ldots, J, Q, K, A\}\{\heartsuit, \diamondsuit, \clubsuit, \spadesuit\}$:

	$\heartsuit$	$\diamondsuit$	$\clubsuit$	$\spadesuit$
2	$2\heartsuit$	$2\diamondsuit$	$2\clubsuit$	$2\spadesuit$
3	$3\heartsuit$	$3\diamondsuit$	$3\clubsuit$	$3\spadesuit$
$\vdots$				
A	$A\heartsuit$	$A\diamondsuit$	$A\clubsuit$	$A\spadesuit$

  Analog $|A_1 \times \ldots \times A_n| = |A_1| \cdots |A_n|$, z.B. $|\{a, b\}^n| = 2^n$.
  - In Kombination mit erster Regel z.B. $|A \times (B \cup C)| = |A| \cdot (|B| + |C|)$, falls $B \cap C = \emptyset$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Seien A, B beliebige Mengen.
  - Gilt $f\colon A \to B$ mit $\left|f^{-1}(b)\right| = m$ konstant für alle $b \in B$, d.h. jedes Element aus B hat genau m Urbilder bzgl. f, dann gilt $|A| = m\,|B|$; d.h. es reicht, die kleinere der beiden Mengen abzuzählen.



  Ist f bijektiv, dann gilt natürlich $|A| = |f(A)| = |B|$ entsprechend der Definition der Kardinalität von Mengen.

- Das ist ein Spezialfall der ersten Regel für $A = \biguplus_{b \in B} f^{-1}(b)$, z.B.:

$$A = f^{-1}(b_1) \cup f^{-1}(b_2) \cup f^{-1}(b_3) = \{a_1, a_6\} \cup \{a_2, a_4\} \cup \{a_3, a_5\}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Lem Für beliebige Funktionen $f\colon A \to B$ gilt:

$$|A| = \sum_{b\in B} \left|f^{-1}(b)\right| \leq |B| \max_{b\in B} \left|f^{-1}(b)\right|$$

- ▷ Sind A und B endlich, folgt außerdem, dass

$$\max_{b\in B} \left|f^{-1}(b)\right| \geq \frac{|A|}{|B|}$$

d.h. es muss mindestens ein b mit mindestens $\lceil |A| \,/\, |B| \rceil$ Urbildern geben.

Def $\lceil x \rceil = \min\{k \in \mathbb{Z} \mid k \geq x\}$, z.B. $\lceil 1.2 \rceil = 2$, $\lceil -1.2 \rceil = -1$

- ▷ Diese einfache, aber häufig sehr nützliche Beobachtung wird als Schubfachprinzip bzw. pigeon-hole principle bezeichnet.

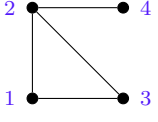
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Beispiele:
- In einem Rudel aus 7 Hunden haben immer mindestens $4 = \lceil 7/2 \rceil$ das gleiche Geschlecht.
- In einer Gruppe von 13 Personen gibt es stets mindestens $2 = \lceil 13/12 \rceil$, die im selben Monat Geburtstag haben.
- In einer Vorlesung mit 1400 Teilnehmern muss es eine Woche geben, in der mindestens $27 = \lceil 1400/52 \rceil$ ihren Geburtstag feiern, und bei jeder Klausur muss es mindestens eine Note (vor Notenbonus) geben, die mindestens $108 = \lceil 1400/13 \rceil$ Mal vergeben wird.
- Es gibt in Deutschland mindestens $415 = \lceil 83.000.000/200.000 \rceil$ Menschen, die exakt die gleiche Zahl von Haaren auf dem Kopf tragen.
- In jedem einfachen Graphen mit mind. 2 Knoten gibt es zwei Knoten mit demselben Grad.

- Im Kapitel über Graphen hatten wir das Handschlaglemma kennengelernt:

$$2\,|E| = \sum_{v \in V} \deg(v)$$

- Stellt man die sogenannte Inzidenzrelation „eIv **gdw** $v \in e$“ tabellarisch dar, dann ist $\deg(v)$ die Summe über die Einträge in der Spalte zu v. Z.B.



I	1	2	3	4
$\{1,2\}$	1	1	0	0
$\{1,3\}$	1	0	1	0
$\{2,3\}$	0	1	1	0
$\{2,4\}$	0	1	0	1

- $\sum_{v \in V} \deg(v)$ ist daher die Summe aller Spaltensummen und die Summe über jede Zeile ist stets gleich 2, da jede Kante zu genau zwei Knoten inzident ist.
- Damit ist die Summe aller Zeilensummen genau $2\,|E|$ und das Handschlaglemma lediglich die Aussage, dass die Summe aller Spaltensummen und die Summe aller Zeilensummen gleich sein müssen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Einfache Zählregeln: Doppeltes Abzählen

- Allgemein: Ist $R \subseteq S \times T$ eine binäre Relation mit $S = \{s_1, \ldots, s_m\}$ und $T = \{t_1, \ldots, t_n\}$ beliebig aufgezählt, dann nennt man die Matrix $(m_{i,j})_{i \in [m], j \in [n]} \in \{0,1\}^{m \times n}$ mit „$m_{i,j} = 1$ **gdw** $s_i R t_j$“ die Inzidenzmatrix zu R.
- Unter doppeltem Abzählen versteht man dann, dass

  die Summe der Spaltensummen $\sum_{j \in [n]} \sum_{i \in [m]} m_{i,j}$ gleich

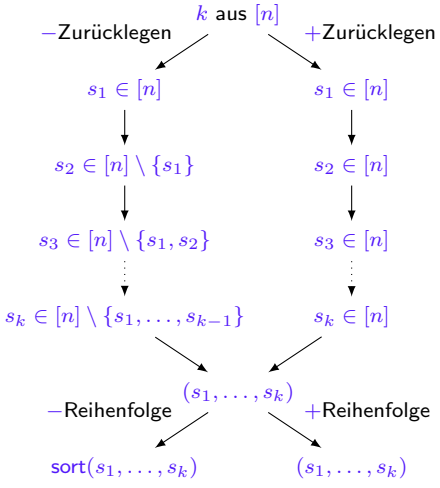
  der Summe der Zeilensummen $\sum_{i \in [m]} \sum_{j \in [n]} m_{i,j}$ ist.
- Mit anderen Worten, für jede binäre Relation $R \subseteq S \times T$ gilt $\sum_{s \in S} |\{t \in T \mid (s,t) \in R\}| = \sum_{t \in T} |\{s \in S \mid (s,t) \in R\}|$.
- Nützlich ist diese Beobachtung, wenn sich entweder die Zeilen- oder Spaltensummen sehr einfach bestimmen lassen, z.B. im Fall des Handschlaglemmas, wo die Zeilensummen stets gleich 2 sind.
- Weiterführendes Beispiel: Das Brettspiel Hex lässt kein Unentschieden zu.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

4 Kombinatorik
Einführung
Kombinatorische Grundprinzipien
Das Urnenmodell: Ziehungen
Ziehen ohne Zurücklegen, mit Beachtung der Reihenfolge
Ziehen ohne Zurücklegen, ohne Beachtung der Reihenfolge
Ziehen mit Zurücklegen, ohne Beachtung der Reihenfolge
Beispiele
Weitere Verteilungsprobleme
Zusammenfassung

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Ziehen aus einer Urne



- Urne: fiktiver Behälter mit n Kugeln, die mit den Zahlen $1, \ldots, n$ beschriftet sind.
- Es werden k Kugeln hintereinander gezogen. Verschiedene Regeln für die Ziehung führen zu verschiedenen Ergebnismengen.
  - Ziehen ohne Zurücklegen, mit Beachtung der Reihenfolge.

    $$A_{n,k} := \{(s_1, \ldots, s_k) \in [n]^k \mid |\{s_1, s_2, \ldots, s_k\}| = k\}$$

  - Ziehen ohne Zurücklegen, ohne Beachtung der Reihenfolge.

    $$B_{n,k} := \{(s_1, \ldots, s_k) \in [n]^k \mid s_1 < s_2 < \ldots < s_k\}$$

    „ohne Beachtung der Reihenfolge“ äquivalent zu „darf beliebige Anordnung fixieren“.
  - Ziehen mit Zurücklegen, ohne Beachtung der Reihenfolge.

    $$C_{n,k} := \{(s_1, \ldots, s_k) \in [n]^k \mid s_1 \leq s_2 \leq \ldots \leq s_k\}$$

  - (Ziehen mit Zurücklegen, mit Beachtung der Reihenfolge: $[n]^k$)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Ziehen aus einer Urne

- Wie viele mögliche Lottoziehungen gibt es?

  49 Bälle und 6 Ziehungen. Ziehen ohne Zurücklegen, ohne Beachtung der Reihenfolge.

- Ein Wettbewerb vergibt einen ersten, einen zweiten, ... und einen k-ten Preis. Wie viele Möglichkeiten gibt es, die Preise unter den n Wettbewerbsteilnehmern zu verteilen?

  n Bälle und k Ziehungen. Ziehen ohne Zurücklegen, mit Beachtung der Reihenfolge.

- Wie viele Möglichkeiten gibt es, k Fußballspiele auf n Austragungsorte zu verteilen?

  n Bälle und k Ziehungen. Ziehen mit Zurücklegen, mit Beachtung der Reihenfolge.

- Wie viele Möglichkeiten gibt es, k Euro unter n Personen zu verteilen?

  n Bälle und k Ziehungen. Ziehen mit Zurücklegen, ohne Beachtung der Reihenfolge.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

$$A_{n,k} := \{(s_1, \ldots, s_k) \in [n]^k \mid |\{s_1, s_2, \ldots, s_k\}| = k\}$$

„k-Tupel mit k verschiedenen Einträgen aus $[n] = \{1, 2, \ldots, n\}$“

- Bsp.: (geordnete) Poker-Hände: $\{(k_1, \ldots, k_5) \in [52]^5 \mid |\{k_1, \ldots, k_5\}| = 5\}$.
- Mögliche Fälle in Abhängigkeit von n, k:
  - Falls $k > n$, ist die Menge leer, da die Anforderungen nicht erfüllt werden können, also $|A_{n,k}| = 0$.
  - Falls $k = 0 \leq n$, dann erfüllt nur das leere Tupel die Anforderungen, also $|A_{n,0}| = 1$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

$$A_{n,k} := \{(s_1, \ldots, s_k) \in [n]^k \mid |\{s_1, s_2, \ldots, s_k\}| = k\}$$

„k-Tupel mit k verschiedenen Einträgen aus $[n] = \{1, 2, \ldots, n\}$“

- Bsp.: (geordnete) Poker-Hände: $\{(k_1, \ldots, k_5) \in [52]^5 \mid |\{k_1, \ldots, k_5\}| = 5\}$.
  - Falls $0 < k \leq n$, dann können wir rekursiv vorgehen, d.h. das Problem auf sich selbst zurückführen, nur mit kleineren Parametern:

    Jedes k-Tupel $(s_1, \ldots, s_k) \in A_{n,k}$ unterteilt sich in s_1 und in ein Tupel $(s_2, \ldots, s_k)$ von $k-1$ verschiedenen Einträgen aus $[n] \setminus \{s_1\}$.

    Daher $|A_{n,k}| = n \cdot |A_{n-1,k-1}|$, wobei wir bereits wissen, dass $|A_{n,0}| = 1$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

$$A_{n,k} := \{(s_1, \ldots, s_k) \in [n]^k \mid |\{s_1, s_2, \ldots, s_k\}| = k\}$$

„k-Tupel mit k verschiedenen Einträgen aus $[n] = \{1, 2, \ldots, n\}$“

- Bsp.: (geordnete) Poker-Hände: $\{(k_1, \ldots, k_5) \in [52]^5 \mid |\{k_1, \ldots, k_5\}| = 5\}$.
  - Mittels der Rekursionsgleichung

$$|A_{n,k}| = n \cdot |A_{n-1,k-1}| \text{ mit } |A_{n,0}| = 1$$

erhalten wir z.B. für die Anzahl der möglichen Hände beim Poker:

$$|A_{52,5}| = 52 \cdot |A_{51,4}| = 52 \cdot 51 \cdot |A_{50,3}| = \ldots = 52 \cdot 51 \cdot 50 \cdot 49 \cdot 48 \cdot |A_{47,0}| = \frac{52!}{47!}$$

Im Allgemeinen: $|A_{n,k}| = n \cdot (n-1) \cdot \ldots \cdot (n-k+1) = \frac{n!}{(n-k)!}$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Die Menge aller möglichen Ziehungen ist

$$\{\{s_1, \ldots, s_k\} \subseteq [n] \mid |\{s_1, \ldots, s_k\}| = k\} =: \binom{[n]}{k}$$

„k-elementige Teilmengen von $[n]$“

- Jede k-elementige Teilmenge von $[n]$ lässt sich durch Sortieren eindeutig mit einem sortierten k-Tupel identifizieren, z.B.

$$\{3, 4, 2, 6\} \mapsto (2, 3, 4, 6)$$

- D.h. es gibt eine Bijektion zwischen der Menge $\binom{[n]}{k}$ der k-elementigen Teilmengen von $[n]$ und der Menge

$$B_{n,k} := \{(s_1, \ldots, s_k) \in [n]^k \mid s_1 < s_2 < \ldots < s_k\}$$

„Aufsteigend sortierte k-Tupel mit k verschiedenen Einträgen aus $[n]$“

- Beide Mengen sind somit gleich mächtig, und es reicht $B_{n,k}$ abzuzählen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

$$B_{n,k} := \{(s_1, \ldots, s_k) \in [n]^k \mid s_1 < s_2 < \ldots < s_k\}$$

▷ Bsp.: Anzahl der ungeordneten Hände beim Poker mit 52 Karten.

Fixiere beliebige Aufzählung $\beta \colon \mathcal{K} \xrightarrow{\text{bij.}} [52]$, z.B.:

$$\beta(2\heartsuit) = 1, \ldots, \beta(A\heartsuit) = 13, \beta(2\diamondsuit) = 14, \ldots, \beta(2\clubsuit) = 27, \ldots, \beta(A\spadesuit) = 52$$

Damit: Ungeordnete Hand $\xrightarrow{\text{bij.}}$ Teilmenge von $[52]$ $\xrightarrow{\text{bij.}}$ sortiertes Tupel.

$$\{8\diamondsuit, A\spadesuit, 2\heartsuit, 3\heartsuit, 4\diamondsuit\} \mapsto \{20, 52, 1, 2, 16\} \mapsto (1, 2, 16, 20, 52)$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

$$B_{n,k} := \{(s_1, \ldots, s_k) \in [n]^k \mid s_1 < s_2 < \ldots < s_k\}$$

- Jedes $(s_1, \ldots, s_k) \in B_{n,k}$ kann auf $k!$ unterschiedliche Arten umsortiert werden.
  - Gegeben ein sortiertes Tupel $(s_1, \ldots, s_k) \in B_{n,k}$ setze $S := \{s_1, \ldots, s_k\}$ und betrachte die Menge $T := \{(t_1, \ldots, t_k) \in S^k \mid |\{t_1, \ldots, t_k\}| = k\}$
  - Das sind genau alle möglichen Permutationen der Einträge von $(s_1, \ldots, s_k)$.
  - T steht in Bijektion mit $A_{k,k}$; also $|T| = |A_{k,k}| = k!$.
  - Umgekehrt entsprechen jeweils $k!$ Tupel aus $A_{n,k}$ genau einem Tupel aus $B_{n,k}$, also $|B_{n,k}| = \frac{|A_{n,k}|}{k!}$ (mit $0! := 1$).
  - Bsp.: $\frac{52!}{47!}$ geordnete Hände, $\frac{52!}{47! \cdot 5!}$ ungeordnete Hände.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

$$B_{n,k} := \{(s_1, \ldots, s_k) \in [n]^k \mid s_1 < s_2 < \ldots < s_k\}$$

- Mit $|A_{n,k}| = \dfrac{n!}{(n-k)!}$ für $0 \leq k \leq n$ erhält man insgesamt

$$|B_{n,k}| = \frac{n!}{(n-k)!\,k!}\,.$$

- Allgemein definiert man den Binomialkoeffizienten

$$\binom{n}{k} := \begin{cases} \dfrac{n!}{(n-k)!\,k!} & \text{falls } 0 \leq k \leq n \\ 0 & \text{sonst} \end{cases}$$

so dass $|B_{n,k}| = \binom{n}{k}$, d.h. $\binom{n}{k}$ ist die Anzahl der k-elementigen Teilmengen einer n-elementigen Menge.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

$$B_{n,k} := \{(s_1, \ldots, s_k) \in [n]^k \mid s_1 < s_2 < \ldots < s_k\}$$

- Wir betrachten noch einen alternativen Weg zur Bestimmung von $|B_{n,k}|$ mittels einer Rekursionsgleichung.
- Jedes sortierte k-Tupel $(s_1, \ldots, s_k) \in B_{n,k}$ mit Einträgen aus $[n]$ fällt in genau eine der beiden Klassen:
  - n kommt nicht vor, d.h. $s_k \leq n-1$, also $(s_1, \ldots, s_k) \in B_{n-1,k}$.
  - n kommt vor, d.h. $s_k = n$ und damit $(s_1, \ldots, s_{k-1}) \in B_{n-1,k-1}$.

    (D.h. $\{(s_1, \ldots, s_{k-1}, n) \in B_{n,k}\}$ steht in Bijektion mit $B_{n-1,k-1}$.)

  Damit können wir statt $B_{n,k}$ die beiden Mengen $B_{n-1,k}$ und $B_{n-1,k-1}$ jeweils zählen und mit Hilfe der Summenregel einfach addieren:

$$|B_{n,k}| = |B_{n-1,k}| + |B_{n-1,k-1}| \text{ mit } |B_{n,0}| = 1 \text{ und } |B_{n,n}| = 1$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

$$B_{n,k} := \{(s_1, \ldots, s_k) \in [n]^k \mid s_1 < s_2 < \ldots < s_k\}$$

- Damit erhält man die Rekursionsgleichung für die Binomialkoeffizienten, auch bekannt als Pascalsche Identität:

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

die eine einfache rekursive Berechnung von $\binom{n}{k}$ erlaubt (siehe dynamische Programmierung in Einführung in die Informatik).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Lem Die Binomialformel, gültig für alle $x, y \in \mathbb{R}$, $n \in \mathbb{N}_0$, lautet:

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k}$$

Begründung: Multipliziert man mechanisch $\overbrace{(x+y)\cdots(x+y)}^{n}$ aus, so gilt

$$(x+y)^n = \sum_{(a_1,\ldots,a_n)\in\{x,y\}^n} a_1 a_2 \cdots a_n$$

d.h. man erhält jedes Wort aus $\{x,y\}^n$ genau einmal als Summanden, da man aus jedem der n Faktoren $(x+y)$ entweder x oder y wählen darf.

- Es gilt $a_1 \cdots a_n = x^k y^{n-k}$ **gdw** x genau k-mal in $a_1 \cdots a_n$ vorkommt. Damit gibt es eine Bijektion zwischen den Tupeln $(a_1, \ldots, a_n)$ mit $a_1 \cdots a_n = x^k y^{n-k}$ und den k-elementigen Teilmengen von $[n]$.
- Es gibt also $\binom{n}{k}$ Summanden mit Wert $x^k y^{n-k}$ (daher „Binomialkoeffizient“).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Mit $y := 1$ ergibt sich:

$$(1+x)^n = \sum_{k=0}^{n} \binom{n}{k} x^k$$

Leitet man beide Seiten nach x ab, so erhält man hiermit:

$$n(1+x)^{n-1} = \sum_{k=1}^{n} \binom{n}{k} k x^{k-1}$$

Insbesondere für $x := 1$ folgt daher:

$$n \cdot 2^{n-1} = \sum_{k=1}^{n} \binom{n}{k} k$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Entsprechend ergibt sich einerseits

$$(1+x)^m(1+x)^n = (1+x)^{m+n} = \sum_{k=0}^{m+n} \binom{m+n}{k} x^k$$

und andererseits

$$(1+x)^m(1+x)^n = \underbrace{\left(\sum_{i=0}^{m} \binom{m}{i} x^i\right)\left(\sum_{j=0}^{n} \binom{n}{j} x^j\right)}_{*}$$

Der Koeffizient von x^k in $*$ ergibt sich durch Summation der Produkte der Koeffizienten von x^i und x^j für $i+j=k$:

$$\binom{m+n}{k} x^k = \sum_{i=0}^{k} \binom{m}{i} x^i \binom{n}{k-i} x^{k-i}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Mit $x := 1$ erhält man die Vandermondesche Identität:

$$\binom{m+n}{k} = \sum_{i=0}^{k} \binom{m}{i}\binom{n}{k-i}$$

- Alternative Herleitung der Vandermondeschen Identität:

  Um k Elemente aus $[m+n]$ ohne Zurücklegen ohne Beachtung der Reihenfolge zu ziehen, kann folgendermaßen vorgegangen werden:

  - Wähle eine Zahl $0 \leq i \leq k$.
  - Ziehe i Elemente aus $[m]$.
  - Ziehe die verbleibenden $k-i$ Elemente aus $[m+n] \setminus [m]$.

  Für jedes i gibt es $\binom{m}{i}\binom{n}{k-i}$ mögliche Ziehungen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Ebenfalls mittels der Binomialformel erhält man

$$0 = \sum_{k=0}^{n} \binom{n}{k} (-1)^k \quad \text{für } n \geq 1$$

und

$$2^n = \sum_{k=0}^{n} \binom{n}{k}$$

Letzteres bedeutet, dass $[n]$ genau 2^n Teilmengen besitzt, da $\binom{n}{k}$ die Anzahl der k-elementigen Teilmengen von $[n]$ ist.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

$$C_{n,k} := \{(s_1, \ldots, s_k) \in [n]^k \mid s_1 \leq s_2 \leq \ldots \leq s_k\}$$

„Aufsteigend sortierte k-Tupel über $[n]$ mit Wiederholungen“

- Bsp.: 5 Karten ziehen mit Zurücklegen in den Stapel nach jeder Ziehung

  Karten wieder mit $[52]$ identifiziert.

  Geordnete Hand: $\left|\{(s_1, s_2, \ldots, s_5) \in [52]^5\}\right| = 52^5$.

  Ungeordnete Hand:
  $\left|\{(s_1, s_2, \ldots, s_5) \in [52]^5 \mid s_1 \leq s_2 \leq \ldots \leq s_5\}\right| = |C_{52,5}| = ?$

  (Erinnerung: „ungeordnet“ entspricht „fixiere beliebige Anordnung“)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

$$C_{n,k} := \{(s_1, \ldots, s_k) \in [n]^k \mid s_1 \leq s_2 \leq \ldots \leq s_k\}$$

„Aufsteigend sortierte k-Tupel über $[n]$ mit Wiederholungen“

- Beobachtung: Da die Anordnung vorgegeben ist, reicht es zu wissen, wie häufig jeder Wert $s \in [n]$ in $(s_1, \ldots, s_k)$ vorkommt.
- $D_{k,n} := \{(k_1, \ldots, k_n) \in \mathbb{N}_0^n \mid k_1 + k_2 + \ldots + k_n = k\}$ Zählvektoren mit Summe k.

  Eintrag k_i zählt Vorkommen von Wert i (d.h., wie oft der Wert i gezogen wurde).
- Bsp.: $n = 5, k = 6$

$$C_{5,6} \ni (1,1,2,4,5,5) \overset{\mapsto}{\underset{\mapsto}{\leftrightarrows}} (2,1,0,1,2) \in D_{6,5}$$

- Da $|C_{n,k}| = |D_{k,n}|$ (Positionen von n und k beachten!) zählen wir $D_{k,n}$ statt $C_{n,k}$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

$$D_{n,k} := \{(s_1, \ldots, s_k) \in \mathbb{N}_0^k \mid s_1 + s_2 + \ldots + s_k = n\}$$

„Zählvektoren mit k Einträgen und Summe n“

- Eselsbrücke: Komponenten des Zählvektors unär statt dezimal darstellen

  Aus 0 wird ε, aus 1 wird $|$, aus 2 wird $||$, aus 10 wird $||||||||||$, usw.

- Beispiel:

$$(1,1,2,4,5,5) \overset{\mapsto}{\underset{\leftarrow}{\mapsto}} (2,1,0,1,2) \overset{\mapsto}{\underset{\leftarrow}{\mapsto}} ||,|,,|,||$$

- $D_{n,k}$ entspricht der Menge der Wörter, die aus genau n Strichen und $k-1$ Kommata bestehen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

$$D_{n,k} := \{(s_1, \ldots, s_k) \in \mathbb{N}_0^k \mid s_1 + s_2 + \ldots + s_k = n\}$$

„Zählvektoren mit k Einträgen und Summe n“

- Jedes Wort der Länge $n + k - 1$ mit genau n Strichen und $k - 1$ Kommata ist durch die Positionen der Striche bzw. der Kommata vollständig beschrieben.

Bsp $(1, 1, 2, 4, 5, 5) \overset{\mapsto}{\underset{\mapsto}{\leftrightarrow}} (2, 1, 0, 1, 2) \overset{\mapsto}{\underset{\mapsto}{\leftrightarrow}} ||, |, , |, || \overset{\mapsto}{\underset{\mapsto}{\leftrightarrow}} \{1, 2, 4, 7, 9, 10\}$

- Es gibt $\binom{n+k-1}{n}$ Möglichkeiten, um die Positionen der Striche zu wählen.
- Wir erhalten: $|D_{n,k}| = \binom{n+k-1}{n} = |C_{k,n}|$
- Durch Vertauschen von n und k ergibt sich $|C_{n,k}| = \binom{n+k-1}{k}$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

$$E_{n,k} := \{(s_1, \ldots, s_k) \in \mathbb{N}_0^k \mid s_1 + s_2 + \ldots + s_k \leq n\}$$

„Zählvektoren mit k Einträgen und Summe höchstens n“

- Z.B. Ziehen von maximal 5 Karten mit Zurücklegen ohne Beachtung der Reihenfolge.
- Ansatz: $E_{n,k}$ steht in Bijektion mit $D_{n,k+1}$ („Zählvektoren mit $k+1$ Einträgen und Summe n“)

  Eintrag $s_{k+1} = n - (s_1 + \ldots + s_k)$ sammelt den „Rest“:

$$E_{n,k} \ni (s_1, \ldots, s_k) \overset{\mapsto}{\underset{\leftarrow}{}} (s_1, \ldots, s_k, n - (s_1 + s_2 + \ldots + s_k)) \in D_{n,k+1}$$

  Damit: $|E_{n,k}| = |D_{n,k+1}| = \binom{n+k}{n}$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Eine Variante

- Alternativer Ansatz (zum Vergleich): für jedes $0 \leq m \leq n$ gibt es $|D_{m,k}|$ Zählvektoren mit Summe m.

  Es folgt

$$|E_{n,k}| = \sum_{m=0}^{n} |D_{m,k}| = \sum_{m=0}^{n} \binom{m+k-1}{m}$$

  Also gilt

$$\sum_{m=0}^{n} \binom{m+k-1}{m} = \binom{n+k}{n}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Beispiele

- Wie viele mögliche Lottoziehungen (6 aus 49) gibt es?
  z.B. $\mathsf{sort}(30, 12, 20, 14, 10, 39) = (10, 12, 14, 20, 30, 39)$
  - $\binom{49}{6}$
- Ein Wettbewerb vergibt einen ersten, einen zweiten und einen dritten Preis. Wie viele Möglichkeiten gibt es, die Preise unter den 21 Wettbewerbsteilnehmern zu verteilen?
  z.B. (Verstappen, Hamilton, Bottas)
  - $\binom{21}{3} \cdot 3!$.
- Wie viele Möglichkeiten gibt es, 51 EM-Fußballspiele auf 11 Austragungsorte zu verteilen?
  z.B. TUR:ITA $\mapsto$ Rom, ..., ENG:DEN $\mapsto$ London, ITA:ENG $\mapsto$ London
  - 11^{51}
- Wie viele Möglichkeiten gibt es, 5 Perlen unter Alice, Bob und Charlie aufzuteilen?
  z.B. sort(Bob, Charlie, Alice, Charlie, Bob) = (Alice, Bob, Bob, Charlie, Charlie)
  - $\binom{3+5-1}{5}$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Beispiele

- Wie viele Möglichkeiten gibt es, die Buchstaben ABCDEFGH so anzuordnen, dass die Buchstabenfolge ABC enthalten ist?
  - Wir führen das Problem auf das Ziehen von 6 Elementen ohne Zurücklegen mit Beachtung der Reihenfolge aus der 6-elementigen Menge $\{ \mathsf{ABC}, \mathsf{D}, \mathsf{E}, \mathsf{F}, \mathsf{G}, \mathsf{H} \}$. Damit gibt es $6! = 720$ Möglichkeiten.
- Wie viele verschiedene Wörter erhält man durch Permutation der Buchstaben von „hallo“?
  - Wenn die zwei „l“ unterscheidbar wären ($\mathsf{hal_1l_2o}$), dann gäbe es $5! = 120$ Wörter.
  - Dadurch wird jedes Wort jedoch genau zweimal gezählt: Z.B. ist ohlal mit $\mathsf{ohl_1al_2}$ und $\mathsf{ohl_2al_1}$ zweimal vertreten. Man erhält also $120/2 = 60$ Wörter.
- Wie viele verschiedene Wörter erhält man durch Permutation der Buchstaben von „bewundernswert“?
  - Das Wort enthält drei „e“, zwei „w“, zwei „n“ und zwei „r“. Mit $14!$ wird jedes Wort $3! \cdot 2! \cdot 2! \cdot 2! = 48$ Mal gezählt. Man erhält also $14!/48 = 1.816.214.400$ Wörter.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Stuttgart(dpa/lsw). *Die Staatliche Toto-Lotto GmbH in Stuttgart hat eine Lottosensation gemeldet: Zum ersten Mal in der 40jährigen Geschichte des deutschen Zahlenlottos wurden zwei identische Gewinnreihen festgestellt. Am 21. Juni dieses Jahres [3016. Ausspielung] kam im Lotto am Mittwoch in der Ziehung A die Gewinnreihe 15-25-27-30-42-48 heraus. Genau dieselben Zahlen wurden bei der 1628. Ausspielung im Samstagslotto schon einmal gezogen, nämlich am 20. Dezember 1986. Welch ein Lottozufall: Unter den 49 Zahlen sind fast 14 Millionen verschiedene Sechserreihen möglich.*

▷ Ist das wirklich eine Sensation?

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Die Lottosensation vom 29.6.1995

- Es gibt $M := \binom{49}{6} = 13.983.816$ mögliche (Sechser-)Ziehungen.
- Wie viele Sequenzen von 3016 Ziehungen gibt es, und wie viele davon enthalten irgendeine Ziehung mindestens zweimal?
- Sei Z die Menge aller Sechser-Ziehungen und $|Z| = M$. Wir ziehen 3016 Elemente aus Z mit Zurücklegen und mit Beachtung der Reihenfolge. Die Anzahl S der möglichen Tupel von Sechser-Ziehungen beträgt $S = M^{3016}$.
- Wie viele von diesen Tupeln enthalten irgendeine Sechser-Ziehung mindestens zweimal?

  Trick: wir berechnen die Anzahl *HE* der Tupel, in denen jede Sechser-Ziehung höchstens einmal vorkommt, und subtrahieren sie von S.
  Für die Berechnung von *HE* ziehen wir 3016 Elemente aus Z ohne Zurücklegen mit Beachtung der Reihenfolge. Wir erhalten:

$$HE = \frac{M!}{(M - 3016)!}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Unter der Annahme, dass alle Sequenzen gleich wahrscheinlich sind, haben wir für die W'keit p nach 3016 Ziehungen mindestens eine Wiederholung gesehen zu haben:

$$p = \frac{S - HE}{S} = 1 - \frac{HE}{S} = 1 - \frac{\frac{M!}{(M-3016)!}}{M^{3016}} = 1 - \prod_{i=0}^{3015} \frac{M-i}{M}$$
$$\approx 0,278 = 27,8\%$$

- Mit demselben Prinzip erhält man, dass die W'keit, dass in einer Gruppe von 23 Personen mind. zwei Personen am gleichen Tag Geburtstag haben, größer als 50% ist (Geburtstagsparadoxon).

4 Kombinatorik
Einführung
Kombinatorische Grundprinzipien
Das Urnenmodell: Ziehungen
Weitere Verteilungsprobleme
Stirling-Zahlen 2. Art
Die Siebformel
Anzahl Partitionen bei vorgegebener Klassengröße
Stirling-Zahlen 1. Art
Partitionsfunktionen/Zahlpartitionen
Zusammenfassung

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Wir haben schon einige „Verteilungsprobleme“ gelöst, wie die Berechnung der Möglichkeiten, k Fußball-Spiele auf n Austragungsorte oder k Euromünzen auf n Menschen zu verteilen.
- Wir betrachten nun weitere Probleme dieser Art:
  - Wie viele Möglichkeiten gibt es, k Weihnachtsgeschenke unter n Kindern zu verteilen, so dass jedes Kind mindestens ein Geschenk erhält?

    Sowohl die Kinder als auch die Geschenke sind unterscheidbar!
  - Wie viele Möglichkeiten gibt es, k Weihnachtsgeschenke in n Päckchen aufzuteilen, so dass jedes Päckchen mindestens ein Geschenk enthält?

    Die Päckchen sind nicht unterscheidbar, die Geschenke schon.
  - Wie viele Möglichkeiten gibt es, n Euromünzen unter k Kindern zu verteilen, so dass jedes Kind mindestens eine Münze erhält?

    Die Münzen sind nicht unterscheidbar, die Kinder schon.
  - Wie viele Möglichkeiten gibt es, n Euromünzen in k Päckchen aufzuteilen, so dass jedes Päckchen mindestens eine Münze enthält?

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Wie viele Möglichkeiten gibt es, k (Weihnachts)-Geschenke unter n Kindern zu verteilen, so dass jedes Kind mindestens ein Geschenk erhält?
- Sowohl die Kinder als auch die Geschenke sind unterscheidbar!
- Die Menge der Möglichkeiten ist

$$F_{n,k} := \{(s_1, \ldots, s_k) \in [n]^k \mid |\{s_1, s_2, \ldots, s_k\}| = n\} .$$

  Hier ist s_i das Kind, welches das i-te Geschenk erhält.
- Z.B. für $k = 6, n = 4$ beschreibt das Tupel $(3, 1, 3, 2, 2, 4)$, dass die Geschenke $1, 2, 3, 4, 5, 6$ jeweils an die Kinder $3, 1, 3, 2, 2, 4$ ausgehändigt werden.

  Es gilt z.B. $(3, 3, 2, 2, 4, 4) \notin F_{n,k}$, denn Kind 1 geht leer aus.
- $F_{n,k}$ ist für $k \geq n$ nicht leer; für $k < n$ jedoch $F_{n,k} = \emptyset$. Es gilt $|F_{n,n}| = n!$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Äquivalente Darstellung: wir geben für jedes Kind an, welche nicht-leere Teilmenge der Geschenke es erhält.
- Mathematisch: Ordne dem k-Tupel $(s_1, \ldots, s_k) \in F_{n,k}$ diejenige geordnete Partition von $[k]$ in genau n Klassen zu, die gegeben wird durch: $i, j \in [k]$ liegen in derselben Klasse, falls $s_i = s_j$.
- Z.B. für $n = 4, k = 6$:

$$(3,1,3,2,2,4) \mapsto (\{2\},\{4,5\},\{1,3\},\{6\})$$
$$(3,1,3,4,2,4) \mapsto (\{2\},\{5\},\{1,3\},\{4,6\})$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Stirling-Zahlen 2. Art

- Wie viele Möglichkeiten gibt es, k Geschenke in n Päckchen aufzuteilen, so dass jedes Päckchen mindestens ein Geschenk enthält?
- Die Päckchen sind nicht unterscheidbar!
- Eine Aufteilung entspricht nun einer ungeordneten Partition von $[k]$ in n Äquivalenzklassen:

$$\{\{2\},\{4,5\},\{1,3\},\{6\}\} = \{\{1,3\},\{2\},\{6\},\{4,5\}\}$$

- Die Anzahl der Partitionen von n Elementen in k (nicht leere) Klassen bezeichnet man als Stirling-Zahl zweiter Art $S_{n,k}$.
- Achtung:
  - In $F_{n,k}$ bezeichnet n die Anzahl der Klassen und k die Anzahl der Objekte.
  - In $S_{n,k}$ bezeichnet n die Anzahl der Objekte und k die Anzahl der Klassen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Für jede Partition von n Elementen in k (nicht leere) Klassen gibt es genau $k!$ geordnete Partitionen. Z.B. für $n = 4, k = 3$ haben wir

$$\{\{2\},\{1,4\},\{3\}\} \mapsto \left\{ \begin{array}{lcl} (\{2\},\{1,4\},\{3\}) & , & (\{2\},\{3\},\{1,4\}) \\ (\{1,4\},\{2\},\{3\}) & , & (\{1,4\},\{3\},\{2\}) \\ (\{3\},\{1,4\},\{2\}) & , & (\{3\},\{2\},\{1,4\}) \end{array} \right\}$$

- Somit gilt

$$|F_{n,k}| = n! \cdot S_{k,n}$$

- Wir bestimmen $S_{k,n}$ mit Hilfe einer Rekursionsgleichung.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Rekursionsgleichung für die Stirling-Zahlen 2. Art: Idee

- Betrachten wir die Partitionen von $[5]$ in 4 Klassen:

$$\{\{1\},\{2\},\{3,4\},\{5\}\} \quad \{\{1,5\},\{2\},\{3\},\{4\}\}$$
$$\{\{1\},\{2,3\},\{4\},\{5\}\} \quad \{\{1\},\{2,5\},\{3\},\{4\}\}$$
$$\{\{1\},\{2,4\},\{3\},\{5\}\} \quad \{\{1\},\{2\},\{3,5\},\{4\}\}$$
$$\{\{1,2\},\{3\},\{4\},\{5\}\} \quad \{\{1\},\{2\},\{3\},\{4,5\}\}$$
$$\{\{1,3\},\{2\},\{4\},\{5\}\}$$
$$\{\{1,4\},\{2\},\{3\},\{5\}\}$$

- Wir teilen sie auf in Partitionen, in denen 5 eine eigene Klasse bildet (6 Stück, links) und den Rest (4 Stück, rechts).
- Wenn wir die 5 „entfernen“, erhalten wir links alle Partitionen von $[4]$ in 3 Klassen und rechts alle Partitionen von $[4]$ in 4 Klassen

$$\{\{1\},\{2\},\{3,4\}\} \quad \{\{1\},\{2\},\{3\},\{4\}\}$$
$$\{\{1\},\{2,3\},\{4\}\}$$
$$\{\{1\},\{2,4\},\{3\}\}$$
$$\{\{1,2\},\{3\},\{4\}\}$$
$$\{\{1,3\},\{2\},\{4\}\}$$
$$\{\{1,4\},\{2\},\{3\}\}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Rekursionsgleichung für die Stirling-Zahlen 2. Art: Idee

- Jede Partition von $[4]$ in 3 Klassen ergibt eine Partition von $[5]$ in 4 Klassen:

$$\{\{1\},\{2\},\{3,4\}\} \mapsto \{\{1\},\{2\},\{3,4\},\{5\}\}$$

- Jede Partition von $[4]$ in 4 Klassen ergibt 4 Partitionen von $[5]$ in 4 Klassen:

$$\begin{array}{rcl} \{\{1\},\{2\},\{3\},\{4\}\} & \mapsto & \{\{1,5\},\{2\},\{3\},\{4\}\} \\ & & \{\{1\},\{2,5\},\{3\},\{4\}\} \\ & & \{\{1\},\{2\},\{3,5\},\{4\}\} \\ & & \{\{1\},\{2\},\{3\},\{4,5\}\} \end{array}$$

- Es gilt also: $S_{5,4} = S_{4,3} + 4 \cdot S_{4,4}$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Rekursionsgleichung für die Stirling-Zahlen 2. Art

- Ähnlich zu der Rekursionsgleichung $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$ für die Binomialkoeffizienten partitioniert man nach n.
- Sei $\mathcal{P} = \{P_1, \ldots, P_k\}$ eine Partition von $[n]$ in k Klassen.

  Falls $\{n\} \in \mathcal{P}$, dann ist $\mathcal{P} \setminus \{\{n\}\}$ eine Partition von $[n-1]$ in $k-1$ Klassen.

  Ansonsten ist $\{P \setminus \{n\} \mid P \in \mathcal{P}\}$ eine Partition von $[n-1]$ in k Klassen.
- Dabei lässt sich jede Partition von $[n-1]$ in k Klassen auf k Arten zu einer Partition von $[n]$ in k Klassen erweitern: man muss nur die Klasse wählen, die um n ergänzt werden muss.
- Damit gilt für $n \geq k \geq 0$:

$$\begin{array}{lcll} S_{n,k} & = & S_{n-1,k-1} + k \cdot S_{n-1,k} & \text{für } n > k > 0 \\ S_{n,0} & = & 0 & \text{für } n > 0 \\ S_{n,n} & = & 1 & \end{array}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Eine geschlossene Form für $|F_{n,k}|$ und $S_{n,k}$

- $F_{n,k} := \{(s_1, \ldots, s_k) \in [n]^k \mid |\{s_1, s_2, \ldots, s_k\}| = n\}$.
- Um eine geschlossene Formel für $|F_{n,k}|$ und $S_{n,k}$ zu erhalten, betrachten wir einen zweiten Ansatz, um $F_{n,k}$ abzuzählen. Wir verwenden zwei Techniken:
  - Abzählen des Komplements.
    Problem: Bestimmung von $|X|$ für eine Menge $X \subseteq U$.
    Statt $|X|$ direkt zu berechnen, kann es einfacher sein, $|U|$ und $|X^c|$ für $X^c = U \setminus X$ zu bestimmen und $|X| = |U| - |X^c|$ zu setzen.
  - Prinzip der Inklusion und Exklusion (kurz: Siebformel).
    Eine Formel für die Bestimmung von $|X|$ wenn $X = \bigcup_{i=1}^{n} Y_i$ und die Y_i nicht notwendigerweise disjunkt sind.
- Das Komplement $F_{n,k}^c := [n]^k \setminus F_{n,k}$ besteht genau aus allen k-Tupeln, in denen mindestens ein Element aus $[n]$ nicht vorkommt.
- Offensichtlich gilt $n^k = \left|F_{n,k}^c\right| + \left|F_{n,k}\right|$, es reicht also $\left|F_{n,k}^c\right|$ zu bestimmen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Bestimmung von $\left|F^c_{n,k}\right|$

- Erinnerung: $F^c_{n,k}$ enthält alle k-Tupel, in denen mindestens ein Element aus $[n]$ nicht vorkommt.
- Damit gilt

$$F^c_{n,k} = \bigcup_{x \in [n]} ([n] \setminus \{x\})^k$$

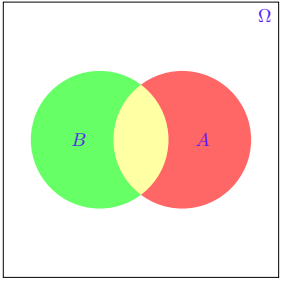
  - $([n] \setminus \{x\})^k$ ist die Menge aller k-Tupel, in denen x nicht vorkommt.
  - Es folgt: Jedes k-Tupel, in dem mindestens ein Element aus $[n]$ nicht vorkommt, gehört zu mindestens einer der Mengen $([n] \setminus \{x\})^k$.

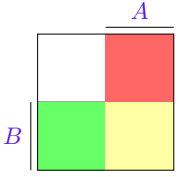
- Problem: Im Allgemeinen gilt nur $\left|F^c_{n,k}\right| \leq \sum_{x \in [n]} |[n] \setminus \{x\}|^k = n \cdot (n-1)^k$, da z.B. alle Tupel $([n] \setminus \{1,2\})^k$ sowohl für $x = 1$ als auch für $x = 2$ mitgezählt werden.
- Die Siebformel erlaubt, die Summation zu korrigieren.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Für die Vereinigung zweier Mengen A, B gilt:

$$|A \cup B| = |A| + |B| - |A \cap B| \,.$$

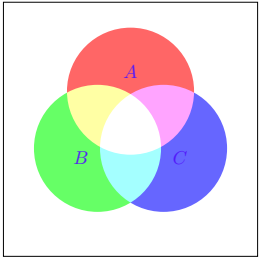


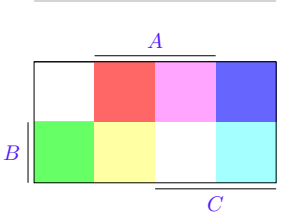


No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Für die Vereinigung dreier Mengen A, B, C ist zu berücksichtigen, dass $|A| + |B| + |C|$ jedes Element in genau zwei der Mengen zweimal zählt, und jedes Element in allen drei Mengen dreimal zählt.
- Damit gilt:

$$\begin{aligned} |A \cup B \cup C| \quad = \quad & |A| + |B| + |C| \\ & - |A \cap B| - |A \cap C| - |B \cap C| \\ & + |A \cap B \cap C| \,. \end{aligned}$$





Die Siebformel

- Für die Vereinigung von vier Mengen A, B, C, D gilt:

$$\begin{aligned} & |A \cup B \cup C \cup D| \\ = \quad & |A| + |B| + |C| + |D| \\ & - |A \cap B| - |A \cap C| - |A \cap D| - |B \cap C| - |B \cap D| - |C \cap D| \\ & + |A \cap B \cap C| + |A \cap B \cap D| + |A \cap C \cap D| + |B \cap C \cap D| \\ & - |A \cap B \cap C \cap D| \,. \end{aligned}$$

- Für $X = \bigcup_{i=1}^{m} Y_i$ gilt die Siebformel:

$$|X| = \sum_{r=1}^{m} (-1)^{r-1} \sum_{I \subseteq [m]\colon |I|=r} \left| \bigcap_{i \in I} Y_i \right|$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Die Siebformel

- Die Siebformel lässt sich mittels Induktion nach m beweisen.
- Ansatz für den Beweis des Induktionsschritts:

$$\bigcup_{i=1}^{m+1} Y_i = Y_{m+1} \cup \underbrace{\bigcup_{i=1}^{m} Y_i}_{=:Z}$$

Dann zuerst die Formel für $m = 2$ auf $Y_{m+1} \cup Z$ anwenden und anschließend die Formel für m auf $\bigcup_{j=1}^{m} Y_j$ und auf $Y_{m+1} \cap Z = \bigcup_{j=1}^{m} (Y_j \cap Y_{m+1})$ anwenden.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Bestimmung von $\left|F_{n,k}^c\right|$ mit Hilfe der Siebformel

- Damit die Siebformel sinnvoll eingesetzt werden kann, müssen die Schnitte $\bigcap_{i\in I} Y_i$ einfach(er) abzuzählen sein.
- Im Fall von $X = F_{n,k}^c$ und $Y_i = ([n] \setminus \{i\})^k$ ist $\bigcap_{i\in I} Y_i$ die Menge aller k-Tupel mit Elementen aus $[n] \setminus I$.
- Es folgt: $\left|\bigcap_{i\in I} Y_i\right| = \left|([n] \setminus I)^k\right| = (n - |I|)^k$.
  Insbesondere, wenn $|I_1| = |I_2|$, dann gilt $\left|\bigcap_{i\in I_1} Y_i\right| = \left|\bigcap_{i\in I_2} Y_i\right|$.
- $[n]$ enthält genau $\binom{n}{r}$ Teilmengen I mit $|I| = r$. Durch Einsetzen in die Siebformel erhält man:

$$\left|F_{n,k}^c\right| = \sum_{r=1}^{n} (-1)^{r+1} \binom{n}{r} (n-r)^k$$

Mittels elementarer Umformungen erhält man hieraus:

$$n! S_{k,n} = |F_{n,k}| = \sum_{i=0}^{n} (-1)^i \binom{n}{i} (n-i)^k = \sum_{i=0}^{n} (-1)^{n-i} \binom{n}{i} i^k$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Anzahl Partitionen bei vorgegebener Klassengröße

- Wir hatten bereits die Stirling-Zahlen 2. Art $S_{n,k}$ kennengelernt:

  $S_{n,k}$ zählt die verschiedenen Partitionen von $[n]$ in genau k Klassen, mit anderen Worten $S_{n,k}$ ist die Anzahl der Äquivalenzrelationen über $[n]$ mit genau k Klassen.

- In manchen Fällen benötigt man jedoch die Anzahl der Äquivalenzrelationen über $[n]$, die genau λ_i viele i-elementige Klassen besitzen.

- Bsp.: Partitionen von $[3]$:

λ_1	λ_2	λ_3	Partitionen
0	0	1	$\{\{1,2,3\}\}$
1	1	0	$\{\{1\},\{2,3\}\}$, $\{\{2\},\{1,3\}\}$, $\{\{3\},\{1,2\}\}$
3	0	0	$\{\{1\},\{2\},\{3\}\}$

- Dabei muss $\sum_{i=1}^{n} i\lambda_i = n$ gelten; insbesondere gibt es genau $|P_{2n,n}|$ mögliche Werte für $(\lambda_1, \ldots, \lambda_n)$ bei festem n (siehe Zahlpartitionen).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Anzahl Partitionen bei vorgegebener Klassengröße

- Seien $n, \lambda_1, \ldots, \lambda_n \in \mathbb{N}_0$ mit $\sum_{i=1}^{n} i\lambda_i = n$ gegeben.
- Wir ordnen jeder Permutation $(s_1, \ldots, s_n)$ von $[n]$ (d.h. $\{s_1, \ldots, s_n\} = [n]$) eine Partition mit genau λ_i vielen i-elementigen Klassen zu, indem wir für anwachsendes i von links nach rechts jeweils λ_i viele i-Tupel abspalten und als Klassen lesen:
- Z.B. für $n = 6$ und $\lambda_1 = 1, \lambda_2 = 1, \lambda_3 = 1$:

$$(1,3,6,2,5,4) \mapsto (1)(3,6)(2,5,4) \mapsto \{\{1\},\{3,6\},\{2,5,4\}\}$$

  während für $\lambda_2 = 1, \lambda_4 = 1$:

$$(1,3,6,2,5,4) \mapsto (1,3)(6,2,5,4) \mapsto \{\{1,3\},\{6,2,5,4\}\}$$

- Formal:
  - Setze $k := 1, \mathcal{P} := \emptyset$
  - Für i von 1 bis n:
    - Für j von 1 bis λ_i: setze $\mathcal{P} := \mathcal{P} \cup \{\{s_k, \ldots, s_{k+i-1}\}\}$ und $k := k + i$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Dabei können sowohl die Einträge der abgetrennten Tupel beliebig umgeordnet werden, als auch die Reihenfolge der Tupel derselben Länge, z.B. für $n = 6$ und $\lambda_1 = 2, \lambda_2 = 2$:

$$(1,3,6,2,5,4) \mapsto (1)(3)(6,2)(5,4) \mapsto \{\{1\},\{3\},\{2,6\},\{4,5\}\}$$
$$(3,1,4,5,2,6) \mapsto (3)(1)(4,5)(2,6) \mapsto \{\{1\},\{3\},\{2,6\},\{4,5\}\}$$
$$(3,4,1,2,5,6) \mapsto (3)(4)(1,2)(5,6) \mapsto \{\{3\},\{4\},\{1,2\},\{5,6\}\}$$

  Jedes der λ_i-vielen i-Tupel kann auf $i!$ Weisen umgeordnet werden;

  die λ_i-vielen i-Tupel können auf $\lambda_i!$ Arten angeordnet werden;

  d.h. je $(1!)^{\lambda_1} \cdots (n!)^{\lambda_n} \cdot \lambda_1! \cdots \lambda_n!$ viele Permutationen werden auf dieselbe Partition abgebildet:

- Damit gibt es genau

$$\frac{n!}{(1!)^{\lambda_1} \cdots (n!)^{\lambda_n} \cdot \lambda_1! \cdots \lambda_n!}$$

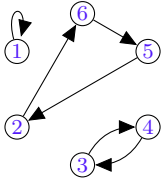
  Partitionen von $[n]$ mit λ_i vielen i-elementigen Klassen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Sei $f\colon [n] \overset{\text{bij}}{\to} [n]$ eine Permutation von $[n]$, z.B.:

$$f(1) = 1,\ f(2) = 6,\ f(3) = 4,\ f(4) = 3,\ f(5) = 2,\ f(6) = 5$$

- Wir können f als Kantenrelation über der Knotenmenge $[n]$ lesen, z.B.



- Dabei fällt auf, dass $([n], f)$ stets in eine Menge disjunkter (einfacher) Kreise zerfällt.

  (Den formalen Beweis hierfür lernen wir später im Kapitel Algebra kennen.)

- Die Kreise entsprechen dabei den „Pfaden“ $x, f(x), f(f(x)), \ldots$, die ein Element $x \in [n]$ durchläuft, wenn man f iterativ auf sich selbst anwendet.

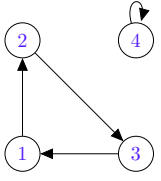
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

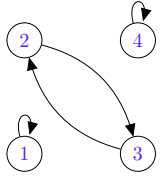
- Diese Beobachtung motiviert die sogenannte Zykelschreibweise für Permutationen:

$$f = (x_0^{(1)} x_1^{(1)} \dots x_{l_1-1}^{(1)})(x_0^{(2)} x_1^{(2)} \dots x_{l_2-1}^{(2)}) \dots (x_0^{(k)} x_1^{(k)} \dots x_{l_k-1}^{(k)})$$

  mit $f(x_j^{(i)}) = x_{(j+1) \bmod l_i}^{(i)}$.

- Die einzelnen Tupel $(x_0^{(i)} x_1^{(i)} \dots x_{l_i-1}^{(i)})$ (Zykel genannt) beschreiben die Kreise, in die $([n], f)$ zerfällt.
- Um das Bild $f(x)$ von x zu finden, wandert man einfach zyklisch um eine Komponente im Zykel weiter.
- Beispiel:

$f = (1\,2\,3)\,(4) =$ 

$g = (1)\,(2\,3)\,(4) =$ 

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

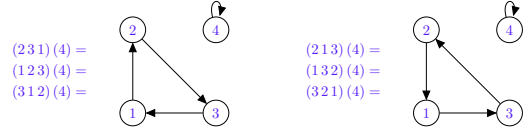
- Bei der Bestimmung der Anzahl der Partitionen von $[n]$ bei vorgegebener Anzahl λ_i der i-elementigen Klassen hatten wir bereits eine der Zykelschreibweise entsprechende Zwischendarstellung verwendet:

$$(1,3,6,2,5,4) \mapsto (1)(3)(6,2)(5,4)$$
$$(3,1,4,5,2,6) \mapsto (3)(1)(4,5)(2,6)$$
$$(3,4,1,2,5,6) \mapsto (3)(4)(1,2)(5,6)$$

- Man muss die dort verwendete Argumentation nur leicht anpassen, um die Anzahl aller Permutationen über $[n]$ zu bestimmen, die in je λ_i viele Zykel der Länge i zerfallen:

$$\frac{n!}{1^{\lambda_1} \cdots n^{\lambda_n} \cdot \lambda_1! \cdots \lambda_n!}$$

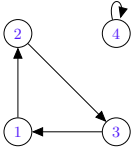
Die Einträge eines Zykels dürfen nur noch zyklisch rotiert, jedoch nicht mehr beliebig vertauscht werden.



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Die Anzahl der Permutationen von $[n]$ mit genau k Zykeln wird als Stirling-Zahl 1. Art $s_{n,k} = \begin{bmatrix} n \\ k \end{bmatrix}$ bezeichnet.
- Eine Rekursionsgleichung für $s_{n,k}$ lässt sich analog zu $S_{n,k}$ herleiten:

  Wieder unterscheidet man nach $n+1$.
  - Sei f eine Permutation von $[n+1]$ in Zykelschreibweise mit k Zykeln.
  - Ist $(n+1)$ ein Zykel von f, d.h. gilt $f(n+1) = n+1$, dann erhält man durch Entfernen von $(n+1)$ eine Permutation von $[n]$ in Zykelschreibweise mit $k-1$ Zykeln.

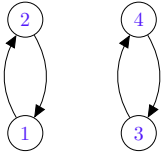
$f = (1\,2\,3)\,(4) =$ 

Jede Permutation von $[n]$ mit $k-1$ Zykeln kann andererseits durch Hinzufügen des Zykels $(n+1)$ zu einer Permutation von $[n+1]$ mit k Zykeln eindeutig erweitert werden.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Ist $n+1$ hingegen Teil eines Zykels der Länge > 1, dann erhält man durch Entfernen von $n+1$ aus diesem Zykel eine Permutation von $[n]$ mit immer noch k Zykeln.

$g = (1\,2)\,(3\,4) =$



Will man eine Permutation f' von $[n]$ mit k Zykeln um $n+1$ erweitern, so dass kein neuer Zykel entsteht, muss man eine der Kanten von $([n], f')$ mittels $n+1$ aufspalten. Da f' eine Funktion ist, hat $([n], f')$ genau n Kanten. D.h. f' lässt sich auf n Arten erweitern.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Damit erhält man:

$$\begin{bmatrix} n+1 \\ k \end{bmatrix} = \begin{bmatrix} n \\ k-1 \end{bmatrix} + n \cdot \begin{bmatrix} n \\ k \end{bmatrix} \text{ mit } \begin{bmatrix} n \\ n \end{bmatrix} = 1, \begin{bmatrix} n+1 \\ 0 \end{bmatrix} = 0$$

- Für die Stirling-Zahlen 2. Art schreibt man auch $S_{n,k} = \begin{Bmatrix} n \\ k \end{Bmatrix}$.

- Zum Vergleich:

$$\begin{Bmatrix} n+1 \\ k \end{Bmatrix} = \begin{Bmatrix} n \\ k-1 \end{Bmatrix} + k \cdot \begin{Bmatrix} n \\ k \end{Bmatrix} \text{ mit } \begin{Bmatrix} n \\ n \end{Bmatrix} = 1, \begin{Bmatrix} n+1 \\ 0 \end{Bmatrix} = 0$$

und

$$\binom{n+1}{k} = \binom{n}{k-1} + \binom{n}{k} \text{ mit } \binom{n}{n} = 1, \binom{n+1}{0} = 1$$

Geordnete Zahlpartitionen

- Wie viele Möglichkeiten gibt es, n Euro unter k Kindern zu verteilen, so dass jedes Kind mindestens 1 Euro erhält?
- Die Euros sind nicht unterscheidbar, die Kinder schon!

$$G_{n,k} := \{(s_1, \ldots, s_k) \in \mathbb{N}^k \mid s_1 + \ldots + s_k = n\}$$

„Zählvektoren mit k positiven Komponenten und Summe n“

- Jedes Tupel $(s_1, \ldots, s_k)$ kann geschrieben werden als

$$n = \overbrace{\underbrace{1+\cdots+1}_{s_1}+\underbrace{1+\cdots+1}_{s_2}+\cdots+\underbrace{1+\cdots+1}_{s_k}}^{n}$$

Damit wird das Tupel eindeutig durch die Plus-Zeichen bestimmt, die die s_i trennen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Die Anzahl der geordneten Partitionen ist damit gleich der Anzahl der Möglichkeiten, $k-1$ Plus-Zeichen aus den insgesamt $n-1$ Plus-Zeichen auszuwählen.

$$|G_{n,k}| = \binom{n-1}{k-1}$$

- Aus
  - Eine $(n-1)$-elementige Menge hat $\binom{n-1}{k-1}$ Teilmengen mit $(k-1)$ Elementen.
  - Eine $(n-1)$-elementige Menge hat 2^{n-1} Teilmengen.

  folgt:

$$\sum_{k=1}^{n} |G_{n,k}| = \sum_{k=1}^{n} \binom{n-1}{k-1} = 2^{n-1}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

(Ungeordnete) Zahlpartitionen

- Wie viele Möglichkeiten gibt es, n Euro in k Päckchen aufzuteilen, so dass jedes Päckchen mindestens 1 Euro enthält?
- Sowohl die Euros als auch die Päckchen sind nicht unterscheidbar!
- Das ist die Anzahl der Zahlpartitionen von n in k positive Summanden

$$P_{n,k} := \{(s_1, \ldots, s_k) \in \mathbb{N}^k \mid s_1 + \ldots + s_k = n, s_1 \leq s_2 \leq \ldots \leq s_k\}$$

- $|P_{n,k}|$ und insbesondere $|P_n| := \sum_{k=1}^{n} |P_{n,k}|$ wurden u.a. von den Mathematikern Hardy und Ramanujan untersucht.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

(Ungeordnete) Zahlpartitionen

$$P_{n,k} = \{(s_1, \ldots, s_k) \in \mathbb{N}^k \mid s_1 + \ldots + s_k = n, s_1 \leq \ldots \leq s_k\}$$

„Partitionen von n“

- Wir leiten für $|P_{n,k}|$ wieder eine Rekursionsgleichung her, indem wir danach unterscheiden, ob $s_1 = 1$ gilt:
  - $\{(s_1, s_2, \ldots, s_k) \in P_{n,k} \mid s_1 = 1\}$ steht in Bijektion mit $P_{n-1,k-1}$ mittels
    $$(1, s_2, \ldots, s_k) \mapsto (s_2, \ldots, s_k)$$
  - $\{(s_1, s_2, \ldots, s_k) \in P_{n,k} \mid s_1 > 1\}$ steht in Bijektion mit $P_{n-k,k}$ mittels
    $$(s_1, \ldots, s_k) \mapsto (s_1 - 1, \ldots, s_k - 1)$$
- Es folgt somit für $0 \leq k, n$

$$\begin{array}{lcll} |P_{n,k}| & = & |P_{n-1,k-1}| + |P_{n-k,k}| & \text{für } n \geq k > 0 \\ |P_{n,0}| & = & 0 & \text{für } n > 0 \\ |P_{n,k}| & = & 0 & \text{für } k > n \\ |P_{n,n}| & = & 1 & \end{array}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

(Ungeordnete) Zahlpartitionen

- Wie viele Möglichkeiten gibt es, n Euro in k Päckchen aufzuteilen, wenn Päckchen auch 0 Euro enthalten dürfen?

$$H_{n,k} = \{(s_1, \ldots, s_k) \in {\mathbb{N}_0}^k \mid s_1 + \ldots + s_k = n, s_1 \leq \ldots \leq s_k\}$$

- $H_{n,k}$ und $P_{n+k,k}$ stehen in Bijektion mittels:

$$H_{n,k} \ni (s_1, \ldots, s_k) \mapsto (s_1 + 1, \ldots, s_k + 1) \in P_{n+k,k}$$

also $|H_{n,k}| = |P_{n+k,k}|$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

4 Kombinatorik

Einführung

Kombinatorische Grundprinzipien

Das Urnenmodell: Ziehungen

Weitere Verteilungsprobleme

Zusammenfassung

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Kombinatorik: Zusammenfassung

Kombinatorische Grundprinzipien:

- $\left|\biguplus_{i\in\mathbb{N}_0} A_i\right| = \sum_{i\in\mathbb{N}_0} |A_i|$
  „Summenregel“
- $\left|\bigcup_{i\in\mathbb{N}_0} A_i\right| \leq \sum_{i\in\mathbb{N}_0} |A_i|$.
  „union bound“
- $|A_1 \times A_2 \times \ldots \times A_n| = \prod_{i\in[n]} |A_i|$
  „Produktregel“
- $\left|\bigcup_{i\in[n]} A_i\right| = \sum_{I\subseteq[n]\colon I\neq\emptyset} (-1)^{|I|+1} \left|\bigcap_{i\in I} A_i\right|$
  „Siebformel“
- $|A| = m\,|B|$, falls es ein $f\colon A \to B$ mit $\left|f^{-1}(b)\right| = m$ konstant gibt.
- Für jedes $f\colon A \to B$ (A, $B \neq \emptyset$ endlich) gibt es ein $b \in B$, so dass $\left|f^{-1}(b)\right| \geq \lceil |A| \,/\, |B| \rceil$.
  „Schubfachprinzip“
- Für jedes $R \subseteq S \times T$ gilt $\sum_{s\in S} |\{t \in T \mid (s,t) \in R\}| = \sum_{t\in T} |\{s \in S \mid (s,t) \in R\}|$.
  „Doppeltes Abzählen“

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Kombinatorik: Zusammenfassung

Urnenmodell:

- $\left|\{(s_1, \ldots, s_k) \in [n]^k\}\right| = n^k$
  „Ziehen mit Zurücklegen, mit Beachtung der Reihenfolge“
- $\left|\{(s_1, \ldots, s_k) \in [n]^k \mid |\{s_1, s_2, \ldots, s_k\}| = k\}\right| = \frac{n!}{(n-k)!} =: n^{\underline{k}}$
  „Ziehen ohne Zurücklegen, mit Beachtung der Reihenfolge“
- $\left|\{(s_1, \ldots, s_k) \in [n]^k \mid s_1 < s_2 < \ldots < s_k\}\right| = \frac{n!}{k!(n-k)!} = \binom{n}{k}$
  „Ziehen ohne Zurücklegen, ohne Beachtung der Reihenfolge“
- $\left|\{(s_1, \ldots, s_k) \in [n]^k \mid s_1 \leq s_2 \leq \ldots \leq s_k\}\right| = \binom{k+n-1}{k}$
  „Ziehen mit Zurücklegen, ohne Beachtung der Reihenfolge“

Stirling-Zahlen:

- $s_{n+1,k} = s_{n,k-1} + n s_{n,k}$, $s_{n,n} = 1$, $s_{n+1,0} = 0$
  „Stirling-Zahlen 1. Art: Anzahl Permutationen von $[n]$ mit k Zykeln.“
- $\left|\{(s_1, \ldots, s_k) \in [n]^k \mid |\{s_1, s_2, \ldots, s_k\}| = n\}\right| = n! S_{k,n}$
  mit $S_{n+1,k} = S_{n,k-1} + k S_{n,k}$, $S_{n,n} = 1$, $S_{n+1,0} = 0$
  „Stirling-Zahlen 2. Art: Anzahl Partitionen von $[n]$ in k Klassen.“

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Zahlpartitionen:

- $\left|\{(s_1, \ldots, s_k) \in \mathbb{N}_0^k \mid s_1 + s_2 + \ldots + s_k = n\}\right| = \binom{n+k-1}{n}$
- $\left|\{(s_1, \ldots, s_k) \in \mathbb{N}_0^k \mid s_1 + s_2 + \ldots + s_k \leq n\}\right| = \binom{n+k}{n}$
- $\left|\{(s_1, \ldots, s_k) \in \mathbb{N}^k \mid s_1 + s_2 + \ldots + s_k = n\}\right| = \binom{n-1}{k-1}$
- $\left|\{(s_1, \ldots, s_k) \in \mathbb{N}^k \mid s_1 + s_2 + \ldots + s_k = n, s_1 \leq s_2 \leq \ldots \leq s_k\}\right| = |P_{n,k}|$
  mit $|P_{n,k}| = |P_{n-1,k-1}| + |P_{n-k,k}|$, $|P_{n,n}| = 1$, $|P_{n+1,0}| = 0$ und $|P_{n,k}| = 0$ falls $k > n$.
  „Anzahl Zahlpartitionen von n in k positive Summanden.“

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

1 Einführung

2 Grundlagen

3 Graphentheorie

4 Kombinatorik

5 Logik

6 Grundlagen der Gruppen- und Zahlentheorie

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

5 Logik

Aussagenlogik

Syntax und Semantik

Gültige, widersprüchliche und erfüllbare Formeln

Semantische Äquivalenzumformungen

Normalformen

Erfüllbarkeitsäquivalente KNF

Operator-Basen

DPLL: Davis-Putnam-Logemann-Loveland

Resolution

Ausblick: Prädikatenlogik

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Aussagenlogik: Formale Einführung

- Die Aussagenlogik verwendet (wie jede Logik) eine künstliche Sprache, definiert durch ihre Syntax und ihre Semantik.
- Die Syntax der Sprache legt durch Regeln fest, welche Zeichenketten wohlgeformte Ausdrücke sind.
- Die wohlgeformten Ausdrücke einer Logik heißen Formeln.
- Die Semantik legt die Bedeutung der Formeln fest.
- Eine formale Semantik ordnet jedem Ausdruck ein mathematisches Objekt zu, welches als Bedeutung des Ausdrucks interpretiert wird.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Eine formale Syntax besteht aus einem Vokabular und einer Menge von Formationsregeln.
- Das Vokabular legt fest, welche Zeichen in Ausdrücken vorkommen dürfen.
- Die Formationsregeln legen fest, welche Zeichenketten über dem Vokabular zulässig oder wohlgeformt sind und welche nicht.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Syntax: Vokabular und Formationsregeln

- Das Vokabular setzt sich aus folgenden Zeichenklassen zusammen:

Wahrheitskonstanten:	true, false
Unendliche Menge V von Aussagenvariablen:	p, q, r, s, t, ...
Logische Operatoren:	$\neg$, $\wedge$, $\vee$, $\rightarrow$
Hilfssymbole:	$($, $)$

- Die Formationsregeln sind:

Regel 0:	true und false sind Formeln.
Regel 1:	Eine Aussagenvariable ist eine Formel.
Regel 2:	Ist F eine Formel, dann ist auch $\neg F$ eine Formel.
Regel 3:	Sind F und G Formeln, dann sind $(F \wedge G)$, $(F \vee G)$ und $(F \rightarrow G)$ ebenfalls Formeln.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Syntax: Formationsregeln

- Die Definitionen kann man kurz schreiben als

$$F ::= \mathsf{true} \mid \mathsf{false} \mid p \mid \neg F \mid (F \vee F) \mid (F \wedge F) \mid (F \to F) \quad \text{mit } p \in V$$

  was man als Konstruktionsvorschrift bzw. rekursives Programm liest.

- In null Schritten (bzw. Rekursionstiefe 0) lassen sich true und false und alle atomaren Formeln $p, q, r, \ldots$ konstruieren.

- In einem Schritt konstruiert man z.B.:

$$\neg p \quad \neg q \quad \neg r \quad (p \wedge q) \quad (q \wedge p) \quad (p \vee q) \quad (q \vee p) \quad \ldots$$

- In zwei Schritten z.B.:

$$\neg\neg p \quad \neg(p \wedge q) \quad ((p \wedge p) \vee r) \quad (\neg p \vee q) \quad \ldots$$

- Weitere Formeln:

$$((p \wedge q) \to r) \quad ((p \wedge (\mathsf{false} \vee q)) \to \neg r) \quad ((\neg p \vee \neg q) \to (q \wedge p))$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

(1)	p	Regel 1
(2)	q	Regel 1
(3)	$(p \wedge q)$	Regel 3a, (1), (2)
(4)	$\neg(p \wedge q)$	Regel 2, (3)
(5)	$\neg q$	Regel 2, (2)
(6)	$(q \vee \neg q)$	Regel 3b, (2), (5)
(7)	$(\neg(p \wedge q) \rightarrow (q \vee \neg q))$	Regel 3c, (4), (6)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Syntax: Syntaxbaum

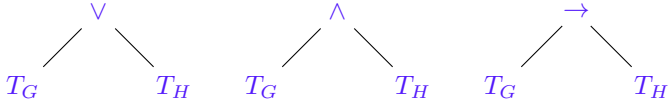
- Die Konstruktion einer Formel stellt man als Syntaxbaum dar.
- Der Syntaxbaum von true, false und einer atomaren Formel p ist einfach:

true false p

- Der Syntaxbaum einer Formel $\neg G$ ergibt sich aus dem Syntaxbaum T_G von G:

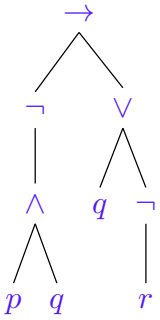


- Der Syntaxbaum einer Formel $F = (G \vee H)$ bzw. $F = (G \wedge H)$ bzw. $F = (G \to H)$ ergibt sich aus den Syntaxbäumen T_G, T_H von G, H:



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Bsp.: Syntaxbaum zu $(\neg(p \wedge q) \to (q \vee \neg r))$



- Strikte Klammerung gibt die Konstruktion vollständig vor.
- Wichtig: Syntaxbäume sind geordnete Wurzelbäume, d.h. man unterscheidet zwischen rechtem und linkem Nachfolger, deren innere Knoten mit logischen Operatoren und deren Blätter mit Variablen oder Konstanten beschriftet sind.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Sind F, G aussagenlogische Formeln, so bezeichnet man G als Teilformel von F, wenn G als Zeichenkette vollständig und zusammenhängend in F vorkommt.

- ▷ Alternativ: Der Syntaxbaum zu G ist ein Teilbaum von F.

Def Die in F vorkommenden Variablen nennt man auch atomare Teilformeln von F.

Def V_F bezeichnet die Menge aller in F vorkommenden Variablen.

- ▷ Beispiel: Alle Teilformeln zu $F = ((p \vee \neg q) \wedge \neg(q \vee \neg\neg r))$

$$p \qquad q \qquad \neg q \qquad (p \vee \neg q)$$

$$r \qquad \neg r \qquad \neg\neg r \qquad (q \vee \neg\neg r) \qquad \neg(q \vee \neg\neg r)$$

$$((p \vee \neg q) \wedge \neg(q \vee \neg\neg r))$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Syntax: Bindungsregeln

- Formeln, die strikt nach der Syntax gebaut sind, können für Menschen schwer zu parsen sein, da sie zu viele Klammern enthalten.
- Bindungsregeln zur Vereinfachung von Klammerausdrücken:

$$\begin{array}{lll} \neg & \text{bindet stärker als} & \wedge \\ \wedge & \text{bindet stärker als} & \vee \\ \vee & \text{bindet stärker als} & \rightarrow \end{array}$$

- Beispiele:
  - $\neg p \wedge q$ steht für $(\neg p \wedge q)$, nicht für $\neg(p \wedge q)$
  - $p \wedge q \rightarrow r$ steht für $((p \wedge q) \rightarrow r)$, nicht für $(p \wedge (q \rightarrow r))$
  - $p \wedge q \vee r \wedge s$ steht für $((p \wedge q) \vee (r \wedge s))$
  - In den Folien: Weder $p \wedge q \vee r \wedge s$ noch $((p \wedge q) \vee (r \wedge s))$, sondern $(p \wedge q) \vee (r \wedge s)$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Aussagenlogik: Semantik (Intuition)

- Was ist die Bedeutung einer Formel?
- Die Formel $F := (p \wedge q)$ kann in vier „Welten“ ausgewertet werden
  - Welt 1: p und q sind beide wahr. F ist wahr.
  - Welt 2: p ist wahr, q ist falsch. F ist falsch.
  - Welt 3: p ist falsch, q ist wahr. F ist falsch.
  - Welt 4: p ist falsch, q ist falsch. F ist falsch.
- Die Bedeutung einer Formel ist die Funktion, die jeder möglichen „Welt“ den Wahrheitswert der Formel in dieser Welt zuordnet:
  - 1 (die Formel ist wahr) oder
  - 0 (die Formel ist falsch).
- Man nennt $\{0, 1\}$ die Booleschen Werte.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Belegungen

Def Eine Belegung ist eine Funktion $\beta\colon V' \to \{0,1\}$ mit $V' \subseteq V$.

Die Belegung $\beta\colon V' \to \{0,1\}$ passt zu einer aussagenlogischen Formel F, falls $V_F \subseteq V'$ (jede in F vorkommende Variable ist in V' enthalten).

Eine zu F passende Belegung $\beta\colon V' \to \{0,1\}$ heißt minimal, falls $V_F = V'$ (V' ist die Menge aller in F vorkommenden Variablen).

Def Sei F eine aussagenlogische Formel, β eine zu F passende Belegung. Der Wahrheitswert $[F](\beta)$ von F unter β ist definiert durch:

$$[F](\beta) := \begin{cases} 1 & \text{falls } F = \mathsf{true} \\ 0 & \text{falls } F = \mathsf{false} \\ \beta(p) & \text{falls } F = p \text{ für } p \in V \\ 1 - [G](\beta) & \text{falls } F = \neg G \\ \max\{[G](\beta), [H](\beta)\} & \text{falls } F = (G \vee H) \\ \min\{[G](\beta), [H](\beta)\} & \text{falls } F = (G \wedge H) \\ \max\{1 - [G](\beta), [H](\beta)\} & \text{falls } F = (G \to H) \end{cases}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Wahrheitstabellen

- Die Definition der Funktion $[F]$ kann mit Hilfe von Wahrheitstabellen (wie zu Beginn) visualisiert werden.
- Wahrheitstabellen der Grundoperatoren Negation ($\neg$), Konjunktion ($\wedge$), Disjunktion ($\vee$) und Implikation ($\rightarrow$):

Negation

G	$\neg G$
1	0
0	1

Konjunktion

G	H	$G \wedge H$
0	0	0
0	1	0
1	0	0
1	1	1

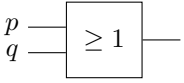
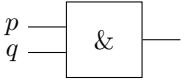
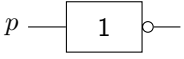
Disjunktion

G	H	$G \vee H$
0	0	0
0	1	1
1	0	1
1	1	1

Implikation

G	H	$G \rightarrow H$
0	0	1
0	1	1
1	0	0
1	1	1

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Operator	Bezeichnung	DS	ERA	Gatterdarstellung
Disjunktion	OR	$p \vee q$	$p + q$	
Konjunktion	AND	$p \wedge q$	$p \cdot q$	
Negation	NOT	$\neg p$	$\overline{p}$	

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Semantik einer Formel

- Die Funktion $[F]$, die eine zu F passende Belegung auf $\{0,1\}$ abbildet, ist die Semantik von F.
- Gilt $[F](\beta) = 1$, so sagt man:
  - F wertet sich unter β zu „wahr“ aus, oder kurz
  - F ist unter β wahr.
- Gilt $[F](\beta) = 0$, so sagt man:
  - F wertet sich unter β zu „falsch“ aus, oder kurz
  - F ist unter β falsch.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Semantik der Implikation

- Die formale Semantik der Implikation ist sehr unterschiedlich von dem Gebrauch von „wenn . . . dann“ in der Alltagssprache.
- $p \to q$ sagt nicht, dass p eine Ursache für q ist.

  Pinguine schwimmen $\to$ Pferde wiehern ist wahr in unserer Welt.
- $p \to q$ sagt nichts darüber, ob p wahr oder falsch ist.

  Friedrich Merz wurde bestochen $\to$ Friedrich Merz muss ins Gefängnis ist wahr in unserer Welt.
- Wenn in einer Welt p falsch ist, dann ist in dieser Welt $p \to q$ wahr, unabhängig davon, was q aussagt.

  Pinguine fliegen $\to$ Pferde sprechen ist wahr in unserer Welt.
- $p \to q$ sagt nichts darüber, ob $\neg p \to \neg q$ wahr ist.

  In der Alltagssprache folgt aus

  Wenn Du brav bist, dann bekommst Du ein Bonbon

  oft implizit

  Wenn Du nicht brav bist, dann bekommst Du kein Bonbon.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Implikation: Wahr oder falsch in unserer Welt?

- $1+1=6 \longrightarrow$ Friedrich Merz ist Bundeskanzler
- Deutschland ist größer als Spanien $\longrightarrow$ Die Sonne ist heute aufgegangen
- Dienstag ist ein Wochentag $\longrightarrow$ Donald Trump ist ein Pinguin
- Der Mond besteht aus grünem Käse $\longrightarrow$ Frankreich hat einen König

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- In der Praxis macht man die Aussagenlogik benutzerfreundlich:
- Üblicherweise gibt man einfach explizit an, welche Zeichen(ketten) als Variablen im Weiteren aufzufassen sind. (Vgl. Variablendeklaration.)
- Zusätzlich erweitert man die Syntax um weitere Operatorsymbole z.B.:
  - Exklusiv-Oder: $\oplus$, XOR.

    Entspricht dem sprachlichen Konstrukt „entweder ... oder“ — genau einer der beiden Operanden muss wahr sein. Die Möglichkeit, dass beide Operanden wahr sind, wird ausgeschlossen.
  - Bikonditional $\leftrightarrow$.

    Entspricht dem sprachlichen Konstrukt „genau dann ... wenn“
  - If-Then-Else-Operator $\mathrm{ITE}(F, G, H)$ (der Arität 3).

    Entspricht dem sprachlichen Konstrukt „wenn F, dann G, sonst H“

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Und erweitert die Semantik entsprechend für diese neuen Operatoren, hier mittels Wahrheitstabellen:

F	G	$F \oplus G$
0	0	0
0	1	1
1	0	1
1	1	0

F	G	$F \leftrightarrow G$
0	0	1
0	1	0
1	0	0
1	1	1

F	G	H	$\mathsf{ITE}(F,G,H)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- Allgemein kann jede Wahrheitstabelle als Definition eines Operators aufgefasst werden.
- Aufgabe: Wie viele mögliche binäre Operatoren gibt es? Und wie viele der Arität n?

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

p	q	r	$((p \vee \neg q) \rightarrow r) \leftrightarrow \neg r$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Berechnung von $[F]$ für eine gegebene Formel F

p	q	r	$((p$	$\vee$	$\neg$	$q)$	$\rightarrow$	$r)$	$\leftrightarrow$	$\neg$	r
0	0	0									
0	0	1									
0	1	0									
0	1	1									
1	0	0									
1	0	1									
1	1	0									
1	1	1									

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Berechnung von $[F]$ für eine gegebene Formel F

p	q	r	$((p$	$\vee$	$\neg$	$q)$	$\rightarrow$	$r)$	$\leftrightarrow$	$\neg$	r
0	0	0	0			0		0			0
0	0	1	0			0		1			1
0	1	0	0			1		0			0
0	1	1	0			1		1			1
1	0	0	1			0		0			0
1	0	1	1			0		1			1
1	1	0	1			1		0			0
1	1	1	1			1		1			1

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

p	q	r	$((p$	$\vee$	$\neg$	$q)$	$\rightarrow$	$r)$	$\leftrightarrow$	$\neg$	r
0	0	0	0		1	0		0		1	0
0	0	1	0		1	0		1		0	1
0	1	0	0		0	1		0		1	0
0	1	1	0		0	1		1		0	1
1	0	0	1		1	0		0		1	0
1	0	1	1		1	0		1		0	1
1	1	0	1		0	1		0		1	0
1	1	1	1		0	1		1		0	1

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

p	q	r	$((p$	$\vee$	$\neg$	$q)$	$\rightarrow$	$r)$	$\leftrightarrow$	$\neg$	r
0	0	0	0	1	1	0		0		1	0
0	0	1	0	1	1	0		1		0	1
0	1	0	0	0	0	1		0		1	0
0	1	1	0	0	0	1		1		0	1
1	0	0	1	1	1	0		0		1	0
1	0	1	1	1	1	0		1		0	1
1	1	0	1	1	0	1		0		1	0
1	1	1	1	1	0	1		1		0	1

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Berechnung von $[F]$ für eine gegebene Formel F

p	q	r	$((p$	$\vee$	$\neg$	$q)$	$\rightarrow$	$r)$	$\leftrightarrow$	$\neg$	r
0	0	0	0	1	1	0	0	0		1	0
0	0	1	0	1	1	0	1	1		0	1
0	1	0	0	0	0	1	1	0		1	0
0	1	1	0	0	0	1	1	1		0	1
1	0	0	1	1	1	0	0	0		1	0
1	0	1	1	1	1	0	1	1		0	1
1	1	0	1	1	0	1	0	0		1	0
1	1	1	1	1	0	1	1	1		0	1

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Berechnung von $[F]$ für eine gegebene Formel F

p	q	r	$((p$	$\vee$	$\neg$	$q)$	$\rightarrow$	$r)$	$\leftrightarrow$	$\neg$	r
0	0	0	0	1	1	0	0	0	0	1	0
0	0	1	0	1	1	0	1	1	0	0	1
0	1	0	0	0	0	1	1	0	1	1	0
0	1	1	0	0	0	1	1	1	0	0	1
1	0	0	1	1	1	0	0	0	0	1	0
1	0	1	1	1	1	0	1	1	0	0	1
1	1	0	1	1	0	1	0	0	0	1	0
1	1	1	1	1	0	1	1	1	0	0	1

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Kurzschlussauswertung

- Werte rechten Teilbaum nur aus, wenn nötig.

Def Kurzschlussauswertung von $\wedge, \vee, \rightarrow$.

G	H	$(G$	$\wedge$	$H)$	$(G$	$\vee$	$H)$	$(G$	$\rightarrow$	$H)$
0	0	0	0	$*$	0	0	0	0	1	$*$
0	1	0	0	$*$	0	1	1	0	1	$*$
1	0	1	0	0	1	1	$*$	1	0	0
1	1	1	1	1	1	1	$*$	1	1	1

Im Fall von $(G \leftrightarrow H)$ bzw. $(G \oplus H)$ müssen immer beide Teilformeln G, H ausgewertet werden.

- Verwendet z.B. in C++ und Java für `&&`, `||`.
- Beispiel:

p	q	$((p$	$\wedge$	$q)$	$\rightarrow$	$(p$	$\vee$	$q))$
0	0	0	0	$*$	1	$*$	$*$	$*$
0	1	0	0	$*$	1	$*$	$*$	$*$
1	0	1	0	0	1	$*$	$*$	$*$
1	1	1	1	1	1	1	1	$*$

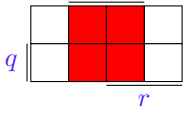
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Berechnung von $[F]$ mit KV-Diagrammen

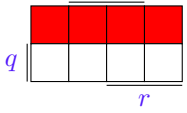
- Sei F eine Formel. Sei B_F die Menge aller minimalen Belegungen $\beta\colon V_F \to \{0,1\}$ von F. Wir nehmen B_F als Universum, d.h. $\overline{X} = B_F \setminus X$.
- Für jede Teilformel G von F sei $\mathit{Erf}_G = \{\beta \in B_F \mid [G](\beta) = 1\}$ (Menge der minimalen Belegungen, die G erfüllen).
- Für jede Teilformel G von F gilt:
  - Wenn $G = G_1 \wedge G_2$ dann $\mathit{Erf}_G = \mathit{Erf}_{G_1} \cap \mathit{Erf}_{G_2}$.
  - Wenn $G = G_1 \vee G_2$ dann $\mathit{Erf}_G = \mathit{Erf}_{G_1} \cup \mathit{Erf}_{G_2}$.
  - Wenn $G = \neg H$ dann $\mathit{Erf}_G = \overline{\mathit{Erf}_H}$.
  - Wenn $G = G_1 \to G_2$ dann $\mathit{Erf}_G = \overline{\mathit{Erf}_{G_1}} \cup \mathit{Erf}_{G_2}$.
  - Wenn $G = G_1 \leftrightarrow G_2$ dann $\mathit{Erf}_G = (\mathit{Erf}_{G_1} \cap \mathit{Erf}_{G_2}) \cup (\overline{\mathit{Erf}_{G_1}} \cap \overline{\mathit{Erf}_{G_2}})$.
- Damit kann die Menge Erf_F mit Hilfe eines KV-Diagramms berechnet werden

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

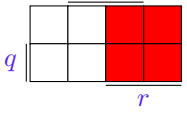
p



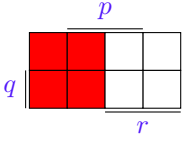
$\neg q$



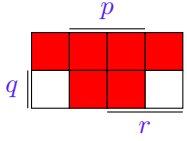
r



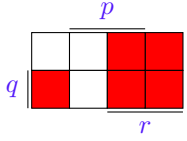
$\neg r$



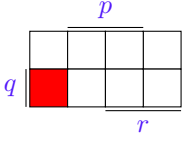
$p \vee \neg q$



$(p \vee \neg q) \rightarrow r$



$((p \vee \neg q) \rightarrow r) \leftrightarrow \neg r$



Gültige Formeln

- Eine Formel F ist (allgemein)gültig, wenn für jede Belegung β, die zu F passt, gilt: $[F](\beta) = 1$.
  - Intuitiv: eine Formel ist gültig, wenn sie überall („in allen Welten“) wahr ist.
  - Wir sagen auch: F ist eine Tautologie.
  - Achtung: „Eine Formel ist gültig“ ist nicht dasselbe wie „Eine Formel ist wahr“.
- Lemma: Eine Formel F ist gültig genau dann, wenn für jede minimale Belegung β, die zu F passt, gilt: $[F](\beta) = 1$.
- Tautologietest: berechne die Wahrheitstabelle und prüfe, ob die Spalte für F ausschließlich Einsen enthält.
- Beispiele von Tautologien:

$$p \vee \neg p \qquad (p \wedge q) \to p \qquad (p \to (q \to r)) \to ((p \to q) \to (p \to r))$$
$$p \to (p \vee q) \qquad p \to (q \to p)$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Widersprüchliche Formeln

- Eine Formel F ist widersprüchlich (oder unerfüllbar), wenn für jede Belegung β, die zu F passt, gilt: $[F](\beta) = 0$.
  - Intuitiv: eine Formel ist widersprüchlich, wenn sie überall („in allen Welten“) falsch ist.
  - Wir sagen auch: F ist ein Widerspruch.
  - Achtung: „Eine Formel ist widersprüchlich“ ist nicht dasselbe wie „Eine Formel ist falsch“.
- Lemma: Eine Formel F ist widersprüchlich genau dann, wenn für jede minimale Belegung β, die zu F passt, gilt: $[F](\beta) = 0$.
- Widerspruchstest: berechne die Wahrheitstabelle und prüfe, ob die Spalte für F ausschließlich Nullen enthält.
- Beispiele von Widersprüchen:

$$p \wedge \neg p \qquad p \wedge (p \to \neg p) \qquad p \leftrightarrow \neg p$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Erfüllbare Formeln

- Eine Formel F ist erfüllbar, wenn es eine Belegung β gibt, die zu F passt, und $[F](\beta) = 1$ erfüllt.
  - Intuitiv: eine Formel ist erfüllbar, wenn sie in mindestens einer Welt wahr ist.
- Erfüllbarkeitstest: berechne die Wahrheitstabelle und prüfe, ob die Spalte für F mindestens eine Eins enthält.
- Beispiele von erfüllbaren, aber nicht gültigen Formeln:

$$p \qquad p \to q \qquad p \to \neg p$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Aufgabe

	gültig	erfüllbar	unerfüllbar
p			
$p \vee q$			
$p \vee \neg p$			
$p \wedge \neg p$			
$p \rightarrow \neg p$			
$p \rightarrow q$			
$p \rightarrow (q \rightarrow p)$			
$p \rightarrow (p \rightarrow q)$			

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

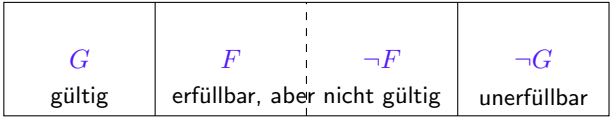
Aufgabe

	gültig	erfüllbar	unerfüllbar
p		$\times$	
$p \vee q$		$\times$	
$p \vee \neg p$	$\times$	$\times$	
$p \wedge \neg p$			$\times$
$p \rightarrow \neg p$		$\times$	
$p \rightarrow q$		$\times$	
$p \rightarrow (q \rightarrow p)$	$\times$	$\times$	
$p \rightarrow (p \rightarrow q)$		$\times$	

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gültige, erfüllbare, und unerfüllbare Formeln

- Aussagenlogische Formeln teilen sich auf in gültige, unerfüllbare und erfüllbare, aber nicht gültige Formeln.



- F gültig **gdw** $\neg F$ unerfüllbar
  - bzw. F unerfüllbar **gdw** $\neg F$ gültig
- F nicht gültig **gdw** $\neg F$ erfüllbar
  - bzw. F erfüllbar **gdw** $\neg F$ nicht gültig
- F nicht gültig, aber erfüllbar **gdw** $\neg F$ nicht gültig, aber erfüllbar

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Das Erfüllbarkeitsproblem (SAT)

Def Das Erfüllbarkeitsproblem (satisfiability problem, SAT) ist eines der am meisten untersuchten Probleme der Informatik:

Gegeben: *aussagenlogische Formel* F
Frage: *Ist* F *erfüllbar? Finde ggf. erfüllende Belegung.*

- SAT ist NP-vollständig:
  - NP: „Probleme, für die man effizient eine Lösung raten und überprüfen kann“
  - Einfach: Rate minimale F-Belegung β und überprüfe, ob $[F](\beta) = 1$.

    Offen: Wie berechnet man i.A. schnell eine erfüllende F-Belegung?
  - vollständig: Alle Probleme in NP lassen sich effizient als SAT-Instanzen darstellen,

    z.B.: Hamiltonkreise, Graphfärbbarkeit

    (Hamiltonkreise, Graphfärbbarkeit sind selbst NP-vollständig.)
- Trotzdem enorme Fortschritte im Bereich der SAT-Solver seit 2000.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Problem: Gegeben ein Sudoku S, konstruiere eine Formel F_S, die folgende (noch nicht formal definierte) Eigenschaft erfüllt:
  Eine minimale, zu F_S passende Belegung macht F_S wahr, genau dann, wenn sie der Lösung des Sudokus entspricht.

4				9				2
		1				5		
	9		3	4	5		1	
		8				2	5	
7		5		3		4	6	1
	4	6				9		8
	6		1	5	9		8	
		9				6		
5				7				4

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Aussagenlogische Variablen: $\{X_{Z,S} \mid X, Z, S \in [9]\}$
- Bedeutung von $X_{Z,S}$ („intended meaning“):

  Auf dem Feld Z, S steht die Zahl X.
- Anforderungen: Jede Zahl kommt in jeder Zeile vor.
  Jede Zahl kommt in jeder Spalte vor.
  Jede Zahl kommt in jedem Quadrat vor.
  Jedes Feld enthält höchstens eine Zahl (!)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Jede Zahl kommt in jeder Zeile vor
  - Formel für „Zahl X kommt in Zeile Z vor“:

$$(X_{Z,1} \vee X_{Z,2} \vee \cdots \vee X_{Z,9})$$

  - Formel für „Zahl X kommt in jeder Zeile vor“:

$$\bigwedge_{Z=1}^{9} (X_{Z,1} \vee X_{Z,2} \vee \cdots \vee X_{Z,9})$$

  - Formel für „Jede Zahl kommt in jeder Zeile vor“:

$$\bigwedge_{X=1}^{9} \bigwedge_{Z=1}^{9} (X_{Z,1} \vee X_{Z,2} \vee \cdots \vee X_{Z,9})$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Jede Zahl kommt in jeder Spalte vor.

$$\bigwedge_{X=1}^{9} \bigwedge_{S=1}^{9} (X_{1,S} \vee X_{2,S} \vee \cdots \vee X_{9,S})$$

- Jede Zahl kommt in jedem Quadrat vor.

$$\bigwedge_{X=1}^{9} \Bigg(\begin{array}{l} \left(\bigvee_{Z=1}^{3} \bigvee_{S=1}^{3} X_{Z,S} \right) \wedge \left(\bigvee_{Z=1}^{3} \bigvee_{S=4}^{6} X_{Z,S} \right) \wedge \left(\bigvee_{Z=1}^{3} \bigvee_{S=7}^{9} X_{Z,S} \right) \wedge \\ \left(\bigvee_{Z=4}^{6} \bigvee_{S=1}^{3} X_{Z,S} \right) \wedge \left(\bigvee_{Z=4}^{6} \bigvee_{S=4}^{6} X_{Z,S} \right) \wedge \left(\bigvee_{Z=4}^{6} \bigvee_{S=7}^{9} X_{Z,S} \right) \wedge \\ \left(\bigvee_{Z=7}^{9} \bigvee_{S=1}^{3} X_{Z,S} \right) \wedge \left(\bigvee_{Z=7}^{9} \bigvee_{S=4}^{6} X_{Z,S} \right) \wedge \left(\bigvee_{Z=7}^{9} \bigvee_{S=7}^{9} X_{Z,S} \right) \end{array} \Bigg)$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Jedes Feld enthält höchstens eine Zahl
  - Formel für „Feld Z, S enthält nicht X und Y“:

$$\neg(X_{Z,S} \wedge Y_{Z,S})$$

  - Formel für „Feld Z, S enthält höchstens eine Zahl“:

$$\bigwedge_{X=1}^{9} \bigwedge_{\substack{Y=1 \\ Y \neq X}}^{9} \neg(X_{Z,S} \wedge Y_{Z,S})$$

  - Formel für „Jedes Feld enthält höchstens eine Zahl “:

$$\bigwedge_{Z=1}^{9} \bigwedge_{S=1}^{9} \bigwedge_{X=1}^{9} \bigwedge_{\substack{Y=1 \\ Y \neq X}}^{9} \neg(X_{Z,S} \wedge Y_{Z,S})$$

- Formel für die Anfangskonfiguration:

$$4_{1,1} \wedge 9_{1,5} \wedge \ldots \wedge 4_{9,9}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Modellierung des Sudoku-Problems mit SAT

- Sei F_S die Formel, die aus der Konjunktion der Formeln für die Anforderungen und für die Anfangskonfiguration entsteht.
- In F_S kommen 729 (9^3) Variablen vor.
- Eine Belegung β dieser Variablen entspricht einer Lösung des Sudokus genau dann, wenn $[F_S](\beta) = 1$.
- Wenn das Sudoku wohlformuliert ist, dann hat F_S genau eine erfüllende Belegung.
- SAT-Solver können eine erfüllende Belegung (und damit Lösung) finden.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Hamiltonkreisproblem in SAT

- Hamiltonkreis: einfacher Kreis $v_0, \ldots, v_{n-1}, v_n$, der alle Knoten besucht.
- Für jedes konkrete Wertepaar v, i bezeichne die aussagenlogische Variable $A_{v,i}$ den Wahrheitswert der Aussage „Zum Zeitpunkt i bei Knoten v“.
- Wir dürfen $V = [n]$ und $E \subseteq [n] \times [n]$ annehmen.

  1. $\bigwedge_{i \in \{0,1,\ldots,n\}} \bigwedge_{u<v \in V} (\neg A_{u,i} \vee \neg A_{v,i})$
  2. $\bigwedge_{i \in \{0,1,\ldots,n-1\}} \bigwedge_{u \in V} \left(\neg A_{u,i} \vee \bigvee_{w \in uE} A_{w,i+1}\right)$
  3. $\bigwedge_{v \in V} \bigvee_{i \in \{0,1,\ldots,n-1\}} A_{v,i}$
  4. $A_{1,0} \wedge A_{1,n}$

  Die eigentliche Formel erhält man erst nach Expansion für das gegebene n. Z.B. wird aus 3 für $n = 3$ die eigentliche Formel:

  $$(A_{1,0} \vee A_{1,1} \vee A_{1,2}) \wedge (A_{2,0} \vee A_{2,1} \vee A_{2,2}) \wedge (A_{3,0} \vee A_{3,1} \vee A_{3,2})$$

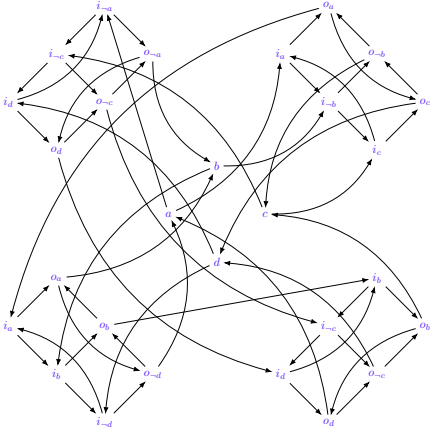
  (Üblicherweise überlässt man die Schreibarbeit dem Computer.)

- ⚠ Falls $|V| = n$, dann $n(n+1)$ aussagenlogische Variablen, d.h. für $n = 16$ bereits 2^{272} minimale Belegungen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Ausblick: SAT als graphtheoretisches Problem (*)

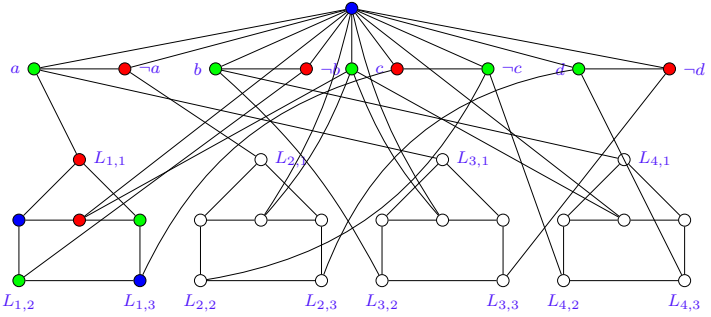
- Mittels SAT kann ein Hamiltonkreis berechnet werden (falls es einen gibt), aber mittels Hamiltonkreisproblem kann auch eine SAT-Instanz gelöst werden:



Die „Satelliten“ kodieren eine Formel $F = \bigwedge_{i=1}^{4}(L_{i,1} \vee L_{i,2} \vee L_{i,3})$.

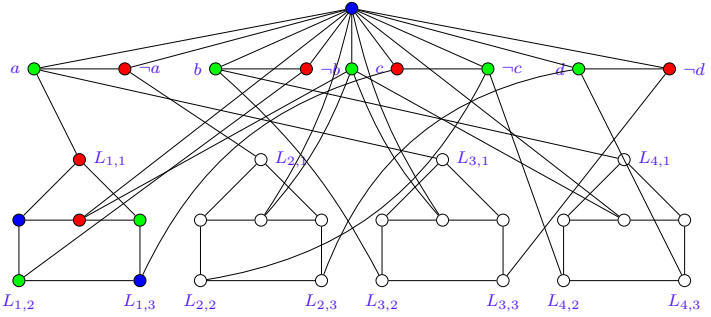
Eine erf. Belegung übersetzt sich in einen Hamiltonkreis und umgekehrt.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science



- Gegeben ein Graph $G = (V, E)$: Übersetzen der Frage „$\chi(G) \leq k$?“ in Aussagenlogik (Übung):
  - Für jeden Knoten $v \in V$ und jede Farbe $i \in [k]$ eine Variable $C_{v,i}$.
  - Bedingungen:

    (a) Jeder Knoten mind. eine Farbe,
    (b) jeder Knoten max. eine Farbe, und
    (c) jede Kante verbindet stets Knoten unterschiedlicher Farbe.



- Rückrichtung geht auch: jede aussagenlogische Formel lässt sich als Graph kodieren, so dass eine 3-Färbung einer erfüllenden Belegung entspricht.
  - Gegeben eine Formel in 3-KNF, kodiere jede Klausel $(L_{i,1} \vee L_{i,2} \vee L_{i,3})$ als ein „Haus“.
  - Zusätzliche Knoten und Kanten erzwingen, dass jede Färbung eine Belegung kodiert.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Logische Äquivalenz

- Zwei Formeln F und G sind logisch äquivalent (symbolisch: $F \equiv G$) genau dann, wenn für jede Belegung β, die zu F und zu G passt, gilt: $[F](\beta) = [G](\beta)$.
  - Intuitiv: wenn F und G äquivalent sind, dann sind sie zwei verschiedene Schreibweisen derselben Aussage.
- Lemma: Sei $V_{F,G}$ die Menge der Variablen, die in F oder G vorkommen. F und G sind äquivalent genau dann, wenn für jede Belegung $\beta\colon V_{F,G} \to \{0,1\}$ gilt: $[F](\beta) = [G](\beta)$.
- Äquivalenztest: Konstruiere die Wahrheitstabelle von $F \leftrightarrow G$ und prüfe, ob die Spalte für $F \leftrightarrow G$ ausschließlich Einsen enthält.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

F	G	Gilt $F \equiv G$?
$p \wedge (p \vee q)$	p	
$\neg(p \vee q) \wedge (\neg p \vee \neg q)$	p	
$p \wedge (q \vee r)$	$(p \wedge q) \vee r$	
$p \wedge (q \vee r)$	$(p \wedge q) \vee (p \wedge r)$	

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

F	G	Gilt $F \equiv G$?
$p \wedge (p \vee q)$	p	Ja
$\neg(p \vee q) \wedge (\neg p \vee \neg q)$	p	Nein
$p \wedge (q \vee r)$	$(p \wedge q) \vee r$	Nein
$p \wedge (q \vee r)$	$(p \wedge q) \vee (p \wedge r)$	Ja

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Eine logische Inferenz ist eine Formel der Gestalt $F \to G$. Dabei ist F die Annahme und G die Konklusion.
- Eine logische Inferenz ist korrekt, wenn sie gültig ist.
- Notation: $F \models G$ (in Worten: G folgt aus F) bezeichnet, dass $F \to G$ gültig ist.
- Oft schreibt man $F_1, \ldots, F_n \models G$ statt $F_1 \wedge \cdots \wedge F_n \models G$.
- Achtung! In der Alltagssprache haben „G folgt aus F“ und „F impliziert G“ oft dieselbe Bedeutung. In dieser Vorlesung
  - „F impliziert G“ bezeichnet die Formel $F \to G$.
  - „G folgt aus F“ bezeichnet, dass die Formel $F \to G$ gültig ist.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

F	G	$F \models G$?
p	$p \vee q$	
p	$p \wedge q$	
p, q	$p \vee q$	
p, q	$p \wedge q$	
$p \wedge q$	p	
$p \vee q$	p	
$p \wedge (p \to q)$	q	

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

F	G	$F \models G$?
p	$p \vee q$	Ja
p	$p \wedge q$	Nein
p, q	$p \vee q$	Ja
p, q	$p \wedge q$	Ja
$p \wedge q$	p	Ja
$p \vee q$	p	Nein
$p \wedge (p \to q)$	q	Ja

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Einige Beziehungen

- F ist gültig (eine Tautologie) genau dann, wenn
  - $\neg F$ ist ein Widerspruch.
  - $F \equiv \mathsf{true}$
  - $\mathsf{true} \models F$ (Notation: $\models F$)
- F ist ein Widerspruch genau dann, wenn
  - $\neg F$ ist gültig.
  - $F \equiv \mathsf{false}$
  - $F \models \mathsf{false}$
- $F \equiv G$ gilt, genau dann, wenn $F \leftrightarrow G$ eine Tautologie ist.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Äquivalenzumformungen

- Bereits bekannt:

  Umformen von arithmetischen oder mengentheoretischen Termen (Ausdrücken) unter Verwendung von

  Assoziativität, Kommutativität, Distributivität, ...

- Beispiel: Für $x, y \in \mathbb{R}$:

$$(x^3 + x^2y + xy^2 + y^3)(x - y) = (x^4 - y^4)$$

- („Schul“)Beweis:

$$\begin{aligned} & (x^3 + x^2y + xy^2 + y^3)(x - y) \\ = \; & x^4 + x^3y + x^2y^2 + xy^3 - x^3y - x^2y^2 - xy^3 - y^4 \\ = \; & x^4 - y^4 \end{aligned}$$

- Verwendet ein Distributivgesetz: für beliebige Ausdrücke e, f, g gilt $e(f - g) = ef - eg$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Beispiel: Für $x, y \in \mathbb{R}$

$$(x^3 + x^2y + xy^2 + y^3)(x - y) = (x^4 - y^4)$$

- Wenn $=$ nur für dieselbe Zeichenkette verwendet werden dürfte, dann wäre die Behauptung falsch.

  Die Bedeutung von $=$ ist jedoch:

  Egal mit welchen reellen Zahlen man x und y ersetzt (belegt), die so gewonnenen arithmetischen Ausdrücke werten sich (unter der üblichen Interpretation von $+$ und $\cdot$) zu derselben reellen Zahl aus.

- Vergleiche mit $F \equiv G$ für aussagenlogische Formeln F, G:

  Unter jeder zu F und G passenden Belegung β werten sich beide Formeln zum selben Wahrheitswert aus, kurz $[F](\beta) = [G](\beta)$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

▷ **Standardäquivalenzen I**:

Für beliebige aussagenlogische Formeln F, G, H gilt:

$$
\begin{array}{lcll}
(F \wedge F) & \equiv & F & \text{(Idempotenz)} \\
(F \vee F) & \equiv & F & \\
(F \wedge G) & \equiv & (G \wedge F) & \text{(Kommutativität)} \\
(F \vee G) & \equiv & (G \vee F) & \\
((F \wedge G) \wedge H) & \equiv & (F \wedge (G \wedge H)) & \text{(Assoziativität)} \\
((F \vee G) \vee H) & \equiv & (F \vee (G \vee H)) & \\
(F \wedge (F \vee G)) & \equiv & F & \text{(Absorption)} \\
(F \vee (F \wedge G)) & \equiv & F & \\
(F \wedge (G \vee H)) & \equiv & ((F \wedge G) \vee (F \wedge H)) & \text{(Distributivität)} \\
(F \vee (G \wedge H)) & \equiv & ((F \vee G) \wedge (F \vee H)) &
\end{array}
$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

▷ **Standardäquivalenzen II**:

Für beliebige aussagenlogische Formeln F, G, H gilt:

$\neg\neg F$	$\equiv$	F	(Doppelnegation)
$\neg(F \wedge G)$	$\equiv$	$(\neg F \vee \neg G)$	(De Morgan)
$\neg(F \vee G)$	$\equiv$	$(\neg F \wedge \neg G)$	
$(F \wedge \neg F)$	$\equiv$	false	(Triviale Kontradiktion)
$(F \vee \neg F)$	$\equiv$	true	(Triviale Tautologie)
$(F \wedge \text{false})$	$\equiv$	false	(Dominanz)
$(F \vee \text{true})$	$\equiv$	true	
$(F \wedge \text{true})$	$\equiv$	F	(Identität)
$(F \vee \text{false})$	$\equiv$	F	

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- **Standardäquivalenzen III**:

  Für beliebige aussagenlogische Formeln F, G, H gilt:

$$
\begin{array}{lcll}
(F \to G) & \equiv & (\neg F \vee G) & \text{(Implikation)} \\
(F \leftrightarrow G) & \equiv & ((F \to G) \wedge (G \to F)) & \text{(Bikonditional)} \\
 & \equiv & \neg(F \oplus G) & \\
(F \oplus G) & \equiv & ((F \vee G) \wedge (\neg F \vee \neg G)) & \text{(XOR)} \\
 & \equiv & ((F \wedge \neg G) \vee (\neg F \wedge G)) &
\end{array}
$$

- Bemerkung: Häufig verwendet man diese Äquivalenzen auch, um direkt Implikation, Bikonditional und XOR zu definieren.
- Die Äquivalenz für die Implikation ist häufig nützlich!

- Beachte: Die Äquivalenzen gelten für beliebige aussagenlogische Formeln.
- Vergleiche: Distributivitätsgesetz auf $\mathbb{R}$

$$x(y+z) = xy + xz$$

gilt natürlich auch noch, wenn man $x := (z+5)^2, y := (u \cdot v), z := u+3$ durch beliebige arithmetische Ausdrücke ersetzt:

$$(z+5)^2((u \cdot v) + (u+3)) = (z+5)^2(u \cdot v) + (z+5)^2(u+3)$$

- Ganz analog für aussagenlogische Formeln.

„Instanziieren“ von $(F \vee F) \equiv F$ mit $F := (p \wedge (q \to \neg r))$:

$$(\underbrace{(p \wedge (q \to \neg r))}_{=:F} \vee \underbrace{(p \wedge (q \to \neg r))}_{=:F}) \equiv \underbrace{(p \wedge (q \to \neg r))}_{=:F}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Nach Behauptung muss z.B. $(F \wedge (F \vee G)) \equiv F$ für beliebige Formeln F, G gelten.
- Also auch für Instanz mit $F := p$ und $G := q$, d.h. für $(p \wedge (p \vee q)) \equiv p$
- Überprüfung mittels Wahrheitstabelle (oder KV-Diagramm):

p	q	$(p$	$\wedge$	$(p$	$\vee$	$q))$
0	0		0		0	
0	1		0		1	
1	0		1		1	
1	1		1		1	

- Alle Standardäquivalenzen lassen sich mittels Wahrheitstabellen sehr einfach überprüfen, da maximal drei aussagenlogische Variablen.
- Warum reicht es, die Äquivalenzen für atomare Formeln $F := p$, $G := q$, $H := r$ zu überprüfen?
- Deckt bereits alle möglichen Kombinationen von Wahrheitswerten $[F](\beta), [G](\beta), [H](\beta)$ unter jeder passenden Belegung β ab – egal welche Formeln F, G, H konkret sind.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Äquivalenzumformungen

- Äquivalenzumformung: Ersetzen einer Teilformel F' von F durch eine zu F' äquivalente Formel G', um eine neue Formel G zu gewinnen.
- Notation: $G = F[F'/G']$.
- Äquivalenzumformungen macht man bei arithmetischen Ausdrücken ganz automatisch. Für den Beweis von

$$(x + (y \cdot (z - z))) = x$$

geht man folgendermaßen vor:

1. Ersetze $(z - z)$ durch 0, da $(z - z) \equiv_{\mathbb{R}} 0$:

$$(x + (y \cdot (z - z))) \equiv_{\mathbb{R}} (x + (y \cdot 0))$$

2. Ersetze $(y \cdot 0)$ durch 0, da $(y \cdot 0) \equiv_{\mathbb{R}} 0$:

$$(x + (y \cdot 0)) \equiv_{\mathbb{R}} (x + 0)$$

3. Ersetze $(x + 0)$ durch x, da $(x + 0) \equiv_{\mathbb{R}} x$:

$$(x + 0) \equiv_{\mathbb{R}} x$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Äquivalenzumformungen

- Dass man semantisch äquivalente (Teil-)Formeln rein syntaktisch gegeneinander austauschen darf, bezeichnet man in der Aussagenlogik als

  Ersetzbarkeitstheorem: Seien F, G, H aussagenlogische Formeln mit $F \equiv G$ und F eine Teilformel von H.

  Sei H' eine aussagenlogische Formel, welche aus H entsteht, indem man ein beliebiges Vorkommen von F in H durch G ersetzt.

  Dann gilt $H \equiv H'$.

- $\equiv$ ist eine Kongruenz auf den aussagenlogischen Formeln: eine Äquivalenzrelation, für die das Ersetzbarkeitstheorem gilt.

- Das Ersetzbarkeitstheorem erlaubt Nebenrechnungen (wie bei arithmetischen Ausdrücken).
  - Vereinfache in Nebenrechnung erst eine Teilformel mittels Äquivalenzumformungen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Äquivalenzumformungen: Beispiel

$$
\begin{aligned}
& (p \wedge (p \to q)) \\
\equiv\ & (p \wedge (\neg p \vee q)) && \text{mit } \underline{\qquad\qquad} \\
\equiv\ & ((p \wedge \neg p) \vee (p \wedge q)) && \text{mit } \underline{\qquad\qquad} \\
\equiv\ & (\mathsf{false} \vee (p \wedge q)) && \text{mit } \underline{\qquad\qquad} \\
\equiv\ & ((p \wedge q) \vee \mathsf{false}) && \text{mit } \underline{\qquad\qquad} \\
\equiv\ & (p \wedge q) && \text{mit } \underline{\qquad\qquad}
\end{aligned}
$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Äquivalenzumformungen: Noch ein Beispiel

$$
\begin{aligned}
& ((p \vee q) \wedge (\neg p \vee r) \wedge (s \vee \neg s)) \rightarrow (q \vee r) \\
\equiv\ & ((p \vee q) \wedge (\neg p \vee r) \wedge \mathsf{true}) \rightarrow (q \vee r) && \text{mit } F \vee \neg F \equiv \mathsf{true} \\
\equiv\ & ((p \vee q) \wedge (\neg p \vee r)) \rightarrow (q \vee r) && \text{mit } F \wedge \mathsf{true} \equiv F \\
\equiv\ & \neg((p \vee q) \wedge (\neg p \vee r)) \vee (q \vee r) && \text{mit } F \rightarrow G \equiv \neg F \vee G \\
\equiv\ & (\neg(p \vee q) \vee \neg(\neg p \vee r)) \vee (q \vee r) && \text{mit } \neg(F \wedge G) \equiv \neg F \vee \neg G \\
\equiv\ & (\neg p \wedge \neg q) \vee (\neg\neg p \wedge \neg r) \vee q \vee r && \text{mit } \neg(F \vee G) \equiv \neg F \wedge \neg G \\
\equiv\ & (\neg p \wedge \neg q) \vee (p \wedge \neg r) \vee q \vee r && \text{mit } \neg\neg F \equiv F \\
\equiv\ & (\neg p \wedge \neg q) \vee q \vee (p \wedge \neg r) \vee r && \text{mit } F \vee G \equiv G \vee F \\
\equiv\ & ((\neg p \vee q) \wedge (\neg q \vee q)) \vee ((p \vee r) \wedge (\neg r \vee r)) && \text{mit } (F \wedge G) \vee H \equiv (F \vee H) \wedge (G \vee H) \\
\equiv\ & ((\neg p \vee q) \wedge \mathsf{true}) \vee ((p \vee r) \wedge \mathsf{true}) && \text{mit } \neg F \vee F \equiv \mathsf{true} \\
\equiv\ & \neg p \vee q \vee p \vee r && \text{mit } F \wedge \mathsf{true} \equiv F \\
\equiv\ & \neg p \vee p \vee q \vee r && \text{mit } F \vee G \equiv G \vee F \\
\equiv\ & \mathsf{true} \vee q \vee r && \text{mit } \neg F \vee F \equiv \mathsf{true} \\
\equiv\ & \mathsf{true} && \text{mit } \mathsf{true} \vee F \equiv \mathsf{true}
\end{aligned}
$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Äquivalenzumformungen

- Wann zeigt man $F \equiv G$ mittels Wahrheitstabellen, wann mittels Äquivalenzumformungen?
- Wahrheitstabellen für Menschen bis maximal 5 Variablen noch praktikabel.
- Mit wie vielen Variablen können Computer umgehen?
- Für $|V_F| = n$ muss man 2^n Belegungen auswerten.
- Annahme: 1ms zum Auswerten von F pro Belegung (Länge egal).
  - In einer Minute $6 \cdot 10^4$ Belegungen; maximal 15 Variablen.
  - An einem Tag $86,4 \cdot 10^6$ Belegungen; maximal 26 Variablen.
  - In einem Jahr $31,536 \cdot 10^9$ Belegungen; maximal 34 Variablen.
- Äquivalenzumformungen daher ab spätestens 5 Variablen meist der bessere Ansatz, um Äquivalenzen zu zeigen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Sei F eine aussagenlogische Formel mit Variablen V_F.

- Die Menge der Literale (bzgl. F) ist $\{p, \neg p \mid p \in V_F\}$.
  (D.h. ein Literal ist entweder eine Variable $p \in V_F$ oder ihre Negation $\neg p$.)
- F ist in Negationsnormalform (NNF) **:gdw**
  1. nur die Junktoren $\neg, \wedge, \vee$ in F verwendet werden, und
  2. $\neg$ ausschließlich auf Variablen angewendet wird.
- F ist in disjunktiver Normalform (DNF) **:gdw** F folgende Struktur hat:

$$\bigvee_{i=1}^{m} \left(\bigwedge_{j=1}^{m_i} L_{i,j} \right) \qquad \text{mit } L_{i,j} \in \{p, \neg p \mid p \in V_F\}$$

- F ist in konjunktiver Normalform (KNF) **:gdw** F folgende Struktur hat:

$$\bigwedge_{i=1}^{m} \left(\bigvee_{j=1}^{m_i} L_{i,j} \right) \qquad \text{mit } L_{i,j} \in \{p, \neg p \mid p \in V_F\}$$

- Weiterhin werden auch false, true als Formeln in KNF, DNF, NNF aufgefasst.

⚠ Nur für Literale definiert man $\overline{p} := \neg p$ und $\overline{\neg p} := p$, so dass „$\neg \bigvee \bigwedge L \equiv \bigwedge \bigvee \overline{L}$“.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

KNF und DNF

Bsp Eine Formel in DNF: $(p \wedge \neg q) \vee (\neg p \wedge \neg q \wedge r) \vee q$.

Bsp Eine Formel in KNF: $(p \vee \neg q) \wedge (\neg p \vee \neg q \vee r) \wedge q$.

- Eine DNF-Formel ist erfüllbar **gdw** mindestens ein Disjunkt erfüllbar ist.

  Führt zu einem effizienten Algorithmus für Erfüllbarkeit von DNF-Formeln.

- Eine KNF-Formel ist gültig **gdw** alle Konjunkte gültig sind.

  Führt zu einem effizienten Algorithmus für Gültigkeit von KNF-Formeln.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

DNF

Thm Für jede Formel F gibt es eine Formel G in DNF mit $F \equiv G$.

Beweis: Folgendes Verfahren berechnet die Formel G:

1. $\rightarrow, \leftrightarrow, \oplus$ „auflösen“, d.h. mittels Äquivalenzen bzw. Definitionen durch $\neg, \wedge, \vee$ ausdrücken.
2. $\neg$ mittels De Morgan und Doppelnegation im Syntaxbaum soweit nach unten schieben, bis jedes $\neg$ nur noch an eine Variable bindet.
3. Mittels Distributivität von $\wedge$ über $\vee$

   $(F \wedge (G \vee H)) \equiv ((F \wedge G) \vee (F \wedge H))$ und $((G \vee H) \wedge F) \equiv ((G \wedge F) \vee (H \wedge F))$

   Formeln „ausmultiplizieren“, bis DNF erreicht.

   („$\vee$“ wandert im Syntaxbaum nach oben, „$\wedge$“ nach unten.)

Liefert Formeln $D_F \equiv F$.

DNF Beispiele

▷ $F = ((p \to q) \to r)$

$$
\begin{aligned}
& ((p \to q) \to r) \\
\equiv\ & ((\neg p \vee q) \to r) \\
\equiv\ & (\neg(\neg p \vee q) \vee r) \\
\equiv\ & ((\neg\neg p \wedge \neg q) \vee r) \\
\equiv\ & ((p \wedge \neg q) \vee r) \\
=:\ & D_F
\end{aligned}
$$

▷ $G := (p \to (q \to r))$

$$
\begin{aligned}
& (p \to (q \to r)) \\
\equiv\ & (p \to (\neg q \vee r)) \\
\equiv\ & (\neg p \vee (\neg q \vee r)) \\
\equiv\ & (\neg p \vee \neg q \vee r) \\
=:\ & D_G
\end{aligned}
$$

▷ Normalform für eine Äquivalenzklasse von Formeln nicht eindeutig, z.B.

$$p \equiv (p \wedge q) \vee (p \wedge \neg q)$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Kanonische DNF

- Brute-Force-Bestimmung einer kanonischen Formel in DNF für F:
- Notation: Ist p eine aussagenlogische Variable, so ist $p^1 := p$ das positive Literal von p und $p^0 := \neg p$ das negative Literal von p.
  1. Stelle Wahrheitstabelle zu F auf.
  2. Bestimme alle minimalen erfüllenden Belegungen von F.
  3. Für jede minimale erfüllende Belegung β, bilde die Konjunktion

     $$\bigwedge_{p \in V_F} p^{\beta(p)}$$

     Beispiel: $(p \wedge \neg q \wedge \neg r)$ wenn $V_F = \{p, q, r\}$ und $\beta(p) = 1, \beta(q) = 0, \beta(r) = 0$.

     (Die Konjunktion hat genau eine erfüllende minimale Belegung: β.)
  4. Bilde die Disjunktion über alle diese Konjunktionen.
- Meist kürzere DNF mittels Äquivalenzen oder KV-Diagramm.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Bsp If-Then-Else-Operator

$$\mathrm{ITE}(p, q, r) := ((p \to q) \wedge (\neg p \to r))$$

- Wahrheitstabelle:

p	q	r	$((p$	$\to$	$q)$	$\wedge$	$(\neg$	p	$\to$	$r))$
0	0	0		1		0	1		0	
0	0	1		1		1	1		1	
0	1	0		1		0	1		0	
0	1	1		1		1	1		1	
1	0	0		0		0	0		1	
1	0	1		0		0	0		1	
1	1	0		1		1	0		1	
1	1	1		1		1	0		1	

- DNF:

$$\mathrm{ITE}(p, q, r) \equiv D_F = \underline{\qquad\qquad\qquad\qquad}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Bsp If-Then-Else-Operator

$$\mathrm{ITE}(p,q,r) := ((p \to q) \wedge (\neg p \to r))$$

- Wahrheitstabelle:

p	q	r	$((p$	$\to$	$q)$	$\wedge$	$(\neg$	p	$\to$	$r))$
0	0	0		1		0	1		0	
0	0	1		1		1	1		1	
0	1	0		1		0	1		0	
0	1	1		1		1	1		1	
1	0	0		0		0	0		1	
1	0	1		0		0	0		1	
1	1	0		1		1	0		1	
1	1	1		1		1	0		1	

- DNF:

$$\mathrm{ITE}(p,q,r) \equiv D_F = \quad (\neg p \wedge \neg q \wedge r) \vee (\neg p \wedge q \wedge r) \vee (p \wedge q \wedge \neg r) \vee (p \wedge q \wedge r)$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Thm Für jede Formel F gibt es eine Formel G in KNF mit $F \equiv G$.

Beweis: Vertausche $\wedge$ und $\vee$ in dem Verfahren für die Berechnung einer Formel in DNF (Schritte 1 und 2 ändern sich nicht).

- Brute-Force-Bestimmung von kanonischer KNF:
  - Stelle Wahrheitstabelle zu F auf.
  - Bestimme alle minimalen nicht-erfüllenden Belegungen von F.
  - Für jede minimale nicht-erfüllende Belegung β, bilde die Disjunktion

$$\bigvee_{p \in V_F} p^{(1-\beta(p))} \quad \text{(Maxterm)}$$

Bsp $(\neg p \vee q \vee r)$ wenn $V_F = \{p, q, r\}$ und $\beta(p) = 1, \beta(q) = 0, \beta(r) = 0$.

(Die Disjunktion hat genau eine nicht-erfüllende minimale Belegung: β.)

- Bilde die Konjunktion über alle diese Disjunktionen.
- Intuition: die KNF „sagt“: β_1 ist keine erfüllende Belegung und β_2 ist keine erfüllende Belegung und ...

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Bsp If-Then-Else-Operator

$$\mathrm{ITE}(p,q,r) := ((p \to q) \wedge (\neg p \to r))$$

- Wahrheitstabelle:

p	q	r	$((p$	$\to$	$q)$	$\wedge$	$(\neg$	p	$\to$	$r))$
0	0	0		1		0	1		0	
0	0	1		1		1	1		1	
0	1	0		1		0	1		0	
0	1	1		1		1	1		1	
1	0	0		0		0	0		1	
1	0	1		0		0	0		1	
1	1	0		1		1	0		1	
1	1	1		1		1	0		1	

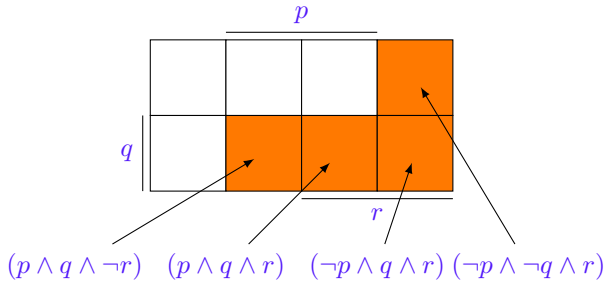
- KNF:

$$\mathrm{ITE}(p,q,r) \equiv K_F = \quad (p \vee q \vee r) \wedge (p \vee \neg q \vee r) \wedge (\neg p \vee q \vee r) \wedge (\neg p \vee q \vee \neg r)$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

DNF mittels KV-Diagrammen: Beispiel

- Beispiel: If-Then-Else-Operator $\mathrm{ITE}(p, q, r) = ((p \to q) \wedge (\neg p \to r))$:



- Kanonische DNF:

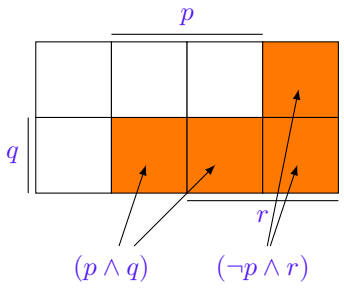
$$(p \wedge q \wedge \neg r) \vee (p \wedge q \wedge r) \vee (\neg p \wedge \neg q \wedge r) \vee (\neg p \wedge q \wedge r)$$

- Kürzere DNF mittels Blockbildung (=Ausklammern):

$$(p \wedge q) \vee (\neg p \wedge r)$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Beispiel: If-Then-Else-Operator $\mathrm{ITE}(p,q,r) = ((p \to q) \wedge (\neg p \to r))$:



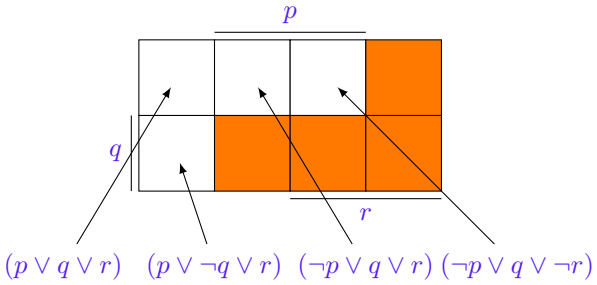
- Kanonische DNF:

$$(p \wedge q \wedge \neg r) \vee (p \wedge q \wedge r) \vee (\neg p \wedge \neg q \wedge r) \vee (\neg p \wedge q \wedge r)$$

- Kürzere DNF mittels Blockbildung (=Ausklammern):

$$(p \wedge q) \vee (\neg p \wedge r)$$

- Beispiel: $\mathsf{ITE}(p, q, r) = ((p \to q) \wedge (\neg p \to r))$:



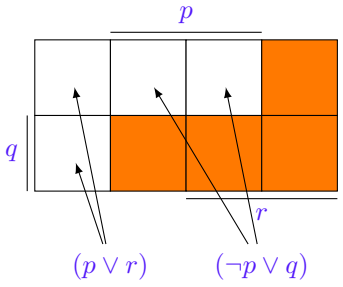
- Kanonische KNF zu $\mathsf{ITE}(p, q, r)$:

$$(p \vee q \vee r) \wedge (p \vee \neg q \vee r) \wedge (\neg p \vee q \vee r) \wedge (\neg p \vee q \vee \neg r)$$

- Kürzere KNF mittels Blockbildung:

$$(p \vee r) \wedge (\neg p \vee q)$$

- Beispiel: $\mathsf{ITE}(p, q, r) = ((p \to q) \wedge (\neg p \to r))$:

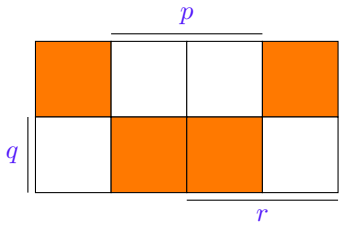


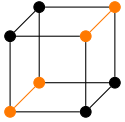
- Kanonische KNF zu $\mathsf{ITE}(p, q, r)$:

$$(p \vee q \vee r) \wedge (p \vee \neg q \vee r) \wedge (\neg p \vee q \vee r) \wedge (\neg p \vee q \vee \neg r)$$

- Kürzere KNF mittels Blockbildung:

$$(p \vee r) \wedge (\neg p \vee q)$$





$$(\neg p \wedge \neg q \wedge \neg r) \vee (p \wedge q \wedge \neg r) \vee (p \wedge q \wedge r) \vee (\neg p \wedge \neg q \wedge r) \equiv (\neg p \wedge \neg q) \vee (p \wedge q) \equiv (p \leftrightarrow q)$$

$$(p \vee \neg q \vee r) \wedge (\neg p \vee q \vee r) \wedge (\neg p \vee q \vee \neg r) \wedge (p \vee \neg q \vee \neg r) \equiv (\neg p \vee q) \wedge (p \vee \neg q) \equiv (p \leftrightarrow q)$$

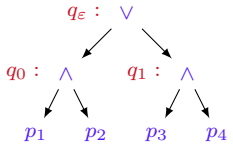
Blöcke können die Ränder überspannen, z.B. $(\neg p \wedge \neg q)$ und $(p \vee \neg q)$.

KNF vs DNF, Effizienz

- Viele Formeln aus der Praxis sind in KNF (oder „fast“ in KNF): Eine Menge von Anforderungen wird durch eine Konjunktion von Formeln modelliert.
- SAT-Solver erwarten als Eingabe eine Formel in KNF.
- ⚠ KNF durch Wahrheitstabelle/KV-Diagramm würde SAT-Solver nutzlos machen.
- Effizienz: (1) Brute force ineffizient. (2) KNF kann exponentiell länger sein: Beispiel: $F = (p_1 \wedge p_2) \vee (p_3 \wedge p_4) \vee \ldots \vee (p_{2k-1} \wedge p_{2k})$
  - KNF zu F besteht aus 2^k verschiedenen Disjunktionen von Literalen (sog. Klauseln), ist also exponentiell länger als F.
- Für jede Formel F gibt es eine zu F erfüllbarkeitsäquivalente Formel G in KNF, die nicht viel länger als F ist. Wichtig für Anwendungen!
  - „$F \equiv_e G$“ kurz für erfüllbarkeitsäquivalent, d.h. es gibt eine erfüllende Belegung β_F für F genau dann, wenn es eine erfüllende Belegung β_G für G gibt.
  - Beachte: β_F muss nicht zu G passen und β_G nicht zu F!
    Beispiel: $A \vee B \equiv_e C \wedge (C \leftrightarrow (A \vee B))$ aber $A \vee B \not\equiv C \wedge (C \leftrightarrow (A \vee B))$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

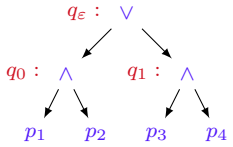
Aufstellen einer KNF-Formel erfüllbarkeitsäquivalent zu F



- Zwischenschritt: Hilfsvariablen „speichern“ Zwischenergebnisse.

$$q_\varepsilon \wedge \underbrace{(q_\varepsilon \leftrightarrow (q_0 \vee q_1))}_{=:G_\varepsilon} \wedge \underbrace{(q_0 \leftrightarrow (p_1 \wedge p_2))}_{=:G_0} \wedge \underbrace{(q_1 \leftrightarrow (p_3 \wedge p_4))}_{=:G_1}$$

Aufstellen einer KNF-Formel erfüllbarkeitsäquivalent zu F

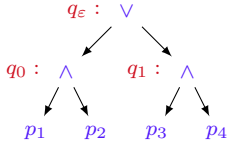


- Zwischenschritt: Hilfsvariablen „speichern“ Zwischenergebnisse.

$$q_\varepsilon \wedge \underbrace{(q_\varepsilon \leftrightarrow (q_0 \vee q_1))}_{=:G_\varepsilon} \wedge \underbrace{(q_0 \leftrightarrow (p_1 \wedge p_2))}_{=:G_0} \wedge \underbrace{(q_1 \leftrightarrow (p_3 \wedge p_4))}_{=:G_1}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Aufstellen einer KNF-Formel erfüllbarkeitsäquivalent zu F



- ▷ Zwischenschritt: Hilfsvariablen „speichern“ Zwischenergebnisse.

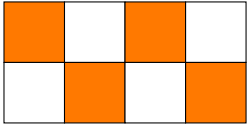
$$q_\varepsilon \wedge \underbrace{(q_\varepsilon \leftrightarrow (q_0 \vee q_1))}_{=:G_\varepsilon} \wedge \underbrace{(q_0 \leftrightarrow (p_1 \wedge p_2))}_{=:G_0} \wedge \underbrace{(q_1 \leftrightarrow (p_3 \wedge p_4))}_{=:G_1}$$

- ▷ Im Fall der Aussagenlogik als Tseytin-Transformation bezeichnet. Vgl.:

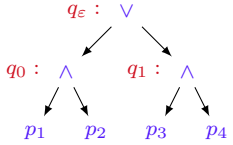
$$x^4 + 2x^2 + 1 = 0 \text{ gdw } y^2 + 2y + 1 = 0 \wedge y = x^2$$

Def Klausel: Disjunktion von Literalen $\bigvee_i L_i$.

⚠ Jede Formel mit 3 Variablen hat eine KNF mit ≤ 4 Klauseln.



Aufstellen einer KNF-Formel erfüllbarkeitsäquivalent zu F



- Zwischenschritt: Hilfsvariablen „speichern“ Zwischenergebnisse.

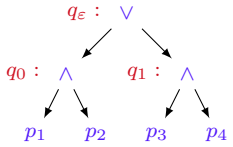
$$q_\varepsilon \wedge \underbrace{(q_\varepsilon \leftrightarrow (q_0 \vee q_1))}_{=:G_\varepsilon} \wedge \underbrace{(q_0 \leftrightarrow (p_1 \wedge p_2))}_{=:G_0} \wedge \underbrace{(q_1 \leftrightarrow (p_3 \wedge p_4))}_{=:G_1}$$

- Ersetzen von G_ε, G_0, G_1 durch sem. äquivalente minimale KNF ergibt:

$$\begin{aligned} E_F := q_\varepsilon \quad & \wedge \quad (\neg q_\varepsilon \vee q_0 \vee q_1) \wedge (q_\varepsilon \vee \neg q_0) \wedge (q_\varepsilon \vee \neg q_1) \\ & \wedge \quad (q_0 \vee \neg p_1 \vee \neg p_2) \wedge (\neg q_0 \vee p_1) \wedge (\neg q_0 \vee p_2) \\ & \wedge \quad (q_1 \vee \neg p_3 \vee \neg p_4) \wedge (\neg q_1 \vee p_3) \wedge (\neg q_1 \vee p_4) \end{aligned}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Aufstellen einer KNF-Formel erfüllbarkeitsäquivalent zu F



- Zwischenschritt: Hilfsvariablen „speichern“ Zwischenergebnisse.

$$q_\varepsilon \wedge \underbrace{(q_\varepsilon \leftrightarrow (q_0 \vee q_1))}_{=:G_\varepsilon} \wedge \underbrace{(q_0 \leftrightarrow (p_1 \wedge p_2))}_{=:G_0} \wedge \underbrace{(q_1 \leftrightarrow (p_3 \wedge p_4))}_{=:G_1}$$

- Ersetzen von G_ε, G_0, G_1 durch sem. äquivalente minimale KNF ergibt:

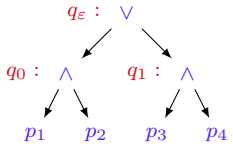
$$\begin{aligned} E_F := q_\varepsilon \quad & \wedge \quad (\neg q_\varepsilon \vee q_0 \vee q_1) \wedge (q_\varepsilon \vee \neg q_0) \wedge (q_\varepsilon \vee \neg q_1) \\ & \wedge \quad (q_0 \vee \neg p_1 \vee \neg p_2) \wedge (\neg q_0 \vee p_1) \wedge (\neg q_0 \vee p_2) \\ & \wedge \quad (q_1 \vee \neg p_3 \vee \neg p_4) \wedge (\neg q_1 \vee p_3) \wedge (\neg q_1 \vee p_4) \end{aligned}$$

- Jede E_F erfüllende Belegung ist eine F erfüllende Belegung.

  z.B. $\beta(p_1) = \beta(p_2) = \beta(p_3) = 1, \beta(p_4) = 0, \beta(q_0) = 1, \beta(q_1) = 0, \beta(q_\varepsilon) = 1$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Aufstellen einer KNF-Formel erfüllbarkeitsäquivalent zu F



- Zwischenschritt: Hilfsvariablen „speichern“ Zwischenergebnisse.

$$q_\varepsilon \wedge \underbrace{(q_\varepsilon \leftrightarrow (q_0 \vee q_1))}_{=:G_\varepsilon} \wedge \underbrace{(q_0 \leftrightarrow (p_1 \wedge p_2))}_{=:G_0} \wedge \underbrace{(q_1 \leftrightarrow (p_3 \wedge p_4))}_{=:G_1}$$

- Ersetzen von G_ε, G_0, G_1 durch sem. äquivalente minimale KNF ergibt:

$$\begin{aligned} E_F := q_\varepsilon \quad & \wedge \quad (\neg q_\varepsilon \vee q_0 \vee q_1) \wedge (q_\varepsilon \vee \neg q_0) \wedge (q_\varepsilon \vee \neg q_1) \\ & \wedge \quad (q_0 \vee \neg p_1 \vee \neg p_2) \wedge (\neg q_0 \vee p_1) \wedge (\neg q_0 \vee p_2) \\ & \wedge \quad (q_1 \vee \neg p_3 \vee \neg p_4) \wedge (\neg q_1 \vee p_3) \wedge (\neg q_1 \vee p_4) \end{aligned}$$

- Man kann eine F erf. Belegung zu einer E_F erf. Belegung erweitern

  z.B. $\beta(p_1) = \beta(p_2) = \beta(p_3) = 1, \beta(p_4) = 0$ um $\beta(q_0) = 1, \beta(q_1) = 0, \beta(q_\varepsilon) = 1$.
  muss man aber nicht

  z.B. $\beta(p_1) = \beta(p_2) = \beta(p_3) = 1, \beta(p_4) = 0$ um $\beta(q_0) = 1, \beta(q_1) = 0, \beta(q_\varepsilon) = 0$.
  Daher nur $F \equiv_e E_F$ und $F \not\equiv E_F$!

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Aufstellen einer KNF-Formel erfüllbarkeitsäquivalent zu F (Algorithmus)

Def Gegeben F konstruieren wir eine Formel E_F in KNF wie folgt.

Sei $\mathcal{F}_F$ die Menge aller Teilformeln von F (mit $F \in \mathcal{F}_F$).

1. Für jede Teilformel $G \in \mathcal{F}_F$ setze:
   Falls $G = p \in V_F$ atomar, sei $q_G := G = p$.
   Falls $G \notin V_F$ nicht atomar, sei q_G eine noch nicht verwendete Hilfsvariable.
2. Definiere für jede nicht atomare Teilformel $G \in \mathcal{F}_F \setminus V_F$ eine Formel K_G:
   Falls $G = \neg H$, dann sei K_G in KNF mit $K_G \equiv (q_G \leftrightarrow \neg q_H)$.
   Falls $G = (H \text{ op } H')$, dann sei K_G in KNF mit $K_G \equiv (q_G \leftrightarrow (q_H \text{ op } q_{H'}))$.
   (Idee: „q_G soll den Wahrheitswert von G speichern“)
3. Setze $E_F := q_F \wedge \bigwedge_{G \in \mathcal{F}_F \setminus V_F} K_G$.

Fkt Nach Konstruktion gilt: $F \equiv_e E_F$.

Genauer gilt für jede Teilformel G (Beweis mittels Induktion):

1. Für eine erfüllende E_F-Belegung muss $\beta(q_G) = [G](\beta)$ gelten.

2. Jede erfüllende F-Belegung kann durch $\beta(q_G) := [G](\beta)$ erweitert werden.

(„E_F simuliert F.“)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- ▷ Anwendung von KNF/DNF: „Beschreibung“ einer Booleschen Funktion (bzw. ihrer Wertetabelle) mittels $\neg, \wedge, \vee$.

Kor Gegeben eine Boolesche Funktion

$$f\colon \{0,1\}^n \to \{0,1\}^m$$

fixiere z.B. die aussagenlogischen Variablen $V_F = \{p_1, p_2, \ldots, p_n\}$.

Dann gibt es Formeln $F_1, \ldots, F_m$ in KNF/DNF, so dass

$$([F_1](\beta), \ldots, [F_m](\beta)) = f\Big(\beta(p_1), \beta(p_2), \ldots, \beta(p_n)\Big)$$

für jede Belegung $\beta\colon V \to \{0,1\}$.

Kor $\{\neg, \wedge, \vee\}$ bildet eine (Operator-)Basis der Aussagenlogik bzw. $\{\neg, \wedge, \vee\}$ ist eine vollständige Menge von Operatoren.

⚠ Wichtig für Prozessoren: nur $\{\neg, \vee, \wedge\}$ müssen physikalisch realisiert werden.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Die Junktoren NAND $\overline{\wedge}$ und NOR $\overline{\vee}$ sind definiert durch:

$$(F \mathbin{\overline{\wedge}} G) := \neg(F \wedge G) \qquad (F \mathbin{\overline{\vee}} G) := \neg(F \vee G)$$

⚠ Beide Junktoren sind nicht assoziativ.

▷ Realisierung von NAND und NOR mittels Transistoren sehr einfach.

Lem Die Mengen $\{\overline{\wedge}\}$ und $\{\overline{\vee}\}$ sind jeweils vollständig.

Folgt für $\{\overline{\wedge}\}$ z.B. aus:

$$\begin{array}{lcll} \neg F & \equiv & \neg(F \wedge F) & \equiv & (F \mathbin{\overline{\wedge}} F) \\ (F \vee G) & \equiv & \neg(\neg F \wedge \neg G) & \equiv & ((F \mathbin{\overline{\wedge}} F) \mathbin{\overline{\wedge}} (G \mathbin{\overline{\wedge}} G)) \\ (F \wedge G) & \equiv & \neg\neg(F \wedge G) & \equiv & ((F \mathbin{\overline{\wedge}} G) \mathbin{\overline{\wedge}} (F \mathbin{\overline{\wedge}} G)) \end{array}$$

und analog für $\{\overline{\vee}\}$.

Kor Damit sind auch $\{\wedge, \neg\}$, $\{\vee, \neg\}$, $\{\rightarrow, \neg\}$ und $\{\rightarrow, \mathsf{false}\}$ vollständig.

⚠ $\{\rightarrow\}$ und $\{\wedge, \vee\}$ sind z.B. nicht vollständig.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

DPLL: Davis-Putnam-Logemann-Loveland

- Grundlage der meisten aktuell verwendeten Löser für das Erfüllbarkeitsproblem der Aussagenlogik (kurz: SAT).
  Gegeben: *aussagenlogische Formel* F.
  Frage: *Ist* F *erfüllbar?*

- Anwendungen:

  Viele Optimierungsprobleme lassen sich sehr einfach mittels Aussagenlogik beschreiben (siehe Übungen).

  Beschreibungen sind häufig schon in KNF, ansonsten erfüllbarkeitsäquivalente Formel in KNF verwenden.

  SAT-Solver sind oft besser als Neuentwicklungen problemspezifischer Algorithmen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Klauseldarstellung von KNF-Formeln

Def Eine Klausel ist eine Disjunktion von Literalen.

- Beispiele: $(p \vee \neg q), (p \vee q \vee r), p$

- Eine Klausel wird durch die Menge ihrer Literale dargestellt, z.B.
  - Die Menge $\{p, \neg q, r\}$ stellt die Klausel $(p \vee \neg q \vee r)$ dar.
  - Die leere Menge von Literalen stellt die Formel false dar.
  - Beachte: $p \equiv (p \vee p)$ und $(p \vee q) \equiv (q \vee p)$ folgen automatisch aus $\{p, p\} = \{p\}$ und $\{p, q\} = \{q, p\}$.
- Eine KNF-Formel wird durch die Menge ihrer Klauseln (genauer: die Menge der Darstellungen ihrer Klauseln) dargestellt.
  - Die Klauselmenge $\{\{\neg p\}, \{p, \neg q, r\}\}$ stellt die Formel $\neg p \wedge (p \vee \neg q \vee r)$ dar.
  - Die leere Menge von Klauseln stellt die Formel true dar.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

DPLL: Davis-Putnam-Logemann-Loveland

- Basisidee: Fallunterscheidung zzgl. „Aufräumen“
  - Gegeben F in KNF, fixiere beliebige Variable x in F
  - Für die Übungen wird meistens vorgegeben, in welcher Reihenfolge die Variablen zu überprüfen sind, um einheitliche Lösungen zu erhalten!
  - Bestimme $F_1 := F[x/\text{true}]$, d.h. ersetze jedes Vorkommen von x in F durch true; dann „räume auf“ mittels der Äquivalenzen (links durch rechts ersetzen):

$$\begin{array}{llllllll} \text{true} \vee G & \equiv & \text{true} & \text{true} \wedge G & \equiv & G & \neg\text{true} & \equiv & \text{false} \\ \text{false} \vee G & \equiv & G & \text{false} \wedge G & \equiv & \text{false} & \neg\text{false} & \equiv & \text{true} \end{array}$$

    und überprüfe, ob F_1 erfüllbar ist; falls ja, fertig, ansonsten selbes Spiel mit $F_0 := F[x/\text{false}]$

Bsp $F = ((p \vee q \vee \neg r) \wedge (\neg p \vee q \vee s))$

$F_1 := F[p/\text{true}] = (q \vee s) \qquad F_0 := F[p/\text{false}] = (q \vee \neg r)$

  - $F_{0/1}$ hat mindestens eine Variable weniger als F; sobald Formel keine Variablen mehr hat, ist Erfüllbarkeit direkt ablesbar.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

DPLL: Davis-Putnam-Logemann-Loveland

- Klauselmengendarstellung erlaubt sehr einfache Implementierung:
  - Die Klauselmenge zu $F[x/\text{true}]$ erhält man, indem man zunächst alle Klauseln verwirft, die x als Element enthalten, und aus den verbleibenden Klauseln jeweils $\neg x$ entfernt:

    $$\mathcal{K}_{F[x/\text{true}]} = \{C \setminus \{\neg x\} \mid C \in \mathcal{K}_F, x \notin C\}.$$

  - Entsprechend:

    $$\mathcal{K}_{F[x/\text{false}]} = \{C \setminus \{x\} \mid C \in \mathcal{K}_F, \neg x \notin C\}.$$

Bsp $\mathcal{K}_F = \{\{p, q, \neg r\}, \{\neg p, q, s\}\}$

$\mathcal{K}_{F[p/\text{true}]} = \{\{q, s\}\}$ $\quad$ $\mathcal{K}_{F[p/\text{false}]} = \{\{q, \neg r\}\}$

- In Klauselmengendarstellung sind die einzigen Formeln ohne Variablen
  $\{\}$ (leere Klauselmenge, nichts zu erfüllen, trivial erfüllt) und
  $\{\ \{\}\ \}$ (Klauselmenge mit leerer Klausel, unerfüllbar).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Basisversion:

  Gegeben Klauselmenge $\mathcal{K}$, totale Ordnung auf den Variablen $\prec$

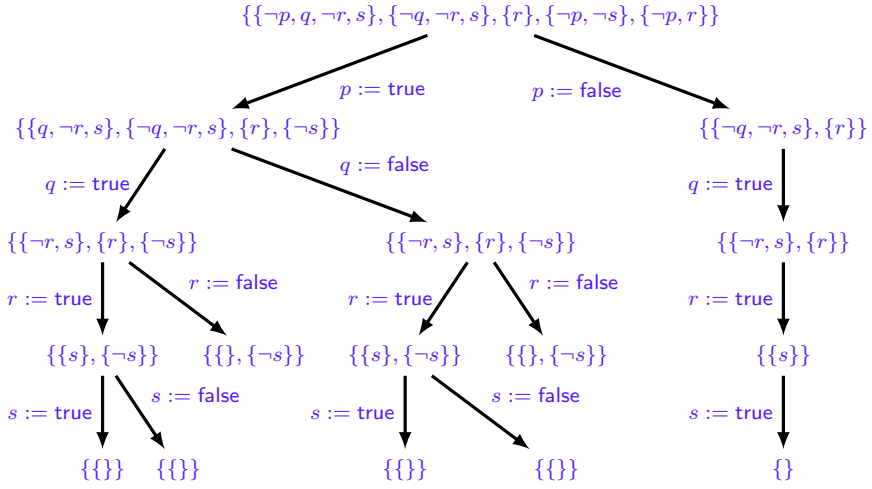
  - Falls $\{\} \in \mathcal{K}$, gib unerfüllbar zurück.
  - Falls $\mathcal{K} = \emptyset$, gib erfüllbar zurück.
  - Ansonsten fixiere die bzgl. $\prec$ erste in $\mathcal{K}$ vorkommende Variable x.

    Prüfe rekursiv, ob $\{C \setminus \{\neg x\} \mid C \in \mathcal{K}, x \notin C\}$ erfüllbar ist;

    falls ja, gib erfüllbar zurück;

    ansonsten gib Ergebnis der Rekursion zu $\{C \setminus \{x\} \mid C \in \mathcal{K}, \neg x \notin C\}$ zurück.

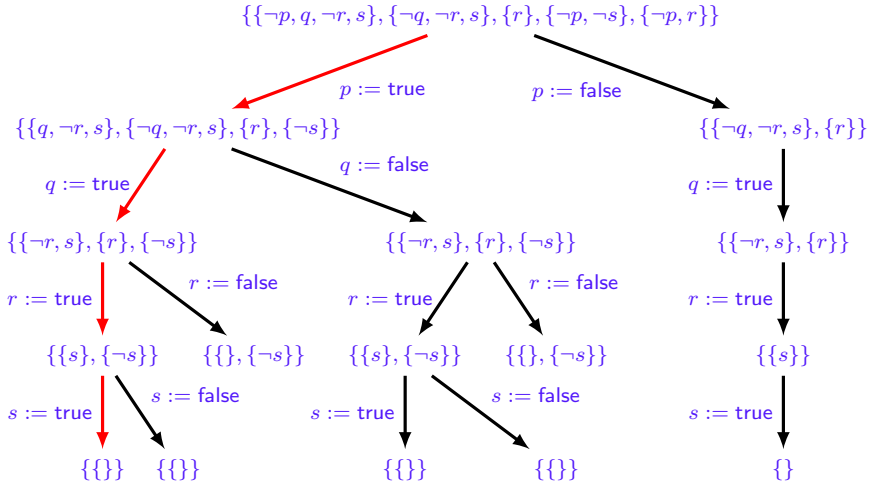
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science



- Variablenordnung im Beispiel: lexikographisch
- Gute Variablenordnung zu finden genauso schwierig wie erfüllende Belegung zu finden.
- In der Basisversion macht DPLL sehr viele unnötige Fallunterscheidungen.

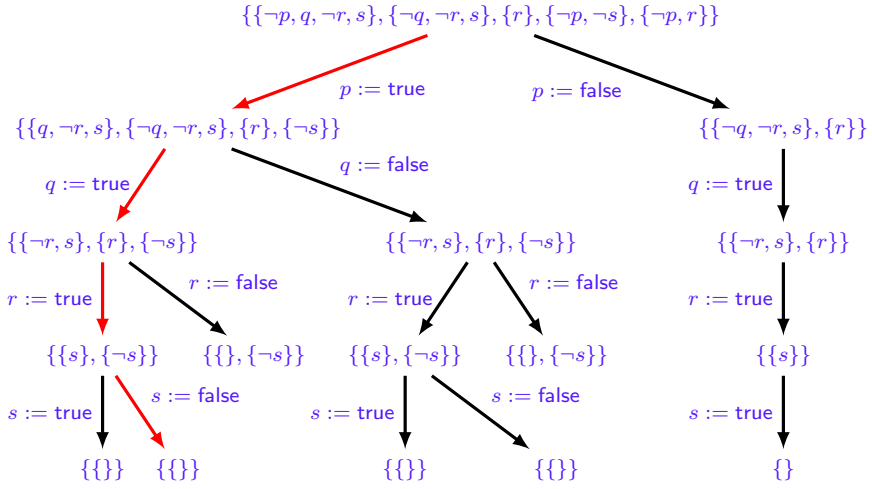
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

DPLL: Davis-Putnam-Logemann-Loveland



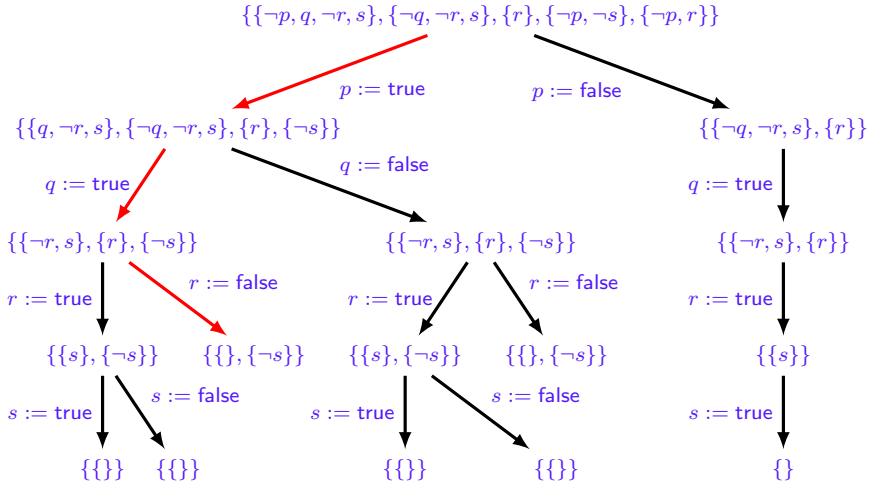
- Variablenordnung im Beispiel: lexikographisch
- Gute Variablenordnung zu finden genauso schwierig wie erfüllende Belegung zu finden.
- In der Basisversion macht DPLL sehr viele unnötige Fallunterscheidungen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science



- Variablenordnung im Beispiel: lexikographisch
- Gute Variablenordnung zu finden genauso schwierig wie erfüllende Belegung zu finden.
- In der Basisversion macht DPLL sehr viele unnötige Fallunterscheidungen.

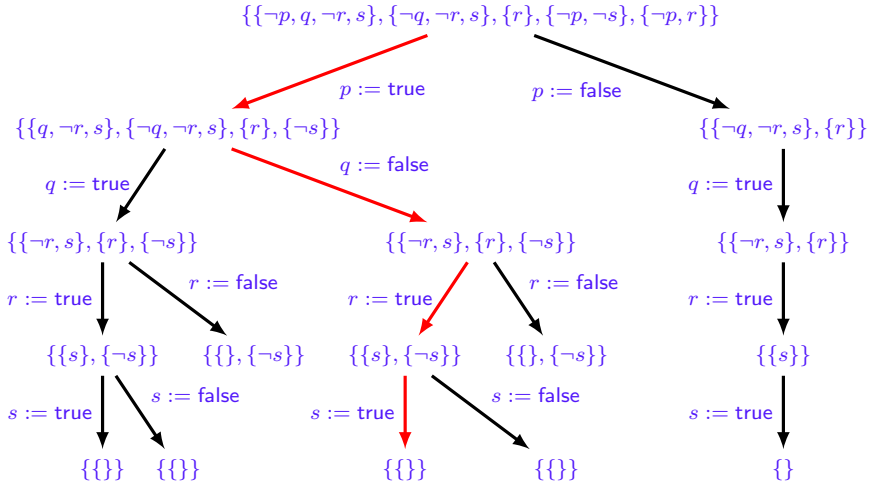
DPLL: Davis-Putnam-Logemann-Loveland



- Variablenordnung im Beispiel: lexikographisch
- Gute Variablenordnung zu finden genauso schwierig wie erfüllende Belegung zu finden.
- In der Basisversion macht DPLL sehr viele unnötige Fallunterscheidungen.

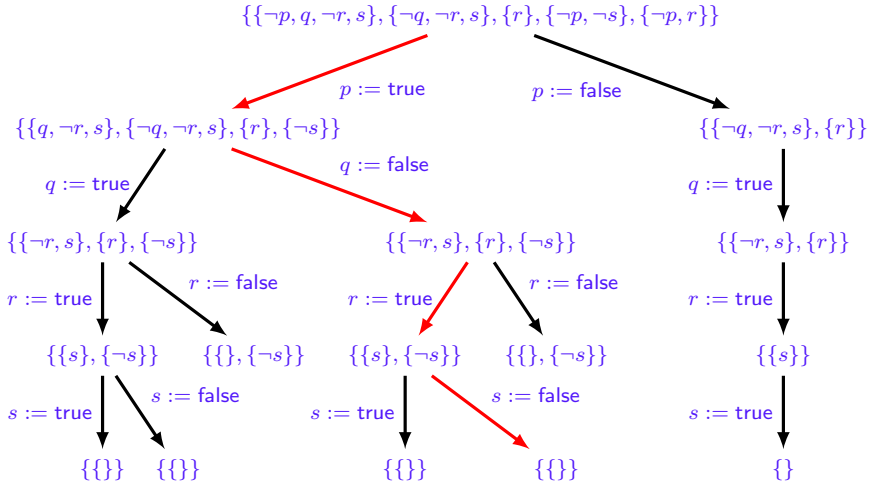
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

DPLL: Davis-Putnam-Logemann-Loveland

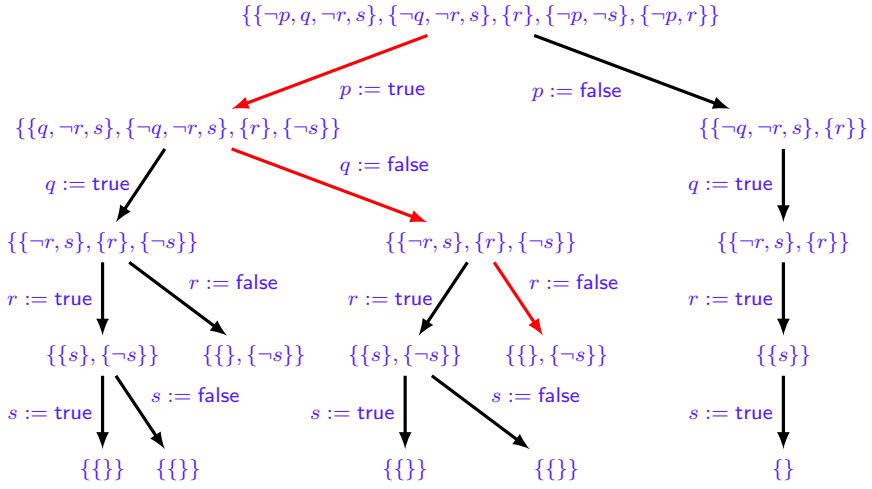


- Variablenordnung im Beispiel: lexikographisch
- Gute Variablenordnung zu finden genauso schwierig wie erfüllende Belegung zu finden.
- In der Basisversion macht DPLL sehr viele unnötige Fallunterscheidungen.

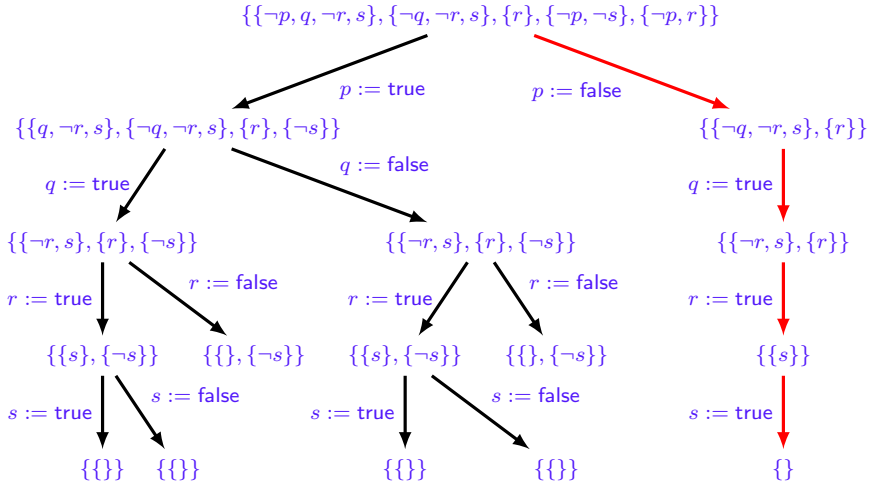
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science



- Variablenordnung im Beispiel: lexikographisch
- Gute Variablenordnung zu finden genauso schwierig wie erfüllende Belegung zu finden.
- In der Basisversion macht DPLL sehr viele unnötige Fallunterscheidungen.



- Variablenordnung im Beispiel: lexikographisch
- Gute Variablenordnung zu finden genauso schwierig wie erfüllende Belegung zu finden.
- In der Basisversion macht DPLL sehr viele unnötige Fallunterscheidungen.



- Variablenordnung im Beispiel: lexikographisch
- Gute Variablenordnung zu finden genauso schwierig wie erfüllende Belegung zu finden.
- In der Basisversion macht DPLL sehr viele unnötige Fallunterscheidungen.

DPLL: Davis-Putnam-Logemann-Loveland

- Sei $\mathcal{K}$ Klauselmenge, L Literal.
- Erweiterungen, um unnötige Fallunterscheidungen zu vermeiden:
  - OLR: one-literal rule

    Falls $\{L\} \in \mathcal{K}$, dann muss L von jeder erfüllenden Belegung von $\mathcal{K}$ auf wahr gesetzt werden. Daher reicht es $\mathcal{K}' := \{C \setminus \{\overline{L}\} \mid L \notin C \wedge C \in \mathcal{K}\}$ weiter zu betrachten.
  - PLR: pure-literal rule

    Falls nur L in den Klauseln von $\mathcal{K}$ auftritt, aber $\overline{L}$ nicht, dann kann man oBdA L auf true setzen, d.h. es reicht $\mathcal{K}' := \{C \mid L \notin C \wedge C \in \mathcal{K}\}$ weiter zu betrachten.
- Erweiterter DPLL: Höchste Priorität OLR, dann PLR, dann erst Fallunterscheidung.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

DPLL: Algorithmus

- DPLL:

  Gegeben Klauselmenge $\mathcal{K}$, totale Ordnung auf den Variablen $\prec$

  - Falls $\{\} \in \mathcal{K}$, gib unerfüllbar zurück.
  - Falls $\mathcal{K} = \emptyset$, gib erfüllbar zurück.
  - Falls OLR anwendbar, wähle bzgl. $\prec$ erstes Literal L mit $\{L\} \in \mathcal{K}$; gib Ergebnis der Rekursion zu $\{C \setminus \{\overline{L}\} \mid L \notin C, C \in \mathcal{K}\}$ zurück.
  - Falls PLR anwendbar, wähle bzgl. $\prec$ erstes in $\mathcal{K}$ vorkommendes Literal L mit $\{C \in \mathcal{K} \mid \overline{L} \in C\} = \emptyset$; gib Ergebnis der Rekursion zu $\{C \mid L \notin C, C \in \mathcal{K}\}$ zurück.
  - Ansonsten fixiere die bzgl. $\prec$ erste in $\mathcal{K}$ vorkommende Variable x.

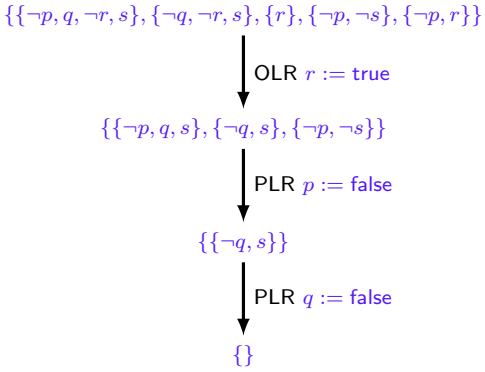
    Prüfe rekursiv, ob $\{C \setminus \{\neg x\} \mid C \in \mathcal{K}, x \notin C\}$ erfüllbar ist;

    falls ja, gib erfüllbar zurück;

    ansonsten gib Ergebnis der Rekursion zu $\{C \setminus \{x\} \mid C \in \mathcal{K}, \neg x \notin C\}$ zurück.
  - zzgl. caching, um selbe Klauselmenge nicht mehrmals zu untersuchen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

DPLL: Davis-Putnam-Logemann-Loveland



- Mit PLR und OLR lassen sich in diesem Beispiel alle Fallunterscheidungen vermeiden.
- Alle in der Praxis verwendeten SAT-Solver verwenden weitere Heuristiken, wie conflict-driven clause learning, und effiziente Datenstrukturen für die Operationen auf den Klauselmengen, um so Formeln mit mehreren tausend Variablen verarbeiten zu können.

DPLL: Evaluierung

- Auf Erfüllbarkeit ausgerichtet:
  - Wenn die Formel erfüllbar ist, dann kann DPLL „früh“ terminieren.
  - DPLL berechnet eine erfüllende Belegung, wenn es eine gibt.
  - Die erfüllende Belegung kann als Zertifikat der Erfüllbarkeit verwendet werden.
  - Erfüllbare Formeln haben kurze Zertifikate: erfüllende Belegung nie länger als die Formel selbst.
- Wir beschreiben nun ein zweites Verfahren, das auf Unerfüllbarkeit ausgerichtet ist: Resolution.
  - Wenn die Formel unerfüllbar ist, dann kann Resolution „früh“ terminieren.
  - Resolution berechnet ein Zertifikat der Unerfüllbarkeit.
  - Für einige Formeln ist das Zertifikat jedoch exponentiell länger als die Formel selbst.
  - Alle bekannten Verfahren haben dieses Problem.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Wissensrepräsentation mit Hilfe von Logik ist eine wichtige Technik der KI.
- Wissen über die Welt (Beziehungen, Empfehlungen, Anweisungen) wird mit Hilfe von Implikationen $F \to G$ dargestellt (Regeln).
  - Wenn der Kunde minderjährig ist, dann darf er keinen Alkohol kaufen.
  - Wenn der Kunde verheiratet ist und Kinder hat, dann soll eine Lebensversicherung empfohlen werden.
  - Wenn Block A eine Kugel und Block B ein Würfel ist, dann darf Block A nicht auf Block B gestellt werden.
- Eine Wissensbasis ist eine Konjunktion von Regeln.
- Frage: Gegeben eine Wissensbasis W, eine Konjunktion von Fakten F, und ein Ziel G, folgt G aus $W \wedge F$, d.h., gilt $W \wedge F \models G$?

  „Gilt G in allen Welten, in denen W und F gelten?“
- Äquivalente Frage: Ist die Formel $W \wedge F \wedge \neg G$ unerfüllbar?

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Beispiel

Wissensbasis:	$(\neg q \wedge p) \rightarrow s\ ,\ r \rightarrow (p \vee \neg s)\ ,\ \neg r \rightarrow \neg s$
Fakten:	p
Ziel:	q

- KNF für $W \wedge F \wedge \neg G$ in Klauselmengendarstellung:

$$\{q, \neg p, s\} \quad \{\neg r, p, \neg s\} \quad \{r, \neg s\} \quad \{p\} \quad \{\neg q\}$$

- ▷ Grundidee: Füge iterativ neue Klauseln zur Formel hinzu, die logische Konsequenzen der ursprünglichen Klauseln sind.

Bsp Wenn die Formel die Klauseln $\{\neg p, q, r\}$ und $\{\neg r, s, t\}$ enthält, dann kann die Klausel $\{\neg p, q, s, t\}$ hinzugefügt werden.

Die Klauseln $\{\neg p, q, r\}$, $\{\neg r, s, t\}$, und $\{\neg p, q, s, t\}$ sind äquivalent zu

$$(p \wedge \neg q) \to r \qquad r \to (s \vee t) \qquad (p \wedge \neg q) \to (s \vee t)$$

und es gilt (Transitivität der Implikation):

$$(p \wedge \neg q) \to r \ , \ r \to (s \vee t) \models (p \wedge \neg q) \to (s \vee t)$$

oder

$$\begin{array}{rl} & ((p \wedge \neg q) \to r) \ \wedge \ (r \to (s \vee t)) \\ \equiv & ((p \wedge \neg q) \to r) \wedge (r \to (s \vee t)) \wedge ((p \wedge \neg q) \to (s \vee t)) \end{array}$$

- ▷ Somit: Hinzufügen der neuen Klausel ändert die Formel nicht, macht nur „implizites Wissen“ explizit. (Vgl. transitiver Abschluss für Relationen.)

Bsp Wenn die Formel die Klauseln $\{p\}$ und $\{\neg p\}$ enthält, dann kann die leere Klausel hinzugefügt werden.

Die Klauseln sind äquivalent zu

$$p \qquad p \to \mathsf{false} \qquad \mathsf{false}$$

- Das Hinzufügen der leeren Klausel zeigt die Unerfüllbarkeit von $W \wedge F \wedge \neg G$, denn in diesem Fall

$$(W \wedge F \wedge \neg G) \equiv (W \wedge F \wedge \neg G) \wedge \mathsf{false} \equiv \mathsf{false}$$

Resolution: Resolvent zweier Klauseln

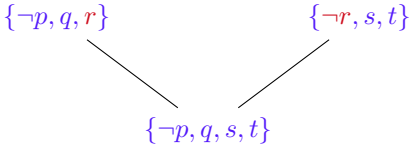
Def Resolventenbildung

Für Literal L, Klauseln K_1, K_2 mit $L \in K_1$, $\overline{L} \in K_2$ gilt:

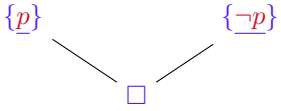
$$K_1, K_2 \models \underbrace{(K_1 \setminus \{L\}) \cup (K_2 \setminus \{\overline{L}\})}_{\text{Resolvent}}$$

wobei $\overline{L}$ das Literal mit $\overline{L} \equiv \neg L$ ist.

- Graphische Darstellung: (Immer genau ein Literal!)



- Die leere Klausel wird bei der Resolution auch mit $\square$ bezeichnet.
- Die leere Klausel kann nur durch



  entstehen, was dem trivialen Widerspruch $p \wedge \neg p \equiv \text{false}$ entspricht.
- Ziel jeder Resolution: die leere Klausel aus der ursprünglichen Klauselmenge resolvieren.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Basisversion der Resolution:

  Gegeben eine Klauselmenge F, fügt man iterativ (ohne nachzudenken) alle möglichen Resolventen hinzu:

$$\begin{aligned}
\mathsf{Res}^0(F) &:= F \\
\mathsf{Res}^{k+1}(F) &:= \mathsf{Res}^k(F) \\
&\cup \{(K_1 \setminus \{L\}) \cup (K_2 \setminus \{\overline{L}\}) \mid K_1, K_2 \in \mathsf{Res}^k(F), L \in K_1, \overline{L} \in K_2\} \\
\mathsf{Res}^*(F) &:= \bigcup_{k \in \mathbb{N}_0} \mathsf{Res}^k(F)
\end{aligned}$$

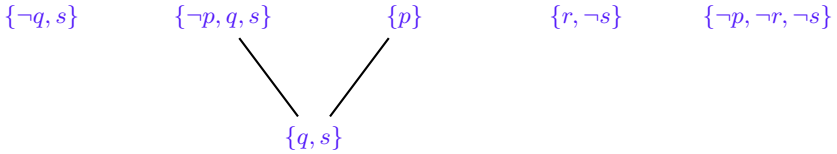
- Insbesondere muss man hierfür nicht wissen, was Aussagenlogik ist, sondern nur mit Mengen umgehen können.
- In der Praxis hört man auf, sobald man $\square$ resolviert hat.
- Verschiedene Strategien, um möglichst schnell/intelligent $\square$ zu resolvieren.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

▷ Resolution in Pseudocode:

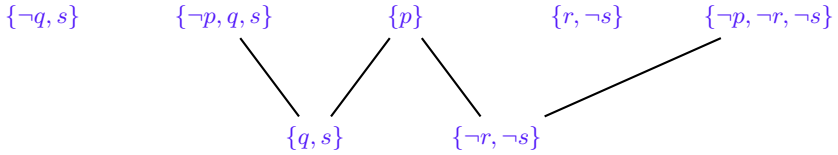
```
def Res*(F): # F als Menge von Klauseln
    todo := F
    neu := ∅
    while todo ≠ ∅:
        for K_1 ∈ todo:
            todo := todo - {K_1}
            for K_2 ∈ F:
                for L ∈ K_1:
                    if ¬L ∈ K_2:
                        R := (K_1 \ {L}) ∪ (K_2 \ {¬L})
                        if R ∉ F:
                            F := F ∪ {R}
                            neu := neu ∪ {R}
                        if R = □:
                            return F
        todo := neu
        neu := ∅
    return F
```

Resolution: Suche nach der leeren Klausel



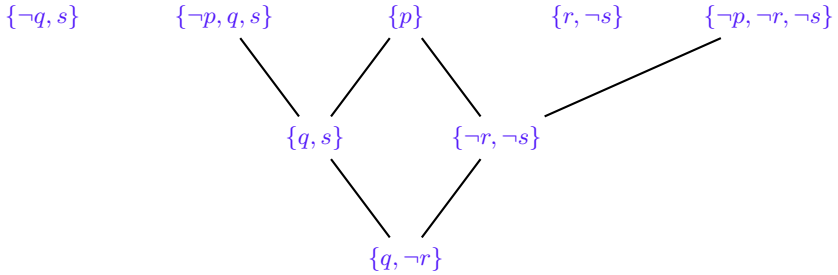
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Resolution: Suche nach der leeren Klausel



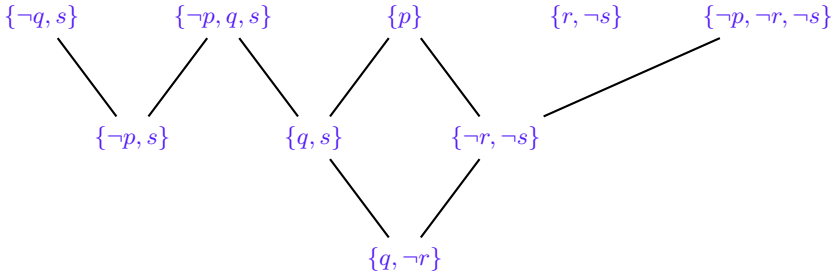
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Resolution: Suche nach der leeren Klausel



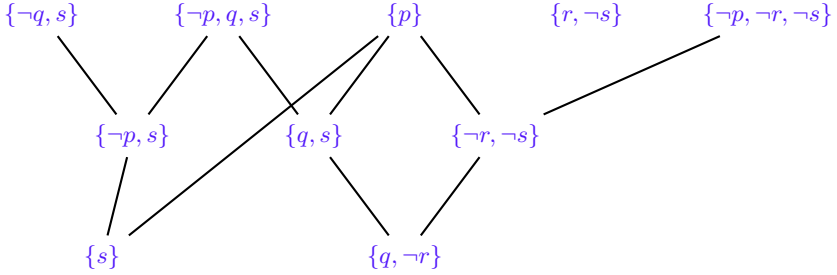
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Resolution: Suche nach der leeren Klausel



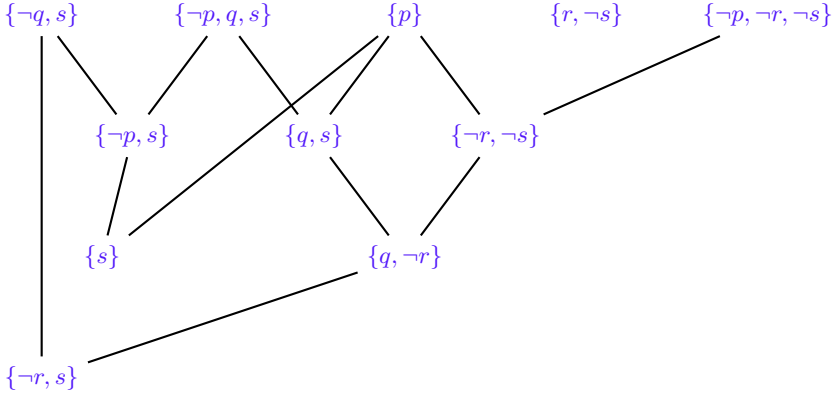
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Resolution: Suche nach der leeren Klausel



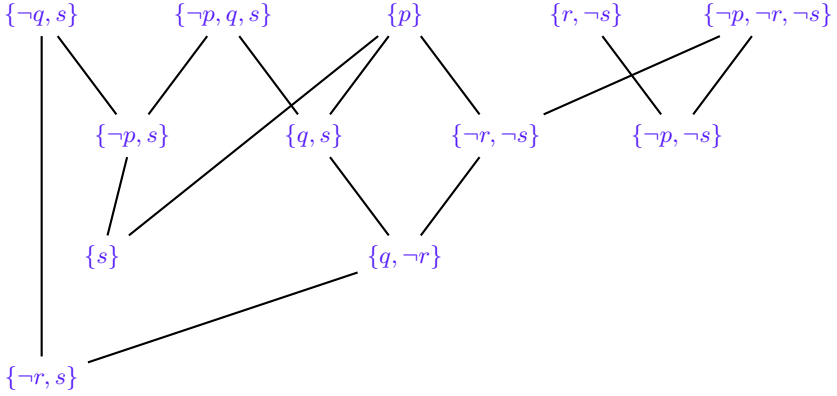
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Resolution: Suche nach der leeren Klausel



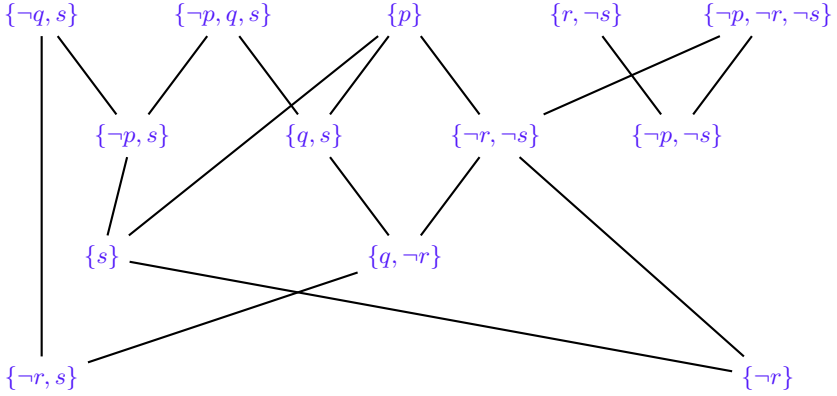
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Resolution: Suche nach der leeren Klausel



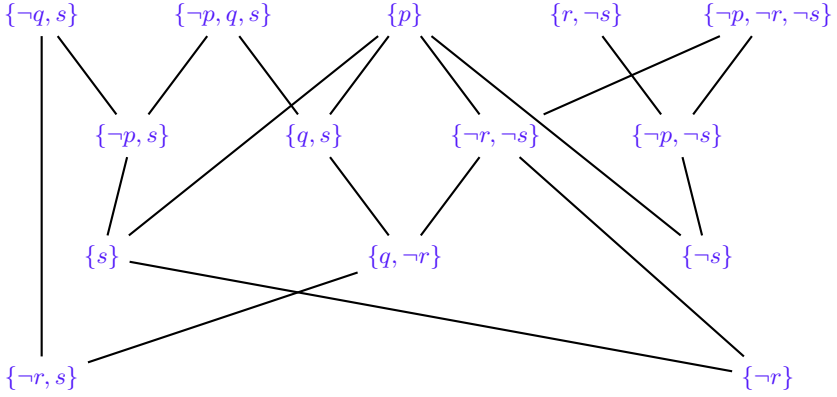
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Resolution: Suche nach der leeren Klausel

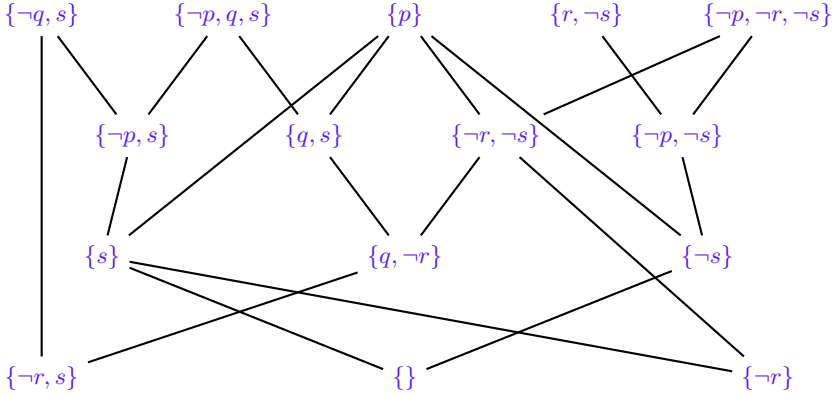


No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

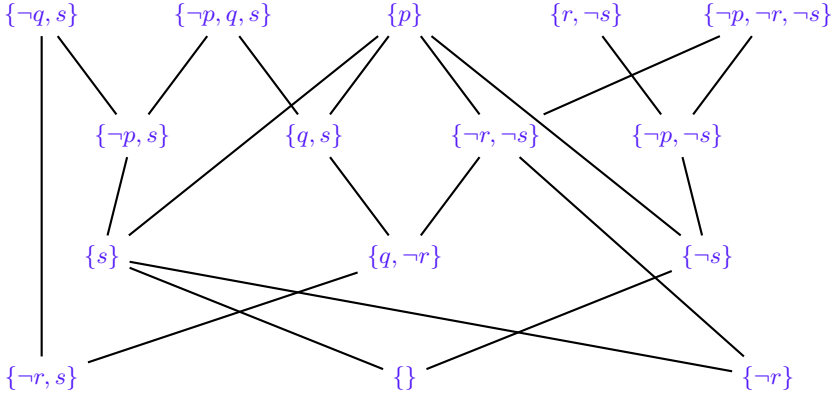
Resolution: Suche nach der leeren Klausel



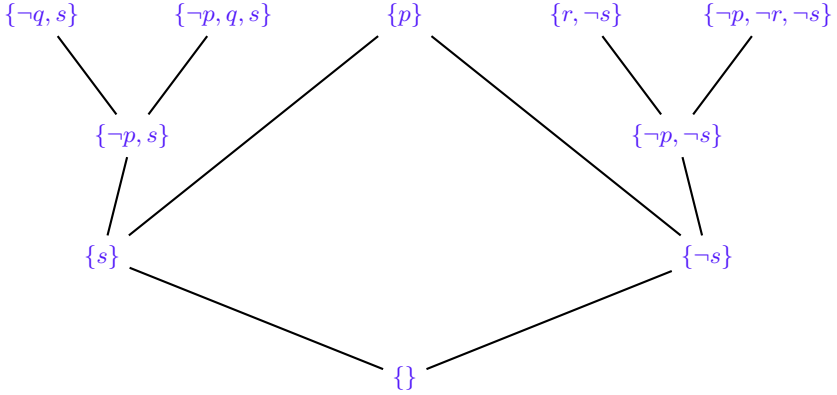
Resolution: Suche nach der leeren Klausel



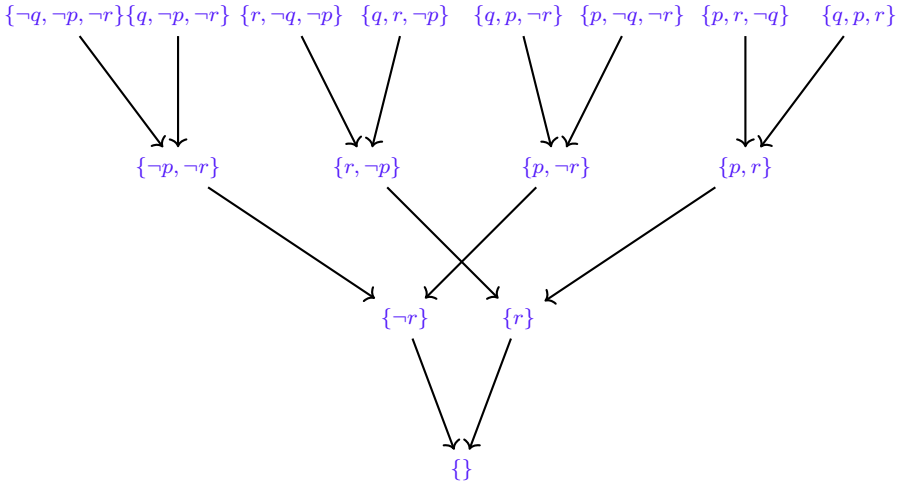
Resolution: Eine optimale Suche



Resolution: Eine optimale Suche



No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science


$\{\neg q, \neg p, \neg r\}$
$\{q, \neg p, \neg r\}$
$\{r, \neg q, \neg p\}$
$\{q, r, \neg p\}$
$\{q, p, \neg r\}$
$\{p, \neg q, \neg r\}$
$\{p, r, \neg q\}$
$\{q, p, r\}$
$\{\neg p, \neg r\}$
$\{r, \neg p\}$
$\{p, \neg r\}$
$\{p, r\}$
$\{\neg r\}$
$\{r\}$
$\{\}$

Resolution: Eigenschaften

Thm Sei F eine Formel in Klauselmengendarstellung/KNF. Dann gilt:

1. Die Berechnung von $\text{Res}^*(F)$ terminiert immer.
2. $\text{Res}^*(F) \equiv F$.
3. $\square \in \text{Res}^*(F)$ genau dann, wenn $F \equiv \text{false}$.

▷ Es folgt: Das Resolutionsverfahren entscheidet für jede gegebene Formel F, ob F erfüllbar oder unerfüllbar ist.

▷ **Beweis**:

1. Nur Klauseln über den ursprünglichen Variablen werden erzeugt, davon gibt es aber nur endlich viele ($2^{2|V_F|}$).
2. Nur logische Folgerungen werden hinzugefügt.
3. (a) Wenn $\square \in \text{Res}^*(F)$ dann $F \equiv \text{false}$: Folgt aus $(G \wedge \text{false} \equiv \text{false})$.
   (b) Wenn $F \equiv \text{false}$, dann $\square \in \text{Res}^*(F)$: siehe die folgenden Folien.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Resolution: Eigenschaften

- Wir zeigen: Wenn $F \equiv \text{false}$, dann $\square \in \text{Res}^*(F)$.
- Sei $F \equiv \text{false}$. Beweis durch Induktion über die Anzahl n der Variablen von F.
- Induktionsbasis: $n = 0$. Es gilt $F \neq \{\}$, denn sonst $F \equiv \text{true}$. Es folgt $F = \{\square\}$ und $\text{Res}^*(F) = \{\square\}$.
- Induktionsschritt: Fixiere beliebiges $n \in \mathbb{N}_0$:
  - Induktionsannahme: Für alle Formeln G mit höchstens n Variablen und $G \equiv \text{false}$ gilt $\square \in \text{Res}^*(G)$.
  - Induktionsbehauptung: Für alle Formeln F mit $n+1$ Variablen und $F \equiv \text{false}$ gilt $\square \in \text{Res}^*(F)$.
  - Beweis der Induktionsbehauptung: Sei $F \equiv \text{false}$ mit $n+1$ Variablen und $p \in V_F$.

    Da $F \equiv \text{false}$, folgt: $F[p/\text{true}]$ und $F[p/\text{false}]$ haben höchstens n Variablen und erfüllen $F[p/\text{true}] \equiv \text{false}$ und $F[p/\text{false}] \equiv \text{false}$.

    Aus der Induktionsannahme mit jeweils $G := F[p/\text{true}]$ und $G := F[p/\text{false}]$ folgen: $\square \in \text{Res}^*(F[p/\text{true}])$ und $\square \in \text{Res}^*(F[p/\text{false}])$.

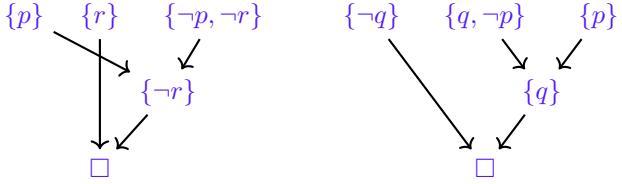
    Daraus folgt $\square \in \text{Res}^*(F)$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

$$F = \{\ \{\neg q, s\}\ ,\ \{\neg p, q, s\}\ ,\ \{p\}\ ,\ \{r, \neg s\}\ ,\ \{\neg p, \neg r, \neg s\}\ \}$$

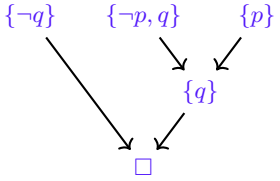
- Resolutionsgraphen für $F[s/\mathsf{true}]$ und $F[s/\mathsf{false}]$ induktiv konstruieren.

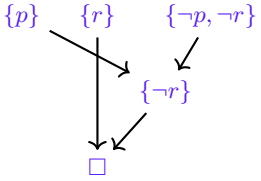
$$\begin{array}{rcl} F[s/\mathsf{true}] & = & \{\ \{p\}\ ,\ \{r\}\ ,\ \{\neg p, \neg r\}\ \} \\ F[s/\mathsf{false}] & = & \{\ \{\neg q\}\ ,\ \{\neg p, q\}\ ,\ \{p\}\ \} \end{array}$$

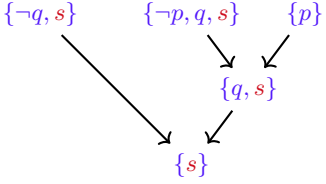


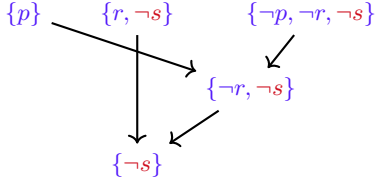
- s und $\neg s$ wieder einführen.

$$F = \{\ \{\neg q, s\}\ ,\ \{\neg p, q, s\}\ ,\ \{p\}\ ,\ \{r, \neg s\}\ ,\ \{\neg p, \neg r, \neg s\}\ \}$$



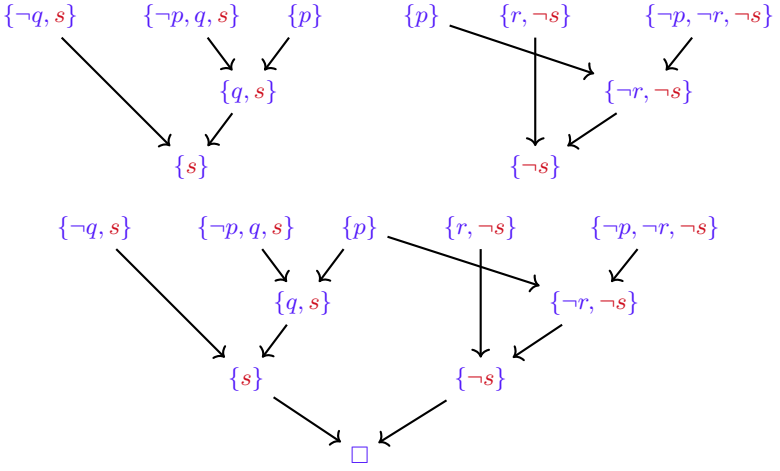






Resolution: Eigenschaften

- Resolutionsgraphen für $\{s\}$ und $\{\neg s\}$ zusammenbinden.



Resolution vs. DPLL

- Aussagenlogik als Problembeschreibungssprache (z.B. Sudoku).
- DPLL: Erfüllbarkeit
  - Sucht eine erfüllende Belegung für F in KNF.
  - In der Praxis meistens schnell, falls eine erfüllende Belegung existiert.
  - Anwendung: Finden eines fehlerhaften Programmablaufs, einer Knotenfärbung, einer Registerzuteilung.
- Resolution: Allgemeingültigkeit/Unerfüllbarkeit
  - Sucht einen Beweis für $F \models G$ bzw. für $F, \neg G \models \text{false}$.

    Jeder Beweisschritt ergänzt Annahmen F um einen Resolventen.

    Resolventenbildung basierend auf Schluss $(F \to G), (G \to H) \models (F \to H)$.
  - Anwendungen: Theorembeweiser, Logikprogrammierung (Prolog), Datenbanken (Datalog), Verifikation (keine fehlerhaften Abläufe), KI.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Beispiel: „murder mystery“

- „Murder mystery“: (ChatGPT failed on 2024-01-19, but succeeded on 2024-09-26)
  Agatha has been murdered by one of the inhabitants of Dreadbury Mansion. The only inhabitants of Dreadbury Mansion are Agatha, the butler and Charles. As is generally known, every murderer hates their victim and is never richer than their victim. Charles does not hate anyone hated by Agatha. Agatha hates everyone except the butler. The butler hates everyone who is not richer than Agatha. The butler hates everyone hated by Agatha. No one hates everyone.
- Aussagenlogische Variablen für Prädikate ($x, y \in \{a, b, c\}$):
  - M_x: „x hat Agatha ermordet/war am Mord beteiligt“
  - $H_{x,y}$: „x hasst y“
  - $R_{x,y}$: „x ist reicher als y“ mit „reicher“ eine Striktordnung.

  Teile der Problemformalisierung:
  - „Jeder Mörder hasst sein Opfer“: $\bigwedge_{x \in \{a,b,c\}} (M_x \to H_{x,a})$
  - „Jeder Mörder ist nicht reicher als sein Opfer“: $\bigwedge_{x \in \{a,b,c\}} (M_x \to \neg R_{x,a})$
  - „Reicher ist transitiv“: $\bigwedge_{x,y,z \in \{a,b,c\}} ((R_{x,y} \wedge R_{y,z}) \to R_{x,z})$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

▷ „Murder mystery“: vollständig ausgeschrieben als Klauselmenge F

$$\left\{\begin{array}{l}
\{\neg M_a, H_{a,a}\}, \{\neg M_b, H_{b,a}\}, \{\neg M_c, H_{c,a}\},\\
\{\neg M_a, \neg R_{a,a}\}, \{\neg M_b, \neg R_{b,a}\}, \{\neg M_c, \neg R_{c,a}\},\\
\{\neg H_{a,a}, \neg H_{c,a}\}, \{\neg H_{a,b}, \neg H_{c,b}\}, \{\neg H_{a,c}, \neg H_{c,c}\},\\
\{H_{a,a}\}, \{\neg H_{a,b}\}, \{H_{a,c}\},\\
\{R_{a,a}, H_{b,a}\}, \{R_{b,a}, H_{b,b}\}, \{R_{c,a}, H_{b,c}\},\\
\{\neg H_{a,a}, H_{b,a}\}, \{\neg H_{a,b}, H_{b,b}\}, \{\neg H_{a,c}, H_{b,c}\},\\
\{\neg H_{a,a}, \neg H_{a,b}, \neg H_{a,c}\}, \{\neg H_{b,a}, \neg H_{b,b}, \neg H_{b,c}\}, \{\neg H_{c,a}, \neg H_{c,b}, \neg H_{c,c}\},\\
\{M_a, M_b, M_c\},\\
\{\neg R_{a,a}\}, \{\neg R_{b,b}\}, \{\neg R_{c,c}\},\\
\{\neg R_{a,b}, \neg R_{b,a}\}, \{\neg R_{a,c}, \neg R_{c,a}\}, \{\neg R_{b,c}, \neg R_{c,b}\},\\
\{\neg R_{a,a}, \neg R_{a,a}, R_{a,a}\}, \{\neg R_{a,a}, \neg R_{a,b}, R_{a,b}\}, \{\neg R_{a,a}, \neg R_{a,c}, R_{a,c}\},\\
\{\neg R_{a,b}, \neg R_{b,a}, R_{a,a}\}, \{\neg R_{a,b}, \neg R_{b,b}, R_{a,b}\}, \{\neg R_{a,b}, \neg R_{b,c}, R_{a,c}\},\\
\{\neg R_{a,c}, \neg R_{c,a}, R_{a,a}\}, \{\neg R_{a,c}, \neg R_{c,b}, R_{a,b}\}, \{\neg R_{a,c}, \neg R_{c,c}, R_{a,c}\},\\
\{\neg R_{b,a}, \neg R_{a,a}, R_{b,a}\}, \{\neg R_{b,a}, \neg R_{a,b}, R_{b,b}\}, \{\neg R_{b,a}, \neg R_{a,c}, R_{b,c}\},\\
\{\neg R_{b,b}, \neg R_{b,a}, R_{b,a}\}, \{\neg R_{b,b}, \neg R_{b,b}, R_{b,b}\}, \{\neg R_{b,b}, \neg R_{b,c}, R_{b,c}\},\\
\{\neg R_{b,c}, \neg R_{c,a}, R_{b,a}\}, \{\neg R_{b,c}, \neg R_{c,b}, R_{b,b}\}, \{\neg R_{b,c}, \neg R_{c,c}, R_{b,c}\},\\
\{\neg R_{c,a}, \neg R_{a,a}, R_{c,a}\}, \{\neg R_{c,a}, \neg R_{a,b}, R_{c,b}\}, \{\neg R_{c,a}, \neg R_{a,c}, R_{c,c}\},\\
\{\neg R_{c,b}, \neg R_{b,a}, R_{c,a}\}, \{\neg R_{c,b}, \neg R_{b,b}, R_{c,b}\}, \{\neg R_{c,b}, \neg R_{b,c}, R_{c,c}\},\\
\{\neg R_{c,c}, \neg R_{c,a}, R_{c,a}\}, \{\neg R_{c,c}, \neg R_{c,b}, R_{c,b}\}, \{\neg R_{c,c}, \neg R_{c,c}, R_{c,c}\},
\end{array}\right\}$$

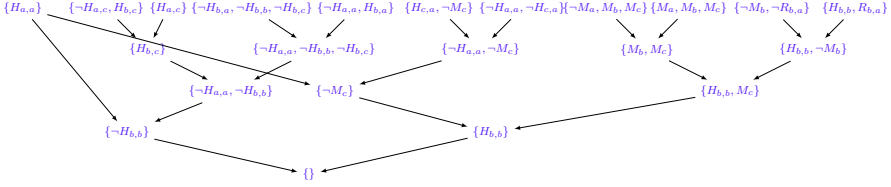
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Beispiel: „murder mystery“

- DPLL angewandt auf die Klauselmenge F führt auf eine erfüllende Belegung (OLR und PLR beginnend bei Fakten $H_{a,a}$, $\neg H_{a,b}$ und $H_{a,c}$ reichen aus), die besagt, dass Agatha Selbstmord begangen hat.

  (OLR und PLR sind Spezialfälle der Resolution.)

- Es bleibt zu überprüfen, ob dies die einzige Auflösung ist. Z.B. mittels Resolution kann geprüft werden, ob $G = (M_a \wedge \neg M_b \wedge \neg M_c)$ eine notwendige Folgerung aus F ist.



(Bereits die angegebene Teilmenge ist unerfüllbar.)

5 Logik

Aussagenlogik

Ausblick: Prädikatenlogik

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Ausblick: Prädikatenlogik

- Die Korrektheit der Inferenz

  Wenn (alle X sind Y) und (Z ist ein X), dann (Z ist ein Y)

  kann mit Aussagenlogik nicht nachgewiesen werden.
- Intuitiver Grund: die Aussagenlogik formalisiert nur die Begriffe „und“, „oder“, „nicht“, „wenn. . . dann“ aber nicht die Begriffe „für alle“ oder „ist“. Die Korrektheit des Argumentes hängt aber von Letzteren ab.
- Vergleiche:
  - Wenn (A und B) dann C
  - Wenn (alle A sind B und C ist A) dann (C ist B)
- Vereinfacht ausgedrückt ist Prädikatenlogik Aussagenlogik plus „für alle“ ($\forall$), „es gibt“ ($\exists$) und „ist“ (sog. Prädikate).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

1 Einführung

2 Grundlagen

3 Graphentheorie

4 Kombinatorik

5 Logik

6 Grundlagen der Gruppen- und Zahlentheorie

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

6 Grundlagen der Gruppen- und Zahlentheorie
Zahlentheoretische Grundlagen
Teilbarkeit, Primzahlen
Modulare Arithmetik
Erweiterter Euklidischer Algorithmus
Gruppen: Grundlagen
Ordnung eines Elements, zyklische Gruppen, Gruppenexponent
Untergruppen und Erzeugendensysteme
Homomorphismus und Isomorphismus
Appendix (*)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Zahlentheorie: Teilbarkeit und Primzahlen (Erinnerung)

Def $a|b$ **:gdw** $b \in a\mathbb{Z} := \{ka \mid k \in \mathbb{Z}\}$

- „a teilt b“, „a Teiler/Faktor von b“, „b teilbar durch a“, „b Vielfaches von a“
- $\mathsf{ggT}(a,b) := \max^{\leq}\{d \in \mathbb{N} : d|a, d|b\}$ und $\mathsf{kgV}(a,b) := \min^{\leq}\{m \in \mathbb{N} : a|m, b|m\}$
- $(\mathbb{N}, |)$ ist ein Verband: $\mathsf{ggT}(a,b) = \inf^{|}(a,b)$ und $\mathsf{kgV}(a,b) = \sup^{|}(a,b)$ (PFZ).
- $(\mathbb{N}_0, |)$ ist ein vollständiger Verband: $n|0$ und $\mathsf{ggT}(n,0) = n$ für alle $n \in \mathbb{N}_0$.

Def Primzahlen $\mathbb{P} := \{p \in \mathbb{N}_{\geq 2} \mid \text{für alle } n \in \mathbb{N}\text{: falls } n|p\text{, dann } n \in \{1,p\}\}$

- $\mathbb{P}$ sind die minimalen Elemente bzgl. $|$ eingeschränkt auf $\mathbb{N}_{\geq 2} = \mathbb{N} \setminus \{1\}$.
- „n prim“ := „$n \in \mathbb{P}$“ (genau zwei Teiler: $1 < n$; „erzeugen“ $|$)
- „n zusammengesetzt“ := „$n \in \mathbb{N} \setminus (\mathbb{P} \cup \{1\})$“ (mind. drei Teiler: $1 < d < n$)
- 1 nimmt Sonderrolle ein (genau ein Teiler; Teiler von allem).

Fkt Für jedes $n > 1$ gibt es ein $p \in \mathbb{P}$ mit $p|n$. (Zeige: $\min\{d > 1 : d|n\} \in \mathbb{P}$.)

Thm Satz des Euklid: $|\mathbb{P}| = |\mathbb{N}|$

- Annahme: Es gibt nur endlich viele Primzahlen $\mathbb{P}' = \{p_1, p_2, \ldots, p_m\} \ni 2$.
  Dann hätte $n := (p_1 \cdot p_2 \cdots p_m) + 1 \geq 3$ einen Primteiler p mit $p \notin \mathbb{P}'$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Primfaktorzerlegung

Thm Fundamentalsatz der Arithmetik: Für alle $n \in \mathbb{N}$ gilt:

$$n = \prod_{p \in \mathbb{P}} p^{\nu_p(n)}$$

mit $\nu_p(n) := \max\{k \in \mathbb{N}_0 \colon p^k | n\}$ die p-adische Ordnung von n.

⚠ Das Produkt ist endlich, da $\nu_p(n) = 0$ für alle $p \in \mathbb{P}$ mit $p > n$.

⚠ Die Primfaktorzerlegung ist (bis auf Kommutativität) eindeutig.

▷ n ist eine Multimenge von Primzahlen mit $\nu_p(n)$ die Vielfachheit von p.

Kor $n|m$ **gdw** $\nu_p(n) \leq \nu_p(m)$ für alle $p \in \mathbb{P}$.

Kor $\mathsf{ggT}(a,b) = \prod_{p \in \mathbb{P}} p^{\min\{\nu_p(a), \nu_p(b)\}} = \inf^{|}(a,b)$ und
$\mathsf{kgV}(a,b) = \prod_{p \in \mathbb{P}} p^{\max\{\nu_p(a), \nu_p(b)\}} = \sup^{|}(a,b)$.

Def $a, b \in \mathbb{Z}$ sind koprim **:gdw** $\mathsf{ggT}(a,b) = 1$ (kein gemeinsamer Primteiler).

▷ Beispiel: $a = 360 = 2^3 \cdot 3^2 \cdot 5^1$, $b = 126 = 2^1 \cdot 3^2 \cdot 7^1$
$\mathsf{ggT}(a,b) = 2^1 \cdot 3^2 \cdot 5^0 \cdot 7^0 = 18$ und $\mathsf{kgV}(a,b) = 2^3 \cdot 3^2 \cdot 5^1 \cdot 7^1 = 2520$
$\mathsf{ggT}(a,b) \cdot \mathsf{kgV}(a,b) = 45360 = ab$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Primfaktorzerlegung

Lem Falls $\mathsf{ggT}(a,b) = 1$, dann $\mathsf{ggT}(a^k, b^l) = 1$ für alle $k, l \in \mathbb{N}$ (siehe EEA).

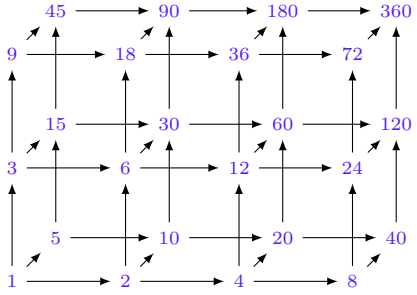
(„Positive Potenzen koprimer Zahlen bleiben koprim.“)

Thm Für alle $n \in \mathbb{N}$ gilt $n = \prod_{p \in \mathbb{P}} p^{\nu_p(n)}$ mit $\nu_p(n) := \max\{k \in \mathbb{N}_0 \colon p^k | n\}$.

- Beweisskizze (starke Induktion bzgl. $n \in \mathbb{N}$):
  - Falls $n = 1$, offensichtlich.
  - Falls $n > 1$, fixiere beliebigen Primteiler $p|n$ und setze $m := n/p$.
    Induktiv hat m eine Primfaktorzerlegung.
    Da $n = pm$ existiert eine Primfaktorzerlegung $n = p_1^{e_1} \cdots p_l^{e_l}$ ($p_i \in \mathbb{P}$, $e_i > 0$).
    (OBdA. sind die Primfaktoren sortiert $p_1 < \ldots < p_l$.)
  - Noch zu zeigen: $N := \prod_{p \in \mathbb{P}} p^{\nu_p(n)} \stackrel{!}{=} n = p_1^{e_1} \cdots p_l^{e_l}$:
    (Beachte: die LS betrachtet jeden Primteiler für sich allein.)
    Da $p_i^{e_i} | n$, folgt $0 < e_i \leq \nu_p(n)$ für $p := p_i$, und damit $n|N$.
    Für $N|n$ betrachte wieder einen bel. Primteiler $p|n$ ($\nu_p(n) > 0$):
    Da $p^{\nu_p(n)}|n$ und alle $p_i^{e_i}$ paarweise koprim, muss es ein eindeutiges i geben mit $p = p_i$ und $\nu_p(n) \leq e_i$ (siehe oben).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Erinnerung: Teilerverband (*)



- Hasse-Diagramm des Teilerverbands von $n = 360$:

  kgV(a, b): kleinste Zahl, die von a und von b erreicht werden kann (sup).

  ggT(a, b): größte Zahl, die a und b erreichen kann (inf).

- $\mathbb{N}$ als „Vektorraum mit $\nu_p(n)$ Koordinate von n bzgl. p“. ($n \in \mathbb{N}$ ist eine Multimenge von Primzahlen.)

  Häufig $0|0$ und ggT$(0, 0) = 0$ und $\nu_p(0) = \infty$ (0 als größtes Element).

  m, n koprim (ggT$(m, n) = 1$) entspricht „orthogonal“.

- $|\mathbb{N}| = |\bigcup_{k\in\mathbb{N}_0} \mathbb{N}_0^k|$ mittels z.B. $(n_1, \ldots, n_k) \mapsto 2^k \cdot 3^{n_1} \cdots p_{k+1}^{n_k}$ (Gödelisierung)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def (Erinnerung) $a \equiv_n b$ **gdw**: $n|(b-a)$ **gdw** $\frac{b-a}{n} \in \mathbb{Z}$ **gdw** $b - a \in n\mathbb{Z}$

- „selber Rest modulo n“, „a kongruent b modulo n“ (Gauß).
- Äquivalenzrelation auf $\mathbb{Z}$ für jedes gegebene $n \in \mathbb{N}$.
- Äquivalenz-/Rest-klassen: $[a]_n := \{a + nz \mid z \in \mathbb{Z}\} =: a + n\mathbb{Z}$
- Standard-Repräsentantensystem: $\mathbb{Z}_n := \{0, 1, \ldots, n-1\}$
  Alternatives Repräsentantensystem: $\{-\lfloor \frac{n}{2} \rfloor, \ldots, \lfloor \frac{n-1}{2} \rfloor\}$
  (Z.B. unsigned int vs. signed int für $n = 2^m$.)
- Quotient: $\mathbb{Z}/n\mathbb{Z} := \mathbb{Z}/\equiv_n = \{r + n\mathbb{Z} \mid r \in \mathbb{Z}_n\}$.
- (i) $\equiv_m \cap \equiv_n = \equiv_{\mathsf{kgV}(m,n)}$; (ii) $n|m$ **gdw** $\equiv_m \subseteq \equiv_n$

Def „$a \bmod n$“ ist der eindeutige Rest $r \in \mathbb{Z}_n$ mit $a \equiv_n r$ („a modulo n“)

⚠ In manchen Prog.-Sprachen abweichende Def., siehe [hier] und [hier].

Fkt $a \bmod n = a - \lfloor \frac{a}{n} \rfloor n = \min\{r \in \mathbb{N}_0 \colon a \equiv_n r\}$ $(n \in \mathbb{N})$

- $\lfloor x \rfloor := \max\{k \in \mathbb{Z} \mid k \leq x\}$ für $x \in \mathbb{R}$ (abrunden, „floor function“).

Fkt $a \equiv_n b$ **gdw** $(a \bmod n) = (b \bmod n)$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

⚠ Addition und Multiplikation auf $\mathbb{Z}$ sind mit $\equiv_n$ verträglich, z.B.:

$$\begin{array}{lll} & (5^5 + 8 \cdot 11) \bmod 13 & \\ \equiv_{13} & 5^5 + 8 \cdot 11 & (11 \equiv_{13} -2) \\ \equiv_{13} & (25)^2 \cdot 5 + 8 \cdot (-2) & (25 \equiv_{13} -1) \\ \equiv_{13} & (-1)^2 \cdot 5 - 16 & (16 \equiv_{13} 3) \\ \equiv_{13} & 5 - 3 & \\ = & 2 & \end{array}$$

d.h. man darf zu jedem Wert a ein beliebiges Vielfaches von n addieren, da nur der Rest $a \bmod n$ wichtig ist.

▷ Beispiel: $n = \sum_{i=0}^{k} d_i 10^i$ mit $d_i \in \{0, 1, \ldots, 9\}$

$$n \equiv_3 \sum_{i=0}^{k} d_i 10^i \equiv_3 \sum_{i=0}^{k} d_i (9+1)^i \equiv_3 \sum_{i=0}^{k} d_i$$

Daher: $3|n$ **gdw** $3|\sum_{i=0}^{k} d_i$.

▷ Zweierkomplement: mit $x = \sum_{i=1}^{m} x_i 2^{m-i}$ ($x_i \in \mathbb{Z}_2$) und $n = 2^m$.

$$-x \equiv_n 2^m - x = 1 + (2^m - 1) - x = 1 + \sum_{i=1}^{m} (1 - x_i) 2^{m-i}$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Lem $\equiv_n$ ist mit $+$ und $\cdot$ verträglich, d.h. mit $a \equiv_n a'$ und $b \equiv_n b'$ gilt

$$a + b \equiv_n a' + b' \equiv_n (a + b) \bmod n \quad \text{und} \quad a \cdot b \equiv_n a' \cdot b' \equiv_n (a \cdot b) \bmod n$$

Beweis: Es gelte $a' = a + kn$ und $b' = b + ln$. Dann folgt:

- $a' + b' = (a + nk) + (b + nl) = a + b + n(k + l) \equiv_n a + b$
- $a'b' = (a + nk) \cdot (b + nl) = ab + n(al + bk + nkl) \equiv_n ab$

Kor Gleichbedeutend: Falls $[a]_n = [a']_n$ und $[b]_n = [b']_n$, dann

$$[a+b]_n = [a'+b']_n = [(a+b) \bmod n]_n \quad \text{und} \quad [a \cdot b]_n = [a' \cdot b']_n = [(a \cdot b) \bmod n]_n$$

d.h. Addition und Multiplikation von Restklassen sind wohldefiniert durch:

$$[a]_n + [b]_n := [a + b]_n \quad \text{und} \quad [a]_n \cdot [b]_n := [a \cdot b]_n$$

das Bild hängt nur von $[a]_n, [b]_n$, aber nicht von $a' \in [a]_n, b' \in [b]_n$ ab.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Koprime Reste modulo n: $\mathbb{Z}_n^*$

Def Sei $n \in \mathbb{N}$ mit $n > 1$. $\mathbb{Z}_n^*$ bezeichnet die koprimen Reste modulo n:

$$\mathbb{Z}_n^* := \{k \in \mathbb{Z}_n \mid \mathsf{ggT}(k,n) = 1\} = \mathbb{Z}_n \setminus \bigcup_{p \in \mathbb{P}\colon p \mid n} p\mathbb{Z}_{\frac{n}{p}}$$

Def Eulersche Phi-Funktion $\varphi(n) := |\mathbb{Z}_n^*|$ (Erinnerung).

- ▷ Z.B.: $\varphi(p^k) = |\mathbb{Z}_{p^k}^*| = |\mathbb{Z}_{p^k} \setminus p\mathbb{Z}_{p^{k-1}}| = p^k - p^{k-1} = p^k(1-p^{-1})$ für $p \in \mathbb{P}$.
- ▷ Z.B.: $\varphi(15) = |\mathbb{Z}_{15}^*| = \left|\mathbb{Z}_{15} \setminus (3\mathbb{Z}_{\frac{15}{3}} \cup 5\mathbb{Z}_{\frac{15}{5}})\right| = |\{1,2,4,7,8,11,13,14\}| = 8$.

Lem $\varphi(n) = |\mathbb{Z}_n^*| = n \prod_{p \in \mathbb{P}\colon p \mid n} (1 - p^{-1})$ (mittels Siebformel)

Kor $\varphi(mn) = \varphi(m)\varphi(n)$ falls $\mathsf{ggT}(m,n) = 1$.

(da $\{p \in \mathbb{P}\colon p \mid mn\} = \{p \in \mathbb{P}\colon p \mid m\} \uplus \{p \in \mathbb{P}\colon p \mid n\}$; „Lemma von Euklid“; siehe EEA)

- ▷ Z.B.: $\varphi(15) = 15(1-\frac{1}{5})(1-\frac{1}{3}) = (5-1)(3-1) = 8$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Erweiterter euklidischer Algorithmus (EEA)

Lem Eigenschaften des ggT:

- ▷ $\mathrm{ggT}(0,a) = |a|$ und $\mathrm{ggT}(1,a) = 1$ (mit $\mathrm{ggT}(0,0) = 0$).
- ▷ $\mathrm{ggT}(a,b) = \mathrm{ggT}(|a|,|b|)$ und $\mathrm{ggT}(a,b) = \mathrm{ggT}(b,a)$.
- ▷ $\mathrm{ggT}(a,b) = \mathrm{ggT}(a,a+b) = \mathrm{ggT}(a,a-b)$.

  Beweis: Für alle $d \in \mathbb{N}$ gilt:

  $a = kd$ und $b = ld$

  gdw $a = kd$ und $a+b = nd$ (mit $n = k+l$)

  gdw $a = kd$ und $a-b = md$ (mit $m = k-l$)

  Damit haben a,b und $a, a \pm b$ dieselben Teiler und damit auch denselben ggT.

Kor $\mathrm{ggT}(a,b) = \mathrm{ggT}(a, b - \lfloor b/a \rfloor a) = \mathrm{ggT}(b \bmod a, a)$ für $a > 0$.

⚠ Damit kann $\mathrm{ggT}(a,b)$ rekursiv ohne Kenntnis der Primfaktorzerlegungen von a und b berechnet werden. Man kann zeigen, dass nur $\mathcal{O}(\log(a+b))$ rekursive Aufrufe benötigt werden.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Erweiterter euklidischer Algorithmus (EEA)

Lem Lemma von Bézout: Für alle $a, b \in \mathbb{Z}$ existieren stets $\alpha, \beta \in \mathbb{Z}$ mit

$$\mathsf{ggT}(a, b) = \alpha \cdot a + \beta \cdot b$$

Beweis: α, β können mittels des erweiterten euklidischen Algorithmus berechnet werden. (Zur Vereinfachung $0 \leq a \leq b$; starke Induktion nach a.)

- Falls $a = 0$: $\mathsf{ggT}(0, b) = b = 0 \cdot a + 1 \cdot b$, d.h. $\alpha = 0, \beta = 1$.
- Falls $a > 0$: Dann gilt $b \bmod a < a$ und es existieren induktiv $\alpha', \beta' \in \mathbb{Z}$ mit

$$\mathsf{ggT}(b \bmod a, a) = \alpha'(b \bmod a) + \beta' a$$

Mit $\mathsf{ggT}(a, b) = \mathsf{ggT}(b \bmod a, a)$ und $b \bmod a = b - ka$ für $k = \lfloor b/a \rfloor$ folgt

$$\mathsf{ggT}(a, b) = \mathsf{ggT}(b \bmod a, a) = \alpha' \underbrace{(b \bmod a)}_{=b-ka} + \beta' a = \underbrace{(\beta' - k\alpha')}_{=:\alpha} a + \underbrace{\alpha'}_{=:\beta} b$$

- Falls $b \bmod a = 0$, dann gilt $\mathsf{ggT}(a, b) = \mathsf{ggT}(0, a) = a = 1 \cdot a + 0 \cdot b$, d.h. einfach $\alpha = 1$ und $\beta = 0$ statt Rekursion zu $\mathsf{ggT}(0, a)$.

Kor $a\mathbb{Z} + b\mathbb{Z} = \mathsf{ggT}(a, b)\mathbb{Z}$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Erweiterter euklidischer Algorithmus (EEA)

Gegeben: $1 \leq a \leq b$ mit $a, b \in \mathbb{N}$

- Falls $(b \bmod a) = 0$, gib $\alpha = 1, \beta = 0$ zurück.

  (Alternativ: falls $a = 0$, gib $\alpha = 0, \beta = 1$ zurück.)

- Ansonsten berechne rekursiv α', β' mit

$$\mathsf{ggT}(b \bmod a, a) = (b \bmod a)\alpha' + a\beta'$$

  Gib dann $\alpha = (\beta' - \lfloor b/a \rfloor \alpha'), \beta = \alpha'$ zurück.

- Beispiel: $a = 5, b = 911$ mit $\mathsf{ggT}(5, 911) = 1$ (911 ist prim)

a	b	$\lfloor b/a \rfloor$	α	β
5	911	182		

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

EEA – Beispiel

Def Erweiterter euklidischer Algorithmus (EEA)

Gegeben: $1 \leq a \leq b$ mit $a, b \in \mathbb{N}$

- Falls $(b \bmod a) = 0$, gib $\alpha = 1, \beta = 0$ zurück.

  (Alternativ: falls $a = 0$, gib $\alpha = 0, \beta = 1$ zurück.)

- Ansonsten berechne rekursiv α', β' mit

$$\mathsf{ggT}(b \bmod a, a) = (b \bmod a)\alpha' + a\beta'$$

  Gib dann $\alpha = (\beta' - \lfloor b/a \rfloor \alpha'), \beta = \alpha'$ zurück.

- Beispiel: $a = 5, b = 911$ mit $\mathsf{ggT}(5, 911) = 1$ (911 ist prim)

a	b	$\lfloor b/a \rfloor$	α	β
5	911	182		
1	5	$-$		

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

EEA – Beispiel

Def Erweiterter euklidischer Algorithmus (EEA)

Gegeben: $1 \leq a \leq b$ mit $a, b \in \mathbb{N}$

- Falls $(b \bmod a) = 0$, gib $\alpha = 1, \beta = 0$ zurück.

  (Alternativ: falls $a = 0$, gib $\alpha = 0, \beta = 1$ zurück.)

- Ansonsten berechne rekursiv α', β' mit

$$\mathsf{ggT}(b \bmod a, a) = (b \bmod a)\alpha' + a\beta'$$

  Gib dann $\alpha = (\beta' - \lfloor b/a \rfloor \alpha'), \beta = \alpha'$ zurück.

- Beispiel: $a = 5, b = 911$ mit $\mathsf{ggT}(5, 911) = 1$ (911 ist prim)

a	b	$\lfloor b/a \rfloor$	α	β
5	911	182		
1	5	–	1	0

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Erweiterter euklidischer Algorithmus (EEA)

Gegeben: $1 \leq a \leq b$ mit $a, b \in \mathbb{N}$

- Falls $(b \bmod a) = 0$, gib $\alpha = 1, \beta = 0$ zurück.

  (Alternativ: falls $a = 0$, gib $\alpha = 0, \beta = 1$ zurück.)

- Ansonsten berechne rekursiv α', β' mit

$$\mathsf{ggT}(b \bmod a, a) = (b \bmod a)\alpha' + a\beta'$$

  Gib dann $\alpha = (\beta' - \lfloor b/a \rfloor \alpha'), \beta = \alpha'$ zurück.

- Beispiel: $a = 5, b = 911$ mit $\mathsf{ggT}(5, 911) = 1$ (911 ist prim)

a	b	$\lfloor b/a \rfloor$	α	β
5	911	182	-182	1
1	5	$-$	1	0

- Test: $1 = \mathsf{ggT}(5, 911) \stackrel{!}{=} 5 \cdot (-182) + 911 \cdot 1 = 1$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Lemma von Euklid

Lem Lemma von Euklid: Falls $d|ab$ und $\mathsf{ggT}(d,a)=1$, dann $d|b$.

da: Aus $d|ab$ und $\mathsf{ggT}(d,a)=\alpha a+\delta d=1$ folgt $b=b(\alpha a+\delta d)=(\alpha\frac{ab}{d}+\delta b)d\in d\mathbb{Z}$.

Kor

1. Falls $p|ab$ und $p\in\mathbb{P}$, dann $p|a$ oder $p|b$.
2. Falls $\mathsf{ggT}(a,b)=1$, dann $\mathsf{ggT}(a^k,b^l)=1$ für alle $k,l\in\mathbb{N}$.
   (Kontraposition: Falls $p|a^k$ und $p|b^l$, dann induktiv $p|a$ und $p|b$.)
3. $\mathsf{ggT}(a,n)=\mathsf{ggT}(b,n)=1$ **gdw** $\mathsf{ggT}(ab,n)=1$ **gdw** $\mathsf{ggT}(ab \bmod n,n)=1$.
   (Kontraposition: $p|n$ und ($p|a$ oder $p|b$) **gdw** $p|n$ und $p|ab$.)
4. Falls $1=\mathsf{ggT}(a,n)=a\alpha+n\beta$, dann $a\alpha\equiv_n 1$ und

$$ax+b\equiv_n 0 \quad \textbf{gdw} \quad x\equiv_n -\alpha b$$

Bsp $5x\equiv_{911}12$ **gdw** $x\equiv_{911}(-182)\cdot 12\equiv_{911}549$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

6 Grundlagen der Gruppen- und Zahlentheorie

Zahlentheoretische Grundlagen

Gruppen: Grundlagen

Ordnung eines Elements, zyklische Gruppen, Gruppenexponent

Untergruppen und Erzeugendensysteme

Homomorphismus und Isomorphismus

Appendix (*)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Definition einer Gruppe

Def Gruppe: multiplikative vs. additive Notation ($1 \in \mathbb{G}$ bzw. $0 \in \mathbb{G}$)

$$\mathbb{G} \hat{=} (\mathbb{G}, \cdot, 1) \qquad \mathbb{G} \hat{=} (\mathbb{G}, +, 0)$$

ist eine Gruppe, falls

1		$\cdot : \mathbb{G} \times \mathbb{G} \to \mathbb{G}$	$+ : \mathbb{G} \times \mathbb{G} \to \mathbb{G}$	(abgeschlossen)
2	f.alle a, b, c:	$(a \cdot b) \cdot c = a \cdot (b \cdot c)$	$(a + b) + c = a + (b + c)$	(assoziativ)
3	f.alle a:	$a \cdot 1 = 1 \cdot a = a$	$a + 0 = 0 + a = a$	(Neutrales)
4	f.alle a ex. b:	$a \cdot b = b \cdot a = 1$	$a + b = b + a = 0$	(Inverses)

$\mathbb{G}$ ist kommutativ (abelsch), falls zusätzlich

5	f.alle a, b:	$a \cdot b = b \cdot a$	$a + b = b + a$	(kommutativ)

⚠ $\cdot$ und $+$ bezeichnen i.A. beliebige Operationen auf $\mathbb{G}$.

▷ Additive Notation $(\mathbb{G}, +, 0)$ nur für kommutative (abelsche) Gruppen.
Multiplikative Notation $(\mathbb{G}, \cdot, 1)$ für allgemeine (d.h. u.U. nicht kommutative) Gruppen; wie gewohnt, ab kurz für $a \cdot b$. Im Folgenden daher meistens $(\mathbb{G}, \cdot, 1)$.

Def $\mathbb{G}$ ist endlich **:gdw** $|\mathbb{G}| < \infty$. $|\mathbb{G}|$ wird auch die (Gruppen)ordnung genannt.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Definition einer Gruppe

Def Gruppe: multiplikative vs. additive Notation ($1 \in \mathbb{G}$ bzw. $0 \in \mathbb{G}$)

$$\mathbb{G} \hat{=} (\mathbb{G}, \cdot, 1) \qquad \mathbb{G} \hat{=} (\mathbb{G}, +, 0)$$

ist eine Gruppe, falls

1		$\cdot\colon \mathbb{G} \times \mathbb{G} \to \mathbb{G}$	$+\colon \mathbb{G} \times \mathbb{G} \to \mathbb{G}$	(abgeschlossen)
2	f.alle a, b, c:	$(a \cdot b) \cdot c = a \cdot (b \cdot c)$	$(a + b) + c = a + (b + c)$	(assoziativ)
3	f.alle a:	$a \cdot 1 = 1 \cdot a = a$	$a + 0 = 0 + a = a$	(Neutrales)
4	f.alle a ex. b:	$a \cdot b = b \cdot a = 1$	$a + b = b + a = 0$	(Inverses)

$\mathbb{G}$ ist kommutativ (abelsch), falls zusätzlich

5	f.alle a, b:	$a \cdot b = b \cdot a$	$a + b = b + a$	(kommutativ)

Def $H \subseteq \mathbb{G}$ ist eine Untergruppe von $(\mathbb{G}, \cdot, 1)$ (kurz: $H \leq \mathbb{G}$) **:gdw** $(H, \cdot, 1)$ ist eine Gruppe (selbe Operation).

Def Eine Abbildung $h\colon \mathbb{G}_1 \to \mathbb{G}_2$ zwischen zwei Gruppen $(\mathbb{G}_i, \cdot_i, 1_i)$ ist ein (Gruppen-)Homomorphismus **:gdw** $h(a \cdot_1 b) = h(a) \cdot_2 h(b)$ für alle $a, b \in \mathbb{G}_1$.

Ein (Gruppen-)Isomorphismus ist ein bijektiver Homomorphismus.

$\mathbb{G}_1$ und $\mathbb{G}_2$ sind isomorph ($\mathbb{G}_1 \cong \mathbb{G}_2$) **:gdw** es gibt einen Isomorphismus.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gruppen bzgl. kanonischer Addition und Multiplikation

- Bzgl. kanonischer Addition $+\colon \mathbb{R} \times \mathbb{R} \to \mathbb{R}$:

$$(n\mathbb{Z}, +, 0) \leq (\mathbb{Z}, +, 0) \leq (\mathbb{Q}, +, 0) \leq (\mathbb{R}, +, 0)$$

  Alle kommutativ/abelsch, unendlich, mit $-x$ als Inverses.

  - $n\mathbb{Z} = \{nz \mid z \in \mathbb{Z}\}$ für festes $n \in \mathbb{N}$.
  - $(\mathbb{N}_0, +, 0)$ ist keine Gruppe: nur 0 hat ein Inverses.
  - $(\mathbb{Z}_n, +, 0)$ ist keine Gruppe: nicht abgeschlossen.

- Bzgl. kanonischer Multiplikation $\cdot\colon \mathbb{R} \times \mathbb{R} \to \mathbb{R}$:

$$(\mathbb{Q}_{>0}, \cdot, 1) \leq \begin{matrix}(\mathbb{Q} \setminus \{0\}, \cdot, 1) \\ (\mathbb{R}_{>0}, \cdot, 1)\end{matrix} \leq (\mathbb{R} \setminus \{0\}, \cdot, 1)$$

  Alle kommutativ, unendlich, x^{-1} mittels schriftlicher Division.

  - $(\mathbb{R}, +, 0) \cong (\mathbb{R}_{>0}, \cdot, 1)$ mittels $h(x) = e^x$ bzw. $h^{-1}(x) = \log x$
  - $(\mathbb{Z} \setminus \{0\}, \cdot, 1)$ ist keine Gruppe: nur ± 1 besitzen Inverse.
  - $(\mathbb{Z}_n^*, \cdot, 1)$ ist keine Gruppe: nicht abgeschlossen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gruppen auf $\mathbb{Z}_n$

Def $(\mathbb{Z}_n, +_n, 0)$ additive Gruppe modulo $n \in \mathbb{N}$

- Grundmenge: $\mathbb{Z}_n = \{0, 1, \ldots, n-1\}$
- Gruppenoperation: $a +_n b := (a + b) \bmod n$
- Neutrales: 0
- Inverse: 0 für 0, sonst $n - a$ für $a \in \mathbb{Z}_n \setminus \{0\}$

Def $(\mathbb{Z}_n^*, \cdot_n, 1)$ multiplikative Gruppe modulo $n \geq 2$

- Grundmenge: $\mathbb{Z}_n^* = \{x \in \mathbb{Z}_n \mid \mathsf{ggT}(x, n) = 1\}$
- Gruppenoperation: $a \cdot_n b := (a \cdot b) \bmod n$
  Abschluss: $\mathsf{ggT}(a, n) = 1 = \mathsf{ggT}(b, n)$ **gdw** $1 = \mathsf{ggT}(ab, n) = \mathsf{ggT}(ab \bmod n, n)$
- Neutrales: 1
- Inverse: $\alpha \bmod n$ invers zu a mit $1 = \mathsf{ggT}(a, n) = \alpha \cdot a + \beta \cdot n$ (EEA)

- Assoziativ: $a +_n (b +_n c) \equiv_n a + b + c$ und $a \cdot_n (b \cdot_n c) \equiv_n a \cdot b \cdot c$
- Wie im Beweis der Assoziativität, ist es häufig einfacher, bzgl. $\equiv_n$ zu rechnen, statt ein festes Repräsentantensystem wie $\mathbb{Z}_n$ zu verwenden.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gruppen auf $\mathbb{Z}/n\mathbb{Z}$

Def $(\mathbb{Z}/n\mathbb{Z}, +, [0]_n)$

- ▷ Grundmenge: $\mathbb{Z}/n\mathbb{Z} = \{[x]_n : x \in \mathbb{Z}\} = \{0 + n\mathbb{Z}, 1 + n\mathbb{Z}, \ldots, (n-1) + n\mathbb{Z}\}$
- ▷ Gruppenoperation: $[a]_n + [b]_n := [a + b]_n$
- ▷ Neutrales: $[0]_n = 0 + n\mathbb{Z}$
- ▷ Inverse: $[-a]_n$

Def $((\mathbb{Z}/n\mathbb{Z})^*, \cdot, [1]_n)$

- ▷ Grundmenge: $(\mathbb{Z}/n\mathbb{Z})^* = \{[x]_n : x \in \mathbb{Z}, \mathsf{ggT}(x, n) = 1\}$
- ▷ Gruppenoperation: $[a]_n \cdot [b]_n := [a \cdot b]_n$
- ▷ Neutrales: $[1]_n = 1 + n\mathbb{Z}$
- ▷ Inverses von $[a]_n$ mittels EEA: falls $1 = \mathsf{ggT}(a, n) = \alpha \cdot a + \beta \cdot n$, dann $[\alpha]_n$.

Lem $(\mathbb{Z}_n, +_n, 0)$ und $(\mathbb{Z}/n\mathbb{Z}, +, [0]_n)$ bzw. $(\mathbb{Z}_n^*, \cdot_n, 1)$ und $((\mathbb{Z}/n\mathbb{Z})^*, \cdot, [1]_n)$ sind bis auf Isomorphie dieselben Gruppen. Wechsel zwischen Gruppen mittels $[a]_n \mapsto a \bmod n$ und $a \mapsto [a]_n = a + n\mathbb{Z}$. (Restklasse vs. konkreter Rest als Repräsentant seiner Restklasse.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Multiplikative Gruppe modulo n: $(\mathbb{Z}_n^*, \cdot_n, 1)$

⚠ Nur die koprimen Reste modulo n haben ein Inverses bzgl. $\cdot_n$:

$\cdot_6$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	1	2	3	4	5
2	0	2	4	0	2	4
3	0	3	0	3	0	3
4	0	4	2	0	4	2
5	0	5	4	3	2	1

Erinnerung (siehe EEA/Bézout):

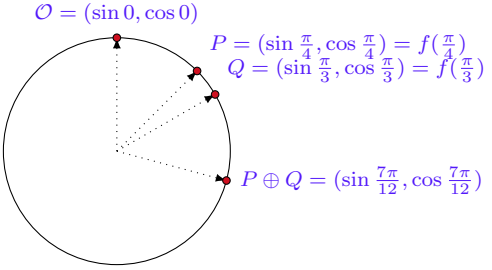
- ggT$(a, n) =$ ggT$(b, n) = 1$ **gdw** ggT$(ab, n) = 1$.
- ggT$(a \cdot_n b, n) =$ ggT$(ab \bmod n, n) =$ ggT(ab, n)

⚠ Aus $a \cdot_n b = 1$ folgt daher ggT$(a, n) =$ ggT$(b, n) = 1$.

Und mit $a, b \in \mathbb{Z}_n$ also auch $a, b \in \mathbb{Z}_n^*$.

- Falls $1 <$ ggT$(a, n) =: d$, dann $a \cdot_n \frac{n}{d} = \frac{a}{d} \cdot_n n = 0$. (Nullteiler)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

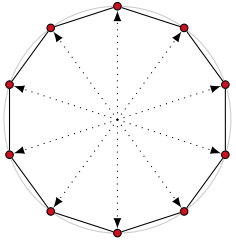


Def Kreisgruppe: Addition auf Einheitskreis bzgl. $f(t) := (\sin t, \cos t)$

- Grundmenge: $\mathsf{S}^1 := \{(x, y) \in \mathbb{R}^2 \mid x^2 + y^2 = 1\} = \{(\sin t, \cos t) \mid t \in \mathbb{R}\}$
- Gruppenoperation: $f(t_1) \oplus f(t_2) := f(t_1 + t_2)$
  Mittels Additionstheoremen für $\sin$ und $\cos$ direkt ohne $f(t)$:
  $(x_1, y_1) \oplus (x_2, y_2) := (x_1 y_2 + x_2 y_1, y_1 y_2 - x_1 x_2)$
- Neutrales: $\mathcal{O} := f(0) = (0, 1)$ („12 Uhr“)
- Inverse: $-f(t) := f(-t)$

Isomorph zu $([0, 2\pi), +_{2\pi}, 0)$ mittels f und zu $([0, 1), +_1, 0)$ mittels $f(2\pi t)$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science



⚠ Für jedes $n \in \mathbb{N}$ definiert

$$\mathsf{S}^1[n] := \{(\sin(\tfrac{2\pi k}{n}), \cos(\tfrac{2\pi k}{n})) \mid k \in \mathbb{Z}_n\} = \{P \in \mathsf{S}^1 \mid nP = \mathcal{O}\}$$

eine Untergruppe von S^1, welche mittels $h_n(k) := (\sin(\tfrac{2\pi k}{n}), \cos(\tfrac{2\pi k}{n}))$ isomorph zu $(\mathbb{Z}_n, +_n, 0)$ ist. (Siehe zyklische Gruppen später.)

▷ Siehe auch Einheitswurzeln (lineare Algebra), Elliptische Kurven, Edwards-Kurven.

Direkte Produkte von Gruppen

▷ $(\mathbb{R}^n, +, 0)$ ist eine kommutative Gruppe bzgl. der komponentenweisen Addition:

$$(x_1, \ldots, x_n) + (y_1, \ldots, y_n) := (x_1 + y_1, \ldots, x_n + y_n)$$

Nach Definition rechnet man in jeder Komponente unabhängig (parallel) von den anderen Komponenten einfach in $(\mathbb{R}, +, 0)$. Die Konstruktion lässt sich somit direkt auf beliebige Gruppen übertragen:

Def Sei I eine Menge und für jedes $i \in I$ sei $(\mathbb{G}_i, \cdot_i, 1_i)$ eine Gruppe.
Dann ist das direkte Produkt $\prod_{i \in I} \mathbb{G}_i$ die Gruppe mit

- ▷ Grundmenge: $\bigtimes_{i \in I} \mathbb{G}_i$
- ▷ Gruppenoperation: $(x_i)_{i \in I} \cdot (y_i)_{i \in I} = (x_i \cdot_i y_i)_{i \in I}$
- ▷ Neutrales: $1 = (1_i)_{i \in I}$
- ▷ Inverse: $(x_i)_{i \in I}^{-1} = (x_i^{-1})_{i \in I}$

Ist $I = [n]$, dann schreibt man dafür auch einfach $\mathbb{G}_1 \times \ldots \times \mathbb{G}_n$.
Spezialfälle: Die Menge der Funktionen $\mathbb{G}^I$ und $\mathbb{G}^n$.
(Im Fall endlich vieler Gruppen spricht man auch von der direkten Summe; im unendlichen Fall ist die direkte Summe eine echte Untergruppe des direkten Produkts.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Symmetrische Gruppe

Def $(\mathcal{S}_A, \circ, \mathrm{Id}_A)$ symmetrische Gruppe über A.

- ▷ Grundmenge: $\mathcal{S}_A = \{f \colon A \to A \mid f \text{ bijektiv } \}$
- ▷ Gruppenoperation: $f \circ g \colon A \to A, a \mapsto f(g(a))$ (Funktionskomposition)
- ▷ Neutrales: Identität $\mathrm{Id}_A \colon A \to A, a \mapsto a$
- ▷ Inverse: Umkehrabbildung

$\mathcal{S}_n$ kurz für die Menge $\mathcal{S}_{[n]}$ aller Permutationen von $[n]$.

⚠ Die symmetrische Gruppe ist nur für $|A| < 3$ kommutativ.

▷ Mittels der Zykelnotation kann man $\mathcal{S}_A$ auch als Untergruppe von $\mathcal{S}_B$ auffassen, falls $A \subseteq B$. (Elemente aus $B \setminus A$ bleiben fix.)

▷ Ist $\pi \in \mathcal{S}_A$ in Zykelnotation gegeben, erhält man das Inverse durch Umkehren der Zykel.

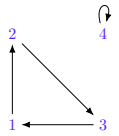
⚠ Die Automorphismen (Symmetrien) eines Graphen mit n Knoten bilden eine Untergruppe von $\mathcal{S}_n$. (Siehe z.B. die Diedergruppen als Symmetrien des Kreisgraphen).

Thm Satz von Cayley: Jede Gruppe $\mathbb{G}$ ist isomorph zu einer Untergruppe von $\mathcal{S}_{\mathbb{G}}$. (Siehe nach Kürzungsregel.)

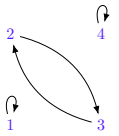
No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Erinnerung: Zykelschreibweise

$$f = \begin{pmatrix} 1 \mapsto 2 \\ 2 \mapsto 3 \\ 3 \mapsto 1 \\ 4 \mapsto 4 \end{pmatrix} = (1,2,3)\,(4) =$$



$$g = \begin{pmatrix} 1 \mapsto 1 \\ 2 \mapsto 3 \\ 3 \mapsto 2 \\ 4 \mapsto 4 \end{pmatrix} = (1)\,(2,3)\,(4) =$$



- Die Zykel können beliebig angeordnet werden, die Elemente in einem Zykel können zyklisch rotiert werden, z.B.:

$$(2)(1,3,5)(4) = (2)(4)(5,1,3) = (3,5,1)(4)(2) = (1,3,5)$$

  häufig gibt man Fixpunkte (Kreise der Länge 1) nicht explizit an.

- Beispiel: (Funktionskomposition $\circ$ als Gruppenoperation!)

$$f \circ g = (1,2,3) \circ (2,3) = (2,1) \qquad g \circ f = (2,3) \circ (1,2,3) = (1,3)$$

$$f \circ f = (1,2,3) \circ (1,2,3) = (1,3,2) \qquad g \circ g = (2,3) \circ (2,3) = \mathsf{Id}$$

- Inverses ergibt sich durch Umdrehen der Tupel/Kreise:

$$f^{-1} = (3,2,1) = (1,3,2) \qquad g^{-1} = (3,2) = (2,3)$$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

▷ Beispiel: $(\mathcal{S}_3, \circ, \mathsf{Id})$ mit Fixpunkte (Kreise der Länge 1) nur implizit.

	$\mathsf{Id}_{[3]}$	$(1,2)$	$(1,3)$	$(2,3)$	$(1,2,3)$	$(1,3,2)$
$\mathsf{Id}_{[3]}$	$\mathsf{Id}_{[3]}$	$(1,2)$	$(1,3)$	$(2,3)$	$(1,2,3)$	$(1,3,2)$
$(1,2)$	$(1,2)$	$\mathsf{Id}_{[3]}$	$(1,3,2)$	$(1,2,3)$	$(2,3)$	$(1,3)$
$(1,3)$	$(1,3)$	$(1,2,3)$	$\mathsf{Id}_{[3]}$	$(1,3,2)$	$(1,2)$	$(2,3)$
$(2,3)$	$(2,3)$	$(1,3,2)$	$(1,2,3)$	$\mathsf{Id}_{[3]}$	$(1,3)$	$(1,2)$
$(1,2,3)$	$(1,2,3)$	$(1,3)$	$(2,3)$	$(1,2)$	$(1,3,2)$	$\mathsf{Id}_{[3]}$
$(1,3,2)$	$(1,3,2)$	$(2,3)$	$(1,2)$	$(1,3)$	$\mathsf{Id}_{[3]}$	$(1,2,3)$

mit z.B.

$$(1,2) \circ (2,3) = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 1 & 3 \end{pmatrix} \circ \begin{pmatrix} 1 & 2 & 3 \\ 1 & 3 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 3 & 1 \end{pmatrix} = (1,2,3)$$

und

$$(2,3) \circ (1,2) = \begin{pmatrix} 1 & 2 & 3 \\ 1 & 3 & 2 \end{pmatrix} \circ \begin{pmatrix} 1 & 2 & 3 \\ 2 & 1 & 3 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 3 \\ 3 & 1 & 2 \end{pmatrix} = (1,3,2)$$

Fkt $\mathcal{S}_n$ ist nicht kommutativ für $n \geq 3$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Eindeutigkeit von Neutralem und Inversen

Lem Sei $\mathbb{G} \hat{=} (\mathbb{G}, \cdot, 1)$ eine Gruppe. Dann gilt stets (ab für $a \cdot b$):

1. Das Neutrale ist eindeutig da:

   Annahme: Es gibt $1' \in \mathbb{G}$ mit $1'a = a = a1'$ für alle $a \in \mathbb{G}$.

   Dann: $1 = 1'1 = 1'$.

2. Das Inverse ist eindeutig da:

   Annahme: Es gibt $a, b, c \in \mathbb{G}$ mit $ab = ba = ac = ca = 1$.

   Dann: $b = b1 = b(ac) = (ba)c = 1c = c$

Def Für multiplikative Gruppen $(\mathbb{G}, \cdot, 1)$ schreibt man a^{-1} für das (jetzt) eindeutige Inverse von $a \in \mathbb{G}$; für additive Gruppen $(\mathbb{G}, +, 0)$ entsprechend $-a$.

3. $(a^{-1})^{-1} = a$ und $(ab)^{-1} = b^{-1}a^{-1}$ da:

   Einfach nachrechnen, dass L.S. und R.S. jeweils invers zu a^{-1} bzw. ab sind.

⚠ In additiver Notation: $-(-a) = a$ und $-(a + b) = (-b) + (-a)$.

(Wurde bereits für Permutationen gezeigt und überträgt sich auf Grund des Satzes von Cayley.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

⚠ Inverses stets von der betrachteten Gruppe/Operation abhängig, z.B.:

Inverses von 5 in	$(\mathbb{Z}_{911}, +_{911}, 0)$	:	906
Inverses von 5 in	$(\mathbb{Z}^*_{911}, \cdot_{911}, 1)$	:	729
Inverses von 5 in	$(\mathbb{R}_{>0}, \cdot, 1)$	:	0.2
Inverses von 5 in	$(\mathbb{R}, +, 0)$	:	-5
Inverses von 5 in	„AES“	:	82

(AES rechnet in GF(2^8) bzgl. $1 + x + x^3 + x^4 + x^8$.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Kürzungsregel

Lem Sei $\mathbb{G} \hat{=} (\mathbb{G}, \cdot, 1)$ eine Gruppe. Dann darf man mittels a^{-1} kürzen:

- ▷ Falls $ab = ac$, dann $b = c$.
- ▷ Falls $ba = ca$, dann $b = c$

da: $b = 1b = (a^{-1}a)b = a^{-1}(ab) = a^{-1}(ac) = (a^{-1}a)c = 1c = c$

Kor

- ▷ $a^{-1}b$ ist die eindeutige Lösung von $ax = b$ da:

  $ax = b = a(a^{-1}b)$, also $x = a^{-1}b$ nach Kürzungsregel.

  Entsprechend ba^{-1} eindeutige Lösung von $xa = b$.

- ▷ Für jedes $a \in \mathbb{G}$ sind $l_a(x) := ax$ und $r_a(x) := xa$ Permutationen von $\mathbb{G}$.

  Insbesondere: $r_a^{-1} = r_{a^{-1}}$ und $r_b \circ r_a = r_{ab}$. (Entsprechend für l_a.)

  $l_a = r_a$ **gdw** a kommutiert mit jedem $x \in \mathbb{G}$.

  (D.h. $l_a \neq r_a$ i.A. in nicht-kommutativen Gruppen wie z.B. Matrizen.)

  $h \colon \mathbb{G} \to \mathcal{S}_{\mathbb{G}}, a \mapsto l_a$ ist ein injektiver Gruppenhomomorphismus.

  ($h^{-1}(l_a) := l_a(1) = a$; Satz von Cayley.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

$(\mathbb{Z}_9^*, \cdot_9, 1)$	1	2	4	5	7	8
1	1	2	4	5	7	8
2	2	4	8	1	5	7
4	4	8	7	2	1	5
5	5	1	2	7	8	4
7	7	5	1	8	4	2
8	8	7	5	4	2	1

$(\mathcal{S}_3, \circ, \mathsf{Id}_3)$	$\mathsf{Id}_{[3]}$	$(1,2)$	$(1,3)$	$(2,3)$	$(1,2,3)$	$(1,3,2)$
$\mathsf{Id}_{[3]}$	$\mathsf{Id}_{[3]}$	$(1,2)$	$(1,3)$	$(2,3)$	$(1,2,3)$	$(1,3,2)$
$(1,2)$	$(1,2)$	$\mathsf{Id}_{[3]}$	$(1,3,2)$	$(1,2,3)$	$(2,3)$	$(1,3)$
$(1,3)$	$(1,3)$	$(1,2,3)$	$\mathsf{Id}_{[3]}$	$(1,3,2)$	$(1,2)$	$(2,3)$
$(2,3)$	$(2,3)$	$(1,3,2)$	$(1,2,3)$	$\mathsf{Id}_{[3]}$	$(1,3)$	$(1,2)$
$(1,2,3)$	$(1,2,3)$	$(1,3)$	$(2,3)$	$(1,2)$	$(1,3,2)$	$\mathsf{Id}_{[3]}$
$(1,3,2)$	$(1,3,2)$	$(2,3)$	$(1,2)$	$(1,3)$	$\mathsf{Id}_{[3]}$	$(1,2,3)$

- r_a (l_a) entspricht gerade der Spalte (Zeile) zu a in der Verknüpfungstafel (engl. Cayley table), d.h. die Zeilen und Spalten müssen Permutationen von $\mathbb{G}$ sein.
- Konsequenz: r_a und l_a können in Zykelschreibweise angegeben werden, z.B.:

$$l_5 = r_5 = (1,5,7,8,4,2) \qquad r_5^2 = r_{25 \bmod 9} = r_7 = l_7 = (1,7,4)(2,5,8)$$

(Satz von Cayley: Jede Gruppe $\mathbb{G}$ ist isomorph zu einer Untergruppe von $\mathcal{S}_{\mathbb{G}}$: $r_b \circ r_a = r_{ab}$).

- Anwendung One-Time-Pad:
  1. Wähle $a \in \mathbb{G}$ zufällig und geheim.
  2. Einmal-Verschlüsselung von $x \in \mathbb{G}$ als $y := l_a(x) = ax$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Mit Gruppen verwandte algebraische Strukturen

Def Monoid $(M, \cdot, 1)$

$\cdot$ assoziativ, 1 Neutrales (Inverse nicht verlangt).

▷ z.B. $(\mathbb{N}_0, +, 0)$, $(\mathbb{N}, \cdot, 1)$, $(\mathbb{R}_{\geq 0}, +, 0)$, $(\mathbb{Z}, \cdot, 1)$, $(\Sigma^*, \circ, \varepsilon)$

Def Kommutativer Ring mit Eins $(R, +, \cdot, 0, 1)$

$(R, +, 0)$: kom. Gruppe; $(R, \cdot, 1)$: kom. Monoid;
verbunden durch Distributivität: $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$.

▷ $(\mathbb{Z}, +, \cdot, 0, 1)$, $(\mathbb{Z}_n, +_n, \cdot_n, 0, 1)$, $(\mathbb{R}[X], +, \cdot, 0, 1)$ (Polynome in X)

▷ Siehe Lineare Algebra: Matrizen bilden nicht-kommutativen Ring.

Def Körper $(F, +, \cdot, 0, 1)$

$(F, +, \cdot, 0, 1)$: kom. Ring mit 1; $(F \setminus \{0\}, \cdot, 1)$: kom. Gruppe.

▷ z.B. $(\mathbb{Q}, +, \cdot, 0, 1)$, $(\mathbb{R}, +, \cdot, 0, 1)$

Lem $(\mathbb{Z}_n, +_n, \cdot_n, 0, 1)$ ist ein Körper **gdw** $n \in \mathbb{P}$.

Def F-Vektorraum $(V, +, \cdot, 0)$:

$(V, +, 0)$: kom. Gruppe; F: Körper; skalare Mul.: $\lambda x + \mu y \in V$ ($x, y \in V$, $\lambda, \mu \in F$).

Bsp $(F^n, +, \cdot, 0)$ für $F = \mathbb{Z}_p, \mathbb{Q}, \mathbb{R}$ („Koordinatenvektorräume“, siehe direkte Summe).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Lineare Gleichungssysteme modulo p

Fkt Sei p prim. LGS über $\mathbb{Z}_p$ löst man wie über $\mathbb{R}$, mit $x/y := x \cdot y^{-1}$ für $y \not\equiv_p 0$.
Dann gilt z.B. $x/y + u/v \equiv_p (xv/yv) + (uy/vy) \equiv_p (xv + uy)/(vy)$.

Bsp Lösung unter der Annahme, dass alle Inversen modulo p existieren!:

$$
\begin{array}{l}
5x + 2y \equiv_p 1 \\
3x + 4y \equiv_p 2 \\
\hline
x \equiv_p (1 - 2y)/5 \qquad \text{(benötigt } 5 \not\equiv_p 0\text{)} \\
\hline
(3 - 6y)/5 + 4y \equiv_p 2 \\
\hline
3 - 6y + 20y \equiv_p 10 \qquad (*) \\
\hline
y \equiv_p 7/14 \qquad \text{(benötigt } 14 \not\equiv_p 0\text{)} \\
\hline
x \equiv_p 0
\end{array}
$$

Modulo $p = 3$ bekommen wir $y \equiv_3 2$.

Aber modulo $p = 7$ würde Gleichung (*) zu $0 \equiv_7 0$ werden, d.h. y kann beliebig gewählt werden.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

6 Grundlagen der Gruppen- und Zahlentheorie

Zahlentheoretische Grundlagen

Gruppen: Grundlagen

Ordnung eines Elements, zyklische Gruppen, Gruppenexponent

Untergruppen und Erzeugendensysteme

Homomorphismus und Isomorphismus

Appendix (*)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Potenzen in Gruppen

Def Sei $\mathbb{G}$ eine Gruppe. Für alle $a \in \mathbb{G}$ und $k \in \mathbb{N}_0$:

mul.: a^k (a^{-k}) bezeichnet das Produkt von k Kopien von a (a^{-1}).

$$a^0 := 1 \qquad a^{k+1} := a^k \cdot a \qquad a^{-(k+1)} := a^{-k} \cdot a^{-1}$$

$$\langle a \rangle := a^{\mathbb{Z}} := \{a^k \mid k \in \mathbb{Z}\} = \{\ldots, a^{-3}, a^{-2}, a^{-1}, 1, a, a^2, a^3, \ldots\}$$

add.: ka ($-ka$) bezeichnet die Summe von k Kopien von a ($-a$).

$$0a := 0 \qquad (k+1)a := ka + a \qquad -(k+1)a := -ka + (-a)$$

$$\langle a \rangle := a\mathbb{Z} := \{ka \mid k \in \mathbb{Z}\} = \{\ldots, -3a, -2a, -a, 0, a, 2a, 3a, \ldots\}$$

⚠ Der Exponent k in a^k gibt nur die Vielfachheit von a bzw. a^{-1} an.

Fkt Im Exponent rechnet man daher in $\mathbb{Z}$, d.h. für alle $a \in \mathbb{G}$, $k, l \in \mathbb{Z}$:

$$(a^k)^l = a^{kl} = (a^l)^k \qquad a^k a^l = a^{k+l} = a^l a^k \qquad \langle a \rangle = \langle a^{-1} \rangle$$

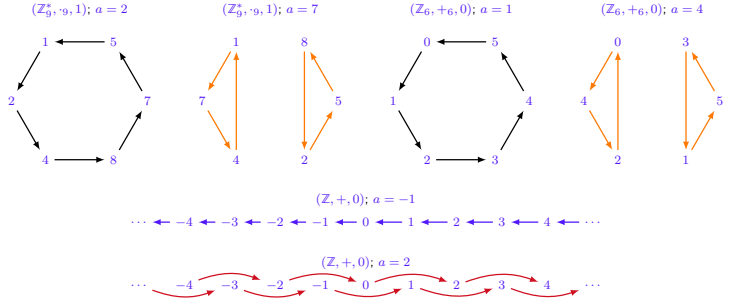
(Vgl.: R^k, f^k für Relationen und Funktionen, insb. $r_a^k = r_{a^k}$ und $r_a^{-1} = r_{a^{-1}}$.)

Kor $h\colon (\mathbb{Z}, +, 0) \to \mathbb{G}, k \mapsto a^k$ ist ein Homomorphismus, $\operatorname{im} h = \langle a \rangle$ eine kommutative Untergruppe von $\mathbb{G}$.

⚠ $a^k b^k = (ab)^k$ für alle $k \in \mathbb{Z}$ **gdw** $(ab)^2 = a^2 b^2$ **gdw** $ab = ba$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

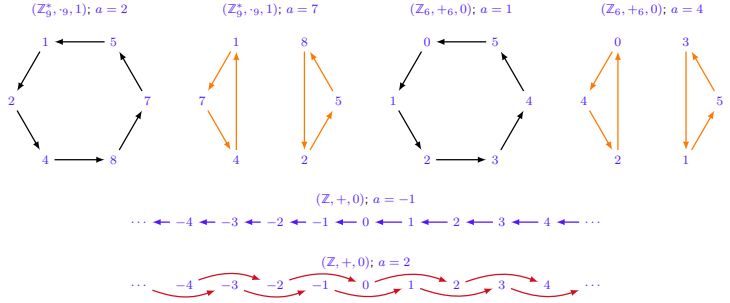
Gruppen als Graphen und ord(a)



- Sei $(\mathbb{G}, \cdot, 1)$ eine Gruppe.
- Jedes $a \in \mathbb{G}$ definiert eine Permutation $r_a(x) = xa$ bzgl. einer Kantenrelation $r_a = \{(x, xa) \mid x \in \mathbb{G}\}$. (Jedes x hat genau einen Nachfolger xa und Vorgänger xa^{-1}.)
- $x\langle a\rangle = \{xa^k \mid k \in \mathbb{Z}\}$ ist die Zshgs-Komponente (CC) von x in $(\mathbb{G}, r_a)$ mit $1\langle a\rangle = \langle a\rangle$ die CC von 1.
- $x\langle a\rangle$ ist ein endlicher einfacher Kreis oder unendlicher einfacher Pfad (Zykelnotation). Falls $\mathbb{G}$ endlich, dann ist $x\langle a\rangle$ immer ein endlicher einfacher Kreis.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gruppen als Graphen und $\mathsf{ord}(a)$



Def $\mathsf{ord}(a) := |\langle a \rangle|$ ist die Ordnung von a.
Wenn $|\langle a \rangle| = |\mathbb{N}|$, dann $\mathsf{ord}(a) = \infty$. (Nach Definition: $|\langle a \rangle| \leq |\mathbb{N}| = |\mathbb{Z}|$.)

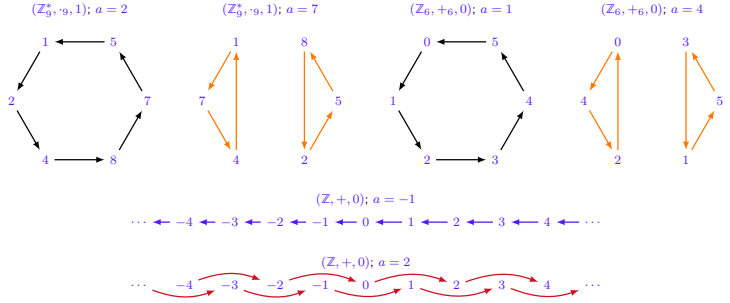
Fkt Mit $h \colon (\mathbb{Z}, +, 0) \to (\mathbb{G}, \cdot, 1), k \mapsto a^k$ gilt entweder $\mathbb{Z} \cong \langle a \rangle$ (wenn h injektiv und $\equiv_h = \mathsf{Id}_{\mathbb{Z}}$) oder $\mathbb{Z}/\,\mathsf{ord}(a)\mathbb{Z} \cong \mathbb{Z}_{\mathsf{ord}(a)} \cong \langle a \rangle$ (sonst $\equiv_h = \equiv_{\mathsf{ord}(a)}$).

Fkt Es gilt $|x\langle a \rangle| = |\langle a \rangle| = \mathsf{ord}(a)$ (da $x^{-1} x a^k = a^k$).

Kor Wenn $\mathbb{G}$ endlich ist, dann hat $(\mathbb{G}, r_a)$ genau $\frac{|\mathbb{G}|}{\mathsf{ord}(a)}$ CCs; insbesondere: $\mathsf{ord}(a)$ teilt $|\mathbb{G}|$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gruppen als Graphen und ord(a)

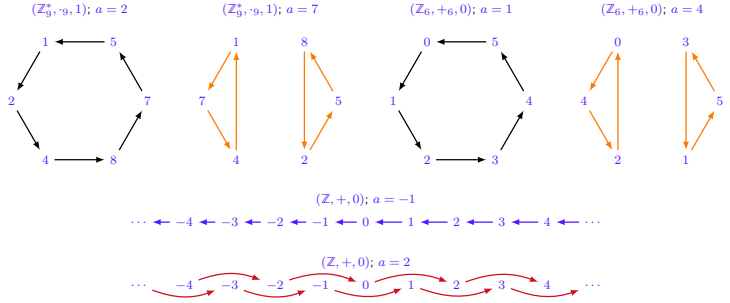


Kor Für endliche ord(a):

- ▷ ord$(a) = 1$ **gdw** $a = 1$ das neutrale Element ist.
- ▷ ord$(a) = |\{a^k \mid k \in \mathbb{N}\}| = \min\{k \in \mathbb{N} \colon a^k = 1\}$.
  (Zeitpunkt der ersten Rückkehr: $1, a, a^2, \ldots, a^{\mathsf{ord}(a)} = 1$ („nur vorwärts“).)
- ▷ $a^{\mathsf{ord}(a)} = 1$, $a^{-1} = a^{\mathsf{ord}(a)-1}$, $a^k = a^{k \bmod \mathsf{ord}(a)}$; $a^k = 1$ **gdw** ord$(a) | k$.
- ▷ $\mathsf{ord}(a^m) = \min\{k \in \mathbb{N} \colon a^{mk} = 1\} = \frac{\mathsf{kgV}(m,\mathsf{ord}(a))}{m} = \frac{\mathsf{ord}(a)}{\mathsf{ggT}(m,\mathsf{ord}(a))}$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gruppen als Graphen und ord(a)



Def $g \in \mathbb{G}$ ist ein Erzeuger von $\mathbb{G}$ (g erzeugt $\mathbb{G}$.) **:gdw** $\langle g \rangle = \mathbb{G}$.
$\mathbb{G}$ heißt zyklisch **:gdw** $\mathbb{G}$ hat einen Erzeuger.
(Wenn $\mathbb{G}$ unendlich, dann ist $\langle g \rangle$ ein unendlicher Pfad: „unendlicher Kreis“)

Kor Angenommen g erzeugt $\mathbb{G} \hat{=} (\mathbb{G}, \cdot, 1)$. Dann $|\mathbb{G}| = |\langle g \rangle| \leq |\mathbb{N}|$.
Wenn $|\mathbb{G}| = |\mathbb{N}|$, dann $(\mathbb{Z}, +, 0) \cong \mathbb{G}$ durch $h(k) = g^k$.
Wenn $|\mathbb{G}| = n$, dann $(\mathbb{Z}_n, +_n, 0) \cong \mathbb{G}$ durch $h(k) = g^{k \bmod n}$ für $n = |\mathbb{G}|$.
($g^{k+l} = g^k g^l$; $g^k = g^l$ **gdw** $g^{k-l} = 1$; im $h = \mathbb{G}$ da zyklisch; s. auch diskreter Logarithmus.)

Ordnung eines Elements: Beispiel $\mathcal{S}_3$

▷ Beispiel: $(\mathcal{S}_3, \circ, \mathsf{Id})$:

$$\begin{aligned}
\langle \mathsf{Id} \rangle &= \{\mathsf{Id}\} \\
\langle (2,3) \rangle &= \{(2,3), \mathsf{Id}\} \\
\langle (1,2) \rangle &= \{(1,2), \mathsf{Id}\} \\
\langle (1,3) \rangle &= \{(1,3), \mathsf{Id}\} \\
\langle (1,2,3) \rangle &= \{(1,2,3), (1,3,2), \mathsf{Id}\} \\
\langle (1,3,2) \rangle &= \langle (1,2,3) \rangle
\end{aligned}$$

Erinnerung: $\mathcal{S}_n$ ist nicht kommutativ für $n \geq 3$ und damit nicht zyklisch.

Fkt Ist $\pi \in \mathcal{S}_n$ eine Permutation, die in Zykelschreibweise die Form

$$\pi = (x_1^{(1)}, \ldots, x_{l_1}^{(1)})(x_1^{(2)}, \ldots, x_{l_2}^{(2)}) \ldots (x_1^{(r)}, \ldots, x_{l_r}^{(r)})$$

hat, dann gilt:

$$\mathsf{ord}(\pi) = |(\pi)| = \mathsf{kgV}(l_1, l_2, \ldots, l_r)$$

(Intuition: r Uhren, i-te Uhr braucht l_i Zeiteinheiten für einen Zyklus (!).)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Zyklische Gruppen

- Beispiele:
  - $(\mathbb{Z}, +, 0)$ ist zyklisch; Erzeuger 1 und -1.
  - $(\mathbb{Z}_n, +_n, 0)$ ist zyklisch; Erzeuger $\mathbb{Z}_n^*$ (folgt).
  - $(\mathcal{S}_n, \circ, \mathsf{Id})$ ist nicht zyklisch i.A., da nicht kommutativ für $n \geq 3$.
  - $\langle a \rangle$ ist immer eine zyklische Untergruppe.
  - $(\mathbb{Q}_{>0}, \cdot, 1)$ ist nicht zyklisch.

    (Fixiere $a = r/s \in \mathbb{Q}_{>0}$ mit $\mathsf{ggT}(r, s) = 1$; zeige dann $\langle a \rangle \neq \mathbb{Q}_{>0}$.)
  - $(\mathbb{R}_{>0}, \cdot, 1)$ ist nicht zyklisch. (Eine zyklische Gruppe ist immer abzählbar.)

Thm $(\mathbb{Z}_n^*, \cdot_n, 1)$ zykl. **gdw** $n \in \{2, 4, p^r, 2p^r \mid p \in \mathbb{P} \setminus \{2\}, r \in \mathbb{N}\}$ (ohne Beweis).

- Erster Beweis durch Gauß; Erzeuger von $\mathbb{Z}_n^*$ werden auch Primitivwurzeln genannt (siehe Kreisgruppe).
- $\mathbb{Z}_1^* = \{0\}$ wird manchmal als zyklisch betrachtet mit $0 \equiv_1 1$.

- Jede endliche kommutative Gruppe lässt sich in ein endliches direktes Produkt (direkte Summe) von zyklischen Gruppen zerlegen. (Siehe Hauptsatz über endlich erzeugte abelsche Gruppen.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Erzeuger und Bestimmung von $\mathsf{ord}(a)$

Fkt Jede Gruppe $(\mathbb{G}, \cdot, 1)$ mit $p := |\mathbb{G}| \in \mathbb{P}$ ist zyklisch und nur 1 ist kein Erzeuger.

da: (1) $\mathsf{ord}(a) \in \{1, p\}$ und (2) $\mathsf{ord}(a) = 1$ **gdw** $a = 1$. Also: $\langle a \rangle = \mathbb{G}$ **gdw** $a \neq 1$.

Lem Falls $\mathbb{G}$ zyklisch und $n = |\mathbb{G}|$ endlich, dann hat $\mathbb{G}$ genau $\varphi(n)$ Erzeuger.

da: Es gelte $\mathbb{G} = \langle g \rangle$, d.h. $\mathsf{ord}(g) = n$. Es reicht g^k für $k \in \mathbb{Z}_n$ zu betrachten.

Erinnerung: $\mathsf{ord}(g^k) = \frac{\mathsf{ord}(g)}{\mathsf{ggT}(k, \mathsf{ord}(g))} = \frac{n}{\mathsf{ggT}(k,n)}$.

Damit: $\mathsf{ord}(g^k) = n$ **gdw** $\mathsf{ggT}(k, n) = 1$ **gdw** $k \in \mathbb{Z}_n^*$

Kor

- $\mathbb{Z}_n^*$ ist genau die Menge der Erzeuger von $(\mathbb{Z}_n, +_n, 0)$.
  (Spezialfall: g^k (mult) wird zu $k = k \cdot 1$ (add) mit $\langle 1 \rangle = \mathbb{Z}_n$.)
- $\mathbb{Z}_{p^r}^*$ und $\mathbb{Z}_{2p^r}^*$ haben $\varphi(\varphi(p^r))$ viele Erzeuger ($p > 2$ prim).
  ($n = |\mathbb{Z}_{2p^r}^*| = |\mathbb{Z}_{p^r}^*| = \varphi(p^r)$.)

Lem Sei $(\mathbb{G}, \cdot, 1)$ eine endliche Gruppe mit $n := |\mathbb{G}|$. Dann gilt für $a \in \mathbb{G}$:
$\langle a \rangle = \mathbb{G}$ **gdw** $a^{n/p} \neq 1$ für alle $p \in \mathbb{P}$ mit $p | n$.

(Kontraposition: $\langle a \rangle \neq \mathbb{G}$ **gdw** $\mathsf{ord}(a) < n$ **gdw** es ex. $p \in \mathbb{P}$ mit $\mathsf{ord}(a) | \frac{n}{p}$.)

Fkt Falls $\mathsf{ord}(a) | k$ und $\mathsf{ord}(a) | l$, dann $\mathsf{ord}(a) | \mathsf{ggT}(k, l)$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gruppenexponent, Euler, Fermat, RSA (1)

Def Der Exponent einer endlichen Gruppe $(\mathbb{G}, \cdot, 1)$ ist:

$$\lambda_{\mathbb{G}} := \mathsf{kgV}(\{\mathsf{ord}(a) \mid a \in \mathbb{G}\}) = \min\{\lambda \in \mathbb{N} \mid \text{ für alle } a \in \mathbb{G}\colon a^\lambda = 1\}$$

($\lambda_{\mathbb{G}}$ teilt $|\mathbb{G}|$ (falls endlich); kgV entspricht sup im Teilerverband.)

Kor In jeder endlichen Gruppe $(\mathbb{G}, \cdot, 1)$ gilt für jedes $a \in \mathbb{G}$:

$$a^k = a^{k \bmod |\mathbb{G}|} = a^{k \bmod \lambda_{\mathbb{G}}} = a^{k \bmod \mathsf{ord}(a)}$$

d.h. im Exponenten darf man bzgl. $\equiv_{|\mathbb{G}|}$ bzw. $\equiv_{\lambda_{\mathbb{G}}}$ rechnen.

Speziell: $1 = a^{|\mathbb{G}|} = a^{\lambda_{\mathbb{G}}} = a^{\mathsf{ord}(a)}$ und $a^{-1} = a^{|\mathbb{G}|-1} = a^{\lambda_{\mathbb{G}}-1} = a^{\mathsf{ord}(a)-1}$.

Fkt Falls $\mathbb{G} = \langle g \rangle$ zyklisch, dann $\lambda_{\mathbb{G}} = |\mathbb{G}| = \mathsf{ord}(g)$.

▷ Beispiel: In $(\mathbb{Z}_{11}^*, \cdot_{11}, 1)$ mit $|\mathbb{Z}_{11}^*| = \varphi(11) = 11 - 1$

$$2^{1001} \equiv_{11} 2^{1001 \bmod |\mathbb{Z}_{11}^*|} = 2^1 = 2$$

Weiterhin gilt $\mathsf{ord}(2) = 10$ und daher $\lambda_{\mathbb{Z}_{11}^*} = |\mathbb{Z}_{11}^*| = 10$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gruppenexponent, Euler, Fermat, RSA (2)

Def Der Exponent einer endlichen Gruppe $(\mathbb{G}, \cdot, 1)$ ist:

$$\lambda_{\mathbb{G}} := \mathsf{kgV}(\{\mathsf{ord}(a) \mid a \in \mathbb{G}\}) = \min\{\lambda \in \mathbb{N} \mid \text{ für alle } a \in \mathbb{G}\colon a^\lambda = 1\}$$

($\lambda_{\mathbb{G}}$ teilt $|\mathbb{G}|$ (falls endlich); kgV entspricht sup im Teilerverband.)

Kor In jeder endlichen Gruppe $(\mathbb{G}, \cdot, 1)$ gilt für jedes $a \in \mathbb{G}$:

$$a^k = a^{k \bmod |\mathbb{G}|} = a^{k \bmod \lambda_{\mathbb{G}}} = a^{k \bmod \mathsf{ord}(a)}$$

d.h. im Exponenten darf man bzgl. $\equiv_{|\mathbb{G}|}$ bzw. $\equiv_{\lambda_{\mathbb{G}}}$ rechnen.

Speziell: $1 = a^{|\mathbb{G}|} = a^{\lambda_{\mathbb{G}}} = a^{\mathsf{ord}(a)}$ und $a^{-1} = a^{|\mathbb{G}|-1} = a^{\lambda_{\mathbb{G}}-1} = a^{\mathsf{ord}(a)-1}$.

Fkt $\mathsf{ord}((1, 2, \ldots, k)) = k$ in $\mathcal{S}_n$ für $k \leq n$. Daher: $\lambda_{\mathcal{S}_n} = \mathsf{kgV}([n])$.

▷ Beispiel: In $(\mathcal{S}_6, \circ, \mathsf{Id})$ mit $|\mathcal{S}_6| = 6! = 720$ für $\pi = (1, 2, 3, 4, 5, 6)$:

$$\pi^{787} = \pi^{787 \bmod |\mathcal{S}_6|} = \pi^{67 \bmod \lambda_{\mathcal{S}_6}} = \pi^{7 \bmod \mathsf{ord}(\pi)} = \pi$$

Weiterhin gilt $\mathsf{ord}(\pi) = 6$ und $\lambda_{\mathcal{S}_6} = \mathsf{kgV}([6]) = 60$.

⚠ $|\mathcal{S}_3| = 6 = \lambda_{\mathcal{S}_3}$, aber $\mathcal{S}_3$ ist nicht kommutativ und damit auch nicht zyklisch.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gruppenexponent, Euler, Fermat, RSA (3)

Def Der Exponent einer endlichen Gruppe $(\mathbb{G}, \cdot, 1)$ ist:

$$\lambda_{\mathbb{G}} := \mathsf{kgV}(\{\mathsf{ord}(a) \mid a \in \mathbb{G}\}) = \min\{\lambda \in \mathbb{N} \mid \text{ für alle } a \in \mathbb{G}\colon a^{\lambda} = 1\}$$

($\lambda_{\mathbb{G}}$ teilt $|\mathbb{G}|$ (falls endlich); kgV entspricht sup im Teilerverband.)

Kor In jeder endlichen Gruppe $(\mathbb{G}, \cdot, 1)$ gilt für jedes $a \in \mathbb{G}$:

$$a^k = a^{k \bmod |\mathbb{G}|} = a^{k \bmod \lambda_{\mathbb{G}}} = a^{k \bmod \mathsf{ord}(a)}$$

d.h. im Exponenten darf man bzgl. $\equiv_{|\mathbb{G}|}$ bzw. $\equiv_{\lambda_{\mathbb{G}}}$ rechnen.

Speziell: $1 = a^{|\mathbb{G}|} = a^{\lambda_{\mathbb{G}}} = a^{\mathsf{ord}(a)}$ und $a^{-1} = a^{|\mathbb{G}|-1} = a^{\lambda_{\mathbb{G}}-1} = a^{\mathsf{ord}(a)-1}$.

Fkt Der Exponent von $\mathbb{G}_1 \times \ldots \times \mathbb{G}_n$ ist das KGV der Exponenten aller $\mathbb{G}_i$.

(Wieder n Uhren; i-te Uhr braucht $\lambda_{\mathbb{G}_i}$ Zeiteinheiten für einen Zyklus; siehe direktes Produkt von gerichteten Kreisgraphen; Permutationen in Zykelnotation $\pi = \zeta_1 \ldots \zeta_l$ sind ein Spezialfall.)

Kor $(\mathbb{Z}_m, +_m, 0) \times (\mathbb{Z}_n, +_n, 0)$ ist zyklisch **gdw** $\mathsf{ggT}(m, n) = 1$.

($\mathsf{kgV}(m, n) = mn$ **gdw** $\mathsf{ggT}(m, n) = 1$; und falls $\mathsf{ggT}(m, n) = 1$, dann $\mathsf{ord}((1, 1)) = mn$.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Gruppenexponent, Euler, Fermat, RSA (4)

Def Der Exponent einer endlichen Gruppe $(\mathbb{G}, \cdot, 1)$ ist:

$$\lambda_{\mathbb{G}} := \mathsf{kgV}(\{\mathsf{ord}(a) \mid a \in \mathbb{G}\}) = \min\{\lambda \in \mathbb{N} \mid \text{ für alle } a \in \mathbb{G}\colon a^\lambda = 1\}$$

($\lambda_{\mathbb{G}}$ teilt $|\mathbb{G}|$ (falls endlich); kgV entspricht sup im Teilerverband.)

Kor In jeder endlichen Gruppe $(\mathbb{G}, \cdot, 1)$ gilt für jedes $a \in \mathbb{G}$:

$$a^k = a^{k \bmod |\mathbb{G}|} = a^{k \bmod \lambda_{\mathbb{G}}} = a^{k \bmod \mathsf{ord}(a)}$$

d.h. im Exponenten darf man bzgl. $\equiv_{|\mathbb{G}|}$ bzw. $\equiv_{\lambda_{\mathbb{G}}}$ rechnen.

Speziell: $1 = a^{|\mathbb{G}|} = a^{\lambda_{\mathbb{G}}} = a^{\mathsf{ord}(a)}$ und $a^{-1} = a^{|\mathbb{G}|-1} = a^{\lambda_{\mathbb{G}}-1} = a^{\mathsf{ord}(a)-1}$.

Kor Satz von Euler in $(\mathbb{Z}_n^*, \cdot_n, 1)$ bzw. in $((\mathbb{Z}/n\mathbb{Z})^*, \cdot, [1]_n)$:

$$a^{\varphi(n)} = 1 \quad \text{bzw.} \quad a^{\varphi(n)} \equiv_n 1 \text{ falls } \mathsf{ggT}(a, n) = 1$$

Kor Kleiner Fermatscher Satz in $(\mathbb{Z}_p^*, \cdot_p, 1)$ bzw. in $((\mathbb{Z}/p\mathbb{Z})^*, \cdot, [1]_p)$ für $p \in \mathbb{P}$:

$$a^{p-1} = 1 \quad \text{bzw.} \quad a^{p-1} \equiv_p 1 \text{ falls } \mathsf{ggT}(a, p) = 1$$

Insbesondere $a^p \equiv_p a$ für alle $a \in \mathbb{Z}$.

Gruppenexponent, Euler, Fermat, RSA (5)

Def Der Exponent einer endlichen Gruppe $(\mathbb{G}, \cdot, 1)$ ist:

$$\lambda_{\mathbb{G}} := \mathsf{kgV}(\{\mathsf{ord}(a) \mid a \in \mathbb{G}\}) = \min\{\lambda \in \mathbb{N} \mid \text{ für alle } a \in \mathbb{G}\colon a^\lambda = 1\}$$

($\lambda_{\mathbb{G}}$ teilt $|\mathbb{G}|$ (falls endlich); kgV entspricht sup im Teilerverband.)

Kor In jeder endlichen Gruppe $(\mathbb{G}, \cdot, 1)$ gilt für jedes $a \in \mathbb{G}$:

$$a^k = a^{k \bmod |\mathbb{G}|} = a^{k \bmod \lambda_{\mathbb{G}}} = a^{k \bmod \mathsf{ord}(a)}$$

d.h. im Exponenten darf man bzgl. $\equiv_{|\mathbb{G}|}$ bzw. $\equiv_{\lambda_{\mathbb{G}}}$ rechnen.

Speziell: $1 = a^{|\mathbb{G}|} = a^{\lambda_{\mathbb{G}}} = a^{\mathsf{ord}(a)}$ und $a^{-1} = a^{|\mathbb{G}|-1} = a^{\lambda_{\mathbb{G}}-1} = a^{\mathsf{ord}(a)-1}$.

Kor Falls $\mathsf{ggT}(e, |\mathbb{G}|) = 1$, dann ist $f\colon \mathbb{G} \to \mathbb{G}, x \mapsto x^e$ eine Permutation mit $f^{-1}(x) = x^d$ für jedes d mit $ed \equiv_{|\mathbb{G}|} 1$ **da** für alle $x \in \mathbb{G}$:

$$(x^e)^d = x^{ed} = x^{ed \bmod |\mathbb{G}|} = x^1$$

- ▷ Analog für $\mathsf{ggT}(e, \lambda_{\mathbb{G}}) = 1$ und $ed \equiv_{\lambda_{\mathbb{G}}} 1$.
- ▷ Anwendungen: RSA (Rivest-Shamir-Adleman) Verschlüsselungsverfahren

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Exponent einer endlichen abelschen Gruppe

Lem In jeder endlichen abelschen Gruppe $(\mathbb{G}, \cdot, 1)$ gibt es ein Element der Ordnung $\lambda_{\mathbb{G}}$.

Ü Ist $\mathsf{ggT}(\mathsf{ord}(a), \mathsf{ord}(b)) = 1$, dann $\mathsf{ord}(ab) = \mathsf{ord}(a)\,\mathsf{ord}(b)$.

- ▷ Allgemein: Zerlege $\mathsf{kgV}(\mathsf{ord}(a), \mathsf{ord}(b)) = mn$ mit $\mathsf{ggT}(m, n) = 1$, $m \mid \mathsf{ord}(a)$, $n \mid \mathsf{ord}(b)$; dann hat $a^{\mathsf{ord}(a)/m}\, b^{\mathsf{ord}(b)/n}$ die Ordnung mn.
- ▷ Damit kann man induktiv ein Element der Ordnung $\lambda_{\mathbb{G}} = \mathsf{kgV}(\mathsf{ord}(a) \mid a \in \mathbb{G})$ konstruieren.

  (Vgl. Verbände: $\sup(x, y, z) = \sup(x, \sup(y, z))$.)

Kor Ist $\mathbb{G}$ endlich und abelsch mit $\lambda_{\mathbb{G}} = |\mathbb{G}|$, dann ist $\mathbb{G}$ zyklisch.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Exponent einer endlichen abelschen Gruppe

Kor $(\mathbb{Z}_p^*, \cdot_p, 1)$ ist zyklisch.

- Sei $\lambda := \lambda_{\mathbb{Z}_p^*}$. Es gilt $x^\lambda \equiv_p 1$ für alle $x \in \mathbb{Z}_p^* = \mathbb{Z}_p \setminus \{0\}$, d.h. das Polynom $X^\lambda - 1$ hat mindestens $p-1$ verschiedene Nullstellen.
- Da $(\mathbb{Z}_p, +_p, \cdot_p, 0, 1)$ ein Körper ist, hat $X^\lambda - 1$ höchstens λ Nullstellen (wie über $\mathbb{R}$; ohne Beweis).
- Also $\lambda \geq p-1$ und, da $\lambda \mid p-1$, folgt $\lambda = p-1 = |\mathbb{Z}_p^*|$.

Kor Sei $\mathbb{G}$ endlich und abelsch. Dann ist $f(x) = x^e$ eine Permutation **gdw** $\mathsf{ggT}(e, \lambda_{\mathbb{G}}) = 1$.

- Falls $\mathsf{ggT}(e, \lambda_{\mathbb{G}}) = 1$, dann ist $g(x) = x^d$ die Umkehrabbildung für $ed \equiv_{\lambda_{\mathbb{G}}} 1$.
- Sei $t = \mathsf{ggT}(e, \lambda_{\mathbb{G}}) > 1$. Da es ein Element der Ordnung $\lambda_{\mathbb{G}}$ gibt, gibt es auch ein Element x der Ordnung $t > 1$. Also $x \neq 1$, aber $f(x) = 1 = f(1)$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

RSA (Rivest-Shamir-Adleman) Verschlüsselungsverfahren

1. Alice:
   - Wählt zwei große (verschiedene) Primzahlen p, q z.B. $p = 13$, $q = 17$.
   - Berechnet $N = p \cdot q$ z.B. $N = 221$.
   - Berechnet $\lambda(N) = \mathsf{kgV}(p-1, q-1)$ z.B. $\lambda(221) = 48$.
   - Wählt $e \in \{3, \ldots, \lambda(N)\}$ mit $\mathsf{ggT}(e, \lambda(N)) = 1$ z.B. $e = 5$.
   - Berechnet d mit $(e \cdot d) \bmod \lambda(N) = 1$ z.B. $d = 29$.
   - Sendet e, N an Bob.
2. Bob:
   - Verschlüsselung von m mit $\mathsf{ggT}(m, N) = 1$, z.B. $m = 101$:
   - Berechnet $m' := m^e \bmod N$ z.B. $m' = 186$.
   - Sendet m' an Alice.
3. Alice: Entschlüsselung
   - Berechnet $m = (m')^d \bmod N$ z.B. $m = 101$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

6 Grundlagen der Gruppen- und Zahlentheorie

Zahlentheoretische Grundlagen

Gruppen: Grundlagen

Ordnung eines Elements, zyklische Gruppen, Gruppenexponent

Untergruppen und Erzeugendensysteme

Homomorphismus und Isomorphismus

Appendix (*)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Untergruppen

Def $H \subseteq \mathbb{G}$ ist eine Untergruppe von $(\mathbb{G}, \cdot, 1)$ (kurz: $H \leq \mathbb{G}$), falls $(H, \cdot, 1)$ eine Gruppe ist. (Erinnerung.)

⚠ D.h. man muss zeigen, dass $\cdot$ auch auf H abgeschlossen ist, $1 \in H$ und für jedes $a \in H$ auch $a^{-1} \in H$ gilt.

▷ H muss dabei das Neutrale und die Inversen von $\mathbb{G}$ übernehmen:

Falls $1' \in H$ mit $1' = 1' \cdot 1'$, dann erhält man $1 = 1'$ in $\mathbb{G}$ durch Kürzen von $1'$.

Falls $a, b \in H$ mit $ab = 1$ existieren, dann folgt $a = b^{-1}$ durch Kürzen in $\mathbb{G}$.

Lem Sei $(\mathbb{G}, \cdot, 1)$ Gruppe und $\emptyset \neq H \subseteq \mathbb{G}$.

Dann gilt: $H \leq \mathbb{G}$ **gdw** $ab^{-1} \in H$ für alle $a, b \in H$.

▷ Beweis:

▷ Falls $H \leq \mathbb{G}$, dann mit $a, b \in H$ auch $b^{-1} \in H$ und $ab^{-1} \in H$.

▷ Es gelte $ab^{-1} \in H$ für alle $a, b \in H$.

Da $H \neq \emptyset$, gibt es ein $a \in H$ und damit $1 = aa^{-1} \in H$.

Mit $a \in H$ gilt auch $a^{-1} = 1a^{-1} \in H$.

Mit $a, b \in H$ gilt auch $ab = a(b^{-1})^{-1} \in H$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Untergruppen zyklischer Gruppen

Lem Jede Untergruppe $H \leq \mathbb{G}$ einer zyklischen Gruppe $(\mathbb{G}, \cdot, 1)$ ist zyklisch.

da: Wir nehmen erst $\mathbb{G} = (\mathbb{Z}, +, 0)$ an. OBdA $H \neq \{0\}$.

Mit $x \in H$ auch $-x \in H$.

Sei daher $m = \min(H \cap \mathbb{N})$ und damit $\langle m \rangle = m\mathbb{Z} \subseteq H$.

Dann gilt bereits $H = \langle m \rangle = m\mathbb{Z}$.

da: Ansonsten gibt es ein $n \in H \setminus m\mathbb{Z}$.

Damit aber auch $n + m\mathbb{Z} \subseteq H$ und somit $n \bmod m \in H$.

Für allgemeine zyklische Gruppen betrachte $I = \{k \in \mathbb{Z} \mid g^k \in H\}$.

Dann ist I eine Untergruppe von $(\mathbb{Z}, +, 0)$, so dass $I = m\mathbb{Z}$ und $H = \langle g^m \rangle$.

Lem Sei $\mathbb{G} = \langle g \rangle$ eine endliche zyklische Gruppe mit $N = |\mathbb{G}|$.

Dann ist $\langle g^d \rangle$ für jeden Teiler d von N die einzige Untergruppe der Ordnung $\frac{N}{d}$.

da: Es reicht g^k für $k \in \mathbb{Z}_N$ zu betrachten, da alle Untergruppen zyklisch.

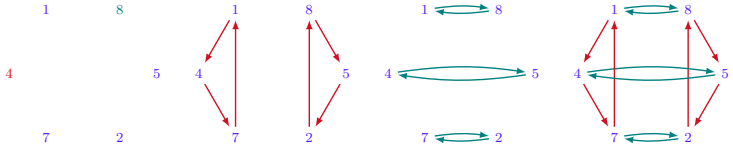
Dann $\mathsf{ord}(g^k) = \frac{\mathsf{ord}(g)}{\mathsf{ggT}(k, \mathsf{ord}(g))} = \frac{N}{\mathsf{ggT}(k,N)} = \frac{N}{d}$ für $d = \mathsf{ggT}(k, N)$.

Mittels EEA: $d = \mathsf{ggT}(k, N) = \lambda k + \mu N$ für geeignete $\lambda, \mu \in \mathbb{Z}$.

Daher $g^d = g^{\lambda k + \mu N} = (g^k)^\lambda \in \langle g^k \rangle$, aber auch $\mathsf{ord}(g^d) = \frac{N}{\mathsf{ggT}(d,N)} = \frac{N}{d}$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Bsp $(\mathbb{Z}_9^*, \cdot_9, 1)$ für $\Sigma = \{4, 8\}$:



- Im Allgemeinen ist eine beliebige Menge $\Sigma \subseteq \mathbb{G}$ keine Untergruppe.
  Aber man kann die kleinste Untergruppe $\langle\Sigma\rangle$ konstruieren, die Σ enthält.

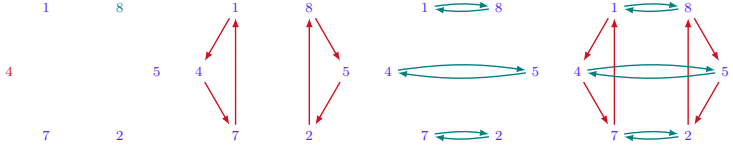
Def Die von Σ erzeugte Untergruppe in $(\mathbb{G}, \cdot, 1)$ ist

$$\langle\Sigma\rangle := \{1a_1^{e_1} \cdots a_l^{e_l} \mid l \in \mathbb{N}_0, a_i \in \Sigma, e_i \in \{-1, 1\}\}$$

- Alle endlichen Produkte der Elemente in Σ oder ihrer Inversen.
- Produkt zweier endlicher Produkte ist endlich, daher ist $\langle\Sigma\rangle$ abgeschlossen.
- Entspricht symmetrischem reflexiv-transitivem Abschluss (CCs): endliche Produkte sind endliche Pfade.
- Lineare Algebra: Vektorraum erzeugt (aufgespannt) von einer Menge Vektoren.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Bsp $(\mathbb{Z}_9^*, \cdot_9, 1)$ für $\Sigma = \{4, 8\}$:



Def Sei $(\mathbb{G}, \cdot, 1)$ eine Gruppe und $\Sigma \subseteq \mathbb{G}$. Der Cayleygraph von $\mathbb{G}$ bzgl. Σ ist

$$\Gamma(\mathbb{G}, \Sigma) := (\mathbb{G}, \{(x, xa) \mid x \in \mathbb{G}, a \in \Sigma\})$$

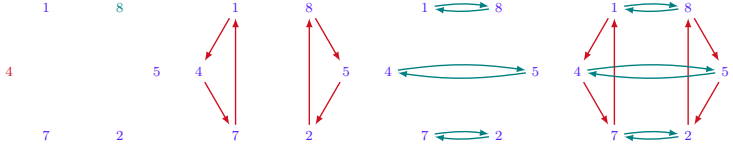
- $\Gamma(\mathbb{G}, \Sigma)$ ist die Vereinigung der Graphen $(\mathbb{G}, r_a)$ für $a \in \Sigma$.
  Jede Kante (x, y) hat die eindeutige „Farbe“ $a = x^{-1}y$.
- $\langle\Sigma\rangle$ ist die CC von 1 in $\Gamma(\mathbb{G}, \Sigma)$ (Spezialfall: $\langle a\rangle$ für $\Sigma = \{a\}$).
  Die CC von x ist $x\langle\Sigma\rangle = \{xa_1^{e_1} \cdots a_l^{e_l} \mid l \in \mathbb{N}_0, a_i \in \Sigma, e_i \in \{-1, 1\}\}$.
  Also $|x\langle\Sigma\rangle| = |\langle\Sigma\rangle|$ und genau $\frac{|\mathbb{G}|}{|\langle\Sigma\rangle|}$ CCs.

Fkt $H \leq \mathbb{G}$ **gdw** $H = \langle H\rangle$.

Thm Satz von Lagrange: Wenn $H \leq \mathbb{G}$, dann ist $|H|$ ein Teiler von $|\mathbb{G}|$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Bsp $(\mathbb{Z}_9^*, \cdot_9, 1)$ für $\Sigma = \{4, 8\}$:



Def Sei $(\mathbb{G}, \cdot, 1)$ eine Gruppe und $\Sigma \subseteq \mathbb{G}$. Der Cayleygraph von $\mathbb{G}$ bzgl. Σ ist

$$\Gamma(\mathbb{G}, \Sigma) := (\mathbb{G}, \{(x, xa) \mid x \in \mathbb{G}, a \in \Sigma\})$$

- $\Gamma(\mathbb{G}, \Sigma)$ ist die Vereinigung der Graphen $(\mathbb{G}, r_a)$ für $a \in \Sigma$.
  Jede Kante (x, y) hat die eindeutige „Farbe“ $a = x^{-1}y$.
- $\langle\Sigma\rangle$ ist die CC von 1 in $\Gamma(\mathbb{G}, \Sigma)$ (Spezialfall: $\langle a\rangle$ für $\Sigma = \{a\}$).
  Die CC von x ist $x\langle\Sigma\rangle = \{xa_1^{e_1} \cdots a_l^{e_l} \mid l \in \mathbb{N}_0, a_i \in \Sigma, e_i \in \{-1, 1\}\}$.
  Also $|x\langle\Sigma\rangle| = |\langle\Sigma\rangle|$ und genau $\frac{|\mathbb{G}|}{|\langle\Sigma\rangle|}$ CCs.

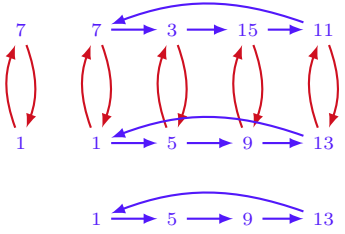
Fkt $H \leq \mathbb{G}$ **gdw** $H = \langle H\rangle$.

Thm Satz von Lagrange: Wenn $H \leq \mathbb{G}$, dann ist $|H|$ ein Teiler von $|\mathbb{G}|$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Σ ist ein Erzeugendensystem von $\mathbb{G}$ **:gdw** $\langle\Sigma\rangle = \mathbb{G}$ (**gdw** $\Gamma(\mathbb{G}, \Sigma)$ ist zshgd.)

▷ Beispiel: $(\mathbb{Z}_{16}^*, \cdot_{16}, 1)$ mit $\Sigma = \{5, 7\}$



Damit $\mathbb{Z}_{16}^* \cong \mathbb{Z}_2 \times \mathbb{Z}_4$ mittels $h(i,j) = (7^i 5^j) \bmod 16$, da $7, 5$ kommutieren.
(Siehe kartesisches Graphprodukt: h wertet $(7^i, 5^j)$ modulo 16 zu $7^i 5^j \bmod 16$ aus.)

Thm Hauptsatz über endlich erzeugte abelsche Gruppen (ohne Beweis)
Falls $\mathbb{G}$ endlich und abelsch ist, dann $\mathbb{G} \cong \mathbb{Z}_{m_1} \times \ldots \times \mathbb{Z}_{m_n}$ (alle additiv) für geeignete Primpotenzen $m_1 \leq \ldots \leq m_n$.
(D.h. es existiert stets ein Erzeugendensystem $\Sigma = \{a_1, \ldots, a_n\}$, so dass $\mathbb{G} = \langle\Sigma\rangle$ und $\Gamma(\mathbb{G}, \Sigma)$ isomorph zu dem kartesischen Produkt der durch $\langle a_i \rangle$ gegebenen Kreisgraphen ist.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

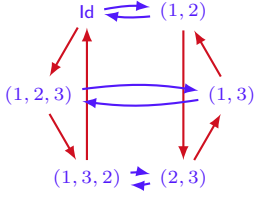
Erzeugendensysteme

Def Σ ist ein Erzeugendensystem von $\mathbb{G}$ **:gdw** $\langle\Sigma\rangle = \mathbb{G}$ (**gdw** $\Gamma(\mathbb{G}, \Sigma)$ ist zshgd.)

Lem $\Sigma = \{(1,2), (1,2,\ldots,n)\}$ ist ein Erzeugendensystem von $(\mathcal{S}_n, \circ, \mathsf{Id})$.

(Entspricht dem Sortieren einer zyklischen Liste mittels Bubblesort.)

- ▷ Beispiel für $n = 3$ mit $(1,2)$ in Blau und $(1,2,3)$ in Rot:



  $\mathcal{S}_3$ hat aber kein Erzeugendensystem mit nur einem Element, da nicht kommutativ und damit auch nicht zyklisch.

- ▷ Falls $\langle\Sigma\rangle = \mathbb{G}$, dann ergibt ein Spannbaum von $\Gamma(\mathbb{G}, \Sigma)$ für jedes $x \in \mathbb{G}$ eine Faktorisierung bzgl. $\Sigma \cup \Sigma^{-1}$. (Z.B. $(1,3) = (1,2) \circ (1,2,3)^2$.)

  Ist $\mathbb{G}$ endlich, dann reicht Σ aus, da $a^{-1} = a^{\mathsf{ord}(a)-1}$ für jedes $a \in \Sigma$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Erzeugendensysteme

Def Σ ist ein Erzeugendensystem von $\mathbb{G}$ **:gdw** $\langle\Sigma\rangle = \mathbb{G}$ (**gdw** $\Gamma(\mathbb{G}, \Sigma)$ ist zshgd.)

- Für jede Gruppe $\mathbb{G}$ ist $\Sigma = \mathbb{G}$ ein Erzeugendensystem.
- Falls $\mathbb{G}$ endlich ist, findet man (irgend)ein Erzeugendensystem, indem man schrittweise $\{a_1, \ldots, a_i\}$ um ein $a_{i+1} \notin \langle a_1, \ldots, a_i\rangle$ erweitert (vgl. Spannbaum).
- Aber wie groß muss $|\Sigma|$ mindestens sein?
  - 1 erzeugt sowohl $(\mathbb{Z}, +, 0)$ als auch $(\mathbb{Z}_n, +_n, 0)$ (da zyklisch).
  - $(\mathbb{Q}, +, 0)$ wird von $\{\frac{1}{p^k} \mid p \in \mathbb{P}, k \in \mathbb{N}_0\}$ erzeugt.
    (EEA: falls $1 = \mathsf{ggT}(m, n) = \mu m + \nu n$, dann $\frac{1}{mn} = \frac{\mu}{n} + \frac{\nu}{m}$.)
  - $(\mathbb{Q}_{>0}, \cdot, 1)$ wird von $\mathbb{P}$ erzeugt.
  - $(\mathbb{R}, +, 0)$ besitzt kein abzählbares Erzeugendensystem:
    Ist Σ abzählbar, dann auch $\langle\Sigma\rangle$.
  - $(\mathbb{Z}_m, +_m, 0) \times (\mathbb{Z}_n, +_n, 0)$ hat z.B. $\{(1, 0), (0, 1)\}$ als Erzeugendensystem.
    Falls $\mathsf{ggT}(m, n) = 1$, dann ist auch $\{(1, 1)\}$ ein Erzeugendensystem.
    ($\Gamma(\mathbb{Z}_m \times \mathbb{Z}_n, \{(1, 0), (0, 1)\})$ ist das kartesische Produkt von $\Gamma(\mathbb{Z}_m, \{1\})$ und $\Gamma(\mathbb{Z}_n, \{1\})$, während $\Gamma(\mathbb{Z}_m \times \mathbb{Z}_n, \{(1, 1)\})$ ihr direktes Produkt ist (als Graphen).)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

6 Grundlagen der Gruppen- und Zahlentheorie

Zahlentheoretische Grundlagen

Gruppen: Grundlagen

Ordnung eines Elements, zyklische Gruppen, Gruppenexponent

Untergruppen und Erzeugendensysteme

Homomorphismus und Isomorphismus

Appendix (*)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Seien $(\mathbb{G}_A, \cdot_A, 1_A)$ und $(\mathbb{G}_B, \cdot_B, 1_B)$ Gruppen.
$h\colon \mathbb{G}_A \to \mathbb{G}_B$ heißt Homomorphismus, falls für alle $a, a' \in \mathbb{G}_A$

$$h(a \cdot_A a') = h(a) \cdot_B h(a')$$

Falls h bijektiv, nennt man h Isomorphismus
und $\mathbb{G}_A, \mathbb{G}_B$ isomorph ($\mathbb{G}_A \cong \mathbb{G}_B$).

- Beispiele: Homomorphismen („von ähnlicher Gestalt“)
  - $h\colon (\mathbb{Z}, +, 0) \xrightarrow{\text{surj}} (\mathbb{Z}/n\mathbb{Z}, +, [0]_n) \xrightarrow{\text{bij}} (\mathbb{Z}_n, +_n, 0), z \mapsto [z]_n \mapsto z \bmod n$
  - $h\colon (\mathbb{R}, +, 0) \xrightarrow{\text{surj}} (\{(x,y) \in \mathbb{R}^2 \mid x^2 + y^2 = 1\}, \oplus, (0,1)), t \mapsto (\sin t, \cos t)$
  - $h\colon (\mathbb{G}, \cdot, 1) \xrightarrow{\text{inj}} (\mathcal{S}_{\mathbb{G}}, \cdot, \mathsf{Id}), a \mapsto l_a$ (Satz von Cayley; $l_a(x) := ax$).
  - $h\colon (\mathbb{Z}, +, 0) \xrightarrow{\text{inj}} (\mathbb{R}_{>0}, \cdot, 1), x \mapsto e^x$.
  - $h\colon (\mathbb{Z}, +, 0) \to (\mathbb{G}, \cdot, 1), x \mapsto a^x$ mit $h(\mathbb{Z}) = \langle a \rangle$ (für bel. $a \in \mathbb{G}$).
    Falls $n := \mathsf{ord}(a) < \infty$, dann „$h(k) = a^k = a^l = h(l)$ **gdw** $k \equiv_n l$“, s.d.:
    $h\colon (\mathbb{Z}, +, 0) \xrightarrow{\text{surj}} (\mathbb{Z}_n, +_n, 0) \xrightarrow{\text{inj}} (\mathbb{G}, \cdot, 1), k \mapsto (k \bmod n) \mapsto a^{k \bmod n}$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Seien $(\mathbb{G}_A, \cdot_A, 1_A)$ und $(\mathbb{G}_B, \cdot_B, 1_B)$ Gruppen.
$h\colon \mathbb{G}_A \to \mathbb{G}_B$ heißt Homomorphismus, falls für alle $a, a' \in \mathbb{G}_A$

$$h(a \cdot_A a') = h(a) \cdot_B h(a')$$

Falls h bijektiv, nennt man h Isomorphismus
und $\mathbb{G}_A, \mathbb{G}_B$ isomorph ($\mathbb{G}_A \cong \mathbb{G}_B$).

- Beispiele: Isomorphismen („von gleicher Gestalt“)
  - $(\mathbb{Z}/n\mathbb{Z}, +, [0]_n) \cong (\mathbb{Z}_n, +_n, 0)$ und $((\mathbb{Z}/n\mathbb{Z})^*, \cdot, [1]_n) \cong (\mathbb{Z}_n^*, \cdot_n, 1)$
    mittels $h([a]_n) = a \bmod n$.
  - $(\mathbb{R}, +, 0) \cong (\mathbb{R}_{>0}, \cdot, 1)$ mittels $h(x) = e^x$ und $h^{-1}(x) = \log x$.
  - $(\mathbb{Z}, +, 0) \cong (\mathbb{G}, \cdot, 1)$ mittels $h(x) = g^x$, falls $\langle g \rangle = \mathbb{G}$ und $|\mathbb{G}| = \infty$.
  - $(\mathbb{Z}_n, +_n, 0) \cong (\mathbb{G}, \cdot, 1)$ mittels $h(x) = g^x$, falls $\langle g \rangle = \mathbb{G}$ und $n = |\mathbb{G}|$.
  - $(\mathbb{Z}_n, +_n, 0) \cong (\{(\sin(\frac{2\pi k}{n}), \cos(\frac{2\pi k}{n})) \mid k \in \mathbb{Z}_n\}, \oplus, (0,1))$
    mittels $h(k) = (\sin(\frac{2\pi k}{n}), \cos(\frac{2\pi k}{n}))$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Def Seien $(\mathbb{G}_A, \cdot_A, 1_A)$ und $(\mathbb{G}_B, \cdot_B, 1_B)$ Gruppen.

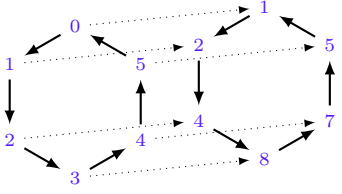
$h\colon \mathbb{G}_A \to \mathbb{G}_B$ heißt Homomorphismus, falls für alle $a, a' \in \mathbb{G}_A$

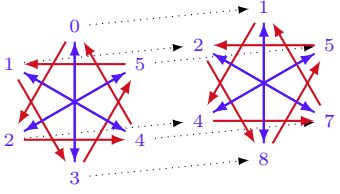
$$h(a \cdot_A a') = h(a) \cdot_B h(a')$$

Falls h bijektiv, nennt man h Isomorphismus

und $\mathbb{G}_A, \mathbb{G}_B$ isomorph ($\mathbb{G}_A \cong \mathbb{G}_B$).

▷ Anmerkung: Falls $\mathbb{G}_1 \cong \mathbb{G}_2$ mittels h, dann ergibt h Kantenfarben erhaltenden Graphisomorphismus mit $\Gamma(\mathbb{G}_1, \Sigma) \cong \Gamma(\mathbb{G}_2, h(\Sigma))$.





Zu jeder endlichen Gruppe $\mathbb{G}$ existiert auch ein (einfacher) Graph G, s.d. $\mathbb{G}$ isomorph zu der Gruppe der Graphautomorphismen von G ist (siehe Anhang bzw. Harary).

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Thm Ist $h\colon \mathbb{G}_A \to \mathbb{G}_B$ ein Gruppenhomomorphismus, dann gilt:

1. $h(1_A) = 1_B$ (**da**: $1_B \cdot_B h(1_A) = h(1_A) = h(1_A \cdot_A 1_A) = h(1_A) \cdot_B h(1_A)$)
2. $h(a^{-1}) = h(a)^{-1}$ (**da**: $h(a) \cdot_B h(a)^{-1} = 1_B = h(a \cdot_A a^{-1}) = h(a) \cdot_B h(a^{-1})$)
3. $h(\langle a\rangle) = \langle h(a)\rangle$ (**da**: $h(\langle a\rangle) = \{h(a^k) \mid k \in \mathbb{Z}\} = \{h(a)^k \mid k \in \mathbb{Z}\} = \langle h(a)\rangle$)
4. $\mathsf{ord}(h(a)) \mid \mathsf{ord}(a)$; falls h bijektiv, dann $\mathsf{ord}(a) = \mathsf{ord}(h(a))$.

   (**da**: $h(a)^{\mathsf{ord}(a)} = h(a^{\mathsf{ord}(a)}) = 1_B$)
5. $\mathsf{im}\, h = h(\mathbb{G}_A) \leq \mathbb{G}_B$ (**da**: $h(x) \cdot_B h(y)^{-1} = h(x \cdot_A y^{-1}) \in h(\mathbb{G}_A)$)
6. $\ker h := h^{-1}(1_B) \leq \mathbb{G}_A$ (**da**: $h(x \cdot_A y^{-1}) = h(x) \cdot_B h(y)^{-1} = 1_B \cdot 1_B = 1_B$)

   h ist injektiv **gdw** $\ker h = \{1_A\}$. („Kern von h“)
7. Ist h bijektiv, dann ist auch die Umkehrabbildung h^{-1} ein Isomorphismus.

   (**da**: $h^{-1}(h(x) \cdot_B h(y)) = h^{-1}(h(x \cdot_A y)) = x \cdot_A y = h^{-1}(h(x)) \cdot_A h^{-1}(h(y))$)
8. Ist h bij., dann: $\langle a\rangle = \mathbb{G}_A$ **gdw** $\langle h(a)\rangle = \mathbb{G}_B$ (**da**: $\langle h(a)\rangle = h(\langle a\rangle) = h(\mathbb{G}_A) = \mathbb{G}_B$)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

$\equiv_h$ und Homomorphismen (*)

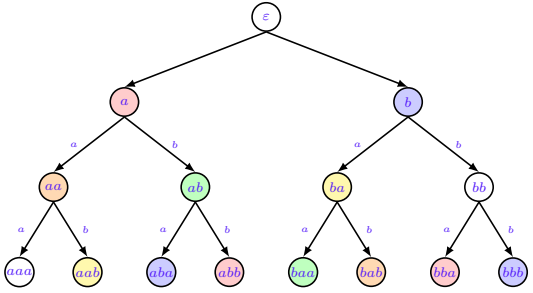
- Sei $h\colon \mathbb{G} \to \mathbb{G}'$ ein Homomorphismus:
  - Als Abbildung definiert h eine Äquivalenzrelation $\equiv_h$ („selbes Bild“) auf $\mathbb{G}$ und eine Bijektion $\tilde{h}\colon \mathbb{G}/\equiv_h \to \mathsf{im}(h), [a] \mapsto h(a)$.
  - $[a] = h^{-1}(h(a)) = a \cdot h^{-1}(1) = a \cdot \mathsf{ker}(h)$ mit $\mathsf{ker}(h) \leq \mathbb{G}$.
  - $\equiv_h$ ist eine Kongruenz, d.h.: Aus $a \equiv_h a'$ und $b \equiv_h b'$ folgt $ab \equiv_h a'b'$.
  - $\tilde{h}$ ist ein Isomorphismus von $\mathbb{G}/\equiv_h$ nach $\mathsf{im}(h) \leq \mathbb{G}'$.
- Jeder Homomorphismus h ist somit die Komposition eines surjektiven Homomorphismus $\mathbb{G} \to \mathbb{G}/\equiv_h$ („Gruppieren nach Bild“) und eines Isomorphismus $\mathbb{G}/\equiv_h \to \mathsf{im}(h)$.
- Das überträgt sich auf den Cayleygraphen $G = \Gamma(\mathbb{G}, \Sigma)$: Der Quotientengraph $G/\equiv_h$ ist isomorph zu $\langle h(\Sigma)\rangle$ als Teilgraph von $\Gamma(\mathbb{G}', h(\Sigma))$.

Bsp Für $h\colon \mathbb{Z} \xrightarrow{\text{surj}} \mathbb{Z}_n, x \mapsto x \bmod n$ erhält man $\equiv_n$ mit $[a] = a + n\mathbb{Z}$, $\mathsf{ker}(h) = n\mathbb{Z}$ und $\mathbb{Z}/\equiv_n \cong \mathbb{Z}_n$ ($\mathbb{Z}$ als Helix im $\mathbb{R}^3$, $\mathbb{Z}_n$ als ihre Projektion entlang $n\mathbb{Z}$ auf die Ebene).

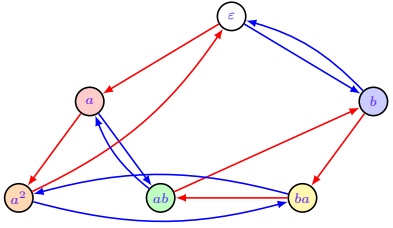
Bsp Betrachte $h\colon (\mathbb{Z}_{10}, +_{10}, 0) \to (\mathbb{Z}_{25}^*, \cdot_{25}, 1), k \mapsto 16^k \bmod 25$ bzgl. $\Sigma = \{1\}$.

Bsp Betrachte $h\colon (\mathbb{R}, +, 0) \to \mathsf{S}^1, t \mapsto (\sin t, \cos t)$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science



$h(a) = (1, 2, 3) \quad h(b) = (1, 2) \qquad a^3 \equiv_h \varepsilon \quad b^2 \equiv_h \varepsilon \quad (ab)^2 \equiv_h \varepsilon$



Chinesischer Restsatz (CRT)

Fkt Direkte Produkte (Erinnerung): Sei $\mathbb{G}_i = (\mathbb{G}_i, \cdot_i, 1_i)$ eine Gruppe für jedes $i \in [n]$.
Dann ist $\mathbb{G}_1 \times \ldots \times \mathbb{G}_n$ auch eine Gruppe bzgl.

$$(a_1, \ldots, a_n) \cdot (b_1, \ldots, b_n) := (a_1 \cdot_1 b_1, \ldots, a_n \cdot_n b_n) \quad 1 := (1_1, \ldots, 1_n)$$

(D.h. man rechnet einfach parallel/komponentenweise in allen Gruppen.)

Thm Seien m, n koprim mit $1 = \mathsf{ggT}(m,n) = \mu m + \nu n$. Dann gilt

$$\mathbb{Z}_{mn} \cong \mathbb{Z}_m \times \mathbb{Z}_n \quad \text{und} \quad \mathbb{Z}_{mn}^* \cong \mathbb{Z}_m^* \times \mathbb{Z}_n^*$$

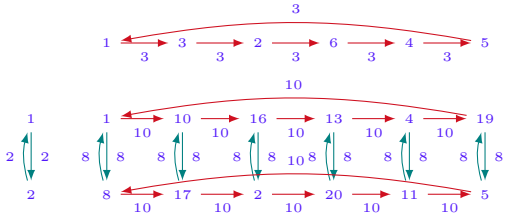
mittels $h\colon \mathbb{Z}_{mn} \to \mathbb{Z}_m \times \mathbb{Z}_n, a \mapsto (a \bmod m, a \bmod n)$, wobei

$$h^{-1}(u,v) = (u\nu n + v\mu m) \bmod mn$$

- Das ist eine Erweiterung von $\equiv_m \cap \equiv_n = \equiv_{\mathsf{kgV}(m,n)}$ im Fall $\mathsf{kgV}(m,n) = mn$.
- Wendet man den CRT rekursiv auf eine Primfaktorzerlegung von $n = p_1^{k_1} \cdots p_r^{k_r}$ an, so erhält man $\mathbb{Z}_n \cong \mathbb{Z}_{p_1^{k_1}} \times \ldots \times \mathbb{Z}_{p_r^{k_r}}$ und $\mathbb{Z}_n^* \cong \mathbb{Z}_{p_1^{k_1}}^* \times \ldots \times \mathbb{Z}_{p_r^{k_r}}^*$ ($p_i < p_{i+1}$).
  Weiterhin gilt (ohne Beweis), dass $\mathbb{Z}_{2^k}^* \cong \mathbb{Z}_2 \times \mathbb{Z}_{2^{k-2}}$, falls $k > 2$ (s. $\mathbb{Z}_{16}^*$), sonst $\mathbb{Z}_{p^k}^* \cong \mathbb{Z}_{p^{k-1}(p-1)} \cong \mathbb{Z}_{p^{k-1}} \times \mathbb{Z}_{p-1}$.
  Damit erhält man einen Spezialfall des Basistheorems für $\mathbb{Z}_n^*$ (und $\mathbb{Z}_n$).
  (Erlaubt auch den Gruppenexponenten von $\mathbb{Z}_n^*$ mittels $n = p_1^{k_1} \cdots p_r^{k_r}$ zu bestimmen.)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 2ow, TUM CIT Department of Computer Science

CRT: $\mathbb{Z}_{21}^* \cong \mathbb{Z}_3^* \times \mathbb{Z}_7^* \cong \mathbb{Z}_2 \times \mathbb{Z}_6$

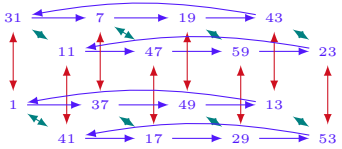


Bsp

- $\mathbb{Z}_{21}^* \cong \mathbb{Z}_3^* \times \mathbb{Z}_7^*$ mittels $h^{-1}(x_3, x_7) = (7x_3 - 6x_7) \bmod 21$.
- $\mathbb{Z}_3^* = \langle 2 \rangle \cong \mathbb{Z}_2$ und $\mathbb{Z}_7^* = \langle 3 \rangle \cong \mathbb{Z}_6$.
- $2 \in \mathbb{Z}_3^*$ entspricht $h^{-1}(2, 1) = 8 \in \mathbb{Z}_{21}^*$.
- $3 \in \mathbb{Z}_7^*$ entspricht $h^{-1}(1, 3) = 10 \in \mathbb{Z}_{21}^*$.
- Also $\Gamma(\mathbb{Z}_3^*, \{2\}) \square \Gamma(\mathbb{Z}_7^*, \{3\}) \cong \Gamma(\mathbb{Z}_{21}^*, \{8, 10\})$ mittels h, h^{-1}, da $\{10, 8\}$ ein kommutierendes Erzeugendensystem ist $(\mathbb{Z}_2 \times \mathbb{Z}_6 \ni (i, j) \mapsto (2^i \bmod 3, 3^j \bmod 7) \mapsto 8^i \cdot_{21} 10^j)$.
- Mittels CRT könnte man sogar $\mathbb{Z}_7^* \cong \mathbb{Z}_6 \cong \mathbb{Z}_2 \times \mathbb{Z}_3$ weiter zerlegen, mit $\mathbb{Z}_7^* = \langle 3^2, 3^3 \rangle$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

CRT: $\mathbb{Z}_{60}^* \cong \mathbb{Z}_4^* \times \mathbb{Z}_3^* \times \mathbb{Z}_5^* \cong \mathbb{Z}_2 \times \mathbb{Z}_2 \times \mathbb{Z}_4$



Bsp

- $\mathbb{Z}_{60}^* \cong \mathbb{Z}_4^* \times \mathbb{Z}_{15}^*$ mit $h_1^{-1}(x_4, x_{15}) = (-15x_4 + 16x_{15}) \bmod 60$
- $\mathbb{Z}_{15}^* \cong \mathbb{Z}_3^* \times \mathbb{Z}_5^*$ mit $h_2^{-1}(x_3, x_5) = (-5x_3 + 6x_5) \bmod 15$
- $\mathbb{Z}_{60}^* \cong \mathbb{Z}_4^* \times \mathbb{Z}_3^* \times \mathbb{Z}_5^*$ mittels $h(x) := (x \bmod 4, x \bmod 3, x \bmod 5)$ und $h^{-1}(x_4, x_3, x_5) = h_1^{-1}(x_4, h_2^{-1}(x_3, x_5))$.
- Es gilt $\mathbb{Z}_4^* = \langle 3 \rangle$, $\mathbb{Z}_3^* = \langle 2 \rangle$, $\mathbb{Z}_5^* = \langle 2 \rangle$ und damit $\mathbb{Z}_2 \times \mathbb{Z}_2 \times \mathbb{Z}_4 \cong \mathbb{Z}_{60}^*$ mittels $H((i,j,k)) := h^{-1}(3^i \bmod 4, 2^j \bmod 3, 2^k \bmod 5)$.
- Erzeugendensystem („Basisvektoren“): $H((1,0,0)) = h^{-1}(3,1,1) = 31$, $H((0,1,0)) = h^{-1}(1,2,1) = 41$, $H((0,0,1)) = h^{-1}(1,1,2) = 37$.
- Z.B. ist $x \in \mathbb{Z}$ eine Lösung von

$$x \equiv_4 2 \qquad x \equiv_3 1 \qquad x \equiv_5 3$$

gdw $x \equiv_{60} h^{-1}(2,1,3) = 58$, d.h. $58 + 60\mathbb{Z}$ ist die Menge aller Lösungen.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

Chinesischer Restsatz: Beweis

▷ Es gelte $1 = \mathsf{ggT}(m,n) = \mu m + \nu n$ und
$g(x_m, x_n) := (x_m \nu n + x_n \mu m) \bmod mn$.
Für $\diamond \in \{+, \cdot\}$ gilt:

$$\begin{aligned} h(x \diamond_{mn} y) &= h(x \diamond y - \lfloor \tfrac{x \diamond y}{mn} \rfloor mn) \\ &= ((x \diamond y) \bmod m, (x \diamond y) \bmod n) \\ &= (x \diamond_m y, x \diamond_n y) = h(x) \diamond h(y) \end{aligned}$$

g, h sind bijektiv mit $g = h^{-1}$:

$$\begin{aligned} g(h(x)) &\equiv_{mn} ((x \bmod m)\nu n + (x \bmod n)\mu m) \\ &\equiv_{mn} ((x - \lfloor \tfrac{x}{m} \rfloor m)\nu n + (x - \lfloor \tfrac{x}{n} \rfloor n)\mu m) \\ &\equiv_{mn} x(\nu n + \mu m) = x \in \mathbb{Z}_{mn} \end{aligned}$$

$h(\mathbb{Z}_{mn}^*) = \mathbb{Z}_m^* \times \mathbb{Z}_n^*$ da:
$x = g(u,v) \in \mathbb{Z}_{mn}^*$ **gdw** $\mathsf{ggT}(x, mn) = 1$
gdw $\mathsf{ggT}(x,m) = 1 = \mathsf{ggT}(x,n)$ (da $\mathsf{ggT}(m,n) = 1$)
gdw $\mathsf{ggT}(x \bmod m, m) = 1 = \mathsf{ggT}(x \bmod n, n)$ (da $\mathsf{ggT}(a,b) = \mathsf{ggT}(a \bmod b, b)$)
gdw $h(x) = (u,v) \in \mathbb{Z}_m^* \times \mathbb{Z}_n^*$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

6 Grundlagen der Gruppen- und Zahlentheorie

Zahlentheoretische Grundlagen

Gruppen: Grundlagen

Ordnung eines Elements, zyklische Gruppen, Gruppenexponent

Untergruppen und Erzeugendensysteme

Homomorphismus und Isomorphismus

Appendix (*)

RSA, Diffie-Hellman/ElGamal (*)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

RSA (*)

Lem Sei $(\mathbb{G}, \cdot, 1)$ eine endliche Gruppe.

Für jedes feste $k \in \mathbb{Z}$ sei $E_k \colon \mathbb{G} \to \mathbb{G}, a \mapsto a^k$.

1. Falls $k \equiv_{|\mathbb{G}|} k'$, dann sind E_k und $E_{k'}$ identisch da

   dann $a^k = a^{k \bmod |\mathbb{G}|} = a^{k' \bmod |\mathbb{G}|} = a^{k'}$ für alle $a \in \mathbb{G}$.

2. Falls $\mathsf{ggT}(k, |\mathbb{G}|) = 1$, dann ist E_k bijektiv und

   es gilt $E_k^{-1} = E_{k'}$ für jedes $k' \in \mathbb{Z}$ mit $k \cdot k' \equiv_{|\mathbb{G}|} 1$ da:

   $E_k(E_{k'}(a)) = a^{kk'} = a^{kk' \bmod |\mathbb{G}|} = a$.

▷ Anmerkung:

- ▷ Auch für $k \not\equiv_{|\mathbb{G}|} k'$ können E_k und $E_{k'}$ identisch sein, z.B.:

  Es gilt $|\mathbb{Z}_{77}^*| = \varphi(77) = 60$ und $7 \not\equiv_{60} 37$.

  Aber $x^7 \equiv_{77} x^{37}$ für alle $x \in \mathbb{Z}_{77}^*$.

- ▷ Genauer gilt: E_k ist bijektiv **gdw** $\mathsf{ggT}(k, \lambda_{\mathbb{G}}) = 1$

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

- Generische Basisversion von RSA:
  1. Empfänger wählt $\mathbb{G}$ so, dass er $M = |\mathbb{G}|$ (oder $M = \lambda_{\mathbb{G}}$) berechnen kann;

     wählt dann $k \in \mathbb{Z}_M^*$ und berechnet $k' \in \mathbb{Z}_M^*$ mit $kk' \equiv_M 1$.
  2. Empfänger veröffentlicht $\mathbb{G}$ und k in geeigneter Form.
  3. Sender codiert Nachricht m in Gruppenelement a;

     schickt das permutierte Element $E_k(a)$ über öffentlichen Kanal.
  4. Empfänger berechnet $a = E_{k'}(E_k(a))$ und gewinnt daraus wieder m.
- Zentrale Anforderung an $\mathbb{G}$:

  Allein unter der Kenntnis von $\mathbb{G}$ und $E_k(a)$ ist es in einem relevanten Zeitraum „nicht möglich“ a (und daher auch nicht $|\mathbb{G}|, \lambda_{\mathbb{G}}, k'$) zu berechnen
- Kandidaten für geeignete Gruppen $\mathbb{G}$:

  $(\mathbb{Z}_{pq}^*, \cdot_{pq}, 1)$ mit $p \neq q$ sehr große Primzahlen. Es gilt $\left|\mathbb{Z}_{pq}^*\right| = (p-1)(q-1)$.

  Empfänger veröffentlicht nur k und ausgewertetes Produkt $N := pq$.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

▷ Beispiel für $p = 41, q = 43, N = pq = 1763$:

Dann gilt $\varphi(41 \cdot 43) = \left|\mathbb{Z}_{pq}^*\right| = (41-1)(43-1) = 2^3 \cdot 5 \cdot 2 \cdot 3 \cdot 7 = 1680$.

Wähle z.B. $k = 11 \in \mathbb{Z}_{1680}^*$.

Mittels EEA:

a	b	$\lfloor b/a \rfloor$	α	β
11	1680	152	611	-4
8	11	1	-4	3
3	8	2	3	-1
2	3	1	-1	1
1	2	2	1	0

Also $k' = 611$ mit $11 \cdot_{1680} 611 = 1$.

„Verschlüsseln“ von 71: $E_{11}(71) = (71^{11}) \bmod 1763 = 270$

„Entschlüsseln“ von 270: $E_{611}(270) = (270^{611}) \bmod 1763 = 71$

⚠ RSA ohne geeignete Randomisierung ist kein sicheres Verschlüsselungsverfahren!

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

ElGamal (*)

- Generische Version von ElGamal:
  1. Empfänger wählt endliche zyklische Gruppe $(\mathbb{G}, \cdot, 1)$ samt Erzeuger g.

     Wählt weiterhin zufälliges $x \in \mathbb{Z}_{|\mathbb{G}|}$ und berechnet $h_x = g^x$ in $\mathbb{G}$.
  2. Empfänger veröffentlicht h_x, g und „$\mathbb{G}$“ in geeigneter Form
  3. Sender codiert Nachricht m als Gruppenelement a.

     Wählt weiterhin zufälliges $y \in \mathbb{Z}_{|\mathbb{G}|}$ und berechnet $h_y = g^y$ und $c = h_x^y \cdot a$ in $\mathbb{G}$.

     Verschickt dann h_y und c über öffentlichen Kanal.
  4. Empfänger berechnet $h_y^{-x} \cdot c = g^{-xy} \cdot g^{xy} \cdot a = a$ in $\mathbb{G}$.
- Notwendige Forderung an $\mathbb{G}$:

  Invertieren von $\mathbb{Z}_{|\mathbb{G}|} \to \mathbb{G}, k \mapsto g^k$ muss „schwer (außer Raten)“ zu berechnen sein (DLP). (In $(\mathbb{Z}_n, +_n, 0)$ einfach $gk \equiv_n y$.)
- Kandidaten für geeignete Gruppen $\mathbb{G}$:

  Bestimmte Untergruppen von $(\mathbb{Z}_p^*, \cdot_p, 1)$ für p prim; bestimmte Untergruppen elliptischer Kurven.

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science

RSA, ElGamal – Anmerkungen (*)

- RSA wird in der vorgestellten Form nur als Baustein für Verschlüsselungsverfahren benutzt.
- Üblicherweise verwendet man RSA nur, um den Schlüssel für z.B. AES-basierte Verschlüsselungsverfahren auszutauschen.
- Für die Einbettung von Binärdaten in die Gruppe wird z.B. das randomisierte Optimal Asymmetric Encryption Padding verwendet.
- Für ElGamal verwendet man meistens die Untergruppe $\langle g^2 \rangle$ von $(\mathbb{Z}_p^*, \cdot_p, 1)$ für $\langle g \rangle = \mathbb{Z}_p^*$ ($p \in \mathbb{P}$).
- ElGamal wird ebenfalls höchstens zum Schlüsselaustausch verwendet – meistens allerdings nur das zugrundeliegende Diffie-Hellman-Protokoll:

  Hier wählt der Sender nur y und sendet nur h_y an den Empfänger. Aus $h_x^y = g^{xy} = h_y^x$ bestimmen beide Parteien dann den gemeinsamen Schlüssel für z.B. AES-basierte Verschlüsselungsverfahren.
- [Aktuelle Empfehlung] bis 2038 (ECRYPT):
  Für RSA $p, q \geq 2^{1500}$.
  Für DLP in $(\mathbb{Z}_p^*, \cdot_p, 1)$: $p \geq 2^{3000}$ ($p - 1$ sollte dominanten Primfaktor haben)

No commercial use, only for use in Discrete Structures (IN0015,INHN0004), 26w, TUM CIT Department of Computer Science